MINA

Product documentation

Mediation, Integration and Automation

Version 1.0.3 - 104 activities, 11 groups, 103 functions and 12 shared connection types.

This manual and the searchable web guide share the same content. Activity tables reflect base editor contracts; selected schemas and provider modes can add fields or requirements. Read the Security section before shared or production deployment.

Use the PDF bookmarks or the linked table of contents to navigate. Content does not certify external providers, guarantee performance, or establish production security readiness.

Contents

MINA documentation 12

Tasks, sub-tasks and event starters 13

Input mapping, schemas and expressions 14

Transformation contracts and validation 15

Transformation tests, templates and durable mediation 16

AI-assisted task and project development 17

Workspace, projects and samples 18

Start 19

Scheduler 21

Call Sub Task 23

End 25

Read File 27

Write File 30

List Files 33

Delete File 36

Rename File 38

Copy File 40

File Poller 42

FTP Get 44

FTP Put 47

FTP Delete 50

FTP Directory 53

FTP Change Directory 56

SFTP Get 59

SFTP Put 62

SFTP Delete 65

SFTP Directory 68

SFTP Change Directory 71

HTTP Listener 74

HTTP Request 77

Read Server-Sent Events 85

WebSocket Exchange 88

HTTP Write Preconditions 91

HTTP Response 94

REST API Receiver 97

REST API Invoke 101

SOAP Service 109

SOAP Request Reply 111

EMS Queue Receiver 114

EMS Topic Subscriber 118

EMS Queue Sender 122

EMS Topic Publisher 126

EMS Request Reply 130

EMS Reply 135

Get JMS Queue Message 140

JMS Receive Message 143

JMS Request Reply 146

JMS Send Message 150

Reply to JMS Message 154

Wait for JMS Request 158

Kafka Receive Message 161

Kafka Send Message 166

Kafka Get Messages 172

Pub/Sub Subscriber 178

Pub/Sub Publisher 181

Dynamic Connection 185

IDoc Acknowledgment 189

IDoc Confirmation 193

IDoc Converter 197

IDoc Listener 200

IDoc Parser 204

IDoc Reader 208

Post IDoc to SAP 211

IDoc Renderer 215

RFC BAPI Listener 218

Invoke RFC BAPI in SAP 222

Reply from RFC BAPI in SAP 226

Read Table 230

JDBC Create 234

JDBC Update 237

JDBC Query 241

JDBC Truncate 245

JDBC Delete 248

Stored Procedure 251

Dynamic SQL 254

Snowflake Insert 258

Snowflake Query 262

Snowflake Update 265

Snowflake Delete 268

Snowflake Bulk Load 271

AMQP Receive Message 276

AMQP Get Message 280

AMQP Send Message 285

AMQP Dead Letter Message 289

Parse XML 292

Render XML 294

Parse JSON 297

Render JSON 299

Parse Data 301

Render Data 305

Read Excel Workbook 308

Mapper 311

Transform 314

Log 316

Catch Exception 318

Throw Exception 320

Rethrow Exception 322

Confirm Message 323

Java Activity 325

Python Invoke 328

External Command 331

Assign 334

Checkpoint 336

Deduplicate Message 338

Message Receipt 341

Replay Messages 344

Aggregate Messages 346

Split Records 349

Sleep 351

Get Shared Variable 353

Set Shared Variable 355

If 357

While True 358

For Each 359

Iterate 361

Repeat Until True 363

Repeat on Error Until True 364

Scope 365

None 366

Critical Section 367

JDBC Transaction 368

Pick First 369

concat 370

trim 371

normalizeSpace 372

upperCase 373

lowerCase 374

capitalize 375

stringLength 376

substring 377

substringBefore 378

substringAfter 379

replace 380

matches 381

startsWith 382

endsWith 383

contains 384

compare 385

translate 386

padLeft 387

padRight 388

tokenize 389

currentDate 390

currentTime 391

currentDateTime 392

parseDate 393

formatDate 394

formatDateTime 395

addDays 396

addMonths 397

addHours 398

addMinutes 399

addSeconds 400

dateDifference 401

year 402

month 403

day 404

hour 405

minute 406

second 407

timezoneFromDateTime 408

equal 409

notEqual 410

greaterThan 411

lessThan 412

greaterOrEqual 413

lessOrEqual 414

add 415

subtract 416

multiply 417

divide 418

mod 419

number 420

integer 421

round 422

roundHalfToEven 423

floor 424

ceiling 425

abs 426

sqrt 427

power 428

modulo 429

clamp 430

min 431

max 432

sum 433

average 434

count 435

lookup 436

lookupTable 437

enrich 438

chunks 439

joinBy 440

group 441

top 442

distinctValues 443

sort 444

reverse 445

first 446

last 447

indexOf 448

subsequence 449

flatten 450

join 451

split 452

deepEqual 453

string 454

boolean 455

jsonParse 456

jsonRender 457

render-xml 458

coalesce 459

default 460

exists 461

empty 462

not 463

ifThenElse 464

uuid 465

base64Encode 466

base64Decode 467

hexEncode 468

hexDecode 469

urlEncode 470

urlDecode 471

hash 472

JDBC shared connection 473

SNOWFLAKE shared connection 477

AMQP shared connection 481

HTTP shared connection 485

FTP shared connection 499

SFTP shared connection 500

EMS shared connection 502

JMS shared connection 505

KAFKA shared connection 508

PUBSUB shared connection 511

SAP shared connection 513

SAP_TID shared connection 516

Environment property defaults 517

MINA documentation site and PDF 551

XML JSON and Data parse and render activities 554

Transformation and mediation capabilities 557

Activity debugging and testing 562

MINA Studio debugging 564

Execution analytics 566

Data and file utilities 567

Runtime logging 569

MINA Control Plane and Data Plane: one-stop guide 570

Python archive modes 579

MINA build and distribution 581

Vendor driver installation 585

MINA environment variables 587

SAP ECC integration runtime guide 591

EMS/JMS sender reuse in Run mode 594

Kafka publishing performance 595

REST APIs and shared HTTP connections 596

HTTP and REST activity verification 607

MINA security review - 2026-09-22 611

Web dependency security update 615

MINA documentation

Getting started

Overview

Mediation, Integration and Automation. This installed reference covers Studio, activities, tasks, groups, mapping functions, shared connections, runtime, deployment and Control Plane. It is available offline and contains no project data or credentials.

Activity fields are extracted from the Studio editor contracts. A shared field marked optional may still be needed for a selected mode or provider. Dynamic schemas are project-specific; the guide explains how Studio derives them rather than inventing their fields.

Use the tree or search to choose a topic. Activity pages have Configuration, Input, Output, Advanced and Errors tabs. Download the complete PDF for the same reference in a portable format.

Scope: this is a product reference, not a claim of provider certification or production security readiness. Read the Security section before shared or production deployment.

Tasks, sub-tasks and event starters

Studio

Configuration

New projects open with a callable Subtask containing Start and End. Add a Starter Task for a configured event such as an HTTP listener or scheduler. Give each task a unique name. Studio replaces spaces in object names with hyphens. Start and End define the Subtask input and output boundaries. Call Sub Task selects a target and may resolve a dynamic override at runtime. Keep an explicit fallback when packaging must discover a called task.

Create activities on the canvas and connect transitions. Conditions use input, environment properties and earlier activity output. Validate Task and Validate Project before Run or Debug. Nested calls are shown in the debug process/activity tree.

Input

Declare the task input using an inline schema or a project schema. Call Sub Task maps into that interface. Event starters receive data from their configured system; they do not require mapping an inbound event from a preceding activity. Real providers still require valid connection and listener configuration.

Output

Map the End activity to the task output interface. A waiting Call Sub Task publishes the called result. Spawn without waiting does not provide the synchronous business result. Project-selected schemas determine actual child fields.

Advanced

Run and ordinary Debug keep continuous starters ready for events until stopped. Activity Testing opens a paused session without starting event listeners. Debug stepping and breakpoints affect the active job; test-bench Run once executes only the selected activity, not a complete process.

Errors

Connect specific error paths or use Catch and Rethrow. Unhandled sub-task errors propagate to the caller. Missing task references, broken mappings, unavailable providers and malformed group graphs require correction before reliable execution.

Input mapping, schemas and expressions

Studio

Mapping workflow

Select an activity and open Input. Drag Data fields, Functions or Constants onto a target, or type an expression. The expanded input editor provides more space and applies edits only after Apply mappings. For typed suggestions enter a dot in a data expression, or press Ctrl+Space; Enter or Tab accepts a suggestion.

```
${input.customer.id}
${properties.routing.destination}
${Previous-Activity.result}
${vars.currentElement}
```

Use the Data tree for the actual saved reference names. A constant string is different from an expression; keep datatype conversion and quoting appropriate for the target. Structural schema nodes are populated through child fields; supported union/scalar targets accept whole values.

Repeating targets support For Each and For Each Group. Conditional mappings support If, When/Otherwise and Choose. String literals require matching single or double quotes; numeric and boolean literals are unquoted. Invalid mappings appear dark red. Enter validates the field, keeps focus on an invalid value, and advances to the next field after success. Conditions support parentheses, not, and, or, comparisons and function calls; quoted text is preserved. For example: (${input.amount} > 10 and contains(${input.status}, "ready")) or false.

Project functions use named parameters such as $value in their definitions, and custom:normalize(${input.name}) when called. Nested project functions work in Mapper, activity Input mappings, Map & Test and both Python archive formats. Calls must supply the declared argument count; missing definitions and excessive recursion produce errors. Built-in function pages list signatures and insertion templates, not guaranteed outputs for every input datatype.

JDBC named parameters such as :orderId create Input parameters.orderId after SQL analysis. Set the parameter datatype before mapping. SAP IDoc fields derive from fetched metadata; XML/JSON/flat data fields derive from the chosen schema. Call Sub Task fields derive from the selected task interface. Refresh metadata and validate after changing contracts.

Transformation contracts and validation

Mapping

Supported contracts

Mapper uses the current selected project schema. JSON schema trees resolve local references and nullable scalar types; recursive structures are bounded. Runtime contract checks support local references, required fields, scalar types, enum/const, additional properties, allOf/anyOf/oneOf/not, array size and uniqueness, tuple items, string length, numeric bounds and multipleOf. Patterns and patternProperties use the bounded regex dependency. Common string formats, conditional contracts, property names, dependencies and contains counts are validated. Remote references and complete JSON Schema conformance remain outside the supported subset.

JSON Parse returns the parsed value directly. Duplicate-key policy supports First wins, Last wins or Error. JSON Render includes all mapped root fields and supports removing null properties. String constants use quotes; numbers, booleans and JSON object/array constants retain their types in Studio and both Python archive formats.

SFTP uses system or configured known-hosts files and verifies host keys by default. Private-key files, passphrases, SSH agent selection and connection timeouts are shared by Studio and Python archives. Explicitly disabling host-key verification permits unknown hosts.

Transformation tests, templates and durable mediation

Mapping

Saved tests and assets

Map & Test includes independently saved input/expected-output cases, field differences, named lookup tables, immutable exported template versions, local version history and contract baseline comparisons. Template imports clear project-specific schema IDs. Both Python archives include run_tests.py and transform_jsonl.py; the latter transforms JSON Lines without executing connectors.

Use lookupTable(value, name, fallback), lookup(value, table, fallback), enrich(record, fields), joinBy(left, right, leftKey, rightKey) and chunks(records, size). Limits bound evaluation steps, execution duration and output size; workflow cancellation signals evaluation checkpoints. Input target search and collapse/expand controls help navigate large schemas.

Durable mediation

General provides Deduplicate Message, Message Receipt, Replay Messages, Aggregate Messages and Split Records. Use the same explicit namespace and durable SQLite file for cooperating activities. Claims return fencing tokens; complete after successful work, fail on the error path and acknowledge the broker last. Replay lists failures or deliberately requeues one stable key. Purge removes expired completed records and retains failed/pending work.

Aggregation requires a stable message ID, a correlation/window key and expected count. Duplicate IDs do not increase the count; forward only when ready=true. A timed-out group needs explicit flush to emit partial records. Use a new correlation/window key for a new group. Local SQLite coordinates processes on one host; use persistent storage in containers. External side effects must be idempotent because these activities do not guarantee exactly-once external writes.

Payload limit is 1 MB, group limit is 64 MB and state capacity defaults to 256 MB. JSONL defaults to 16 MB per batch and 1 MB per output record. Pattern matching times out at 100 ms. These are bounded portable capabilities, not a distributed durable workflow engine or complete schema standards implementation.

AI-assisted task and project development

Studio

Workflow

AI Build accepts a requirement for the current Task or Complete Project. Review the design preview and generated definition before Apply. Generation alone does not deploy or run the application. Without a configured model key, the local blueprint builder is used.

Do not enter credentials or confidential payloads into AI prompts. The current prompt instruction to exclude credentials is not an enforced redaction boundary. Validate and test generated designs, and review the security report before enabling shared access.

Workspace, projects and samples

Studio

Workspace

Use the project tree to manage Tasks, schemas, resources and environment properties. The palette and searchable activity picker add activities to the canvas. Select and move activities, edit their display names, and configure transition conditions. Use Window for the XML/JSON viewers and comparison workspace.

Project export/import preserves editable design artifacts. Deployment export packages an executable application; these are different workflows. Installed samples open as editable project copies. Samples marked connection setup need your own provider configuration and credentials.

Themes include Plain Classic and Plain Studio with light/dark variants. Layout panes can be shown or hidden from menus. The status/footer area exposes background work and execution state.



Start

Activities / Starters & Tasks

Overview

Defines the task input boundary.

For Sub Tasks, the Start schema is the callable input contract. Starter Tasks normally use one external event activity instead.

Activity type: start; operation: start.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

No additional fields declared for this section.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The Task input schema replaces the generic payload boundary when configured.

Field / key	Type / required	Details
Starter payload payload	object Optional	The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes.

Output

The Task input schema replaces the generic payload boundary when configured.

Field / key	Type / required	Details
Initial task payload payload	object Optional	The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

No additional fields declared for this section.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose

sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



Scheduler

Activities / Starters & Tasks

Overview

Runs Scheduler on the task execution path.

Activity type: timer; operation: schedule.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Schedule mode scheduleMode	select Configuration setting	Scheduler configuration. Select date/time, interval or a five-field cron schedule as applicable; runOnceOnLocalStart triggers an immediate local test execution. Choices: dateTime, cron
Scheduled date and time scheduledDateTime	text Configuration setting	Scheduler configuration. Select date/time, interval or a five-field cron schedule as applicable; runOnceOnLocalStart triggers an immediate local test execution.
Cron expression cronExpression	text Configuration setting	Scheduler configuration. Select date/time, interval or a five-field cron schedule as applicable; runOnceOnLocalStart triggers an immediate local test execution.
Time zone timezone	select Configuration setting	Timezone used for database or scheduling date/time interpretation. Match the source timezone and explicitly convert timestamps crossing systems. Choices: local, UTC
Repeat after first run repeatEnabled	boolean Configuration setting	Scheduler configuration. Select date/time, interval or a five-field cron schedule as applicable; runOnceOnLocalStart triggers an immediate local test execution.
Interval interval	number Configuration setting	Polling/scheduling interval for repeated execution. Size the interval to expected arrival rate and processing duration; select the unit where the activity exposes one.
Unit unit	select Configuration setting	Sleep duration and its selected unit, or actual reported sleep in milliseconds. Sleeping pauses this branch, not unrelated parallel branches. Choices: seconds, minutes, hours, days
Run once immediately when started locally runOnceOnLocalStart	boolean Configuration setting	Scheduler configuration. Select date/time, interval or a five-field cron schedule as applicable; runOnceOnLocalStart triggers an immediate local test execution.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

No additional fields declared for this section.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Scheduled time scheduledTime	dateTime Optional	Scheduler output: planned fire time, observed fire time or event sequence. Compare scheduledTime with actualTime when measuring trigger delay.
Actual fire time actualTime	dateTime Optional	Scheduler output: planned fire time, observed fire time or event sequence. Compare scheduledTime with actualTime when measuring trigger delay.
Sequence sequence	integer Optional	Scheduler output: planned fire time, observed fire time or event sequence. Compare scheduledTime with actualTime when measuring trigger delay.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

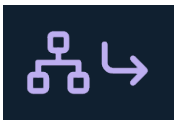
This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
SCHEDULER	The schedule expression or interval is invalid.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



Call Sub Task

Activities / Starters & Tasks

Overview

Invokes a reusable Sub Task and waits for its result.

Mapped values become the Sub Task Start input. Dynamic Sub Task ID/name is resolved per invocation; the selected Sub Task is its design-time schema and runtime fallback. The Sub Task End output returns to this activity.

Activity type: call_task; operation: call.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Sub Task taskId	task Required	Select the Sub Task to invoke. Its Start interface defines the caller input and its End interface defines the returned value.
Dynamic Sub Task ID/name expression dynamicTaskId	text Configuration setting	Resolved for every invocation. Use a literal Sub Task ID/name or an expression such as ${properties.routing.targetTask}; the selected Sub Task remains the fallback.
Spawn without waiting spawn	boolean Configuration setting	Run the selected Sub Task without waiting for its result. Leave disabled when subsequent activities depend on that result.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The selected Sub Task Start and End schemas replace generic payload/result fields.

Field / key	Type / required	Details
Sub Task input payload	object Required	The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes.
Dynamic Sub Task ID/name expression dynamicTaskId	string Optional	Resolved for every invocation. Use a literal Sub Task ID/name or an expression such as ${properties.routing.targetTask}; the selected Sub Task remains the fallback.
Spawn without waiting spawn	boolean Optional	Run the selected Sub Task without waiting for its result. Leave disabled when subsequent activities depend on that result.

Output

The selected Sub Task Start and End schemas replace generic payload/result fields.

Field / key	Type / required	Details
Invoked task ID taskId	string Optional	Select the Sub Task to invoke. Its Start interface defines the caller input and its End interface defines the returned value.

Field / key	Type / required	Details
Sub Task output output	object Optional	Post-operation action or returned output. Use the activity-specific operation/contract to determine whether the value is content, metadata or a completion result.
Spawn status spawned	boolean Optional	Indicates that the Sub Task was started asynchronously; it does not report completion of the child work.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
TASK_NOT_FOUND	The selected or dynamic Sub Task does not exist.
SUBTASK_ERROR	The invoked Sub Task returned an error.
VALIDATION	Required input or mapped data is invalid.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



End

Activities / Starters & Tasks

Overview

Defines the task output boundary.

Mapped values become the final task result. For a Sub Task, this result is returned to its Call Sub Task activity.

Activity type: end; operation: end.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

No additional fields declared for this section.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The Task output schema determines the result mapping.

Field / key	Type / required	Details
Task result result	object Optional	The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes.

Output

The Task output schema determines the result mapping.

Field / key	Type / required	Details
Task output result	object Optional	The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

No additional fields declared for this section.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose

sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



Read File

Activities / File

Overview

Performs the selected filesystem operation.

File paths and operation-specific values accept constants, property expressions, or mappings from earlier execution-path outputs. Runtime output contains operation-specific metadata and content.

Activity type: file; operation: read.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Filename / directory path	text Configuration setting	Absolute paths and project-property expressions are supported.
Include timestamp metadata includeTimestamp	boolean Configuration setting	Include timestamp metadata where the operation supports it, to distinguish or trace generated/polled content.
Read as readAs	select Configuration setting	Representation used for reading, writing or returning the value. Select the mode matching the actual document and downstream consumer contract. Choices: Text, Binary
Exclude file content excludeFileContent	boolean Configuration setting	Omit file content from polling output when only file metadata is needed. Downstream Read File can fetch content explicitly.
Text encoding encoding	text Configuration setting	Character encoding for text conversion, for example UTF-8. Use the sender encoding when parsing; a wrong encoding can corrupt non-ASCII characters.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Filename path	string Required	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Text encoding encoding	string Optional	Character encoding for text conversion, for example UTF-8. Use the sender encoding when parsing; a wrong encoding can corrupt non-ASCII characters.
Include timestamp metadata includeTimestamp	boolean Optional	Include timestamp metadata where the operation supports it, to distinguish or trace generated/polled content.
Read as readAs	string Optional	Representation used for reading, writing or returning the value. Select the mode matching the actual document and downstream consumer contract.

Field / key	Type / required	Details
Exclude file content excludeFileContent	boolean Optional	Omit file content from polling output when only file metadata is needed. Downstream Read File can fetch content explicitly.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Text file content textContent	string Optional	Textual document/content. Supply valid XML/JSON for parsers, or map rendered text for a wire-format destination. Encoding controls conversion to and from bytes.
Binary file content binaryContent	base64 Optional	Binary content represented by the activity contract, commonly base64 for JSON-visible fields. Use binary mode and matching encoding when forwarding bytes.
Full filename fileInfo.fullName	string Optional	File metadata: name, full path, directory, modification time, access flags or size in bytes, according to this field. Values describe the runtime filesystem.
Filename fileInfo.fileName	string Optional	File metadata: name, full path, directory, modification time, access flags or size in bytes, according to this field. Values describe the runtime filesystem.
Location fileInfo.location	string Optional	File metadata: name, full path, directory, modification time, access flags or size in bytes, according to this field. Values describe the runtime filesystem.
File type fileInfo.type	string Optional	Type reported for this value or selected for this operation. Use the corresponding schema datatype and operation-specific choices.
Read protected fileInfo.readProtected	boolean Optional	File metadata: name, full path, directory, modification time, access flags or size in bytes, according to this field. Values describe the runtime filesystem.
Write protected fileInfo.writeProtected	boolean Optional	File metadata: name, full path, directory, modification time, access flags or size in bytes, according to this field. Values describe the runtime filesystem.
Size fileInfo.size	integer Optional	File metadata: name, full path, directory, modification time, access flags or size in bytes, according to this field. Values describe the runtime filesystem.
Last modified fileInfo.lastModified	dateTime Optional	File metadata: name, full path, directory, modification time, access flags or size in bytes, according to this field. Values describe the runtime filesystem.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
FILE_NOT_FOUND	The configured source path does not exist.
FILE_IO	The runtime cannot read, write, move, or delete the file.
PERMISSION	The runtime lacks filesystem permission.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



Write File

Activities / File

Overview

Performs the selected filesystem operation.

File paths and operation-specific values accept constants, property expressions, or mappings from earlier execution-path outputs. Runtime output contains operation-specific metadata and content.

Activity type: file; operation: write.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Filename / directory path	text Configuration setting	Absolute paths and project-property expressions are supported.
Include timestamp metadata includeTimestamp	boolean Configuration setting	Include timestamp metadata where the operation supports it, to distinguish or trace generated/polled content.
Write as writeAs	select Configuration setting	Representation used for reading, writing or returning the value. Select the mode matching the actual document and downstream consumer contract. Choices: Text, Binary
Text encoding encoding	text Configuration setting	Character encoding for text conversion, for example UTF-8. Use the sender encoding when parsing; a wrong encoding can corrupt non-ASCII characters.
Append to existing file append	boolean Configuration setting	Existing-file policy. Append adds content to the existing file; overwrite replaces existing content. Disable replacement where existing output must be preserved.
Overwrite existing file overwrite	boolean Configuration setting	Existing-file policy. Append adds content to the existing file; overwrite replaces existing content. Disable replacement where existing output must be preserved.
Create non-existing directories createDirectories	boolean Configuration setting	Create missing parent directories for the output path. The runtime account still needs directory creation/write permission.
Add line separator addLineSeparator	boolean Configuration setting	Line separator handling for text output. Choose the delimiter expected by the consumer; includeFinalLineSeparator adds a separator after the last record.
Compression compression	select Configuration setting	Compression selection for file or staging content. Match decompression settings at the receiving operation; compression changes the wire bytes. Choices: None, GZip

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Filename path	string Required	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Text content textContent	string Optional	Textual document/content. Supply valid XML/JSON for parsers, or map rendered text for a wire-format destination. Encoding controls conversion to and from bytes.
Binary content binaryContent	base64 Optional	Binary content represented by the activity contract, commonly base64 for JSON-visible fields. Use binary mode and matching encoding when forwarding bytes.
Text encoding encoding	string Optional	Character encoding for text conversion, for example UTF-8. Use the sender encoding when parsing; a wrong encoding can corrupt non-ASCII characters.
Add line separator addLineSeparator	boolean Optional	Line separator handling for text output. Choose the delimiter expected by the consumer; includeFinalLineSeparator adds a separator after the last record.
Include timestamp metadata includeTimestamp	boolean Optional	Include timestamp metadata where the operation supports it, to distinguish or trace generated/polled content.
Write as writeAs	string Optional	Representation used for reading, writing or returning the value. Select the mode matching the actual document and downstream consumer contract.
Append to existing file append	boolean Optional	Existing-file policy. Append adds content to the existing file; overwrite replaces existing content. Disable replacement where existing output must be preserved.
Overwrite existing file overwrite	boolean Optional	Existing-file policy. Append adds content to the existing file; overwrite replaces existing content. Disable replacement where existing output must be preserved.
Create non-existing directories createDirectories	boolean Optional	Create missing parent directories for the output path. The runtime account still needs directory creation/write permission.
Compression compression	string Optional	Compression selection for file or staging content. Match decompression settings at the receiving operation; compression changes the wire bytes.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Affected path path	string Optional	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Operation result success	boolean Optional	Operation outcome returned by the runtime. Inspect error output as well; provider acceptance does not establish processing by a downstream consumer.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.

Field / key	Type / required	Details
		Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
FILE_NOT_FOUND	The configured source path does not exist.
FILE_IO	The runtime cannot read, write, move, or delete the file.
PERMISSION	The runtime lacks filesystem permission.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



List Files

Activities / File

Overview

Performs the selected filesystem operation.

File paths and operation-specific values accept constants, property expressions, or mappings from earlier execution-path outputs. Runtime output contains operation-specific metadata and content.

Activity type: file; operation: list.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Filename / directory path	text Configuration setting	Absolute paths and project-property expressions are supported.
Include timestamp metadata includeTimestamp	boolean Configuration setting	Include timestamp metadata where the operation supports it, to distinguish or trace generated/polled content.
File name pattern / wildcard pattern	text Configuration setting	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Recursive recursive	boolean Configuration setting	Include subdirectories when traversing/listing matching files. Restrict the starting directory to avoid scanning unrelated trees.
List listType	select Configuration setting	File listing selection/order. sortBy chooses the metadata key, sortOrder its direction, and listType limits the kinds of entries returned. Choices: Files and Directories, Only Files, Only Directories
Sort by sortBy	select Configuration setting	File listing selection/order. sortBy chooses the metadata key, sortOrder its direction, and listType limits the kinds of entries returned. Choices: Name, Size, Last Modified
Sort order sortOrder	select Configuration setting	File listing selection/order. sortBy chooses the metadata key, sortOrder its direction, and listType limits the kinds of entries returned. Choices: Ascending, Descending

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Directory / wildcard path	string Required	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.

Field / key	Type / required	Details
Recursive recursive	boolean Optional	Include subdirectories when traversing/listing matching files. Restrict the starting directory to avoid scanning unrelated trees.
Include timestamp metadata includeTimestamp	boolean Optional	Include timestamp metadata where the operation supports it, to distinguish or trace generated/polled content.
File name pattern / wildcard pattern	string Optional	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
List listType	string Optional	File listing selection/order. sortBy chooses the metadata key, sortOrder its direction, and listType limits the kinds of entries returned.
Sort by sortBy	string Optional	File listing selection/order. sortBy chooses the metadata key, sortOrder its direction, and listType limits the kinds of entries returned.
Sort order sortOrder	string Optional	File listing selection/order. sortBy chooses the metadata key, sortOrder its direction, and listType limits the kinds of entries returned.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Matched file information files	array Optional	Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema.
File count count	integer Optional	Number of items returned or affected by this operation. Use zero-result handling where no matching data is a valid outcome.
Detected change eventType	string Optional	Type of file event selected/reported by the poller, such as a created or changed file.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
FILE_NOT_FOUND	The configured source path does not exist.
FILE_IO	The runtime cannot read, write, move, or delete the file.
PERMISSION	The runtime lacks filesystem permission.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose

sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



Delete File

Activities / File

Overview

Performs the selected filesystem operation.

File paths and operation-specific values accept constants, property expressions, or mappings from earlier execution-path outputs. Runtime output contains operation-specific metadata and content.

Activity type: file; operation: delete.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Filename / directory path	text Configuration setting	Absolute paths and project-property expressions are supported.
Include timestamp metadata includeTimestamp	boolean Configuration setting	Include timestamp metadata where the operation supports it, to distinguish or trace generated/polled content.
Remove non-empty directory recursively recursive	boolean Configuration setting	Include subdirectories when traversing/listing matching files. Restrict the starting directory to avoid scanning unrelated trees.
Ignore missing file ignoreMissing	boolean Configuration setting	Allow a missing file/path where supported instead of faulting. Use only when absence is an expected business outcome.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Filename / directory path	string Required	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Recursive recursive	boolean Optional	Include subdirectories when traversing/listing matching files. Restrict the starting directory to avoid scanning unrelated trees.
Include timestamp metadata includeTimestamp	boolean Optional	Include timestamp metadata where the operation supports it, to distinguish or trace generated/polled content.
Ignore missing file ignoreMissing	boolean Optional	Allow a missing file/path where supported instead of faulting. Use only when absence is an expected business outcome.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Affected path path	string Optional	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Operation result success	boolean Optional	Operation outcome returned by the runtime. Inspect error output as well; provider acceptance does not establish processing by a downstream consumer.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
FILE_NOT_FOUND	The configured source path does not exist.
FILE_IO	The runtime cannot read, write, move, or delete the file.
PERMISSION	The runtime lacks filesystem permission.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



Rename File

Activities / File

Overview

Performs the selected filesystem operation.

File paths and operation-specific values accept constants, property expressions, or mappings from earlier execution-path outputs. Runtime output contains operation-specific metadata and content.

Activity type: file; operation: rename.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Filename / directory path	text Configuration setting	Absolute paths and project-property expressions are supported.
Include timestamp metadata includeTimestamp	boolean Configuration setting	Include timestamp metadata where the operation supports it, to distinguish or trace generated/polled content.
To filename destination	text Configuration setting	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Overwrite existing file overwrite	boolean Configuration setting	Existing-file policy. Append adds content to the existing file; overwrite replaces existing content. Disable replacement where existing output must be preserved.
Create non-existing directories createDirectories	boolean Configuration setting	Create missing parent directories for the output path. The runtime account still needs directory creation/write permission.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
From filename path	string Required	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
To filename destination	string Required	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Include timestamp metadata includeTimestamp	boolean Optional	Include timestamp metadata where the operation supports it, to distinguish or trace generated/polled content.
Overwrite existing file overwrite	boolean Optional	Existing-file policy. Append adds content to the existing file; overwrite replaces existing content. Disable replacement where existing output must be preserved.

Field / key	Type / required	Details
Create non-existing directories createDirectories	boolean Optional	Create missing parent directories for the output path. The runtime account still needs directory creation/write permission.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Affected path path	string Optional	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Operation result success	boolean Optional	Operation outcome returned by the runtime. Inspect error output as well; provider acceptance does not establish processing by a downstream consumer.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
FILE_NOT_FOUND	The configured source path does not exist.
FILE_IO	The runtime cannot read, write, move, or delete the file.
PERMISSION	The runtime lacks filesystem permission.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



Copy File

Activities / File

Overview

Performs the selected filesystem operation.

File paths and operation-specific values accept constants, property expressions, or mappings from earlier execution-path outputs. Runtime output contains operation-specific metadata and content.

Activity type: file; operation: copy.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Filename / directory path	text Configuration setting	Absolute paths and project-property expressions are supported.
Include timestamp metadata includeTimestamp	boolean Configuration setting	Include timestamp metadata where the operation supports it, to distinguish or trace generated/polled content.
Destination filename / directory destination	text Configuration setting	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Overwrite existing file overwrite	boolean Configuration setting	Existing-file policy. Append adds content to the existing file; overwrite replaces existing content. Disable replacement where existing output must be preserved.
Create non-existing directories createDirectories	boolean Configuration setting	Create missing parent directories for the output path. The runtime account still needs directory creation/write permission.
Preserve file attributes preserveAttributes	boolean Configuration setting	Request preservation of supported file attributes during the operation. Availability depends on filesystem and runtime permissions.
Copy directory recursively recursive	boolean Configuration setting	Include subdirectories when traversing/listing matching files. Restrict the starting directory to avoid scanning unrelated trees.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Source filename / directory path	string Required	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Destination filename / directory destination	string Required	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.

Field / key	Type / required	Details
Recursive recursive	boolean Optional	Include subdirectories when traversing/listing matching files. Restrict the starting directory to avoid scanning unrelated trees.
Include timestamp metadata includeTimestamp	boolean Optional	Include timestamp metadata where the operation supports it, to distinguish or trace generated/polled content.
Overwrite existing file overwrite	boolean Optional	Existing-file policy. Append adds content to the existing file; overwrite replaces existing content. Disable replacement where existing output must be preserved.
Create non-existing directories createDirectories	boolean Optional	Create missing parent directories for the output path. The runtime account still needs directory creation/write permission.
Preserve file attributes preserveAttributes	boolean Optional	Request preservation of supported file attributes during the operation. Availability depends on filesystem and runtime permissions.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Affected path path	string Optional	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Operation result success	boolean Optional	Operation outcome returned by the runtime. Inspect error output as well; provider acceptance does not establish processing by a downstream consumer.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
FILE_NOT_FOUND	The configured source path does not exist.
FILE_IO	The runtime cannot read, write, move, or delete the file.
PERMISSION	The runtime lacks filesystem permission.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



File Poller

Activities / File

Overview

Performs the selected filesystem operation.

File paths and operation-specific values accept constants, property expressions, or mappings from earlier execution-path outputs. Runtime output contains operation-specific metadata and content.

Activity type: file; operation: poll.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Filename / directory path	text Configuration setting	Absolute paths and project-property expressions are supported.
Include timestamp metadata includeTimestamp	boolean Configuration setting	Include timestamp metadata where the operation supports it, to distinguish or trace generated/polled content.
File name pattern / wildcard pattern	text Configuration setting	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Change type eventType	select Configuration setting	Type of file event selected/reported by the poller, such as a created or changed file. Choices: Created, Modified, Deleted, Any
Poll interval (seconds) pollInterval	number Configuration setting	Polling/scheduling interval for repeated execution. Size the interval to expected arrival rate and processing duration; select the unit where the activity exposes one.
Include existing files includeExisting	boolean Configuration setting	Include files already present when polling begins, rather than only later arrivals. Test restart behavior to avoid unintended reprocessing.
Recursive recursive	boolean Configuration setting	Include subdirectories when traversing/listing matching files. Restrict the starting directory to avoid scanning unrelated trees.
After processing postAction	select Configuration setting	Post-operation action or returned output. Use the activity-specific operation/contract to determine whether the value is content, metadata or a completion result. Choices: None, Delete, Move
Move processed file to moveTo	text Configuration setting	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

No additional fields declared for this section.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Matched file information files	array Optional	Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema.
File count count	integer Optional	Number of items returned or affected by this operation. Use zero-result handling where no matching data is a valid outcome.
Detected change eventType	string Optional	Type of file event selected/reported by the poller, such as a created or changed file.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
FILE_NOT_FOUND	The configured source path does not exist.
FILE_IO	The runtime cannot read, write, move, or delete the file.
PERMISSION	The runtime lacks filesystem permission.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



FTP Get

Activities / FTP

Overview

Performs the selected FTP operation through a shared FTP connection.

Connection defaults come from the resource; paths, names, content, and transfer settings may be mapped dynamically for each invocation.

Activity type: ftp; operation: get.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Shared connection resourceId	resource Configuration setting	Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment. Connection type: ftp
Remote path remotePath	text Configuration setting	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Working directory workingDirectory	text Configuration setting	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Binary transfer binary	boolean Configuration setting	Transfer/read bytes without text decoding when enabled. Map binary content separately from decoded text.
Timeout seconds timeout	number Configuration setting	FTP connection/transfer socket timeout in seconds.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Remote path remotePath	string Required	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Working directory workingDirectory	string Optional	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Binary transfer binary	boolean Optional	Transfer/read bytes without text decoding when enabled. Map binary content separately from decoded text.
Timeout seconds timeout	number Optional	FTP connection/transfer socket timeout in seconds.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Downloaded content contentBase64	base64 Optional	Binary content represented by the activity contract, commonly base64 for JSON-visible fields. Use binary mode and matching encoding when forwarding bytes.
Size size	integer Optional	File metadata: name, full path, directory, modification time, access flags or size in bytes, according to this field. Values describe the runtime filesystem.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Fault	Description
AUTHENTICATION	Remote authentication failed.
REMOTE_NOT_FOUND	The remote path does not exist.
TRANSFER	The remote transfer failed.
CONNECTIVITY	The configured shared connection is unavailable.
RETRY_EXHAUSTED	The reconnection or retry policy was exhausted.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



FTP Put

Activities / FTP

Overview

Performs the selected FTP operation through a shared FTP connection.

Connection defaults come from the resource; paths, names, content, and transfer settings may be mapped dynamically for each invocation.

Activity type: ftp; operation: put.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Shared connection resourceId	resource Configuration setting	Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment. Connection type: ftp
Remote path remotePath	text Configuration setting	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Working directory workingDirectory	text Configuration setting	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Binary transfer binary	boolean Configuration setting	Transfer/read bytes without text decoding when enabled. Map binary content separately from decoded text.
Timeout seconds timeout	number Configuration setting	FTP connection/transfer socket timeout in seconds.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Content content	binary|string Required	Textual document/content. Supply valid XML/JSON for parsers, or map rendered text for a wire-format destination. Encoding controls conversion to and from bytes.
Remote path remotePath	string Required	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Working directory workingDirectory	string Optional	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.

Field / key	Type / required	Details
Binary transfer binary	boolean Optional	Transfer/read bytes without text decoding when enabled. Map binary content separately from decoded text.
Timeout seconds timeout	number Optional	FTP connection/transfer socket timeout in seconds.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Remote path remotePath	string Optional	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Operation result success	boolean Optional	Operation outcome returned by the runtime. Inspect error output as well; provider acceptance does not establish processing by a downstream consumer.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Fault	Description
AUTHENTICATION	Remote authentication failed.
REMOTE_NOT_FOUND	The remote path does not exist.
TRANSFER	The remote transfer failed.
CONNECTIVITY	The configured shared connection is unavailable.
RETRY_EXHAUSTED	The reconnection or retry policy was exhausted.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



FTP Delete

Activities / FTP

Overview

Performs the selected FTP operation through a shared FTP connection.

Connection defaults come from the resource; paths, names, content, and transfer settings may be mapped dynamically for each invocation.

Activity type: ftp; operation: delete.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Shared connection resourceId	resource Configuration setting	Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment. Connection type: ftp
Remote path remotePath	text Configuration setting	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Working directory workingDirectory	text Configuration setting	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Binary transfer binary	boolean Configuration setting	Transfer/read bytes without text decoding when enabled. Map binary content separately from decoded text.
Timeout seconds timeout	number Configuration setting	FTP connection/transfer socket timeout in seconds.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Remote path remotePath	string Required	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Working directory workingDirectory	string Optional	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Binary transfer binary	boolean Optional	Transfer/read bytes without text decoding when enabled. Map binary content separately from decoded text.
Timeout seconds timeout	number Optional	FTP connection/transfer socket timeout in seconds.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Remote path remotePath	string Optional	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Operation result success	boolean Optional	Operation outcome returned by the runtime. Inspect error output as well; provider acceptance does not establish processing by a downstream consumer.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Fault	Description
AUTHENTICATION	Remote authentication failed.
REMOTE_NOT_FOUND	The remote path does not exist.
TRANSFER	The remote transfer failed.
CONNECTIVITY	The configured shared connection is unavailable.
RETRY_EXHAUSTED	The reconnection or retry policy was exhausted.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



FTP Directory

Activities / FTP

Overview

Performs the selected FTP operation through a shared FTP connection.

Connection defaults come from the resource; paths, names, content, and transfer settings may be mapped dynamically for each invocation.

Activity type: ftp; operation: dir.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Shared connection resourceId	resource Configuration setting	Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment. Connection type: ftp
Remote path remotePath	text Configuration setting	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Working directory workingDirectory	text Configuration setting	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Binary transfer binary	boolean Configuration setting	Transfer/read bytes without text decoding when enabled. Map binary content separately from decoded text.
Timeout seconds timeout	number Configuration setting	FTP connection/transfer socket timeout in seconds.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Remote path remotePath	string Optional	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Working directory workingDirectory	string Optional	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Binary transfer binary	boolean Optional	Transfer/read bytes without text decoding when enabled. Map binary content separately from decoded text.
Timeout seconds timeout	number Optional	FTP connection/transfer socket timeout in seconds.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Directory entries entries	array Optional	Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema.
Current directory directory	string Optional	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Fault	Description
AUTHENTICATION	Remote authentication failed.
REMOTE_NOT_FOUND	The remote path does not exist.
TRANSFER	The remote transfer failed.
CONNECTIVITY	The configured shared connection is unavailable.
RETRY_EXHAUSTED	The reconnection or retry policy was exhausted.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



FTP Change Directory

Activities / FTP

Overview

Performs the selected FTP operation through a shared FTP connection.

Connection defaults come from the resource; paths, names, content, and transfer settings may be mapped dynamically for each invocation.

Activity type: ftp; operation: change_dir.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Shared connection resourceId	resource Configuration setting	Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment. Connection type: ftp
Target directory remotePath	text Configuration setting	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Working directory workingDirectory	text Configuration setting	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Binary transfer binary	boolean Configuration setting	Transfer/read bytes without text decoding when enabled. Map binary content separately from decoded text.
Timeout seconds timeout	number Configuration setting	FTP connection/transfer socket timeout in seconds.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Remote path remotePath	string Required	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Working directory workingDirectory	string Optional	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Binary transfer binary	boolean Optional	Transfer/read bytes without text decoding when enabled. Map binary content separately from decoded text.
Timeout seconds timeout	number Optional	FTP connection/transfer socket timeout in seconds.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Remote path remotePath	string Optional	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Operation result success	boolean Optional	Operation outcome returned by the runtime. Inspect error output as well; provider acceptance does not establish processing by a downstream consumer.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Fault	Description
AUTHENTICATION	Remote authentication failed.
REMOTE_NOT_FOUND	The remote path does not exist.
TRANSFER	The remote transfer failed.
CONNECTIVITY	The configured shared connection is unavailable.
RETRY_EXHAUSTED	The reconnection or retry policy was exhausted.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



SFTP Get

Activities / SFTP

Overview

Performs the selected secure file-transfer operation.

Authentication and host-key policy come from the SFTP shared connection. Operation inputs remain dynamically mappable at runtime.

Activity type: sftp; operation: get.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Shared connection resourceId	resource Configuration setting	Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment. Connection type: sftp
Remote path remotePath	text Configuration setting	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Working directory workingDirectory	text Configuration setting	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Binary transfer binary	boolean Configuration setting	Transfer/read bytes without text decoding when enabled. Map binary content separately from decoded text.
Timeout seconds timeout	number Configuration setting	SSH connection timeout in seconds.
Verify host key verifyHostKey	boolean Configuration setting	SSH host identity verification. Supply a trusted known-hosts file for SFTP; disabling verification permits connecting to an unverified server.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Remote path remotePath	string Required	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Working directory workingDirectory	string Optional	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.

Field / key	Type / required	Details
Binary transfer binary	boolean Optional	Transfer/read bytes without text decoding when enabled. Map binary content separately from decoded text.
Timeout seconds timeout	number Optional	SSH connection timeout in seconds.
Verify host key verifyHostKey	boolean Optional	SSH host identity verification. Supply a trusted known-hosts file for SFTP; disabling verification permits connecting to an unverified server.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Downloaded content contentBase64	base64 Optional	Binary content represented by the activity contract, commonly base64 for JSON-visible fields. Use binary mode and matching encoding when forwarding bytes.
Size size	integer Optional	File metadata: name, full path, directory, modification time, access flags or size in bytes, according to this field. Values describe the runtime filesystem.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false

Field / key	Type / required	Details
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

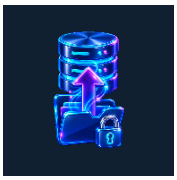
Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Fault	Description
AUTHENTICATION	Remote authentication failed.
REMOTE_NOT_FOUND	The remote path does not exist.
TRANSFER	The remote transfer failed.
CONNECTIVITY	The configured shared connection is unavailable.
RETRY_EXHAUSTED	The reconnection or retry policy was exhausted.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



SFTP Put

Activities / SFTP

Overview

Performs the selected secure file-transfer operation.

Authentication and host-key policy come from the SFTP shared connection. Operation inputs remain dynamically mappable at runtime.

Activity type: sftp; operation: put.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Shared connection resourceId	resource Configuration setting	Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment. Connection type: sftp
Remote path remotePath	text Configuration setting	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Working directory workingDirectory	text Configuration setting	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Binary transfer binary	boolean Configuration setting	Transfer/read bytes without text decoding when enabled. Map binary content separately from decoded text.
Timeout seconds timeout	number Configuration setting	SSH connection timeout in seconds.
Verify host key verifyHostKey	boolean Configuration setting	SSH host identity verification. Supply a trusted known-hosts file for SFTP; disabling verification permits connecting to an unverified server.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Content content	binary|string Required	Textual document/content. Supply valid XML/JSON for parsers, or map rendered text for a wire-format destination. Encoding controls conversion to and from bytes.
Remote path remotePath	string Required	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.

Field / key	Type / required	Details
Working directory workingDirectory	string Optional	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Binary transfer binary	boolean Optional	Transfer/read bytes without text decoding when enabled. Map binary content separately from decoded text.
Timeout seconds timeout	number Optional	SSH connection timeout in seconds.
Verify host key verifyHostKey	boolean Optional	SSH host identity verification. Supply a trusted known-hosts file for SFTP; disabling verification permits connecting to an unverified server.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Remote path remotePath	string Optional	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Operation result success	boolean Optional	Operation outcome returned by the runtime. Inspect error output as well; provider acceptance does not establish processing by a downstream consumer.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0

Field / key	Type / required	Details
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

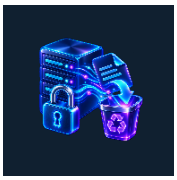
Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Fault	Description
AUTHENTICATION	Remote authentication failed.
REMOTE_NOT_FOUND	The remote path does not exist.
TRANSFER	The remote transfer failed.
CONNECTIVITY	The configured shared connection is unavailable.
RETRY_EXHAUSTED	The reconnection or retry policy was exhausted.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



SFTP Delete

Activities / SFTP

Overview

Performs the selected secure file-transfer operation.

Authentication and host-key policy come from the SFTP shared connection. Operation inputs remain dynamically mappable at runtime.

Activity type: sftp; operation: delete.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Shared connection resourceId	resource Configuration setting	Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment. Connection type: sftp
Remote path remotePath	text Configuration setting	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Working directory workingDirectory	text Configuration setting	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Binary transfer binary	boolean Configuration setting	Transfer/read bytes without text decoding when enabled. Map binary content separately from decoded text.
Timeout seconds timeout	number Configuration setting	SSH connection timeout in seconds.
Verify host key verifyHostKey	boolean Configuration setting	SSH host identity verification. Supply a trusted known-hosts file for SFTP; disabling verification permits connecting to an unverified server.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Remote path remotePath	string Required	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Working directory workingDirectory	string Optional	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.

Field / key	Type / required	Details
Binary transfer binary	boolean Optional	Transfer/read bytes without text decoding when enabled. Map binary content separately from decoded text.
Timeout seconds timeout	number Optional	SSH connection timeout in seconds.
Verify host key verifyHostKey	boolean Optional	SSH host identity verification. Supply a trusted known-hosts file for SFTP; disabling verification permits connecting to an unverified server.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Remote path remotePath	string Optional	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Operation result success	boolean Optional	Operation outcome returned by the runtime. Inspect error output as well; provider acceptance does not establish processing by a downstream consumer.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false

Field / key	Type / required	Details
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Fault	Description
AUTHENTICATION	Remote authentication failed.
REMOTE_NOT_FOUND	The remote path does not exist.
TRANSFER	The remote transfer failed.
CONNECTIVITY	The configured shared connection is unavailable.
RETRY_EXHAUSTED	The reconnection or retry policy was exhausted.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



SFTP Directory

Activities / SFTP

Overview

Performs the selected secure file-transfer operation.

Authentication and host-key policy come from the SFTP shared connection. Operation inputs remain dynamically mappable at runtime.

Activity type: sftp; operation: dir.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Shared connection resourceId	resource Configuration setting	Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment. Connection type: sftp
Remote path remotePath	text Configuration setting	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Working directory workingDirectory	text Configuration setting	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Binary transfer binary	boolean Configuration setting	Transfer/read bytes without text decoding when enabled. Map binary content separately from decoded text.
Timeout seconds timeout	number Configuration setting	SSH connection timeout in seconds.
Verify host key verifyHostKey	boolean Configuration setting	SSH host identity verification. Supply a trusted known-hosts file for SFTP; disabling verification permits connecting to an unverified server.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Remote path remotePath	string Optional	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Working directory workingDirectory	string Optional	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.

Field / key	Type / required	Details
Binary transfer binary	boolean Optional	Transfer/read bytes without text decoding when enabled. Map binary content separately from decoded text.
Timeout seconds timeout	number Optional	SSH connection timeout in seconds.
Verify host key verifyHostKey	boolean Optional	SSH host identity verification. Supply a trusted known-hosts file for SFTP; disabling verification permits connecting to an unverified server.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Directory entries entries	array Optional	Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema.
Current directory directory	string Optional	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false

Field / key	Type / required	Details
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

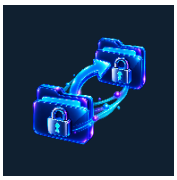
Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Fault	Description
AUTHENTICATION	Remote authentication failed.
REMOTE_NOT_FOUND	The remote path does not exist.
TRANSFER	The remote transfer failed.
CONNECTIVITY	The configured shared connection is unavailable.
RETRY_EXHAUSTED	The reconnection or retry policy was exhausted.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



SFTP Change Directory

Activities / SFTP

Overview

Performs the selected secure file-transfer operation.

Authentication and host-key policy come from the SFTP shared connection. Operation inputs remain dynamically mappable at runtime.

Activity type: sftp; operation: change_dir.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Shared connection resourceId	resource Configuration setting	Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment. Connection type: sftp
Target directory remotePath	text Configuration setting	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Working directory workingDirectory	text Configuration setting	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Binary transfer binary	boolean Configuration setting	Transfer/read bytes without text decoding when enabled. Map binary content separately from decoded text.
Timeout seconds timeout	number Configuration setting	SSH connection timeout in seconds.
Verify host key verifyHostKey	boolean Configuration setting	SSH host identity verification. Supply a trusted known-hosts file for SFTP; disabling verification permits connecting to an unverified server.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Remote path remotePath	string Required	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Working directory workingDirectory	string Optional	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.

Field / key	Type / required	Details
Binary transfer binary	boolean Optional	Transfer/read bytes without text decoding when enabled. Map binary content separately from decoded text.
Timeout seconds timeout	number Optional	SSH connection timeout in seconds.
Verify host key verifyHostKey	boolean Optional	SSH host identity verification. Supply a trusted known-hosts file for SFTP; disabling verification permits connecting to an unverified server.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Remote path remotePath	string Optional	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Operation result success	boolean Optional	Operation outcome returned by the runtime. Inspect error output as well; provider acceptance does not establish processing by a downstream consumer.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false

Field / key	Type / required	Details
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Fault	Description
AUTHENTICATION	Remote authentication failed.
REMOTE_NOT_FOUND	The remote path does not exist.
TRANSFER	The remote transfer failed.
CONNECTIVITY	The configured shared connection is unavailable.
RETRY_EXHAUSTED	The reconnection or retry policy was exhausted.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



HTTP Listener

Activities / HTTP & APIs

Overview

Starts a Starter Task when an inbound HTTP request matches its configured method and path.

Choose the server connection, methods, route, content types, and schema references when creating the task. Setup adds HTTP Send Response for the reply. Run or Debug shows the live endpoint, Swagger link, and Postman download when API documentation is enabled. JSON request fields appear under body. Parsed XSD fields appear under xmlBody, with the original XML under body.

Activity type: http_listener; operation: listen.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
HTTP shared connection resourceId	resource Configuration setting	Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment. Connection type: http
Listener path path	text Configuration setting	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Allowed HTTP methods methods	methods Configuration setting	Select every HTTP method accepted by this listener.
Request parameters requestParameters	requestParameters Configuration setting	Declare typed URI parameters, query parameters, and custom headers.
Request content types requestContentTypes	mediaTypes Configuration setting	Select the accepted request media types. A selected XSD applies to XML and a JSON Schema applies to JSON when several formats are selected.
Response content types responseContentTypes	mediaTypes Configuration setting	Select the offered response media types. With multiple selections, the Accept header selects the format; the first selected type is the default.
Validate request schema validateRequest	boolean Configuration setting	Validate the associated value against the configured contract. A mismatch raises a validation fault; this does not replace validation of business rules.
Request schema requestSchema	schema Configuration setting	Optional contract for the incoming request body. Select a project schema or browse for an XSD or JSON Schema file.
Validate response schema validateResponse	boolean Configuration setting	Validate the associated value against the configured contract. A mismatch raises a validation fault; this does not replace validation of business rules.
Response schema responseSchema	schema Configuration setting	Optional contract for the response body. Select or browse for a schema to make its fields available in HTTP Send Response.
Request execution timeout (seconds) activityTimeoutSeconds	number Configuration setting	Request execution timeout (seconds): bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

No additional fields declared for this section.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Request body body	any Optional	Message or response content. Map the business document rather than a previous publisher receipt. Objects retain their structure; string values carry serialized text.
Base64 request bytes binaryContent	base64 Optional	Binary content represented by the activity contract, commonly base64 for JSON-visible fields. Use binary mode and matching encoding when forwarding bytes.
Temporary uploaded file fileName	string Optional	File metadata: name, full path, directory, modification time, access flags or size in bytes, according to this field. Values describe the runtime filesystem.
Uploaded byte count size	integer Optional	File metadata: name, full path, directory, modification time, access flags or size in bytes, according to this field. Values describe the runtime filesystem.
All received header pairs headerPairs	array Optional	Complete ordered received header name/value pairs, preserving repeated headers. Use this collection when every wire header occurrence is needed.
Client IP clientIp	string Optional	Client IP used for allowlists and throttling. Uses the transport peer unless the shared connection trusts the forwarding proxy.
HTTP trace ID traceId	string Optional	Trace identifier from a valid incoming traceparent, or one generated by the listener. Present when HTTP trace context is enabled.
Request headers headers	object Optional	Header name/value object. Map only headers required by the target protocol; credentials belong in protected authentication settings.
Query parameters query	object Optional	HTTP query parameters. Use a name/value object for structured parameters or QueryString for an already composed query; encode values rather than concatenating untrusted text.
Path parameters pathParameters	object Optional	Values substituted into endpoint path placeholders. Provide a value for each required placeholder in the request URI or selected operation.
HTTP method method	string Optional	HTTP verb or invoked method name, according to this activity. For HTTP select the verb required by the API, such as GET, POST or PUT.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.

Field / key	Type / required	Details
		Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
HTTP_BAD_REQUEST	The inbound HTTP request is invalid.
HTTP_UNAUTHORIZED	Listener authentication rejected the request.
HTTP_NOT_FOUND	No matching listener resource exists.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



HTTP Request

Activities / HTTP & APIs

Overview

Sends an outbound HTTP request and exposes the response to downstream activities.

Select a shared HTTP client connection. In Input -> RestInputRequest, map the URI, method, body, and query parameters under Config. Choose CustomHeaders -> Add header to map each name/value pair independently, or use For Each for a collection. Set Content-Type and Accept under Headers.

Activity type: http; operation: request.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
HTTP shared connection resourceId	resource Configuration setting	Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment. Connection type: http

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
RestInputRequest RestInputRequest	object Required	Root request structure. Map endpoint and request settings under Config, HTTP headers under Headers and multipart content under mimeEnvelopeElement.
Config RestInputRequest.Config	object Optional	Request endpoint and processing settings. Map host, port, requestURI, Method and RequestBody here; shared connection settings provide connection-level defaults.
host RestInputRequest.Config.host	string Optional	Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https.
port RestInputRequest.Config.port	integer Optional	Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https.
scheme RestInputRequest.Config.scheme	string Optional	Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https.
requestURI RestInputRequest.Config.requestURI	string Optional	Endpoint or filesystem path for this operation. For HTTP use the service URI/path; for files use a path accessible to the runtime host, not only the designer machine.

Field / key	Type / required	Details
Method RestInputRequest.Config.Method	string Optional	HTTP verb or invoked method name, according to this activity. For HTTP select the verb required by the API, such as GET, POST or PUT.
RequestBody RestInputRequest.Config.RequestBody	any Optional	HTTP request entity configuration. RequestBody supplies content; bodyFormat selects its encoding/serialization and sendBodyMode controls body transmission. Use multipart parts for multipart requests.
bodyFormat RestInputRequest.Config.bodyFormat	string Optional	HTTP request entity configuration. RequestBody supplies content; bodyFormat selects its encoding/serialization and sendBodyMode controls body transmission. Use multipart parts for multipart requests.
QueryParameters RestInputRequest.Config.QueryParameters	object Optional	HTTP query parameters. Use a name/value object for structured parameters or QueryString for an already composed query; encode values rather than concatenating untrusted text.
URIParameters RestInputRequest.Config.URIParameters	object Optional	Values substituted into endpoint path placeholders. Provide a value for each required placeholder in the request URI or selected operation.
QueryString RestInputRequest.Config.QueryString	string Optional	HTTP query parameters. Use a name/value object for structured parameters or QueryString for an already composed query; encode values rather than concatenating untrusted text.
FilePath RestInputRequest.Config.FilePath	string Optional	File path on the runtime host. Ensure the runtime account can read the file and that the path exists in the deployment environment.
timeout RestInputRequest.Config.timeout	integer Optional	HTTP timeout in milliseconds. Distinguish connect/read timeouts in the shared connection from the whole-activity timeout.
activityTimeoutSeconds RestInputRequest.Config.activityTimeoutSeconds	number Optional	activityTimeoutSeconds: bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.
followRedirects RestInputRequest.Config.followRedirects	boolean Optional	Follow HTTP redirect responses. Disable when the flow must inspect or reject redirects explicitly.
raiseForStatus RestInputRequest.Config.raiseForStatus	boolean Optional	HTTP success policy. raiseForStatus faults responses outside the accepted success status codes; successStatusCodes defines that accepted set.
streamRequest RestInputRequest.Config.streamRequest	boolean Optional	Stream MIME attachment content from files in bounded chunks instead of buffering the complete envelope. File body uploads stream automatically.
responseFile RestInputRequest.Config.responseFile	string Optional	Runtime filename for a streamed download. A temporary file is atomically published only after successful completion; body returns fileName and size. Response schema validation requires buffered mode.
overwriteResponseFile RestInputRequest.Config.overwriteResponseFile	boolean Optional	Permit replacing an existing download destination after a successful transfer. Disabled by default to protect existing files.
responseBodyMode RestInputRequest.Config.responseBodyMode	string Optional	Set binary to preserve the response bytes as the body. File mode uses responseFile; structured output also exposes base64 binaryContent.
multipartPartHeaders RestInputRequest.Config.multipartPartHeaders	object Optional	Object keyed by multipart field name. Map only additional headers declared by that field in the OpenAPI encoding.

Field / key	Type / required	Details
callbackUrl RestInputRequest.Config.callbackUrl	string Optional	Callback target URL supplied to a generated callback subtask. The destination must match a configured HTTPS origin.
openApiDocument RestInputRequest.Config.openApiDocument	any Optional	OpenAPI definition and selected operation identifier. The operation supplies method/path and request/response contracts; reselect it after replacing the specification.
openApiReferenceDocuments RestInputRequest.Config.openApiReferenceDocuments	any Optional	URI-to-document JSON object supplying external API reference documents without a network fetch. Each URI identifies its JSON or YAML contract document.
requestSchema RestInputRequest.Config.requestSchema	any Optional	Selected schema/contract for the request, response or record structure. Refresh mappings after changing schema fields, types or cardinalities.
responseSchema RestInputRequest.Config.responseSchema	any Optional	Selected schema/contract for the request, response or record structure. Refresh mappings after changing schema fields, types or cardinalities.
openApiBaseUrl RestInputRequest.Config.openApiBaseUrl	string Optional	Absolute contract document URL used to resolve relative external references. It does not change the REST service endpoint.
openApiAllowedRefOrigins RestInputRequest.Config.openApiAllowedRefOrigins	string Optional	Explicit HTTPS origins permitted for external contract retrieval. Empty disables network reference retrieval. Redirects and implicit file access are forbidden; downloaded documents have size/count bounds.
operationId RestInputRequest.Config.operationId	string Optional	OpenAPI definition and selected operation identifier. The operation supplies method/path and request/response contracts; reselect it after replacing the specification.
validateRequest RestInputRequest.Config.validateRequest	boolean Optional	Validate the associated value against the configured contract. A mismatch raises a validation fault; this does not replace validation of business rules.
validateResponse RestInputRequest.Config.validateResponse	boolean Optional	Validate the associated value against the configured contract. A mismatch raises a validation fault; this does not replace validation of business rules.
requestEntityProcessing RestInputRequest.Config.requestEntityProcessing	string Optional	HTTP request entity configuration. RequestBody supplies content; bodyFormat selects its encoding/serialization and sendBodyMode controls body transmission. Use multipart parts for multipart requests.
sendBodyMode RestInputRequest.Config.sendBodyMode	string Optional	HTTP request entity configuration. RequestBody supplies content; bodyFormat selects its encoding/serialization and sendBodyMode controls body transmission. Use multipart parts for multipart requests.
successStatusCodes RestInputRequest.Config.successStatusCodes	string Optional	HTTP success policy. raiseForStatus faults responses outside the accepted success status codes; successStatusCodes defines that accepted set.
Headers RestInputRequest.Headers	object Optional	HTTP header container. On requests, use CustomHeaders for additional headers. On responses, DynamicHeaders lists the received headers.
accept RestInputRequest.Headers.accept	string Optional	HTTP Accept header listing response media types the caller can process, for example application/json.
content-type RestInputRequest.Headers.content-type	string Optional	MIME media type describing the content, for example application/json, application/xml or text/plain. Match it to the serialized body.

Field / key	Type / required	Details
Accept-Charset RestInputRequest.Headers.Accept-Charset	string Optional	HTTP character sets acceptable in the response. Keep body encoding and declared media type consistent.
Accept-Encoding RestInputRequest.Headers.Accept-Encoding	string Optional	HTTP content encodings accepted by the caller or applied to the response, such as gzip. This is different from text character encoding.
Cookie RestInputRequest.Headers.Cookie	string Optional	HTTP cookie header. Cookie values can contain session credentials; avoid logging them.
pragma RestInputRequest.Headers.pragma	string Optional	HTTP Pragma header used for legacy cache directives, for example no-cache.
CustomHeaders RestInputRequest.CustomHeaders	object[] Optional	Repeating request headers. Use Add header to map each name/value pair independently, or map a collection with For Each.
name RestInputRequest.CustomHeaders.name	string Optional	Custom HTTP header name, for example X-Correlation-ID. Provide one name/value pair per repeating entry.
value RestInputRequest.CustomHeaders.value	string Optional	Value of the custom HTTP header named in this entry. Map the required string without embedding confidential tokens in logs.
mimeEnvelopeElement RestInputRequest.mimeEnvelopeElement	object Optional	Multipart content container. Supply one part per attachment/body component, with part headers and one supported content representation.
mimePart RestInputRequest.mimeEnvelopeElement.mimePart	object[] Optional	Multipart content container. Supply one part per attachment/body component, with part headers and one supported content representation.
mimeHeaders RestInputRequest.mimeEnvelopeElement.mimePart.mimeHeaders	object Optional	Headers for this multipart part, including media type, content disposition and optional content ID.
content-type RestInputRequest.mimeEnvelopeElement.mimePart.mimeHeaders.content-type	string Optional	MIME media type describing the content, for example application/json, application/xml or text/plain. Match it to the serialized body.
content-transfer-encoding RestInputRequest.mimeEnvelopeElement.mimePart.mimeHeaders.content-transfer-encoding	string Optional	Transfer-encoding metadata for this MIME part. Ensure it matches the supplied content representation.
content-id RestInputRequest.mimeEnvelopeElement.mimePart.mimeHeaders.content-id	string Optional	Identifier of a multipart part, used when the message refers to an attachment by content ID.
content-disposition RestInputRequest.mimeEnvelopeElement.mimePart.mimeHeaders.content-disposition	string Optional	Multipart disposition, for example form-data; name="document"; filename="document.xml". Quote parameter values containing spaces.
binaryContent RestInputRequest.mimeEnvelopeElement.mimePart.binaryContent	string Optional	Binary content represented by the activity contract, commonly base64 for JSON-visible fields. Use binary mode and matching encoding when forwarding bytes.
textContent RestInputRequest.mimeEnvelopeElement.mimePart.textContent	string Optional	Textual document/content. Supply valid XML/JSON for parsers, or map rendered text for a wire-format destination. Encoding controls conversion to and from bytes.

Field / key	Type / required	Details
fileName RestInputRequest.mimeEnvelopeElement.mimePart.fileName	string Optional	File metadata: name, full path, directory, modification time, access flags or size in bytes, according to this field. Values describe the runtime filesystem.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
RestOutputResponse RestOutputResponse	object Required	Complete HTTP response envelope containing statusLine, headers, decoded body and optional text/binary/multipart content.
statusLine RestOutputResponse.statusLine	object Optional	HTTP protocol version, numeric response status and reason phrase returned by the server.
httpVersion RestOutputResponse.statusLine.httpVersion	string Optional	HTTP protocol version reported for the response, for example HTTP/1.1.
statusCode RestOutputResponse.statusLine.statusCode	integer Optional	Numeric HTTP status. Apply success/error routing according to the endpoint contract rather than treating every response as success.
reasonPhrase RestOutputResponse.statusLine.reasonPhrase	string Optional	Human-readable HTTP status description. Use statusCode for programmatic decisions.
body RestOutputResponse.body	any Optional	Message or response content. Map the business document rather than a previous publisher receipt. Objects retain their structure; string values carry serialized text.
asciiContent RestOutputResponse.asciiContent	string Optional	Textual document/content. Supply valid XML/JSON for parsers, or map rendered text for a wire-format destination. Encoding controls conversion to and from bytes.
binaryContent RestOutputResponse.binaryContent	string Optional	Binary content represented by the activity contract, commonly base64 for JSON-visible fields. Use binary mode and matching encoding when forwarding bytes.
Headers RestOutputResponse.Headers	object Optional	HTTP header container. On requests, use CustomHeaders for additional headers. On responses, DynamicHeaders lists the received headers.
Allow RestOutputResponse.Headers.Allow	string[] Optional	HTTP response methods allowed by the endpoint, normally returned with method-not-allowed responses.
Content-Type RestOutputResponse.Headers.Content-Type	string Optional	MIME media type describing the content, for example application/json, application/xml or text/plain. Match it to the serialized body.
Content-Length RestOutputResponse.Headers.Content-Length	string Optional	HTTP content length in bytes. Use the actual encoded body length; it is not a character count.
Content-Encoding RestOutputResponse.Headers.Content-Encoding	string Optional	HTTP content encodings accepted by the caller or applied to the response, such as gzip. This is different from text character encoding.
Date RestOutputResponse.Headers.Date	string Optional	Time reported by the operation/provider. Preserve the original representation when tracing; epoch values and formatted date/time strings need explicit conversion.

Field / key	Type / required	Details
Location RestOutputResponse.Headers.Location	string Optional	HTTP redirect or newly created resource URL returned by the server.
Set-Cookie RestOutputResponse.Headers.Set-Cookie	string[] Optional	HTTP cookie header. Cookie values can contain session credentials; avoid logging them.
Pragma RestOutputResponse.Headers.Pragma	string Optional	HTTP Pragma header used for legacy cache directives, for example no-cache.
DynamicHeaders RestOutputResponse.Headers.DynamicHeaders	object[] Optional	Repeating custom HTTP headers. Add one name/value entry for each header that is not represented by a fixed field.
name RestOutputResponse.Headers.DynamicHeaders.name	string Optional	Custom HTTP header name, for example X-Correlation-ID. Provide one name/value pair per repeating entry.
value RestOutputResponse.Headers.DynamicHeaders.value	string Optional	Value of the custom HTTP header named in this entry. Map the required string without embedding confidential tokens in logs.
mimeEnvelopeElement RestOutputResponse.mimeEnvelopeElement	object Optional	Multipart content container. Supply one part per attachment/body component, with part headers and one supported content representation.
mimePart RestOutputResponse.mimeEnvelopeElement.mimePart	object[] Optional	Multipart content container. Supply one part per attachment/body component, with part headers and one supported content representation.
mimeHeaders RestOutputResponse.mimeEnvelopeElement.mimePart.mimeHeaders	object Optional	Headers for this multipart part, including media type, content disposition and optional content ID.
content-type RestOutputResponse.mimeEnvelopeElement.mimePart.mimeHeaders.content-type	string Optional	MIME media type describing the content, for example application/json, application/xml or text/plain. Match it to the serialized body.
content-transfer-encoding RestOutputResponse.mimeEnvelopeElement.mimePart.mimeHeaders.content-transfer-encoding	string Optional	Transfer-encoding metadata for this MIME part. Ensure it matches the supplied content representation.
content-id RestOutputResponse.mimeEnvelopeElement.mimePart.mimeHeaders.content-id	string Optional	Identifier of a multipart part, used when the message refers to an attachment by content ID.
content-disposition RestOutputResponse.mimeEnvelopeElement.mimePart.mimeHeaders.content-disposition	string Optional	Multipart disposition, for example form-data; name="document"; filename="document.xml". Quote parameter values containing spaces.
binaryContent RestOutputResponse.mimeEnvelopeElement.mimePart.binaryContent	string Optional	Binary content represented by the activity contract, commonly base64 for JSON-visible fields. Use binary mode and matching encoding when forwarding bytes.
textContent RestOutputResponse.mimeEnvelopeElement.mimePart.textContent	string Optional	Textual document/content. Supply valid XML/JSON for parsers, or map rendered text for a wire-format destination. Encoding controls conversion to and from bytes.

Field / key	Type / required	Details
fileName RestOutputResponse.mimeEnvelopeElement.mimePart.fileName	string Optional	File metadata: name, full path, directory, modification time, access flags or size in bytes, according to this field. Values describe the runtime filesystem.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Fault	Description
HTTP_CONNECTIVITY	The HTTP endpoint could not be reached.
HTTP_TIMEOUT	The request exceeded its timeout.
HTTP_CLIENT_ERROR	The endpoint returned a 4xx response.
HTTP_SERVER_ERROR	The endpoint returned a 5xx response.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



Read Server-Sent Events

Activities / HTTP & APIs

Overview

Reads a bounded batch from a Server-Sent Events endpoint.

Map an HTTP or HTTPS URL, maximum events and optional Last-Event-ID. The activity sends Accept: text/event-stream and closes after the requested event count, end of stream or timeout.

Activity type: http; operation: sse_read.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
HTTP shared connection resourceId	resource Configuration setting	Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment. Connection type: http

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
RestInputRequest RestInputRequest	object Required	Root request structure. Map endpoint and request settings under Config, HTTP headers under Headers and multipart content under mimeEnvelopeElement.
Config RestInputRequest.Config	object Optional	Request endpoint and processing settings. Map host, port, requestURI, Method and RequestBody here; shared connection settings provide connection-level defaults.
requestURI RestInputRequest.Config.requestURI	string Optional	Endpoint or filesystem path for this operation. For HTTP use the service URI/path; for files use a path accessible to the runtime host, not only the designer machine.
QueryParameters RestInputRequest.Config.QueryParameters	object Optional	HTTP query parameters. Use a name/value object for structured parameters or QueryString for an already composed query; encode values rather than concatenating untrusted text.
URIParameters RestInputRequest.Config.URIParameters	object Optional	Values substituted into endpoint path placeholders. Provide a value for each required placeholder in the request URI or selected operation.
timeout RestInputRequest.Config.timeout	integer Optional	HTTP timeout in milliseconds. Distinguish connect/read timeouts in the shared connection from the whole-activity timeout.
activityTimeoutSeconds RestInputRequest.Config.activityTimeoutSeconds	number Optional	activityTimeoutSeconds: bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.

Field / key	Type / required	Details
Headers RestInputRequest.Headers	object Optional	HTTP header container. On requests, use CustomHeaders for additional headers. On responses, DynamicHeaders lists the received headers.
accept RestInputRequest.Headers.accept	string Optional	HTTP Accept header listing response media types the caller can process, for example application/json.
content-type RestInputRequest.Headers.content-type	string Optional	MIME media type describing the content, for example application/json, application/xml or text/plain. Match it to the serialized body.
Accept-Charset RestInputRequest.Headers.Accept-Charset	string Optional	HTTP character sets acceptable in the response. Keep body encoding and declared media type consistent.
Accept-Encoding RestInputRequest.Headers.Accept-Encoding	string Optional	HTTP content encodings accepted by the caller or applied to the response, such as gzip. This is different from text character encoding.
Cookie RestInputRequest.Headers.Cookie	string Optional	HTTP cookie header. Cookie values can contain session credentials; avoid logging them.
pragma RestInputRequest.Headers.pragma	string Optional	HTTP Pragma header used for legacy cache directives, for example no-cache.
CustomHeaders RestInputRequest.CustomHeaders	object[] Optional	Repeating request headers. Use Add header to map each name/value pair independently, or map a collection with For Each.
name RestInputRequest.CustomHeaders.name	string Optional	Custom HTTP header name, for example X-Correlation-ID. Provide one name/value pair per repeating entry.
value RestInputRequest.CustomHeaders.value	string Optional	Value of the custom HTTP header named in this entry. Map the required string without embedding confidential tokens in logs.
Maximum events RestInputRequest.Config.maxEvents	integer Optional	Maximum number of complete events or WebSocket messages to read before closing the connection.
Last event ID RestInputRequest.Config.lastEventId	string Optional	Optional Last-Event-ID sent when reconnecting to an SSE source.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Received events or messages events	object[] Optional	Ordered events or messages read before the configured limit. SSE returns event type, data and ID. WebSocket messages include type and data, with binary data in base64.
Number received eventCount	integer Optional	Number of events or messages returned by this activity.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

Field / key	Type / required	Details
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Fault	Description
HTTP_CONNECTIVITY	The HTTP endpoint could not be reached.
HTTP_TIMEOUT	The request exceeded its timeout.
HTTP_CLIENT_ERROR	The endpoint returned a 4xx response.
HTTP_SERVER_ERROR	The endpoint returned a 5xx response.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



WebSocket Exchange

Activities / HTTP & APIs

Overview

Exchanges messages with a WebSocket server.

Map a ws or wss URL, an optional outbound message and the maximum replies. Received binary messages are base64. The activity closes its connection when it finishes.

Activity type: http; operation: websocket_exchange.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
HTTP shared connection resourceId	resource Configuration setting	Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment. Connection type: http

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
RestInputRequest RestInputRequest	object Required	Root request structure. Map endpoint and request settings under Config, HTTP headers under Headers and multipart content under mimeEnvelopeElement.
Config RestInputRequest.Config	object Optional	Request endpoint and processing settings. Map host, port, requestURI, Method and RequestBody here; shared connection settings provide connection-level defaults.
requestURI RestInputRequest.Config.requestURI	string Optional	Endpoint or filesystem path for this operation. For HTTP use the service URI/path; for files use a path accessible to the runtime host, not only the designer machine.
RequestBody RestInputRequest.Config.RequestBody	any Optional	HTTP request entity configuration. RequestBody supplies content; bodyFormat selects its encoding/serialization and sendBodyMode controls body transmission. Use multipart parts for multipart requests.
QueryParameters RestInputRequest.Config.QueryParameters	object Optional	HTTP query parameters. Use a name/value object for structured parameters or QueryString for an already composed query; encode values rather than concatenating untrusted text.
URIParameters RestInputRequest.Config.URIParameters	object Optional	Values substituted into endpoint path placeholders. Provide a value for each required placeholder in the request URI or selected operation.
timeout RestInputRequest.Config.timeout	integer Optional	HTTP timeout in milliseconds. Distinguish connect/read timeouts in the shared connection from the whole-activity timeout.

Field / key	Type / required	Details
activityTimeoutSeconds RestInputRequest.Config.activityTimeoutSeconds	number Optional	activityTimeoutSeconds: bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.
Headers RestInputRequest.Headers	object Optional	HTTP header container. On requests, use CustomHeaders for additional headers. On responses, DynamicHeaders lists the received headers.
accept RestInputRequest.Headers.accept	string Optional	HTTP Accept header listing response media types the caller can process, for example application/json.
content-type RestInputRequest.Headers.content-type	string Optional	MIME media type describing the content, for example application/json, application/xml or text/plain. Match it to the serialized body.
Accept-Charset RestInputRequest.Headers.Accept-Charset	string Optional	HTTP character sets acceptable in the response. Keep body encoding and declared media type consistent.
Accept-Encoding RestInputRequest.Headers.Accept-Encoding	string Optional	HTTP content encodings accepted by the caller or applied to the response, such as gzip. This is different from text character encoding.
Cookie RestInputRequest.Headers.Cookie	string Optional	HTTP cookie header. Cookie values can contain session credentials; avoid logging them.
pragma RestInputRequest.Headers.pragma	string Optional	HTTP Pragma header used for legacy cache directives, for example no-cache.
CustomHeaders RestInputRequest.CustomHeaders	object[] Optional	Repeating request headers. Use Add header to map each name/value pair independently, or map a collection with For Each.
name RestInputRequest.CustomHeaders.name	string Optional	Custom HTTP header name, for example X-Correlation-ID. Provide one name/value pair per repeating entry.
value RestInputRequest.CustomHeaders.value	string Optional	Value of the custom HTTP header named in this entry. Map the required string without embedding confidential tokens in logs.
Maximum events RestInputRequest.Config.maxEvents	integer Optional	Maximum number of complete events or WebSocket messages to read before closing the connection.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Received events or messages events	object[] Optional	Ordered events or messages read before the configured limit. SSE returns event type, data and ID. WebSocket messages include type and data, with binary data in base64.
Number received eventCount	integer Optional	Number of events or messages returned by this activity.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.

Field / key	Type / required	Details
		Default: "${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Fault	Description
HTTP_CONNECTIVITY	The HTTP endpoint could not be reached.
HTTP_TIMEOUT	The request exceeded its timeout.
HTTP_CLIENT_ERROR	The endpoint returned a 4xx response.
HTTP_SERVER_ERROR	The endpoint returned a 5xx response.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



HTTP Write Preconditions

Activities / HTTP & APIs

Overview

Evaluates request validators before a write.

Map inbound request headers and the current ETag, Last-Modified and existence. Branch on allowed, send 412 or 428 on rejection, and guard the actual write atomically in the datastore.

Activity type: http; operation: write_precondition.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

No additional fields declared for this section.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
RestInputRequest RestInputRequest	object Required	Root request structure. Map endpoint and request settings under Config, HTTP headers under Headers and multipart content under mimeEnvelopeElement.
Config RestInputRequest.Config	object Optional	Request endpoint and processing settings. Map host, port, requestURI, Method and RequestBody here; shared connection settings provide connection-level defaults.
Headers RestInputRequest.Headers	object Optional	HTTP header container. On requests, use CustomHeaders for additional headers. On responses, DynamicHeaders lists the received headers.
Current ETag RestInputRequest.Config.currentETag	string Optional	Current datastore entity tag, including the double quotes used in HTTP validators.
Current Last-Modified RestInputRequest.Config.currentLastModified	string Optional	Current datastore Last-Modified value in HTTP date format.
Representation exists RestInputRequest.Config.representationExists	boolean Optional	Whether the resource already exists when evaluating a write precondition.
Require a precondition RestInputRequest.Config.requireWritePrecondition	boolean Optional	Return 428 when the caller supplies no write precondition.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Write allowed allowed	boolean Optional	Whether the write precondition passed. Execute the guarded write only when true.
Suggested HTTP status statusCode	integer Optional	Numeric HTTP status. Apply success/error routing according to the endpoint contract rather than treating every response as success.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Fault	Description
HTTP_CONNECTIVITY	The HTTP endpoint could not be reached.
HTTP_TIMEOUT	The request exceeded its timeout.
HTTP_CLIENT_ERROR	The endpoint returned a 4xx response.

Fault	Description
HTTP_SERVER_ERROR	The endpoint returned a 5xx response.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



HTTP Response

Activities / HTTP & APIs

Overview

Completes the active inbound HTTP exchange.

Place HTTP Send Response on the same execution path as the listener or receiver. In Input, optional HTTPHeaders contains Content-Type, complex StatusLine and repeating Cookies. Map StatusLine.httpVersion, StatusLine.statusCode and StatusLine.reasonPhrase independently. Native HTTP/1 listeners send the mapped version (1.0 or 1.1) and custom reason phrase, including an empty phrase. HTTP/1.0 responses use Content-Length and close the connection. HTTP/2 and HTTP/3 require a matching negotiated version and no nonempty reason phrase; unsupported overrides fail before sending headers. HTTP/1 fallback on an HTTP/2 or HTTP/3 listener does not support custom reason phrases. Unmapped fields use existing defaults, and existing full status-line strings still work. Mapped settings remain visible in statusLine execution output. DynamicHTTPHeaders contains repeating Headers with headerName and headerValue. Dynamic headers override standard/legacy headers; duplicate names remain separate header lines, including Set-Cookie. Legacy statusCode, contentType and headers mappings still work. If the endpoint declares a response schema, its fields are available for body mapping. An explicit Content-Type must be offered by the endpoint and acceptable to the caller.

Activity type: http_response; operation: response.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Status code statusCode	number Configuration setting	Numeric HTTP status. Apply success/error routing according to the endpoint contract rather than treating every response as success.
Response content type contentType	mediaType Configuration setting	Choose a common media type, enter a custom value, or use Automatic to use the listener default.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
HTTPHeaders HTTPHeaders	object Optional	Optional standard response headers and status. Unmapped fields keep the existing response defaults.
Content-Type HTTPHeaders.Content-Type	string Optional	Response media type. Overrides the legacy contentType mapping when supplied.
StatusLine HTTPHeaders.StatusLine	object Optional	Optional response status structure. Map httpVersion, statusCode and reasonPhrase independently, or map an entire object. Existing status-line string mappings remain supported.
httpVersion HTTPHeaders.StatusLine.httpVersion	string Optional	Native HTTP/1 listeners send the selected HTTP/1.0 or HTTP/1.1 version. HTTP/1.0 closes the connection. On HTTP/2 or HTTP/3, the value must match the negotiated protocol; incompatible overrides fail.

Field / key	Type / required	Details
statusCode HTTPHeaders.StatusLine.statusCode	integer Optional	Numeric response status from 100 to 599, for example 201 without quotes. Overrides the legacy statusCode when mapped. Hosted final responses require 200-599.
reasonPhrase HTTPHeaders.StatusLine.reasonPhrase	string Optional	Custom wire reason phrase on native HTTP/1 listeners, including an empty string. Omit for the standard phrase. HTTP/2 and HTTP/3 have no reason phrase; use an empty value or omit it. Unsupported transports and control characters fail validation.
Cookies HTTPHeaders.Cookies	string[] Optional	One or more complete Set-Cookie values, such as session=abc; Path=/; HttpOnly. Each value is sent as its own Set-Cookie header.
DynamicHTTPHeaders DynamicHTTPHeaders	object Optional	Optional custom response headers. Repeat Headers for each headerName/headerValue pair.
Headers DynamicHTTPHeaders.Headers	object[] Optional	Add or duplicate a header entry, map a collection, or use For Each. Dynamic values override standard/legacy headers; repeated names emit separate header lines. Set-Cookie appends to Cookies.
headerName DynamicHTTPHeaders.Headers.headerName	string Optional	HTTP header name, for example X-Correlation-ID. Required when the entry has a value.
headerValue DynamicHTTPHeaders.Headers.headerValue	string Optional	HTTP header value. Empty strings are allowed; line breaks and control characters are rejected.
Response headers headers	object Optional	Header name/value object. Map only headers required by the target protocol; credentials belong in protected authentication settings.
Response body body	any Optional	Message or response content. Map the business document rather than a previous publisher receipt. Objects retain their structure; string values carry serialized text.
Base64 response bytes binaryContent	base64 Optional	Binary content represented by the activity contract, commonly base64 for JSON-visible fields. Use binary mode and matching encoding when forwarding bytes.
Response file to stream fileName	string Optional	File metadata: name, full path, directory, modification time, access flags or size in bytes, according to this field. Values describe the runtime filesystem.
Status code statusCode	number Optional	Numeric HTTP status. Apply success/error routing according to the endpoint contract rather than treating every response as success.
Response content type contentType	string Optional	Choose a common media type, enter a custom value, or use Automatic to use the listener default.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Sent status code statusCode	integer Optional	Numeric HTTP status. Apply success/error routing according to the endpoint contract rather than treating every response as success.
Mapped status line statusLine	object Optional	Mapped status settings retained for debugging. Supported version/reason overrides affect the native HTTP/1 wire response; transport compatibility is checked before sending.

Field / key	Type / required	Details
httpVersion statusLine.httpVersion	string Optional	Mapped protocol metadata, normalized to an HTTP/ version token.
statusCode statusLine.statusCode	integer Optional	Effective numeric response status after applying optional StatusLine overrides.
reasonPhrase statusLine.reasonPhrase	string Optional	Mapped reason phrase, sent on native HTTP/1 listeners. Nonempty reason phrases are unsupported on HTTP/2 and HTTP/3.
Response sent sent	boolean Optional	Operation outcome returned by the runtime. Inspect error output as well; provider acceptance does not establish processing by a downstream consumer.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
HTTP_RESPONSE	The listener response could not be written.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



REST API Receiver

Activities / HTTP & APIs

Overview

Starts a task for a matching REST method and route.

In Request parameters, declare URI path placeholders, query values, and custom headers. Select a type and mark required entries. Invalid or missing required values return HTTP 400. Output provides pathParameters, query, headers with lowercase names, and body or xmlBody. Use HTTP Send Response to reply. Run or Debug exposes the live endpoint and, when enabled, Swagger and Postman.

Activity type: rest; operation: receiver.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
OpenAPI 3 / Swagger 2 JSON or YAML contract openApiDocument	textarea Configuration setting	OpenAPI definition and selected operation identifier. The operation supplies method/path and request/response contracts; reselect it after replacing the specification.
Contract base URL for relative references openApiBaseUrl	text Configuration setting	Absolute contract document URL used to resolve relative external references. It does not change the REST service endpoint.
Supplied external reference documents (URI/document JSON) openApiReferenceDocuments	textarea Configuration setting	URI-to-document JSON object supplying external API reference documents without a network fetch. Each URI identifies its JSON or YAML contract document.
Allowed external reference HTTPS origins openApiAllowedRefOrigins	text Configuration setting	Explicit HTTPS origins permitted for external contract retrieval. Empty disables network reference retrieval. Redirects and implicit file access are forbidden; downloaded documents have size/count bounds.
Contract operation ID operationId	text Configuration setting	OpenAPI definition and selected operation identifier. The operation supplies method/path and request/response contracts; reselect it after replacing the specification.
HTTP shared connection resourceId	resource Configuration setting	Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment. Connection type: http
Resource path path	text Configuration setting	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Allowed HTTP methods methods	methods Configuration setting	Select every HTTP method exposed by this REST resource.
Request parameters requestParameters	requestParameters Configuration setting	Declare typed URI parameters, query parameters, and custom headers.
Request content types requestContentTypes	mediaTypes Configuration setting	Select one or more accepted media types. With an OpenAPI contract, select media types declared by the operation or leave all unchecked to use the contract.

Field / key	Type / required	Details
Response content types responseContentTypes	mediaTypes Configuration setting	Select the offered response media types. With multiple selections, the Accept header selects the format; the first selected type is the default.
Request execution timeout (seconds) activityTimeoutSeconds	number Configuration setting	Request execution timeout (seconds): bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.
Validate request schema validateRequest	boolean Configuration setting	Validate the associated value against the configured contract. A mismatch raises a validation fault; this does not replace validation of business rules.
Request schema requestSchema	schema Configuration setting	Optional contract for the incoming request body. Select a project schema or browse for an XSD or JSON Schema file.
Validate response schema validateResponse	boolean Configuration setting	Validate the associated value against the configured contract. A mismatch raises a validation fault; this does not replace validation of business rules.
Response schema responseSchema	schema Configuration setting	Optional contract for the response body. Select or browse for a schema to make its fields available in HTTP Send Response.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
OpenAPI 3 / Swagger 2 JSON or YAML contract openApiDocument	string Optional	OpenAPI definition and selected operation identifier. The operation supplies method/path and request/response contracts; reselect it after replacing the specification.
Contract base URL for relative references openApiBaseUrl	string Optional	Absolute contract document URL used to resolve relative external references. It does not change the REST service endpoint.
Supplied external reference documents (URI/document JSON) openApiReferenceDocuments	string Optional	URI-to-document JSON object supplying external API reference documents without a network fetch. Each URI identifies its JSON or YAML contract document.
Allowed external reference HTTPS origins openApiAllowedRefOrigins	string Optional	Explicit HTTPS origins permitted for external contract retrieval. Empty disables network reference retrieval. Redirects and implicit file access are forbidden; downloaded documents have size/count bounds.
Contract operation ID operationId	string Optional	OpenAPI definition and selected operation identifier. The operation supplies method/path and request/response contracts; reselect it after replacing the specification.
Resource path path	string Optional	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Allowed HTTP methods methods	string Optional	Select every HTTP method exposed by this REST resource.
Request parameters requestParameters	string Optional	Declare typed URI parameters, query parameters, and custom headers.

Field / key	Type / required	Details
Request content types requestContentTypes	string Optional	Select one or more accepted media types. With an OpenAPI contract, select media types declared by the operation or leave all unchecked to use the contract.
Response content types responseContentTypes	string Optional	Select the offered response media types. With multiple selections, the Accept header selects the format; the first selected type is the default.
Request execution timeout (seconds) activityTimeoutSeconds	number Optional	Request execution timeout (seconds): bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.
Validate request schema validateRequest	boolean Optional	Validate the associated value against the configured contract. A mismatch raises a validation fault; this does not replace validation of business rules.
Request schema requestSchema	string Optional	Optional contract for the incoming request body. Select a project schema or browse for an XSD or JSON Schema file.
Validate response schema validateResponse	boolean Optional	Validate the associated value against the configured contract. A mismatch raises a validation fault; this does not replace validation of business rules.
Response schema responseSchema	string Optional	Optional contract for the response body. Select or browse for a schema to make its fields available in HTTP Send Response.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Request entity body	any Optional	Message or response content. Map the business document rather than a previous publisher receipt. Objects retain their structure; string values carry serialized text.
Base64 request bytes binaryContent	base64 Optional	Binary content represented by the activity contract, commonly base64 for JSON-visible fields. Use binary mode and matching encoding when forwarding bytes.
Temporary uploaded file fileName	string Optional	File metadata: name, full path, directory, modification time, access flags or size in bytes, according to this field. Values describe the runtime filesystem.
Uploaded byte count size	integer Optional	File metadata: name, full path, directory, modification time, access flags or size in bytes, according to this field. Values describe the runtime filesystem.
All received header pairs headerPairs	array Optional	Complete ordered received header name/value pairs, preserving repeated headers. Use this collection when every wire header occurrence is needed.
Client IP clientIp	string Optional	Client IP used for allowlists and throttling. Uses the transport peer unless the shared connection trusts the forwarding proxy.
HTTP trace ID traceId	string Optional	Trace identifier from a valid incoming traceparent, or one generated by the listener. Present when HTTP trace context is enabled.
Headers headers	object Optional	Header name/value object. Map only headers required by the target protocol; credentials belong in protected authentication settings.

Field / key	Type / required	Details
Query parameters query	object Optional	HTTP query parameters. Use a name/value object for structured parameters or QueryString for an already composed query; encode values rather than concatenating untrusted text.
URI parameters pathParameters	object Optional	Values substituted into endpoint path placeholders. Provide a value for each required placeholder in the request URI or selected operation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
REST_VALIDATION	The request does not conform to its API/schema contract.
REST_UNSUPPORTED_MEDIA	The request media type is unsupported.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



REST API Invoke

Activities / HTTP & APIs

Overview

Calls a remote REST API through a shared HTTP client connection.

In Input -> RestInputRequest.Config, map the URI and query parameters, request body, and optional OpenAPI contract and operation ID. Use CustomHeaders -> Add header for repeating name/value pairs, or use For Each for a collection. RestOutputResponse exposes the status, headers, and body.

Activity type: rest; operation: invoke.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
HTTP shared connection resourceId	resource Configuration setting	Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment. Connection type: http

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
RestInputRequest RestInputRequest	object Required	Root request structure. Map endpoint and request settings under Config, HTTP headers under Headers and multipart content under mimeEnvelopeElement.
Config RestInputRequest.Config	object Optional	Request endpoint and processing settings. Map host, port, requestURI, Method and RequestBody here; shared connection settings provide connection-level defaults.
host RestInputRequest.Config.host	string Optional	Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https.
port RestInputRequest.Config.port	integer Optional	Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https.
scheme RestInputRequest.Config.scheme	string Optional	Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https.
requestURI RestInputRequest.Config.requestURI	string Optional	Endpoint or filesystem path for this operation. For HTTP use the service URI/path; for files use a path accessible to the runtime host, not only the designer machine.

Field / key	Type / required	Details
Method RestInputRequest.Config.Method	string Optional	HTTP verb or invoked method name, according to this activity. For HTTP select the verb required by the API, such as GET, POST or PUT.
RequestBody RestInputRequest.Config.RequestBody	any Optional	HTTP request entity configuration. RequestBody supplies content; bodyFormat selects its encoding/serialization and sendBodyMode controls body transmission. Use multipart parts for multipart requests.
bodyFormat RestInputRequest.Config.bodyFormat	string Optional	HTTP request entity configuration. RequestBody supplies content; bodyFormat selects its encoding/serialization and sendBodyMode controls body transmission. Use multipart parts for multipart requests.
QueryParameters RestInputRequest.Config.QueryParameters	object Optional	HTTP query parameters. Use a name/value object for structured parameters or QueryString for an already composed query; encode values rather than concatenating untrusted text.
URIParameters RestInputRequest.Config.URIParameters	object Optional	Values substituted into endpoint path placeholders. Provide a value for each required placeholder in the request URI or selected operation.
QueryString RestInputRequest.Config.QueryString	string Optional	HTTP query parameters. Use a name/value object for structured parameters or QueryString for an already composed query; encode values rather than concatenating untrusted text.
FilePath RestInputRequest.Config.FilePath	string Optional	File path on the runtime host. Ensure the runtime account can read the file and that the path exists in the deployment environment.
timeout RestInputRequest.Config.timeout	integer Optional	HTTP timeout in milliseconds. activityTimeoutSeconds separately bounds the overall activity.
activityTimeoutSeconds RestInputRequest.Config.activityTimeoutSeconds	number Optional	activityTimeoutSeconds: bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.
followRedirects RestInputRequest.Config.followRedirects	boolean Optional	Follow HTTP redirect responses. Disable when the flow must inspect or reject redirects explicitly.
raiseForStatus RestInputRequest.Config.raiseForStatus	boolean Optional	HTTP success policy. raiseForStatus faults responses outside the accepted success status codes; successStatusCodes defines that accepted set.
streamRequest RestInputRequest.Config.streamRequest	boolean Optional	Stream MIME attachment content from files in bounded chunks instead of buffering the complete envelope. File body uploads stream automatically.
responseFile RestInputRequest.Config.responseFile	string Optional	Runtime filename for a streamed download. A temporary file is atomically published only after successful completion; body returns fileName and size. Response schema validation requires buffered mode.
overwriteResponseFile RestInputRequest.Config.overwriteResponseFile	boolean Optional	Permit replacing an existing download destination after a successful transfer. Disabled by default to protect existing files.
responseBodyMode RestInputRequest.Config.responseBodyMode	string Optional	Set binary to preserve the response bytes as the body. File mode uses responseFile; structured output also exposes base64 binaryContent.
multipartPartHeaders RestInputRequest.Config.multipartPartHeaders	object Optional	Object keyed by multipart field name. Map only additional headers declared by that field in the OpenAPI encoding.

Field / key	Type / required	Details
callbackUrl RestInputRequest.Config.callbackUrl	string Optional	Callback target URL supplied to a generated callback subtask. The destination must match a configured HTTPS origin.
openApiDocument RestInputRequest.Config.openApiDocument	any Optional	OpenAPI definition and selected operation identifier. The operation supplies method/path and request/response contracts; reselect it after replacing the specification.
openApiReferenceDocuments RestInputRequest.Config.openApiReferenceDocuments	any Optional	URI-to-document JSON object supplying external API reference documents without a network fetch. Each URI identifies its JSON or YAML contract document.
requestSchema RestInputRequest.Config.requestSchema	any Optional	Selected schema/contract for the request, response or record structure. Refresh mappings after changing schema fields, types or cardinalities.
responseSchema RestInputRequest.Config.responseSchema	any Optional	Selected schema/contract for the request, response or record structure. Refresh mappings after changing schema fields, types or cardinalities.
openApiBaseUrl RestInputRequest.Config.openApiBaseUrl	string Optional	Absolute contract document URL used to resolve relative external references. It does not change the REST service endpoint.
openApiAllowedRefOrigins RestInputRequest.Config.openApiAllowedRefOrigins	string Optional	Explicit HTTPS origins permitted for external contract retrieval. Empty disables network reference retrieval. Redirects and implicit file access are forbidden; downloaded documents have size/count bounds.
operationId RestInputRequest.Config.operationId	string Optional	OpenAPI definition and selected operation identifier. The operation supplies method/path and request/response contracts; reselect it after replacing the specification.
validateRequest RestInputRequest.Config.validateRequest	boolean Optional	Validate the associated value against the configured contract. A mismatch raises a validation fault; this does not replace validation of business rules.
validateResponse RestInputRequest.Config.validateResponse	boolean Optional	Validate the associated value against the configured contract. A mismatch raises a validation fault; this does not replace validation of business rules.
requestEntityProcessing RestInputRequest.Config.requestEntityProcessing	string Optional	HTTP request entity configuration. RequestBody supplies content; bodyFormat selects its encoding/serialization and sendBodyMode controls body transmission. Use multipart parts for multipart requests.
sendBodyMode RestInputRequest.Config.sendBodyMode	string Optional	HTTP request entity configuration. RequestBody supplies content; bodyFormat selects its encoding/serialization and sendBodyMode controls body transmission. Use multipart parts for multipart requests.
successStatusCodes RestInputRequest.Config.successStatusCodes	string Optional	HTTP success policy. raiseForStatus faults responses outside the accepted success status codes; successStatusCodes defines that accepted set.
Headers RestInputRequest.Headers	object Optional	HTTP header container. On requests, use CustomHeaders for additional headers. On responses, DynamicHeaders lists the received headers.
accept RestInputRequest.Headers.accept	string Optional	HTTP Accept header listing response media types the caller can process, for example application/json.
content-type RestInputRequest.Headers.content-type	string Optional	MIME media type describing the content, for example application/json, application/xml or text/plain. Match it to the serialized body.

Field / key	Type / required	Details
Accept-Charset RestInputRequest.Headers.Accept-Charset	string Optional	HTTP character sets acceptable in the response. Keep body encoding and declared media type consistent.
Accept-Encoding RestInputRequest.Headers.Accept-Encoding	string Optional	HTTP content encodings accepted by the caller or applied to the response, such as gzip. This is different from text character encoding.
Cookie RestInputRequest.Headers.Cookie	string Optional	HTTP cookie header. Cookie values can contain session credentials; avoid logging them.
pragma RestInputRequest.Headers.pragma	string Optional	HTTP Pragma header used for legacy cache directives, for example no-cache.
CustomHeaders RestInputRequest.CustomHeaders	object[] Optional	Repeating request headers. Use Add header to map each name/value pair independently, or map a collection with For Each.
name RestInputRequest.CustomHeaders.name	string Optional	Custom HTTP header name, for example X-Correlation-ID. Provide one name/value pair per repeating entry.
value RestInputRequest.CustomHeaders.value	string Optional	Value of the custom HTTP header named in this entry. Map the required string without embedding confidential tokens in logs.
mimeEnvelopeElement RestInputRequest.mimeEnvelopeElement	object Optional	Multipart content container. Supply one part per attachment/body component, with part headers and one supported content representation.
mimePart RestInputRequest.mimeEnvelopeElement.mimePart	object[] Optional	Multipart content container. Supply one part per attachment/body component, with part headers and one supported content representation.
mimeHeaders RestInputRequest.mimeEnvelopeElement.mimePart.mimeHeaders	object Optional	Headers for this multipart part, including media type, content disposition and optional content ID.
content-type RestInputRequest.mimeEnvelopeElement.mimePart.mimeHeaders.content-type	string Optional	MIME media type describing the content, for example application/json, application/xml or text/plain. Match it to the serialized body.
content-transfer-encoding RestInputRequest.mimeEnvelopeElement.mimePart.mimeHeaders.content-transfer-encoding	string Optional	Transfer-encoding metadata for this MIME part. Ensure it matches the supplied content representation.
content-id RestInputRequest.mimeEnvelopeElement.mimePart.mimeHeaders.content-id	string Optional	Identifier of a multipart part, used when the message refers to an attachment by content ID.
content-disposition RestInputRequest.mimeEnvelopeElement.mimePart.mimeHeaders.content-disposition	string Optional	Multipart disposition, for example form-data; name="document"; filename="document.xml". Quote parameter values containing spaces.
binaryContent RestInputRequest.mimeEnvelopeElement.mimePart.binaryContent	string Optional	Binary content represented by the activity contract, commonly base64 for JSON-visible fields. Use binary mode and matching encoding when forwarding bytes.
textContent RestInputRequest.mimeEnvelopeElement.mimePart.textContent	string Optional	Textual document/content. Supply valid XML/JSON for parsers, or map rendered text for a wire-format destination. Encoding controls conversion to and from bytes.

Field / key	Type / required	Details
fileName RestInputRequest.mimeEnvelopeElement.mimePart.fileName	string Optional	File metadata: name, full path, directory, modification time, access flags or size in bytes, according to this field. Values describe the runtime filesystem.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
RestOutputResponse RestOutputResponse	object Required	Complete HTTP response envelope containing statusLine, headers, decoded body and optional text/binary/multipart content.
statusLine RestOutputResponse.statusLine	object Optional	HTTP protocol version, numeric response status and reason phrase returned by the server.
httpVersion RestOutputResponse.statusLine.httpVersion	string Optional	HTTP protocol version reported for the response, for example HTTP/1.1.
statusCode RestOutputResponse.statusLine.statusCode	integer Optional	Numeric HTTP status. Apply success/error routing according to the endpoint contract rather than treating every response as success.
reasonPhrase RestOutputResponse.statusLine.reasonPhrase	string Optional	Human-readable HTTP status description. Use statusCode for programmatic decisions.
body RestOutputResponse.body	any Optional	Message or response content. Map the business document rather than a previous publisher receipt. Objects retain their structure; string values carry serialized text.
asciiContent RestOutputResponse.asciiContent	string Optional	Textual document/content. Supply valid XML/JSON for parsers, or map rendered text for a wire-format destination. Encoding controls conversion to and from bytes.
binaryContent RestOutputResponse.binaryContent	string Optional	Binary content represented by the activity contract, commonly base64 for JSON-visible fields. Use binary mode and matching encoding when forwarding bytes.
Headers RestOutputResponse.Headers	object Optional	HTTP header container. On requests, use CustomHeaders for additional headers. On responses, DynamicHeaders lists the received headers.
Allow RestOutputResponse.Headers.Allow	string[] Optional	HTTP response methods allowed by the endpoint, normally returned with method-not-allowed responses.
Content-Type RestOutputResponse.Headers.Content-Type	string Optional	MIME media type describing the content, for example application/json, application/xml or text/plain. Match it to the serialized body.
Content-Length RestOutputResponse.Headers.Content-Length	string Optional	HTTP content length in bytes. Use the actual encoded body length; it is not a character count.
Content-Encoding RestOutputResponse.Headers.Content-Encoding	string Optional	HTTP content encodings accepted by the caller or applied to the response, such as gzip. This is different from text character encoding.
Date RestOutputResponse.Headers.Date	string Optional	Time reported by the operation/provider. Preserve the original representation when tracing; epoch values and formatted date/time strings need explicit conversion.

Field / key	Type / required	Details
Location RestOutputResponse.Headers.Location	string Optional	HTTP redirect or newly created resource URL returned by the server.
Set-Cookie RestOutputResponse.Headers.Set-Cookie	string[] Optional	HTTP cookie header. Cookie values can contain session credentials; avoid logging them.
Pragma RestOutputResponse.Headers.Pragma	string Optional	HTTP Pragma header used for legacy cache directives, for example no-cache.
DynamicHeaders RestOutputResponse.Headers.DynamicHeaders	object[] Optional	Repeating custom HTTP headers. Add one name/value entry for each header that is not represented by a fixed field.
name RestOutputResponse.Headers.DynamicHeaders.name	string Optional	Custom HTTP header name, for example X-Correlation-ID. Provide one name/value pair per repeating entry.
value RestOutputResponse.Headers.DynamicHeaders.value	string Optional	Value of the custom HTTP header named in this entry. Map the required string without embedding confidential tokens in logs.
mimeEnvelopeElement RestOutputResponse.mimeEnvelopeElement	object Optional	Multipart content container. Supply one part per attachment/body component, with part headers and one supported content representation.
mimePart RestOutputResponse.mimeEnvelopeElement.mimePart	object[] Optional	Multipart content container. Supply one part per attachment/body component, with part headers and one supported content representation.
mimeHeaders RestOutputResponse.mimeEnvelopeElement.mimePart.mimeHeaders	object Optional	Headers for this multipart part, including media type, content disposition and optional content ID.
content-type RestOutputResponse.mimeEnvelopeElement.mimePart.mimeHeaders.content-type	string Optional	MIME media type describing the content, for example application/json, application/xml or text/plain. Match it to the serialized body.
content-transfer-encoding RestOutputResponse.mimeEnvelopeElement.mimePart.mimeHeaders.content-transfer-encoding	string Optional	Transfer-encoding metadata for this MIME part. Ensure it matches the supplied content representation.
content-id RestOutputResponse.mimeEnvelopeElement.mimePart.mimeHeaders.content-id	string Optional	Identifier of a multipart part, used when the message refers to an attachment by content ID.
content-disposition RestOutputResponse.mimeEnvelopeElement.mimePart.mimeHeaders.content-disposition	string Optional	Multipart disposition, for example form-data; name="document"; filename="document.xml". Quote parameter values containing spaces.
binaryContent RestOutputResponse.mimeEnvelopeElement.mimePart.binaryContent	string Optional	Binary content represented by the activity contract, commonly base64 for JSON-visible fields. Use binary mode and matching encoding when forwarding bytes.
textContent RestOutputResponse.mimeEnvelopeElement.mimePart.textContent	string Optional	Textual document/content. Supply valid XML/JSON for parsers, or map rendered text for a wire-format destination. Encoding controls conversion to and from bytes.

Field / key	Type / required	Details
fileName RestOutputResponse.mimeEnvelopeElement.mimePart.fileName	string Optional	File metadata: name, full path, directory, modification time, access flags or size in bytes, according to this field. Values describe the runtime filesystem.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Fault	Description
HTTP_CONNECTIVITY	The HTTP endpoint could not be reached.
HTTP_TIMEOUT	The request exceeded its timeout.
HTTP_CLIENT_ERROR	The endpoint returned a 4xx response.
HTTP_SERVER_ERROR	The endpoint returned a 5xx response.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



SOAP Service

Activities / HTTP & APIs

Overview

Hosts a SOAP service or invokes a request/reply web service.

Service mode publishes the configured contract and begins the task for a matching SOAP request. Request/reply mode uses the selected HTTP connection and WSDL metadata for outbound invocation.

Activity type: soap; operation: service.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
HTTP shared connection resourceId	resource Configuration setting	Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment. Connection type: http
Service path path	text Configuration setting	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Service name serviceName	text Configuration setting	SOAP contract/operation settings. Select the service and operation from the WSDL; soapAction identifies the operation and envelope contains the SOAP XML request.
Operation name operationName	text Configuration setting	SOAP contract/operation settings. Select the service and operation from the WSDL; soapAction identifies the operation and envelope contains the SOAP XML request.
WSDL resource wsdl	text Configuration setting	SOAP contract/operation settings. Select the service and operation from the WSDL; soapAction identifies the operation and envelope contains the SOAP XML request.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Service path path	string Optional	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Service name serviceName	string Optional	SOAP contract/operation settings. Select the service and operation from the WSDL; soapAction identifies the operation and envelope contains the SOAP XML request.
Operation name operationName	string Optional	SOAP contract/operation settings. Select the service and operation from the WSDL; soapAction identifies the operation and envelope contains the SOAP XML request.

Field / key	Type / required	Details
WSDL resource wsdl	string Optional	SOAP contract/operation settings. Select the service and operation from the WSDL; soapAction identifies the operation and envelope contains the SOAP XML request.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
SOAP envelope envelope	xml Optional	SOAP contract/operation settings. Select the service and operation from the WSDL; soapAction identifies the operation and envelope contains the SOAP XML request.
SOAP/HTTP headers headers	object Optional	Header name/value object. Map only headers required by the target protocol; credentials belong in protected authentication settings.
Resolved operation operation	string Optional	Select the operation performed by this activity. The operation determines the available configuration, input contract and output contract.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
SOAP_FAULT	The request generated a SOAP fault.
WSDL_VALIDATION	The message does not conform to the WSDL.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



SOAP Request Reply

Activities / HTTP & APIs

Overview

Hosts a SOAP service or invokes a request/reply web service.

Service mode publishes the configured contract and begins the task for a matching SOAP request. Request/reply mode uses the selected HTTP connection and WSDL metadata for outbound invocation.

Activity type: soap; operation: request_reply.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
HTTP shared connection resourceId	resource Configuration setting	Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment. Connection type: http
Service endpoint url	text Configuration setting	Endpoint or filesystem path for this operation. For HTTP use the service URI/path; for files use a path accessible to the runtime host, not only the designer machine.
SOAP action soapAction	text Configuration setting	SOAP contract/operation settings. Select the service and operation from the WSDL; soapAction identifies the operation and envelope contains the SOAP XML request.
WSDL resource wsdl	text Configuration setting	SOAP contract/operation settings. Select the service and operation from the WSDL; soapAction identifies the operation and envelope contains the SOAP XML request.
Timeout seconds timeout	number Configuration setting	SOAP HTTP transport timeout; configure connection/read waits on the selected HTTP shared connection.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
SOAP envelope envelope	xml Required	SOAP contract/operation settings. Select the service and operation from the WSDL; soapAction identifies the operation and envelope contains the SOAP XML request.
SOAP headers headers	object Optional	Header name/value object. Map only headers required by the target protocol; credentials belong in protected authentication settings.
Service endpoint url	string Optional	Endpoint or filesystem path for this operation. For HTTP use the service URI/path; for files use a path accessible to the runtime host, not only the designer machine.

Field / key	Type / required	Details
SOAP action soapAction	string Optional	SOAP contract/operation settings. Select the service and operation from the WSDL; soapAction identifies the operation and envelope contains the SOAP XML request.
WSDL resource wsdl	string Optional	SOAP contract/operation settings. Select the service and operation from the WSDL; soapAction identifies the operation and envelope contains the SOAP XML request.
Timeout seconds timeout	number Optional	SOAP HTTP transport timeout; configure connection/read waits on the selected HTTP shared connection.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
HTTP status statusCode	integer Optional	Numeric HTTP status. Apply success/error routing according to the endpoint contract rather than treating every response as success.
SOAP response envelope body	xml Optional	Message or response content. Map the business document rather than a previous publisher receipt. Objects retain their structure; string values carry serialized text.
Response headers headers	object Optional	Header name/value object. Map only headers required by the target protocol; credentials belong in protected authentication settings.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0

Field / key	Type / required	Details
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

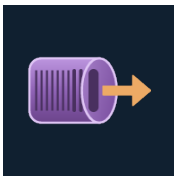
Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Fault	Description
SOAP_FAULT	The remote service returned a SOAP fault.
WSDL_VALIDATION	The request or response violates the WSDL.
CONNECTIVITY	The configured shared connection is unavailable.
RETRY_EXHAUSTED	The reconnection or retry policy was exhausted.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



EMS Queue Receiver

Activities / EMS

Overview

Receive a queue event

Use as a task starter for work arriving on a queue; no upstream message-body input is required.

Prerequisites

Install the licensed, version-compatible JMS client JARs under the runtime driver root/jms or set Driver JAR directory. Use the bundled Java bridge/runtime. A direct EMS connection requires URL, username and password; JNDI additionally requires its context factory, provider URL and connection-factory settings. Keep Java class names exactly as provided by the driver. Test from the execution machine, not only the developer laptop.

Sends, receives, browses, or acknowledges EMS messages.

Destination, delivery, selector, acknowledgement, and message properties follow the selected EMS operation. Client acknowledgement handles can be passed to Confirm Message.

Activity type: ems; operation: queue_receiver.

Configuration

Configuration procedure

Select EMS resource, browse the destination property, and configure selector and receive wait. Client acknowledgement exposes a handle, but acknowledgement timing depends on the runtime path. Session count and flow limit can be mapped from properties; neither is a proven bridge concurrency control.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
EMS connection resourceId	resource Required	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: ems
Messaging style messagingStyle	select Configuration setting	Queue/topic design hint. The current engine primarily chooses destination kind from the operation; do not use this to turn a queue sender into a topic publisher. Choices: Queue, Topic, Generic
Destination queue	text Configuration setting	Queue name, not the broker URL. Browse an environment property to avoid hard-coding environment-specific destinations.
Message type messageType	select Configuration setting	Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented. Choices: Text, Bytes, Map, Object, Object Ref, Simple, Stream, XML Text
JMS message selector messageSelector	text Configuration setting	Provider-side JMS selector expression over supported headers/properties. It does not query inside the JSON message body.

Field / key	Type / required	Details
Acknowledge mode acknowledgeMode	select Configuration setting	Choose automatic completion or explicit acknowledgement. Guarantees differ by connector and execution path; read Advanced before relying on rollback/redelivery. Choices: Auto, Client, Dups OK, Explicit Client, Explicit Client Dups OK, Transactional
Session count maxSessions	number Configuration setting	Passed as a session-count option by the wrapper; current engine bridge does not create a configurable receiver-session pool from it.
Flow limit flowLimit	number Configuration setting	Passed by the wrapper but not enforced as an engine bridge in-flight limit. Do not rely on it as the only backpressure mechanism.
Receive timeout (ms) receiveTimeout	number Configuration setting	Receive wait in milliseconds, default 30000 on the engine bridge path. A timeout can return an empty result; it need not throw a fault.

Input

Mapping example

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
queue = ${properties.messaging.inboundQueue}
receiveTimeout = 30000
Downstream message = ${EMS-Receive.body}
Confirm Message ackId = ${EMS-Receive.ackId}
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Messaging style messagingStyle	string Optional	Queue/topic design hint. The current engine primarily chooses destination kind from the operation; do not use this to turn a queue sender into a topic publisher.
Destination queue	string Optional	Queue name, not the broker URL. Browse an environment property to avoid hard-coding environment-specific destinations.
Message type messageType	string Optional	Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented.
JMS message selector messageSelector	string Optional	Provider-side JMS selector expression over supported headers/properties. It does not query inside the JSON message body.
Acknowledge mode acknowledgeMode	string Optional	Choose automatic completion or explicit acknowledgement. Guarantees differ by connector and execution path; read Advanced before relying on rollback/redelivery.
Session count maxSessions	number Optional	Passed as a session-count option by the wrapper; current engine bridge does not create a configurable receiver-session pool from it.
Flow limit flowLimit	number Optional	Passed by the wrapper but not enforced as an engine bridge in-flight limit. Do not rely on it as the only backpressure mechanism.
Receive timeout (ms) receiveTimeout	number Optional	Receive wait in milliseconds, default 30000 on the engine bridge path. A timeout can return an empty result; it need not throw a fault.

Output

Illustrative runtime output

```
{"body":{"id":"A100"},"headers":{"JMSMessageID":"<provider-id>"},"properties":{},"ackId":"<handle>","co
unt":1}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
JMS message body body	object|string|binary Optional	Received message body. The engine attempts JSON decoding for text that contains valid JSON; other text remains a string.
JMS standard headers headers	object Optional	Header name/value object. The messaging runtime stringifies outgoing values; this is not a guarantee of arbitrary native binary headers or JMS header setter support.
JMS application properties properties	object Optional	Application-defined message properties returned by the broker. Standard JMS headers are returned separately.
Client acknowledgement handle ackId	string Optional	Opaque, runtime-local confirmation token. Pass the original value to Confirm Message; it is not the business message ID and cannot be reconstructed after restart.

Advanced

Delivery, transactions and performance

Persistent sender workers are reused for the same resolved connection and execution scope. The cache is bounded; changing credentials/configuration frequently or exceeding the cache can create new workers. Native text messages and string application properties are the supported send path. Do not treat all editor choices as implemented native features. Retry after an ambiguous timeout can duplicate a message.

Implemented behavior and limitations

Important engine limitation: the one-shot receive bridge acknowledges client-mode delivery before closing, then the engine returns a local confirmation token. Confirm Message cannot undo that early provider acknowledgement. The direct Python listener uses a different persistent transport; do not transfer its guarantees to Studio/engine mode.

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Troubleshooting procedure

Missing JAR/class/native bridge errors: check the installed driver directory, matching versions and packaged Java bridge. Authentication/destination failures: verify credentials, queue/topic permissions and the resolved environment destination. Timeout: inspect empty-receive results as well as faults. A successful send is not proof that another application consumed or processed the message.

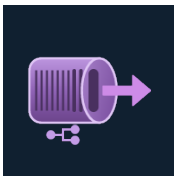
Acceptance test

Acceptance: publish while Run is active, verify automatic flow start, stop during processing and check broker redelivery. Do not approve loss-sensitive processing until acknowledgement timing is verified.

Fault	Description
JMSInvalidInputException	The JMS activity input is invalid.
JMSMessageCreateException	The configured JMS message could not be created.
JMSSessionCreateException	A JMS session could not be created.
JMSReceiveException	The JMS message could not be received.
ActivityTimedOutException	The JMS operation reached its configured timeout.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs



EMS Topic Subscriber

Activities / EMS

Overview

Receive a topic event

Use as a starter for topic messages. Topic delivery differs from queue competing-consumer behavior.

Prerequisites

Install the licensed, version-compatible JMS client JARs under the runtime driver root/jms or set Driver JAR directory. Use the bundled Java bridge/runtime. A direct EMS connection requires URL, username and password; JNDI additionally requires its context factory, provider URL and connection-factory settings. Keep Java class names exactly as provided by the driver. Test from the execution machine, not only the developer laptop.

Sends, receives, browses, or acknowledges EMS messages.

Destination, delivery, selector, acknowledgement, and message properties follow the selected EMS operation. Client acknowledgement handles can be passed to Confirm Message.

Activity type: ems; operation: topic_subscriber.

Configuration

Configuration procedure

Select the EMS resource and topic property. Set selector only when necessary. The UI offers durable/shared options, but the native engine uses a regular consumer and does not wire these subscription constructors.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
EMS connection resourceId	resource Required	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: ems
Messaging style messagingStyle	select Configuration setting	Queue/topic design hint. The current engine primarily chooses destination kind from the operation; do not use this to turn a queue sender into a topic publisher. Choices: Queue, Topic, Generic
Destination topic	text Configuration setting	Broker topic name. Resolve it from the active environment, for example ${properties.messaging.topic}. Consumer topic lists use semicolons.
Message type messageType	select Configuration setting	Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented. Choices: Text, Bytes, Map, Object, Object Ref, Simple, Stream, XML Text
JMS message selector messageSelector	text Configuration setting	Provider-side JMS selector expression over supported headers/properties. It does not query inside the JSON message body.

Field / key	Type / required	Details
Acknowledge mode acknowledgeMode	select Configuration setting	Choose automatic completion or explicit acknowledgement. Guarantees differ by connector and execution path; read Advanced before relying on rollback/redelivery. Choices: Auto, Client, Dups OK, Explicit Client, Explicit Client Dups OK, Transactional
Session count maxSessions	number Configuration setting	Passed as a session-count option by the wrapper; current engine bridge does not create a configurable receiver-session pool from it.
Flow limit flowLimit	number Configuration setting	Passed by the wrapper but not enforced as an engine bridge in-flight limit. Do not rely on it as the only backpressure mechanism.
Receive timeout (ms) receiveTimeout	number Configuration setting	Receive wait in milliseconds, default 30000 on the engine bridge path. A timeout can return an empty result; it need not throw a fault.
Durable subscriber durable	boolean Configuration setting	Editor option not wired to a durable subscriber constructor in the native engine bridge. Offline topic retention cannot be assumed.
Durable subscription name subscriptionName	text Configuration setting	Intended durable subscription identity; current native engine consumer does not use it to create a durable subscription.
Shared subscription sharedSubscription	boolean Configuration setting	Editor option not wired to a shared-subscription constructor in the current native engine bridge.

Input

Mapping example

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
topic = ${properties.messaging.eventsTopic}
Downstream message = ${EMS-Topic.body}
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Messaging style messagingStyle	string Optional	Queue/topic design hint. The current engine primarily chooses destination kind from the operation; do not use this to turn a queue sender into a topic publisher.
Destination topic	string Optional	Broker topic name. Resolve it from the active environment, for example ${properties.messaging.topic}. Consumer topic lists use semicolons.
Message type messageType	string Optional	Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented.
JMS message selector messageSelector	string Optional	Provider-side JMS selector expression over supported headers/properties. It does not query inside the JSON message body.
Acknowledge mode acknowledgeMode	string Optional	Choose automatic completion or explicit acknowledgement. Guarantees differ by connector and execution path; read Advanced before relying on rollback/redelivery.

Field / key	Type / required	Details
Session count maxSessions	number Optional	Passed as a session-count option by the wrapper; current engine bridge does not create a configurable receiver-session pool from it.
Flow limit flowLimit	number Optional	Passed by the wrapper but not enforced as an engine bridge in-flight limit. Do not rely on it as the only backpressure mechanism.
Receive timeout (ms) receiveTimeout	number Optional	Receive wait in milliseconds, default 30000 on the engine bridge path. A timeout can return an empty result; it need not throw a fault.
Durable subscriber durable	boolean Optional	Editor option not wired to a durable subscriber constructor in the native engine bridge. Offline topic retention cannot be assumed.
Durable subscription name subscriptionName	string Optional	Intended durable subscription identity; current native engine consumer does not use it to create a durable subscription.
Shared subscription sharedSubscription	boolean Optional	Editor option not wired to a shared-subscription constructor in the current native engine bridge.

Output

Illustrative runtime output

```
{"body":"event text","headers":{"JMSMessageID":"<provider-id>"},"properties":{},"count":1}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
JMS message body body	object|string|binary Optional	Received message body. The engine attempts JSON decoding for text that contains valid JSON; other text remains a string.
JMS standard headers headers	object Optional	Header name/value object. The messaging runtime stringifies outgoing values; this is not a guarantee of arbitrary native binary headers or JMS header setter support.
JMS application properties properties	object Optional	Application-defined message properties returned by the broker. Standard JMS headers are returned separately.
Client acknowledgement handle ackId	string Optional	Opaque, runtime-local confirmation token. Pass the original value to Confirm Message; it is not the business message ID and cannot be reconstructed after restart.

Advanced

Delivery, transactions and performance

Persistent sender workers are reused for the same resolved connection and execution scope. The cache is bounded; changing credentials/configuration frequently or exceeding the cache can create new workers. Native text messages and string application properties are the supported send path. Do not treat all editor choices as implemented native features. Retry after an ambiguous timeout can duplicate a message.

Implemented behavior and limitations

Do not assume messages published while the subscriber is offline are retained. durable, subscriptionName and sharedSubscription are editor-only on the current native engine path. Client acknowledgement has the same early-ack limitation as Queue Receiver.

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Troubleshooting procedure

Missing JAR/class/native bridge errors: check the installed driver directory, matching versions and packaged Java bridge. Authentication/destination failures: verify credentials, queue/topic permissions and the resolved environment destination. Timeout: inspect empty-receive results as well as faults. A successful send is not proof that another application consumed or processed the message.

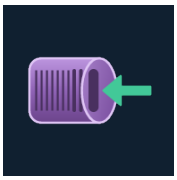
Acceptance test

Acceptance: test online delivery and offline publication separately. Record observed retention and redelivery rather than assuming durable subscription behavior.

Fault	Description
JMSInvalidInputException	The JMS activity input is invalid.
JMSMessageCreateException	The configured JMS message could not be created.
JMSSessionCreateException	A JMS session could not be created.
JMSReceiveException	The JMS message could not be received.
ActivityTimedOutException	The JMS operation reached its configured timeout.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs



EMS Queue Sender

Activities / EMS

Overview

Send a queue message

Use inside a task to send its mapped body to a queue.

Prerequisites

Install the licensed, version-compatible JMS client JARs under the runtime driver root/jms or set Driver JAR directory. Use the bundled Java bridge/runtime. A direct EMS connection requires URL, username and password; JNDI additionally requires its context factory, provider URL and connection-factory settings. Keep Java class names exactly as provided by the driver. Test from the execution machine, not only the developer laptop.

Sends, receives, browses, or acknowledges EMS messages.

Destination, delivery, selector, acknowledgement, and message properties follow the selected EMS operation. Client acknowledgement handles can be passed to Confirm Message.

Activity type: ems; operation: send.

Configuration

Configuration procedure

Select the EMS connection and set queue/destination explicitly. Map message, optional correlationId and application properties. The palette uses operation send; its current base editor contract omits some sender controls, so the supplemental field guide below describes runtime keys as well as editor fields.

Runtime setting	Meaning
destination	Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default".
message	Map the record/message body, not the entire preceding activity envelope unless that envelope is intentionally the payload.
deliveryMode	Persistent or Non-Persistent producer setting. Persistence still depends on broker configuration and is not an exactly-once business guarantee.
priority	JMS priority from 0 to 9; default 4. Ordering and fairness are provider-dependent.
expiration	Message time-to-live in milliseconds; zero means no producer-specified expiry.
correlationId	Correlation identifier attached to the outgoing message. For replies use the request correlation ID, or its message ID when the request protocol requires that fallback.
replyTo	Destination attached as JMSReplyTo. Request Reply creates a temporary queue when this is blank.
dynamicProperties	Object/JSON object of application properties. Current messaging code stringifies outgoing property values.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
EMS connection resourceId	resource Required	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: ems

Input

Mapping example

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
message = ${input.order}
destination = ${properties.messaging.outboundQueue}
correlationId = ${input.order.id}
deliveryMode = Persistent
priority = 4
expiration = 0
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
JMS message body message	object|string|binary Required	Map the record/message body, not the entire preceding activity envelope unless that envelope is intentionally the payload.
Dynamic destination override destination	string Optional	Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default".
Dynamic JMS properties dynamicProperties	object Optional	Object/JSON object of application properties. Current messaging code stringifies outgoing property values.
JMS headers headers	object Optional	Header name/value object. The messaging runtime stringifies outgoing values; this is not a guarantee of arbitrary native binary headers or JMS header setter support.

Output

Illustrative runtime output

```
{"messageId":"<provider-id>","destination":"orders.out","published":true,"senderReused":true}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
JMS message ID messageId	string Optional	Identifier returned by the provider, or a locally generated identifier for buffered publishing. It is not proof that a downstream application processed the message.
Resolved destination destination	string Optional	Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default".

Field / key	Type / required	Details
JMS timestamp timestamp	dateTime Optional	Timestamp reported by the operation. Kafka native record timestamps are epoch milliseconds; do not assume every connector returns the editor-labelled dateTime representation.

Advanced

Delivery, transactions and performance

Persistent sender workers are reused for the same resolved connection and execution scope. The cache is bounded; changing credentials/configuration frequently or exceeding the cache can create new workers. Native text messages and string application properties are the supported send path. Do not treat all editor choices as implemented native features. Retry after an ambiguous timeout can duplicate a message.

Implemented behavior and limitations

Native send creates a text message; object bodies are JSON-encoded. Message-type, disable-message-ID and disable-timestamp editor options are not fully implemented by the bridge. A warm persistent sender can be faster than the first send; senderReused is diagnostic, not a delivery guarantee.

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Troubleshooting procedure

Missing JAR/class/native bridge errors: check the installed driver directory, matching versions and packaged Java bridge. Authentication/destination failures: verify credentials, queue/topic permissions and the resolved environment destination. Timeout: inspect empty-receive results as well as faults. A successful send is not proof that another application consumed or processed the message.

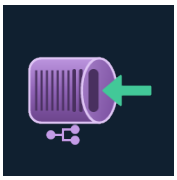
Acceptance test

Acceptance: browse the test queue independently and verify text body, correlation and properties; compare first-send and warm-send timing under the same delivery policy.

Fault	Description
JMSInvalidInputException	The JMS activity input is invalid.
JMSMessageCreateException	The configured JMS message could not be created.
JMSSessionCreateException	A JMS session could not be created.
JMSSendException	The JMS message could not be sent.
ActivityTimedOutException	The JMS operation reached its configured timeout.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs



EMS Topic Publisher

Activities / EMS

Overview

Publish a topic message

Use inside a task to broadcast a mapped body to the configured topic.

Prerequisites

Install the licensed, version-compatible JMS client JARs under the runtime driver root/jms or set Driver JAR directory. Use the bundled Java bridge/runtime. A direct EMS connection requires URL, username and password; JNDI additionally requires its context factory, provider URL and connection-factory settings. Keep Java class names exactly as provided by the driver. Test from the execution machine, not only the developer laptop.

Sends, receives, browses, or acknowledges EMS messages.

Destination, delivery, selector, acknowledgement, and message properties follow the selected EMS operation. Client acknowledgement handles can be passed to Confirm Message.

Activity type: ems; operation: publish.

Configuration

Configuration procedure

Select the EMS connection, topic/destination property and message mapping. Operation publish selects topic semantics. Its current base editor contract omits some sender controls; use the supplemental runtime-key guide below.

Runtime setting	Meaning
destination	Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default".
message	Map the record/message body, not the entire preceding activity envelope unless that envelope is intentionally the payload.
deliveryMode	Persistent or Non-Persistent producer setting. Persistence still depends on broker configuration and is not an exactly-once business guarantee.
priority	JMS priority from 0 to 9; default 4. Ordering and fairness are provider-dependent.
expiration	Message time-to-live in milliseconds; zero means no producer-specified expiry.
correlationId	Correlation identifier attached to the outgoing message. For replies use the request correlation ID, or its message ID when the request protocol requires that fallback.
replyTo	Destination attached as JMSReplyTo. Request Reply creates a temporary queue when this is blank.
dynamicProperties	Object/JSON object of application properties. Current messaging code stringifies outgoing property values.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
EMS connection resourceId	resource Required	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: ems

Input

Mapping example

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
message = ${input.event}
destination = ${properties.messaging.eventsTopic}
deliveryMode = Persistent
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
JMS message body message	object|string|binary Required	Map the record/message body, not the entire preceding activity envelope unless that envelope is intentionally the payload.
Dynamic destination override destination	string Optional	Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default".
Dynamic JMS properties dynamicProperties	object Optional	Object/JSON object of application properties. Current messaging code stringifies outgoing property values.
JMS headers headers	object Optional	Header name/value object. The messaging runtime stringifies outgoing values; this is not a guarantee of arbitrary native binary headers or JMS header setter support.

Output

Illustrative runtime output

```
{"messageId":"<provider-id>","destination":"events","published":true}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
JMS message ID messageId	string Optional	Identifier returned by the provider, or a locally generated identifier for buffered publishing. It is not proof that a downstream application processed the message.
Resolved destination destination	string Optional	Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default".
JMS timestamp timestamp	dateTime Optional	Timestamp reported by the operation. Kafka native record timestamps are epoch milliseconds; do not assume every connector returns the editor-labelled dateTime representation.

Advanced

Delivery, transactions and performance

Persistent sender workers are reused for the same resolved connection and execution scope. The cache is bounded; changing credentials/configuration frequently or exceeding the cache can create new workers. Native text messages and string application properties are the supported send path. Do not treat all editor choices as implemented native features. Retry after an ambiguous timeout can duplicate a message.

Implemented behavior and limitations

A successful topic publish does not establish delivery to every intended subscriber. Offline retention depends on a genuinely durable provider subscription; the native engine subscriber options do not implement that guarantee. Native text-message restrictions match Queue Sender.

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Troubleshooting procedure

Missing JAR/class/native bridge errors: check the installed driver directory, matching versions and packaged Java bridge. Authentication/destination failures: verify credentials, queue/topic permissions and the resolved environment destination. Timeout: inspect empty-receive results as well as faults. A successful send is not proof that another application consumed or processed the message.

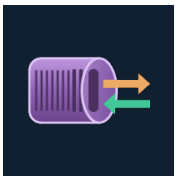
Acceptance test

Acceptance: attach two independent online subscribers, publish a unique event and check each body. Test offline behavior separately.

Fault	Description
JMSInvalidInputException	The JMS activity input is invalid.
JMSMessageCreateException	The configured JMS message could not be created.
JMSSessionCreateException	A JMS session could not be created.
JMSSendException	The JMS message could not be sent.
ActivityTimedOutException	The JMS operation reached its configured timeout.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs



EMS Request Reply

Activities / EMS

Overview

Send a request and await a reply

Use when the task needs a synchronous response from another JMS application.

Prerequisites

Install the licensed, version-compatible JMS client JARs under the runtime driver root/jms or set Driver JAR directory. Use the bundled Java bridge/runtime. A direct EMS connection requires URL, username and password; JNDI additionally requires its context factory, provider URL and connection-factory settings. Keep Java class names exactly as provided by the driver. Test from the execution machine, not only the developer laptop.

Sends, receives, browses, or acknowledges EMS messages.

Destination, delivery, selector, acknowledgement, and message properties follow the selected EMS operation. Client acknowledgement handles can be passed to Confirm Message.

Activity type: ems; operation: request_reply.

Configuration

Configuration procedure

Set the request destination and mapped message. Leave replyTo blank for a temporary reply queue. A fixed reply queue requires external correlation isolation; the current bridge does not install a correlation selector on that queue.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
EMS connection resourceId	resource Required	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: ems
Messaging style messagingStyle	select Configuration setting	Queue/topic design hint. The current engine primarily chooses destination kind from the operation; do not use this to turn a queue sender into a topic publisher. Choices: Queue, Topic, Generic
Destination destination	text Configuration setting	Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default".
Message type messageType	select Configuration setting	Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented. Choices: Text, Bytes, Map, Object, Object Ref, Simple, Stream, XML Text
Delivery mode deliveryMode	select Configuration setting	Persistent or Non-Persistent producer setting. Persistence still depends on broker configuration and is not an exactly-once business guarantee. Choices: Persistent, Non-Persistent

Field / key	Type / required	Details
Priority (0-9) priority	number Configuration setting	JMS priority from 0 to 9; default 4. Ordering and fairness are provider-dependent.
Expiration (ms) expiration	number Configuration setting	Message time-to-live in milliseconds; zero means no producer-specified expiry.
JMS correlation ID correlationId	text Configuration setting	Correlation identifier attached to the outgoing message. For replies use the request correlation ID, or its message ID when the request protocol requires that fallback.
Reply-to destination replyTo	text Configuration setting	Destination attached as JMSReplyTo. Request Reply creates a temporary queue when this is blank.
Disable JMS message ID disableMessageId	boolean Configuration setting	Editor option not forwarded to the current native sender; do not assume identifiers are disabled.
Disable JMS timestamp disableTimestamp	boolean Configuration setting	Editor option not forwarded to the current native sender; do not assume timestamps are disabled.
JMS application / dynamic properties (JSON) dynamicProperties	textarea Configuration setting	Object/JSON object of application properties. Current messaging code stringifies outgoing property values.
Request timeout (ms) requestTimeout	number Configuration setting	Editor request timeout; current engine bridge reads receiveTimeout for its actual wait, not this field. Verify the effective wait in your deployed archive.
Use temporary reply destination temporaryReplyDestination	boolean Configuration setting	Editor flag; the current native request bridge chooses a temporary queue based on replyTo being blank, not this flag.

Input

Mapping example

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
message = ${input.request}
destination = ${properties.messaging.requestQueue}
replyTo = <leave blank for a temporary queue>
correlationId = ${input.request.id}
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
JMS message body message	object|string|binary Required	Map the record/message body, not the entire preceding activity envelope unless that envelope is intentionally the payload.
Dynamic destination override destination	string Optional	Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default".
Dynamic JMS properties dynamicProperties	object Optional	Object/JSON object of application properties. Current messaging code stringifies outgoing property values.
JMS headers headers	object Optional	Header name/value object. The messaging runtime stringifies outgoing values; this is not a guarantee of arbitrary native binary headers or JMS header setter support.

Field / key	Type / required	Details
Messaging style messagingStyle	string Optional	Queue/topic design hint. The current engine primarily chooses destination kind from the operation; do not use this to turn a queue sender into a topic publisher.
Message type messageType	string Optional	Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented.
Delivery mode deliveryMode	string Optional	Persistent or Non-Persistent producer setting. Persistence still depends on broker configuration and is not an exactly-once business guarantee.
Priority (0-9) priority	number Optional	JMS priority from 0 to 9; default 4. Ordering and fairness are provider-dependent.
Expiration (ms) expiration	number Optional	Message time-to-live in milliseconds; zero means no producer-specified expiry.
JMS correlation ID correlationId	string Optional	Correlation identifier attached to the outgoing message. For replies use the request correlation ID, or its message ID when the request protocol requires that fallback.
Reply-to destination replyTo	string Optional	Destination attached as JMSReplyTo. Request Reply creates a temporary queue when this is blank.
Disable JMS message ID disableMessageId	boolean Optional	Editor option not forwarded to the current native sender; do not assume identifiers are disabled.
Disable JMS timestamp disableTimestamp	boolean Optional	Editor option not forwarded to the current native sender; do not assume timestamps are disabled.
Request timeout (ms) requestTimeout	number Optional	Editor request timeout; current engine bridge reads receiveTimeout for its actual wait, not this field. Verify the effective wait in your deployed archive.
Use temporary reply destination temporaryReplyDestination	boolean Optional	Editor flag; the current native request bridge chooses a temporary queue based on replyTo being blank, not this flag.

Output

Illustrative runtime output

```
{"received":true,"body":{"accepted":true},"headers":{},"properties":{},"count":1}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Reply message body body	object|string|binary Optional	Received message body. The engine attempts JSON decoding for text that contains valid JSON; other text remains a string.
Reply JMS headers headers	object Optional	Header name/value object. The messaging runtime stringifies outgoing values; this is not a guarantee of arbitrary native binary headers or JMS header setter support.
Reply properties properties	object Optional	Application-defined message properties returned by the broker. Standard JMS headers are returned separately.

Advanced

Delivery, transactions and performance

Persistent sender workers are reused for the same resolved connection and execution scope. The cache is bounded; changing credentials/configuration frequently or exceeding the cache can create new workers. Native text messages and string application properties are the supported send path. Do not treat all editor choices as implemented native features. Retry after an ambiguous timeout can duplicate a message.

Implemented behavior and limitations

The engine wrapper uses receiveTimeout (default 30000 ms), not the displayed requestTimeout, for the actual reply wait. The temporaryReplyDestination flag is not consumed; blank replyTo controls temporary queue creation. Concurrent callers sharing a fixed reply queue can receive the wrong reply. A timeout can return received=false/count=0 rather than throw.

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Troubleshooting procedure

Missing JAR/class/native bridge errors: check the installed driver directory, matching versions and packaged Java bridge. Authentication/destination failures: verify credentials, queue/topic permissions and the resolved environment destination. Timeout: inspect empty-receive results as well as faults. A successful send is not proof that another application consumed or processed the message.

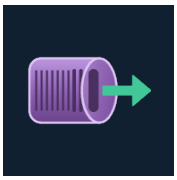
Acceptance test

Acceptance: test a successful reply, no reply, and concurrent requests. Use temporary reply queues until shared-destination correlation is qualified.

Fault	Description
JMSInvalidInputException	The JMS activity input is invalid.
JMSMessageCreateException	The configured JMS message could not be created.
JMSSessionCreateException	A JMS session could not be created.
JMSSendException	The JMS message could not be sent.
ActivityTimedOutException	The JMS operation reached its configured timeout.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs



EMS Reply

Activities / EMS

Overview

Send a response to a JMS request

Use after receiving a request whose reply destination and correlation information are available.

Prerequisites

Install the licensed, version-compatible JMS client JARs under the runtime driver root/jms or set Driver JAR directory. Use the bundled Java bridge/runtime. A direct EMS connection requires URL, username and password; JNDI additionally requires its context factory, provider URL and connection-factory settings. Keep Java class names exactly as provided by the driver. Test from the execution machine, not only the developer laptop.

Sends, receives, browses, or acknowledges EMS messages.

Destination, delivery, selector, acknowledgement, and message properties follow the selected EMS operation. Client acknowledgement handles can be passed to Confirm Message.

Activity type: ems; operation: reply.

Configuration

Configuration procedure

Map the response body and explicit reply destination. For the engine path this is an outgoing send; it does not automatically derive destination from a listener context. Map the request correlation ID, or request message ID if that is the agreed protocol.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
EMS connection resourceId	resource Required	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: ems
Messaging style messagingStyle	select Configuration setting	Queue/topic design hint. The current engine primarily chooses destination kind from the operation; do not use this to turn a queue sender into a topic publisher. Choices: Queue, Topic, Generic
Reply destination destination	text Configuration setting	Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default".
Message type messageType	select Configuration setting	Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented. Choices: Text, Bytes, Map, Object, Object Ref, Simple, Stream, XML Text
Request correlation ID correlationId	text Configuration setting	Correlation identifier attached to the outgoing message. For replies use the request correlation ID, or its message ID when the request protocol requires that fallback.

Field / key	Type / required	Details
Delivery mode deliveryMode	select Configuration setting	Persistent or Non-Persistent producer setting. Persistence still depends on broker configuration and is not an exactly-once business guarantee. Choices: Persistent, Non-Persistent
Priority (0-9) priority	number Configuration setting	JMS priority from 0 to 9; default 4. Ordering and fairness are provider-dependent.
Expiration (ms) expiration	number Configuration setting	Message time-to-live in milliseconds; zero means no producer-specified expiry.
JMS correlation ID correlationId	text Configuration setting	Correlation identifier attached to the outgoing message. For replies use the request correlation ID, or its message ID when the request protocol requires that fallback.
Reply-to destination replyTo	text Configuration setting	Destination attached as JMSReplyTo. Request Reply creates a temporary queue when this is blank.
Disable JMS message ID disableMessageId	boolean Configuration setting	Editor option not forwarded to the current native sender; do not assume identifiers are disabled.
Disable JMS timestamp disableTimestamp	boolean Configuration setting	Editor option not forwarded to the current native sender; do not assume timestamps are disabled.
JMS application / dynamic properties (JSON) dynamicProperties	textarea Configuration setting	Object/JSON object of application properties. Current messaging code stringifies outgoing property values.

Input

Mapping example

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
message = ${input.response}
destination = ${properties.messaging.replyQueue}
correlationId = ${EMS-Receive.headers.JMSCorrelationID}
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
JMS message body message	object|string|binary Required	Map the record/message body, not the entire preceding activity envelope unless that envelope is intentionally the payload.
Dynamic destination override destination	string Optional	Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default".
Dynamic JMS properties dynamicProperties	object Optional	Object/JSON object of application properties. Current messaging code stringifies outgoing property values.
JMS headers headers	object Optional	Header name/value object. The messaging runtime stringifies outgoing values; this is not a guarantee of arbitrary native binary headers or JMS header setter support.
Messaging style messagingStyle	string Optional	Queue/topic design hint. The current engine primarily chooses destination kind from the operation; do not use this to turn a queue sender into a topic publisher.

Field / key	Type / required	Details
Message type messageType	string Optional	Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented.
Request correlation ID correlationId	string Optional	Correlation identifier attached to the outgoing message. For replies use the request correlation ID, or its message ID when the request protocol requires that fallback.
Delivery mode deliveryMode	string Optional	Persistent or Non-Persistent producer setting. Persistence still depends on broker configuration and is not an exactly-once business guarantee.
Priority (0-9) priority	number Optional	JMS priority from 0 to 9; default 4. Ordering and fairness are provider-dependent.
Expiration (ms) expiration	number Optional	Message time-to-live in milliseconds; zero means no producer-specified expiry.
JMS correlation ID correlationId	string Optional	Correlation identifier attached to the outgoing message. For replies use the request correlation ID, or its message ID when the request protocol requires that fallback.
Reply-to destination replyTo	string Optional	Destination attached as JMSReplyTo. Request Reply creates a temporary queue when this is blank.
Disable JMS message ID disableMessageId	boolean Optional	Editor option not forwarded to the current native sender; do not assume identifiers are disabled.
Disable JMS timestamp disableTimestamp	boolean Optional	Editor option not forwarded to the current native sender; do not assume timestamps are disabled.

Output

Illustrative runtime output

```
{"messageId":"<provider-id>","destination":"replies","published":true}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
JMS message ID messageId	string Optional	Identifier returned by the provider, or a locally generated identifier for buffered publishing. It is not proof that a downstream application processed the message.
Resolved destination destination	string Optional	Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default".
JMS timestamp timestamp	dateTime Optional	Timestamp reported by the operation. Kafka native record timestamps are epoch milliseconds; do not assume every connector returns the editor-labelled dateTime representation.

Advanced

Delivery, transactions and performance

Persistent sender workers are reused for the same resolved connection and execution scope. The cache is bounded; changing credentials/configuration frequently or exceeding the cache can create new workers. Native text messages and

string application properties are the supported send path. Do not treat all editor choices as implemented native features.Retry after an ambiguous timeout can duplicate a message.

Implemented behavior and limitations

The request JMSReplyTo value must be usable as a destination name before mapping it. Do not assume the generic headers object sets native JMS correlation/reply headers; use the dedicated correlationId and destination fields. A reply does not prove the requester received it.

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Troubleshooting procedure

Missing JAR/class/native bridge errors: check the installed driver directory, matching versions and packaged Java bridge. Authentication/destination failures: verify credentials, queue/topic permissions and the resolved environment destination. Timeout: inspect empty-receive results as well as faults. A successful send is not proof that another application consumed or processed the message.

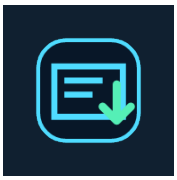
Acceptance test

Acceptance: send a unique correlation ID through a requester/responder pair and verify it on the response.

Fault	Description
JMSInvalidInputException	The JMS activity input is invalid.
JMSMessageCreateException	The configured JMS message could not be created.
JMSSessionCreateException	A JMS session could not be created.
JMSSendException	The JMS message could not be sent.
ActivityTimedOutException	The JMS operation reached its configured timeout.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs



Get JMS Queue Message

Activities / JMS

Overview

Runs Get JMS Queue Message on the task execution path.

Activity type: jms; operation: get_queue_message.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
JMS connection resourceId	resource Required	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: jms
Queue name queue	text Configuration setting	Queue name, not the broker URL. Browse an environment property to avoid hard-coding environment-specific destinations.
Message type messageType	select Configuration setting	Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented. Choices: Text, Bytes, Map, Object, Object Ref, Simple, Stream, XML Text
JMS message selector messageSelector	text Configuration setting	Provider-side JMS selector expression over supported headers/properties. It does not query inside the JSON message body.
Acknowledge mode acknowledgeMode	select Configuration setting	Choose automatic completion or explicit acknowledgement. Guarantees differ by connector and execution path; read Advanced before relying on rollback/redelivery. Choices: Auto, Client, Dups OK, Explicit Client, Explicit Client Dups OK, Transactional
Receive timeout (ms) receiveTimeout	number Configuration setting	Receive wait in milliseconds, default 30000 on the engine bridge path. A timeout can return an empty result; it need not throw a fault.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Queue name queue	string Optional	Queue name, not the broker URL. Browse an environment property to avoid hard-coding environment-specific destinations.
Message type messageType	string Optional	Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented.

Field / key	Type / required	Details
JMS message selector messageSelector	string Optional	Provider-side JMS selector expression over supported headers/properties. It does not query inside the JSON message body.
Acknowledge mode acknowledgeMode	string Optional	Choose automatic completion or explicit acknowledgement. Guarantees differ by connector and execution path; read Advanced before relying on rollback/redelivery.
Receive timeout (ms) receiveTimeout	number Optional	Receive wait in milliseconds, default 30000 on the engine bridge path. A timeout can return an empty result; it need not throw a fault.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
JMS message body body	object|string|binary Optional	Received message body. The engine attempts JSON decoding for text that contains valid JSON; other text remains a string.
JMS standard headers headers	object Optional	Header name/value object. The messaging runtime stringifies outgoing values; this is not a guarantee of arbitrary native binary headers or JMS header setter support.
JMS application properties properties	object Optional	Application-defined message properties returned by the broker. Standard JMS headers are returned separately.
Client acknowledgement handle ackId	string Optional	Opaque, runtime-local confirmation token. Pass the original value to Confirm Message; it is not the business message ID and cannot be reconstructed after restart.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""

Field / key	Type / required	Details
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Fault	Description
JMSInvalidInputException	The JMS activity input is invalid.
JMSMessageCreateException	The configured JMS message could not be created.
JMSSessionCreateException	A JMS session could not be created.
JMSReceiveException	The JMS message could not be received.
ActivityTimedOutException	The JMS operation reached its configured timeout.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx

JMS Receive Message

Activities / JMS

Overview

Runs JMS Receive Message on the task execution path.

Activity type: jms; operation: receive_message.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
JMS connection resourceId	resource Required	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: jms
Messaging style messagingStyle	select Configuration setting	Queue/topic design hint. The current engine primarily chooses destination kind from the operation; do not use this to turn a queue sender into a topic publisher. Choices: Queue, Topic, Generic
Destination destination	text Configuration setting	Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default".
Message type messageType	select Configuration setting	Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented. Choices: Text, Bytes, Map, Object, Object Ref, Simple, Stream, XML Text
JMS message selector messageSelector	text Configuration setting	Provider-side JMS selector expression over supported headers/properties. It does not query inside the JSON message body.
Acknowledge mode acknowledgeMode	select Configuration setting	Choose automatic completion or explicit acknowledgement. Guarantees differ by connector and execution path; read Advanced before relying on rollback/redelivery. Choices: Auto, Client, Dups OK, Explicit Client, Explicit Client Dups OK, Transactional
Maximum sessions maxSessions	number Configuration setting	Passed as a session-count option by the wrapper; current engine bridge does not create a configurable receiver-session pool from it.
Flow limit flowLimit	number Configuration setting	Passed by the wrapper but not enforced as an engine bridge in-flight limit. Do not rely on it as the only backpressure mechanism.
Receive timeout (ms) receiveTimeout	number Configuration setting	Receive wait in milliseconds, default 30000 on the engine bridge path. A timeout can return an empty result; it need not throw a fault.
Durable subscription durable	boolean Configuration setting	Editor option not wired to a durable subscriber constructor in the native engine bridge. Offline topic retention cannot be assumed.
Durable subscription name subscriptionName	text Configuration setting	Intended durable subscription identity; current native engine consumer does not use it to create a durable subscription.

Field / key	Type / required	Details
Shared subscription sharedSubscription	boolean Configuration setting	Editor option not wired to a shared-subscription constructor in the current native engine bridge.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Messaging style messagingStyle	string Optional	Queue/topic design hint. The current engine primarily chooses destination kind from the operation; do not use this to turn a queue sender into a topic publisher.
Destination destination	string Optional	Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default".
Message type messageType	string Optional	Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented.
JMS message selector messageSelector	string Optional	Provider-side JMS selector expression over supported headers/properties. It does not query inside the JSON message body.
Acknowledge mode acknowledgeMode	string Optional	Choose automatic completion or explicit acknowledgement. Guarantees differ by connector and execution path; read Advanced before relying on rollback/redelivery.
Maximum sessions maxSessions	number Optional	Passed as a session-count option by the wrapper; current engine bridge does not create a configurable receiver-session pool from it.
Flow limit flowLimit	number Optional	Passed by the wrapper but not enforced as an engine bridge in-flight limit. Do not rely on it as the only backpressure mechanism.
Receive timeout (ms) receiveTimeout	number Optional	Receive wait in milliseconds, default 30000 on the engine bridge path. A timeout can return an empty result; it need not throw a fault.
Durable subscription durable	boolean Optional	Editor option not wired to a durable subscriber constructor in the native engine bridge. Offline topic retention cannot be assumed.
Durable subscription name subscriptionName	string Optional	Intended durable subscription identity; current native engine consumer does not use it to create a durable subscription.
Shared subscription sharedSubscription	boolean Optional	Editor option not wired to a shared-subscription constructor in the current native engine bridge.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
JMS message body body	object|string|binary Optional	Received message body. The engine attempts JSON decoding for text that contains valid JSON; other text remains a string.

Field / key	Type / required	Details
JMS standard headers headers	object Optional	Header name/value object. The messaging runtime stringifies outgoing values; this is not a guarantee of arbitrary native binary headers or JMS header setter support.
JMS application properties properties	object Optional	Application-defined message properties returned by the broker. Standard JMS headers are returned separately.
Client acknowledgement handle ackId	string Optional	Opaque, runtime-local confirmation token. Pass the original value to Confirm Message; it is not the business message ID and cannot be reconstructed after restart.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
JMSInvalidInputException	The JMS activity input is invalid.
JMSMessageCreateException	The configured JMS message could not be created.
JMSSessionCreateException	A JMS session could not be created.
JMSReceiveException	The JMS message could not be received.
ActivityTimedOutException	The JMS operation reached its configured timeout.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx

JMS Request Reply

Activities / JMS

Overview

Runs JMS Request Reply on the task execution path.

Activity type: jms; operation: request_reply.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
JMS connection resourceId	resource Required	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: jms
Messaging style messagingStyle	select Configuration setting	Queue/topic design hint. The current engine primarily chooses destination kind from the operation; do not use this to turn a queue sender into a topic publisher. Choices: Queue, Topic, Generic
Destination destination	text Configuration setting	Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default".
Message type messageType	select Configuration setting	Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented. Choices: Text, Bytes, Map, Object, Object Ref, Simple, Stream, XML Text
Delivery mode deliveryMode	select Configuration setting	Persistent or Non-Persistent producer setting. Persistence still depends on broker configuration and is not an exactly-once business guarantee. Choices: Persistent, Non-Persistent
Priority (0-9) priority	number Configuration setting	JMS priority from 0 to 9; default 4. Ordering and fairness are provider-dependent.
Expiration (ms) expiration	number Configuration setting	Message time-to-live in milliseconds; zero means no producer-specified expiry.
JMS correlation ID correlationId	text Configuration setting	Correlation identifier attached to the outgoing message. For replies use the request correlation ID, or its message ID when the request protocol requires that fallback.
Reply-to destination replyTo	text Configuration setting	Destination attached as JMSReplyTo. Request Reply creates a temporary queue when this is blank.
Disable JMS message ID disableMessageId	boolean Configuration setting	Editor option not forwarded to the current native sender; do not assume identifiers are disabled.
Disable JMS timestamp disableTimestamp	boolean Configuration setting	Editor option not forwarded to the current native sender; do not assume timestamps are disabled.

Field / key	Type / required	Details
JMS application / dynamic properties (JSON) dynamicProperties	textarea Configuration setting	Object/JSON object of application properties. Current messaging code stringifies outgoing property values.
Request timeout (ms) requestTimeout	number Configuration setting	Editor request timeout; current engine bridge reads receiveTimeout for its actual wait, not this field. Verify the effective wait in your deployed archive.
Use temporary reply destination temporaryReplyDestination	boolean Configuration setting	Editor flag; the current native request bridge chooses a temporary queue based on replyTo being blank, not this flag.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
JMS message body message	object|string|binary Required	Map the record/message body, not the entire preceding activity envelope unless that envelope is intentionally the payload.
Dynamic destination override destination	string Optional	Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default".
Dynamic JMS properties dynamicProperties	object Optional	Object/JSON object of application properties. Current messaging code stringifies outgoing property values.
JMS headers headers	object Optional	Header name/value object. The messaging runtime stringifies outgoing values; this is not a guarantee of arbitrary native binary headers or JMS header setter support.
Messaging style messagingStyle	string Optional	Queue/topic design hint. The current engine primarily chooses destination kind from the operation; do not use this to turn a queue sender into a topic publisher.
Message type messageType	string Optional	Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented.
Delivery mode deliveryMode	string Optional	Persistent or Non-Persistent producer setting. Persistence still depends on broker configuration and is not an exactly-once business guarantee.
Priority (0-9) priority	number Optional	JMS priority from 0 to 9; default 4. Ordering and fairness are provider-dependent.
Expiration (ms) expiration	number Optional	Message time-to-live in milliseconds; zero means no producer-specified expiry.
JMS correlation ID correlationId	string Optional	Correlation identifier attached to the outgoing message. For replies use the request correlation ID, or its message ID when the request protocol requires that fallback.
Reply-to destination replyTo	string Optional	Destination attached as JMSReplyTo. Request Reply creates a temporary queue when this is blank.
Disable JMS message ID disableMessageId	boolean Optional	Editor option not forwarded to the current native sender; do not assume identifiers are disabled.
Disable JMS timestamp disableTimestamp	boolean Optional	Editor option not forwarded to the current native sender; do not assume timestamps are disabled.

Field / key	Type / required	Details
Request timeout (ms) requestTimeout	number Optional	Editor request timeout; current engine bridge reads receiveTimeout for its actual wait, not this field. Verify the effective wait in your deployed archive.
Use temporary reply destination temporaryReplyDestination	boolean Optional	Editor flag; the current native request bridge chooses a temporary queue based on replyTo being blank, not this flag.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Reply message body body	object|string|binary Optional	Received message body. The engine attempts JSON decoding for text that contains valid JSON; other text remains a string.
Reply JMS headers headers	object Optional	Header name/value object. The messaging runtime stringifies outgoing values; this is not a guarantee of arbitrary native binary headers or JMS header setter support.
Reply properties properties	object Optional	Application-defined message properties returned by the broker. Standard JMS headers are returned separately.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false

Field / key	Type / required	Details
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Fault	Description
JMSInvalidInputException	The JMS activity input is invalid.
JMSMessageCreateException	The configured JMS message could not be created.
JMSSessionCreateException	A JMS session could not be created.
JMSSendException	The JMS message could not be sent.
ActivityTimedOutException	The JMS operation reached its configured timeout.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



JMS Send Message

Activities / JMS

Overview

Runs JMS Send Message on the task execution path.

Activity type: jms; operation: send_message.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
JMS connection resourceId	resource Required	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: jms
Messaging style messagingStyle	select Configuration setting	Queue/topic design hint. The current engine primarily chooses destination kind from the operation; do not use this to turn a queue sender into a topic publisher. Choices: Queue, Topic, Generic
Destination destination	text Configuration setting	Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default".
Message type messageType	select Configuration setting	Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented. Choices: Text, Bytes, Map, Object, Object Ref, Simple, Stream, XML Text
Delivery mode deliveryMode	select Configuration setting	Persistent or Non-Persistent producer setting. Persistence still depends on broker configuration and is not an exactly-once business guarantee. Choices: Persistent, Non-Persistent
Priority (0-9) priority	number Configuration setting	JMS priority from 0 to 9; default 4. Ordering and fairness are provider-dependent.
Expiration (ms) expiration	number Configuration setting	Message time-to-live in milliseconds; zero means no producer-specified expiry.
JMS correlation ID correlationId	text Configuration setting	Correlation identifier attached to the outgoing message. For replies use the request correlation ID, or its message ID when the request protocol requires that fallback.
Reply-to destination replyTo	text Configuration setting	Destination attached as JMSReplyTo. Request Reply creates a temporary queue when this is blank.
Disable JMS message ID disableMessageId	boolean Configuration setting	Editor option not forwarded to the current native sender; do not assume identifiers are disabled.
Disable JMS timestamp disableTimestamp	boolean Configuration setting	Editor option not forwarded to the current native sender; do not assume timestamps are disabled.

Field / key	Type / required	Details
JMS application / dynamic properties (JSON) dynamicProperties	textarea Configuration setting	Object/JSON object of application properties. Current messaging code stringifies outgoing property values.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
JMS message body message	object|string|binary Required	Map the record/message body, not the entire preceding activity envelope unless that envelope is intentionally the payload.
Dynamic destination override destination	string Optional	Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default".
Dynamic JMS properties dynamicProperties	object Optional	Object/JSON object of application properties. Current messaging code stringifies outgoing property values.
JMS headers headers	object Optional	Header name/value object. The messaging runtime stringifies outgoing values; this is not a guarantee of arbitrary native binary headers or JMS header setter support.
Messaging style messagingStyle	string Optional	Queue/topic design hint. The current engine primarily chooses destination kind from the operation; do not use this to turn a queue sender into a topic publisher.
Message type messageType	string Optional	Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented.
Delivery mode deliveryMode	string Optional	Persistent or Non-Persistent producer setting. Persistence still depends on broker configuration and is not an exactly-once business guarantee.
Priority (0-9) priority	number Optional	JMS priority from 0 to 9; default 4. Ordering and fairness are provider-dependent.
Expiration (ms) expiration	number Optional	Message time-to-live in milliseconds; zero means no producer-specified expiry.
JMS correlation ID correlationId	string Optional	Correlation identifier attached to the outgoing message. For replies use the request correlation ID, or its message ID when the request protocol requires that fallback.
Reply-to destination replyTo	string Optional	Destination attached as JMSReplyTo. Request Reply creates a temporary queue when this is blank.
Disable JMS message ID disableMessageId	boolean Optional	Editor option not forwarded to the current native sender; do not assume identifiers are disabled.
Disable JMS timestamp disableTimestamp	boolean Optional	Editor option not forwarded to the current native sender; do not assume timestamps are disabled.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
JMS message ID messageId	string Optional	Identifier returned by the provider, or a locally generated identifier for buffered publishing. It is not proof that a downstream application processed the message.
Resolved destination destination	string Optional	Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default".
JMS timestamp timestamp	dateTime Optional	Timestamp reported by the operation. Kafka native record timestamps are epoch milliseconds; do not assume every connector returns the editor-labelled dateTime representation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

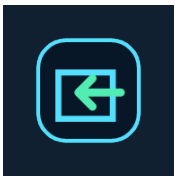
Errors

Fault	Description
JMSInvalidInputException	The JMS activity input is invalid.

Fault	Description
JMSMessageCreateException	The configured JMS message could not be created.
JMSSessionCreateException	A JMS session could not be created.
JMSSendException	The JMS message could not be sent.
ActivityTimedOutException	The JMS operation reached its configured timeout.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



Reply to JMS Message

Activities / JMS

Overview

Runs Reply to JMS Message on the task execution path.

Activity type: jms; operation: reply_message.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
JMS connection resourceId	resource Required	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: jms
Messaging style messagingStyle	select Configuration setting	Queue/topic design hint. The current engine primarily chooses destination kind from the operation; do not use this to turn a queue sender into a topic publisher. Choices: Queue, Topic, Generic
Reply destination destination	text Configuration setting	Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default".
Message type messageType	select Configuration setting	Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented. Choices: Text, Bytes, Map, Object, Object Ref, Simple, Stream, XML Text
Request correlation ID correlationId	text Configuration setting	Correlation identifier attached to the outgoing message. For replies use the request correlation ID, or its message ID when the request protocol requires that fallback.
Delivery mode deliveryMode	select Configuration setting	Persistent or Non-Persistent producer setting. Persistence still depends on broker configuration and is not an exactly-once business guarantee. Choices: Persistent, Non-Persistent
Priority (0-9) priority	number Configuration setting	JMS priority from 0 to 9; default 4. Ordering and fairness are provider-dependent.
Expiration (ms) expiration	number Configuration setting	Message time-to-live in milliseconds; zero means no producer-specified expiry.
JMS correlation ID correlationId	text Configuration setting	Correlation identifier attached to the outgoing message. For replies use the request correlation ID, or its message ID when the request protocol requires that fallback.
Reply-to destination replyTo	text Configuration setting	Destination attached as JMSReplyTo. Request Reply creates a temporary queue when this is blank.
Disable JMS message ID disableMessageId	boolean Configuration setting	Editor option not forwarded to the current native sender; do not assume identifiers are disabled.

Field / key	Type / required	Details
Disable JMS timestamp disableTimestamp	boolean Configuration setting	Editor option not forwarded to the current native sender; do not assume timestamps are disabled.
JMS application / dynamic properties (JSON) dynamicProperties	textarea Configuration setting	Object/JSON object of application properties. Current messaging code stringifies outgoing property values.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
JMS message body message	object|string|binary Required	Map the record/message body, not the entire preceding activity envelope unless that envelope is intentionally the payload.
Dynamic destination override destination	string Optional	Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default".
Dynamic JMS properties dynamicProperties	object Optional	Object/JSON object of application properties. Current messaging code stringifies outgoing property values.
JMS headers headers	object Optional	Header name/value object. The messaging runtime stringifies outgoing values; this is not a guarantee of arbitrary native binary headers or JMS header setter support.
Messaging style messagingStyle	string Optional	Queue/topic design hint. The current engine primarily chooses destination kind from the operation; do not use this to turn a queue sender into a topic publisher.
Message type messageType	string Optional	Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented.
Request correlation ID correlationId	string Optional	Correlation identifier attached to the outgoing message. For replies use the request correlation ID, or its message ID when the request protocol requires that fallback.
Delivery mode deliveryMode	string Optional	Persistent or Non-Persistent producer setting. Persistence still depends on broker configuration and is not an exactly-once business guarantee.
Priority (0-9) priority	number Optional	JMS priority from 0 to 9; default 4. Ordering and fairness are provider-dependent.
Expiration (ms) expiration	number Optional	Message time-to-live in milliseconds; zero means no producer-specified expiry.
JMS correlation ID correlationId	string Optional	Correlation identifier attached to the outgoing message. For replies use the request correlation ID, or its message ID when the request protocol requires that fallback.
Reply-to destination replyTo	string Optional	Destination attached as JMSReplyTo. Request Reply creates a temporary queue when this is blank.
Disable JMS message ID disableMessageId	boolean Optional	Editor option not forwarded to the current native sender; do not assume identifiers are disabled.
Disable JMS timestamp disableTimestamp	boolean Optional	Editor option not forwarded to the current native sender; do not assume timestamps are disabled.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
JMS message ID messageId	string Optional	Identifier returned by the provider, or a locally generated identifier for buffered publishing. It is not proof that a downstream application processed the message.
Resolved destination destination	string Optional	Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default".
JMS timestamp timestamp	dateTime Optional	Timestamp reported by the operation. Kafka native record timestamps are epoch milliseconds; do not assume every connector returns the editor-labelled dateTime representation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Fault	Description
JMSInvalidInputException	The JMS activity input is invalid.
JMSMessageCreateException	The configured JMS message could not be created.
JMSSessionCreateException	A JMS session could not be created.
JMSSendException	The JMS message could not be sent.
ActivityTimedOutException	The JMS operation reached its configured timeout.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx

Wait for JMS Request

Activities / JMS

Overview

Runs Wait for JMS Request on the task execution path.

Activity type: jms; operation: wait_request.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
JMS connection resourceId	resource Required	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: jms
Messaging style messagingStyle	select Configuration setting	Queue/topic design hint. The current engine primarily chooses destination kind from the operation; do not use this to turn a queue sender into a topic publisher. Choices: Queue, Topic, Generic
Destination destination	text Configuration setting	Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default".
Message type messageType	select Configuration setting	Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented. Choices: Text, Bytes, Map, Object, Object Ref, Simple, Stream, XML Text
JMS message selector messageSelector	text Configuration setting	Provider-side JMS selector expression over supported headers/properties. It does not query inside the JSON message body.
Acknowledge mode acknowledgeMode	select Configuration setting	Choose automatic completion or explicit acknowledgement. Guarantees differ by connector and execution path; read Advanced before relying on rollback/redelivery. Choices: Auto, Client, Dups OK, Explicit Client, Explicit Client Dups OK, Transactional
Receive timeout (ms) receiveTimeout	number Configuration setting	Receive wait in milliseconds, default 30000 on the engine bridge path. A timeout can return an empty result; it need not throw a fault.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Messaging style messagingStyle	string Optional	Queue/topic design hint. The current engine primarily chooses destination kind from the operation; do not use this to turn a queue

Field / key	Type / required	Details
		sender into a topic publisher.
Destination destination	string Optional	Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default".
Message type messageType	string Optional	Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented.
JMS message selector messageSelector	string Optional	Provider-side JMS selector expression over supported headers/properties. It does not query inside the JSON message body.
Acknowledge mode acknowledgeMode	string Optional	Choose automatic completion or explicit acknowledgement. Guarantees differ by connector and execution path; read Advanced before relying on rollback/redelivery.
Receive timeout (ms) receiveTimeout	number Optional	Receive wait in milliseconds, default 30000 on the engine bridge path. A timeout can return an empty result; it need not throw a fault.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
JMS message body body	object|string|binary Optional	Received message body. The engine attempts JSON decoding for text that contains valid JSON; other text remains a string.
JMS standard headers headers	object Optional	Header name/value object. The messaging runtime stringifies outgoing values; this is not a guarantee of arbitrary native binary headers or JMS header setter support.
JMS application properties properties	object Optional	Application-defined message properties returned by the broker. Standard JMS headers are returned separately.
Client acknowledgement handle ackId	string Optional	Opaque, runtime-local confirmation token. Pass the original value to Confirm Message; it is not the business message ID and cannot be reconstructed after restart.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
JMSInvalidInputException	The JMS activity input is invalid.
JMSMessageCreateException	The configured JMS message could not be created.
JMSSessionCreateException	A JMS session could not be created.

Fault	Description
JMSReceiveException	The JMS message could not be received.
ActivityTimedOutException	The JMS operation reached its configured timeout.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



Kafka Receive Message

Activities / Kafka

Overview

Start work from Kafka records

Use as an event starter. Run/Debug registers continuous starter processing; one received batch becomes flow input. Pauses/breakpoints can delay downstream work.

Prerequisites

Install the matching confluent-kafka package on the execution host. Configure a Kafka shared connection with reachable bootstrap servers and the required TLS/SASL credentials. Verify advertised broker addresses are reachable from that host; a successful TCP connection to one bootstrap address is not sufficient. These pages describe the Studio/engine runtime; qualify exported direct Python separately.

Produces, receives, or commits Apache Kafka records.

Broker security comes from the shared Kafka connection. Topic, key, headers, value, partitions, offsets, and acknowledgement handles are available for hierarchical mapping.

Activity type: kafka; operation: receive.

Configuration

Configuration procedure

Set topic and groupId explicitly. Select deserializers matching the producer. For Manual mode disable enableAutoCommit and map acknowledgement handles only after processing succeeds. Use a separate test group to avoid consuming production offsets.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Kafka connection resourceId	resource Required	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: kafka
Consumer group ID groupId	text Configuration setting	Consumer group used for assignment and stored offsets. Separate applications requiring independent consumption need separate groups.
Topic names topic	text Configuration setting	Broker topic name. Resolve it from the active environment, for example ${properties.messaging.topic}. Consumer topic lists use semicolons.
Assign custom partitions assignCustomPartition	boolean Configuration setting	Enable explicit partition selection; otherwise the client chooses/subscribes using its normal partitioning and assignment behavior.
Partition IDs / ranges partitionIds	text Configuration setting	Comma/semicolon-separated partition IDs or inclusive ranges used for explicit assignment.
Consumer count consumerCount	number Configuration setting	Exposed by the editor; the engine receive path creates one consumer per call and does not implement a consumer pool from this field.

Field / key	Type / required	Details
Key deserializer keyDeserializer	select Configuration setting	Incoming key decoding; Byte Array exposes base64 text, JSON parses UTF-8 JSON, String decodes text. Choices: String, Byte Array, Avro Schema, JSON
Value deserializer valueDeserializer	select Configuration setting	Incoming record-value decoding. Invalid JSON raises a decode error; Avro is not implemented by this engine path. Choices: String, Byte Array, Avro Schema, JSON
Use Schema Registry useRegistry	boolean Configuration setting	Editor registry flag; setting this does not activate an Avro serializer in the current engine.
Schema Registry URL schemaRegistryUrl	text Configuration setting	Registry endpoint metadata. This engine does not implement registry-backed Avro conversion merely by setting the URL.
Key schema subject : version keySchemaSubject	text Configuration setting	Intended registry key subject/version; not resolved by the current engine serializer.
Value schema subject : version valueSchemaSubject	text Configuration setting	Intended registry value subject/version; not resolved by the current engine serializer.
Acknowledgement mode acknowledgeMode	select Configuration setting	Choose automatic completion or explicit acknowledgement. Guarantees differ by connector and execution path; read Advanced before relying on rollback/redelivery. Choices: Auto, Manual
Enable auto commit enableAutoCommit	boolean Configuration setting	Native periodic offset commit toggle. For manual acknowledgement keep this disabled; otherwise offsets can advance before downstream work finishes.
Auto offset reset autoOffsetReset	select Configuration setting	Policy used when no valid committed offset exists, commonly earliest or latest. It does not rewind a group that already has committed offsets.
Fetch minimum bytes fetchMinBytes	number Configuration setting	Minimum fetch bytes requested from the broker. Larger values can add latency at low traffic.
Maximum poll records maxPollRecords	number Configuration setting	Editor batch-size setting. The engine currently passes max.poll.records to its native consumer; this Java-style key may be rejected by the installed native client. Treat this as a known compatibility issue, not a guaranteed supported control.
Session timeout (ms) sessionTimeoutMs	number Configuration setting	Consumer failure-detection timeout in milliseconds; must satisfy broker limits.
Heartbeat interval (ms) heartbeatIntervalMs	number Configuration setting	Consumer heartbeat interval in milliseconds; coordinate with session timeout.
Additional consumer properties (JSON) additionalProperties	textarea Configuration setting	JSON object of native client configuration values merged into the producer/consumer configuration. Native keys differ from Java client keys; invalid keys are rejected by the installed client.

Input

Mapping example

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
topic = ${properties.messaging.ordersTopic}
groupId = ${properties.messaging.consumerGroup}
valueDeserializer = JSON
acknowledgeMode = Manual
enableAutoCommit = false
Downstream source = ${Kafka-Receive.messages}
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Consumer group ID groupId	string Optional	Consumer group used for assignment and stored offsets. Separate applications requiring independent consumption need separate groups.
Topic names topic	string Optional	Broker topic name. Resolve it from the active environment, for example ${properties.messaging.topic}. Consumer topic lists use semicolons.
Assign custom partitions assignCustomPartition	boolean Optional	Enable explicit partition selection; otherwise the client chooses/subscribes using its normal partitioning and assignment behavior.
Partition IDs / ranges partitionIds	string Optional	Comma/semicolon-separated partition IDs or inclusive ranges used for explicit assignment.
Consumer count consumerCount	number Optional	Exposed by the editor; the engine receive path creates one consumer per call and does not implement a consumer pool from this field.
Key deserializer keyDeserializer	string Optional	Incoming key decoding; Byte Array exposes base64 text, JSON parses UTF-8 JSON, String decodes text.
Value deserializer valueDeserializer	string Optional	Incoming record-value decoding. Invalid JSON raises a decode error; Avro is not implemented by this engine path.
Use Schema Registry useRegistry	boolean Optional	Editor registry flag; setting this does not activate an Avro serializer in the current engine.
Schema Registry URL schemaRegistryUrl	string Optional	Registry endpoint metadata. This engine does not implement registry-backed Avro conversion merely by setting the URL.
Key schema subject : version keySchemaSubject	string Optional	Intended registry key subject/version; not resolved by the current engine serializer.
Value schema subject : version valueSchemaSubject	string Optional	Intended registry value subject/version; not resolved by the current engine serializer.
Acknowledgement mode acknowledgeMode	string Optional	Choose automatic completion or explicit acknowledgement. Guarantees differ by connector and execution path; read Advanced before relying on rollback/redelivery.
Enable auto commit enableAutoCommit	boolean Optional	Native periodic offset commit toggle. For manual acknowledgement keep this disabled; otherwise offsets can advance before downstream work finishes.
Auto offset reset autoOffsetReset	string Optional	Policy used when no valid committed offset exists, commonly earliest or latest. It does not rewind a group that already has committed offsets.
Fetch minimum bytes fetchMinBytes	number Optional	Minimum fetch bytes requested from the broker. Larger values can add latency at low traffic.
Maximum poll records maxPollRecords	number Optional	Editor batch-size setting. The engine currently passes max.poll.records to its native consumer; this Java-style key may be rejected by the installed native client. Treat this as a known compatibility issue, not a guaranteed supported control.
Session timeout (ms) sessionTimeoutMs	number Optional	Consumer failure-detection timeout in milliseconds; must satisfy broker limits.

Field / key	Type / required	Details
Heartbeat interval (ms) heartbeatIntervalMs	number Optional	Consumer heartbeat interval in milliseconds; coordinate with session timeout.
Additional consumer properties (JSON) additionalProperties	string Optional	JSON object of native client configuration values merged into the producer/consumer configuration. Native keys differ from Java client keys; invalid keys are rejected by the installed client.

Output

Illustrative runtime output


```
{"messages":[{"data":{"id":"A100"},"key":"A100","topic":"orders","partition":0,"offset":42,"ackId":"<handle>"}],"count":1,"ackId":"<handle>","ackIds":["<handle>"]}
```


IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Kafka records messages	array Optional	Array of received records with data, key, topic, partition, offset, timestamp, headers and optional ackId.
Record count count	integer Optional	Number of records returned by this receive attempt. A zero count means no message was returned, not successful business processing.
Topic topic	string Optional	Broker topic name. Resolve it from the active environment, for example ${properties.messaging.topic}. Consumer topic lists use semicolons.
Partition partition	integer Optional	Partition number on each returned message. The engine receive result nests it under messages[].
Offset offset	long Optional	Kafka offset within one partition; it is not globally ordered across the topic.
Record timestamp timestamp	dateTime Optional	Timestamp reported by the operation. Kafka native record timestamps are epoch milliseconds; do not assume every connector returns the editor-labelled dateTime representation.
Record headers headers	object Optional	Header name/value object. The messaging runtime stringifies outgoing values; this is not a guarantee of arbitrary native binary headers or JMS header setter support.
Manual acknowledgement handle ackId	string Optional	Opaque, runtime-local confirmation token. Pass the original value to Confirm Message; it is not the business message ID and cannot be reconstructed after restart.

Advanced

Delivery, transactions and performance

Separate local enqueue, broker acknowledgement and downstream processing. Monitor background publisher errors and shutdown draining. Use stable business IDs and idempotent consumers when retries are possible. Producer reuse reduces warm-send overhead, but first connection, TLS, metadata and broker availability still affect latency; no fixed latency is guaranteed.

Implemented behavior and limitations

Runtime records live under messages[], not the editor top-level topic/partition fields. consumerCount is not implemented as a pool in the engine receive call. max.poll.records is forwarded as a native property and may be rejected by confluent-kafka. Manual commits can advance a partition past earlier records, so preserve processing/commit order. Do not claim this receive path is production-qualified without a provider test.

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Troubleshooting procedure

Connection/authentication failures: verify DNS, advertised listeners, TLS chain and ACLs on the runtime host. Serialization failures: compare actual payload type with the chosen serializer. Queue full/timeouts: inspect broker latency and producer buffer pressure before increasing limits. Consumer configuration errors may be caused by Java-style options passed to the native client. Capture the resolved non-secret configuration and the full runtime error.

Acceptance test

Acceptance: send two distinguishable records, inspect messages[].data, test failure before confirmation and verify actual committed offsets and redelivery.

Fault	Description
KafkaConnectionException	The Kafka connection or broker is unavailable.
KafkaSerializationException	The record key or value could not be serialized/deserialized.
KafkaProducerException	The producer could not publish the record.
KafkaConsumerException	The consumer could not fetch or acknowledge records.
KafkaSchemaRegistryException	The Avro schema or registry subject could not be resolved.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs



Kafka Send Message

Activities / Kafka

Overview

Publish a record

Use inside a running flow to enqueue or confirm one record on a Kafka topic.

Prerequisites

Install the matching confluent-kafka package on the execution host. Configure a Kafka shared connection with reachable bootstrap servers and the required TLS/SASL credentials. Verify advertised broker addresses are reachable from that host; a successful TCP connection to one bootstrap address is not sufficient. These pages describe the Studio/engine runtime; qualify exported direct Python separately.

Produces, receives, or commits Apache Kafka records.

Broker security comes from the shared Kafka connection. Topic, key, headers, value, partitions, offsets, and acknowledgement handles are available for hierarchical mapping.

Activity type: kafka; operation: send.

Configuration

Configuration procedure

Select the Kafka resource; bind topic to an environment property. Select JSON for an object payload, or String for already-serialized text. Start with explicit acks and waitForDelivery settings. Partition selection is optional; use a stable key before forcing a partition.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Kafka connection resourceId	resource Required	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: kafka
Topic name topic	text Configuration setting	Broker topic name. Resolve it from the active environment, for example ${properties.messaging.topic}. Consumer topic lists use semicolons.
Assign custom partition assignCustomPartition	boolean Configuration setting	Enable explicit partition selection; otherwise the client chooses/subscribes using its normal partitioning and assignment behavior.
Partition ID partitionId	number Configuration setting	Explicit Kafka partition when assignCustomPartition is enabled. It must exist in the target topic.
Key serializer keySerializer	select Configuration setting	Outgoing key conversion: String, Byte Array or JSON. Avro selection currently raises a registry-runtime error in the engine. Choices: String, Byte Array, Avro Schema, JSON
Value serializer valueSerializer	select Configuration setting	Outgoing value conversion. Select JSON for a structured object; String converts using string representation and is not a JSON encoder. Choices: String, Byte Array, Avro Schema, JSON

Field / key	Type / required	Details
Use Schema Registry useRegistry	boolean Configuration setting	Editor registry flag; setting this does not activate an Avro serializer in the current engine.
Schema Registry URL schemaRegistryUrl	text Configuration setting	Registry endpoint metadata. This engine does not implement registry-backed Avro conversion merely by setting the URL.
Key schema subject : version keySchemaSubject	text Configuration setting	Intended registry key subject/version; not resolved by the current engine serializer.
Value schema subject : version valueSchemaSubject	text Configuration setting	Intended registry value subject/version; not resolved by the current engine serializer.
Wait for broker confirmation waitForDelivery	boolean Configuration setting	False normally returns after local enqueue into a reused producer. True waits for delivery confirmation. Transactions and one-shot producers also force waiting.
Acknowledgements acks	select Configuration setting	Broker acknowledgement policy: 0, 1 or all. This differs from the waitForDelivery switch; acks=0 cannot establish broker durability. Choices: 0, 1, all
Compression type compressionType	select Configuration setting	Client batch compression. Select a codec supported by the installed client and broker; compression trades CPU for network bandwidth. Choices: none, gzip, snappy, lz4, zstd
Producer retries retries	number Configuration setting	Native producer retry setting. Separate from MINA Advanced retries; application retries can repeat a send after an ambiguous timeout.
Buffer memory (bytes) bufferMemory	number Configuration setting	Local producer queue budget in bytes, translated to queue.buffering.max.kbytes. A full queue applies backpressure/errors; it is not durable storage.
Batch size batchSize	number Configuration setting	Target producer batch size in bytes, default 16384 in the engine. Larger batches may improve throughput at the expense of latency.
Linger (ms) lingerMs	number Configuration setting	Maximum batching delay requested from the producer, default 0 in the engine. Increasing it intentionally adds buffering delay.
Maximum request size maxRequestSize	number Configuration setting	Maximum native message size setting, default 1048576 bytes. Broker/topic limits can be lower.
Transactional ID transactionalId	text Configuration setting	Enables native transaction initialization/begin/commit for a send and forces confirmation. Must be unique for concurrently active producers; it is not a multi-activity transaction group.
Enable idempotence enableIdempotence	boolean Configuration setting	Requests native producer idempotence. Must be compatible with other native settings and does not deduplicate arbitrary repeated business calls.
Additional producer properties (JSON) additionalProperties	textarea Configuration setting	JSON object of native client configuration values merged into the producer/consumer configuration. Native keys differ from Java client keys; invalid keys are rejected by the installed client.

Input

Mapping example

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
message = ${input.order}
key = ${input.order.id}
topic = ${properties.messaging.ordersTopic}
```

```
valueSerializer = JSON
waitForDelivery = true
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Kafka record value message	object|string|binary Required	Map the record/message body, not the entire preceding activity envelope unless that envelope is intentionally the payload.
Kafka record key key	string|binary Optional	Kafka record key. Use a stable business key when partition locality matters; preserve the matching serializer.
Kafka headers headers	object Optional	Header name/value object. The messaging runtime stringifies outgoing values; this is not a guarantee of arbitrary native binary headers or JMS header setter support.
Dynamic topic override topic	string Optional	Broker topic name. Resolve it from the active environment, for example ${properties.messaging.topic}. Consumer topic lists use semicolons.
Dynamic partition partitionId	integer Optional	Explicit Kafka partition when assignCustomPartition is enabled. It must exist in the target topic.
Assign custom partition assignCustomPartition	boolean Optional	Enable explicit partition selection; otherwise the client chooses/subscribes using its normal partitioning and assignment behavior.
Key serializer keySerializer	string Optional	Outgoing key conversion: String, Byte Array or JSON. Avro selection currently raises a registry-runtime error in the engine.
Value serializer valueSerializer	string Optional	Outgoing value conversion. Select JSON for a structured object; String converts using string representation and is not a JSON encoder.
Use Schema Registry useRegistry	boolean Optional	Editor registry flag; setting this does not activate an Avro serializer in the current engine.
Schema Registry URL schemaRegistryUrl	string Optional	Registry endpoint metadata. This engine does not implement registry-backed Avro conversion merely by setting the URL.
Key schema subject : version keySchemaSubject	string Optional	Intended registry key subject/version; not resolved by the current engine serializer.
Value schema subject : version valueSchemaSubject	string Optional	Intended registry value subject/version; not resolved by the current engine serializer.
Wait for broker confirmation waitForDelivery	boolean Optional	False normally returns after local enqueue into a reused producer. True waits for delivery confirmation. Transactions and one-shot producers also force waiting.
Acknowledgements acks	string Optional	Broker acknowledgement policy: 0, 1 or all. This differs from the waitForDelivery switch; acks=0 cannot establish broker durability.
Compression type compressionType	string Optional	Client batch compression. Select a codec supported by the installed client and broker; compression trades CPU for network bandwidth.
Producer retries retries	number Optional	Native producer retry setting. Separate from MINA Advanced retries; application retries can repeat a send after an ambiguous timeout.
Buffer memory (bytes) bufferMemory	number Optional	Local producer queue budget in bytes, translated to queue.buffering.max.kbytes. A full queue applies backpressure/errors; it is not durable storage.

Field / key	Type / required	Details
Batch size batchSize	number Optional	Target producer batch size in bytes, default 16384 in the engine. Larger batches may improve throughput at the expense of latency.
Linger (ms) lingerMs	number Optional	Maximum batching delay requested from the producer, default 0 in the engine. Increasing it intentionally adds buffering delay.
Maximum request size maxRequestSize	number Optional	Maximum native message size setting, default 1048576 bytes. Broker/topic limits can be lower.
Transactional ID transactionalId	string Optional	Enables native transaction initialization/begin/commit for a send and forces confirmation. Must be unique for concurrently active producers; it is not a multi-activity transaction group.
Enable idempotence enableIdempotence	boolean Optional	Requests native producer idempotence. Must be compatible with other native settings and does not deduplicate arbitrary repeated business calls.
Additional producer properties (JSON) additionalProperties	string Optional	JSON object of native client configuration values merged into the producer/consumer configuration. Native keys differ from Java client keys; invalid keys are rejected by the installed client.

Output

Illustrative runtime output

```
{"messageId":"<id>","topic":"orders","queued":true,"deliveryConfirmed":true,"partition":0,"offset":42}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Generated record ID messageId	string Optional	Identifier returned by the provider, or a locally generated identifier for buffered publishing. It is not proof that a downstream application processed the message.
Topic topic	string Optional	Broker topic name. Resolve it from the active environment, for example ${properties.messaging.topic}. Consumer topic lists use semicolons.
Accepted by the local Kafka producer queued	boolean Optional	True indicates local producer acceptance, not broker durability or downstream consumption.
Broker delivery confirmed before completion deliveryConfirmed	boolean Optional	True only when the delivery path has completed its confirmation before returning. Check this instead of inferring delivery from queued.
Publisher activity latency in milliseconds publishLatencyMs	number Optional	Publisher activity latency; buffered mode measures enqueue/overhead, not end-to-end broker or consumer latency.
Broker partition (confirmation mode) partition	integer Optional	Partition number on each returned message. The engine receive result nests it under messages[].
Broker offset (confirmation mode) offset	long Optional	Kafka offset within one partition; it is not globally ordered across the topic.
Record timestamp timestamp	dateTime Optional	Timestamp reported by the operation. Kafka native record timestamps are epoch milliseconds; do not assume every connector returns the editor-labelled dateTime representation.

Advanced

Delivery, transactions and performance

Separate local enqueue, broker acknowledgement and downstream processing. Monitor background publisher errors and shutdown draining. Use stable business IDs and idempotent consumers when retries are possible. Producer reuse reduces warm-send overhead, but first connection, TLS, metadata and broker availability still affect latency; no fixed latency is guaranteed.

Implemented behavior and limitations

With waitForDelivery=false, partition/offset can be absent. The generated ID may be local. Avro Schema selection currently raises a registry-runtime fault even when registry fields are populated. A transactionalId starts a transaction for the send; it does not span other activities.

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Troubleshooting procedure

Connection/authentication failures: verify DNS, advertised listeners, TLS chain and ACLs on the runtime host. Serialization failures: compare actual payload type with the chosen serializer. Queue full/timeouts: inspect broker latency and producer buffer pressure before increasing limits. Consumer configuration errors may be caused by Java-style options passed to the

native client. Capture the resolved non-secret configuration and the full runtime error.

Acceptance test

Acceptance: publish a known ID, verify the exact decoded value/key with an independent consumer, then test unavailable-broker behavior in both buffered and confirmed modes.

Fault	Description
KafkaConnectionException	The Kafka connection or broker is unavailable.
KafkaSerializationException	The record key or value could not be serialized/deserialized.
KafkaProducerException	The producer could not publish the record.
KafkaConsumerException	The consumer could not fetch or acknowledge records.
KafkaSchemaRegistryException	The Avro schema or registry subject could not be resolved.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs



Kafka Get Messages

Activities / Kafka

Overview

Poll records during a flow

Use after Start or another activity to perform a bounded receive attempt, rather than registering a new event starter.

Prerequisites

Install the matching confluent-kafka package on the execution host. Configure a Kafka shared connection with reachable bootstrap servers and the required TLS/SASL credentials. Verify advertised broker addresses are reachable from that host; a successful TCP connection to one bootstrap address is not sufficient. These pages describe the Studio/engine runtime; qualify exported direct Python separately.

Produces, receives, or commits Apache Kafka records.

Broker security comes from the shared Kafka connection. Topic, key, headers, value, partitions, offsets, and acknowledgement handles are available for hierarchical mapping.

Activity type: kafka; operation: get.

Configuration

Configuration procedure

Select the consumer resource/group/topics, maxMessages and poll timeout in seconds. For explicit assignment set assignCustomPartition, partitionIds and seekPosition together. Use customOffset only for custom seek.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Kafka connection resourceId	resource Required	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: kafka
Consumer group ID groupId	text Configuration setting	Consumer group used for assignment and stored offsets. Separate applications requiring independent consumption need separate groups.
Topic names topic	text Configuration setting	Broker topic name. Resolve it from the active environment, for example ${properties.messaging.topic}. Consumer topic lists use semicolons.
Assign custom partitions assignCustomPartition	boolean Configuration setting	Enable explicit partition selection; otherwise the client chooses/subscribes using its normal partitioning and assignment behavior.
Partition IDs / ranges partitionIds	text Configuration setting	Comma/semicolon-separated partition IDs or inclusive ranges used for explicit assignment.
Consumer count consumerCount	number Configuration setting	Exposed by the editor; the engine receive path creates one consumer per call and does not implement a consumer pool from this field.

Field / key	Type / required	Details
Key deserializer keyDeserializer	select Configuration setting	Incoming key decoding; Byte Array exposes base64 text, JSON parses UTF-8 JSON, String decodes text. Choices: String, Byte Array, Avro Schema, JSON
Value deserializer valueDeserializer	select Configuration setting	Incoming record-value decoding. Invalid JSON raises a decode error; Avro is not implemented by this engine path. Choices: String, Byte Array, Avro Schema, JSON
Use Schema Registry useRegistry	boolean Configuration setting	Editor registry flag; setting this does not activate an Avro serializer in the current engine.
Schema Registry URL schemaRegistryUrl	text Configuration setting	Registry endpoint metadata. This engine does not implement registry-backed Avro conversion merely by setting the URL.
Key schema subject : version keySchemaSubject	text Configuration setting	Intended registry key subject/version; not resolved by the current engine serializer.
Value schema subject : version valueSchemaSubject	text Configuration setting	Intended registry value subject/version; not resolved by the current engine serializer.
Acknowledgement mode acknowledgeMode	select Configuration setting	Choose automatic completion or explicit acknowledgement. Guarantees differ by connector and execution path; read Advanced before relying on rollback/redelivery. Choices: Auto, Manual
Enable auto commit enableAutoCommit	boolean Configuration setting	Native periodic offset commit toggle. For manual acknowledgement keep this disabled; otherwise offsets can advance before downstream work finishes.
Auto offset reset autoOffsetReset	select Configuration setting	Policy used when no valid committed offset exists, commonly earliest or latest. It does not rewind a group that already has committed offsets.
Fetch minimum bytes fetchMinBytes	number Configuration setting	Minimum fetch bytes requested from the broker. Larger values can add latency at low traffic.
Maximum poll records maxPollRecords	number Configuration setting	Editor batch-size setting. The engine currently passes max.poll.records to its native consumer; this Java-style key may be rejected by the installed native client. Treat this as a known compatibility issue, not a guaranteed supported control.
Session timeout (ms) sessionTimeoutMs	number Configuration setting	Consumer failure-detection timeout in milliseconds; must satisfy broker limits.
Heartbeat interval (ms) heartbeatIntervalMs	number Configuration setting	Consumer heartbeat interval in milliseconds; coordinate with session timeout.
Additional consumer properties (JSON) additionalProperties	textarea Configuration setting	JSON object of native client configuration values merged into the producer/consumer configuration. Native keys differ from Java client keys; invalid keys are rejected by the installed client.
Seek position seekPosition	select Configuration setting	Explicit-assignment start offset. The engine maps beginning to -2 and custom to the supplied offset; other selections currently use -1 (end). This is not a reliable resume-current implementation. Choices: current, beginning, end, custom
Custom offset customOffset	number Configuration setting	Absolute offset used with custom seek and explicit partition assignment.
Maximum messages maxMessages	number Configuration setting	Maximum poll attempts/records requested by one Get call. Each poll can wait for the configured timeout; total latency can grow with this count.
Poll timeout (seconds) timeout	number Configuration setting	Unit depends on the activity: Kafka Get uses seconds; SAP editor fields use milliseconds. Consult the runtime notes because SAP worker calls use timeoutSeconds separately.

Input

Mapping example

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
topic = ${properties.messaging.ordersTopic}
maxMessages = 10
timeout = 1
assignCustomPartition = true
partitionIds = 0
seekPosition = custom
customOffset = 42
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Dynamic topic override topic	string Optional	Broker topic name. Resolve it from the active environment, for example ${properties.messaging.topic}. Consumer topic lists use semicolons.
Dynamic partition partition	integer Optional	Partition number on each returned message. The engine receive result nests it under messages[].
Dynamic offset offset	long Optional	Kafka offset within one partition; it is not globally ordered across the topic.
Consumer group ID groupId	string Optional	Consumer group used for assignment and stored offsets. Separate applications requiring independent consumption need separate groups.
Assign custom partitions assignCustomPartition	boolean Optional	Enable explicit partition selection; otherwise the client chooses/subscribes using its normal partitioning and assignment behavior.
Partition IDs / ranges partitionIds	string Optional	Comma/semicolon-separated partition IDs or inclusive ranges used for explicit assignment.
Consumer count consumerCount	number Optional	Exposed by the editor; the engine receive path creates one consumer per call and does not implement a consumer pool from this field.
Key deserializer keyDeserializer	string Optional	Incoming key decoding; Byte Array exposes base64 text, JSON parses UTF-8 JSON, String decodes text.
Value deserializer valueDeserializer	string Optional	Incoming record-value decoding. Invalid JSON raises a decode error; Avro is not implemented by this engine path.
Use Schema Registry useRegistry	boolean Optional	Editor registry flag; setting this does not activate an Avro serializer in the current engine.
Schema Registry URL schemaRegistryUrl	string Optional	Registry endpoint metadata. This engine does not implement registry-backed Avro conversion merely by setting the URL.
Key schema subject : version keySchemaSubject	string Optional	Intended registry key subject/version; not resolved by the current engine serializer.
Value schema subject : version valueSchemaSubject	string Optional	Intended registry value subject/version; not resolved by the current engine serializer.

Field / key	Type / required	Details
Acknowledgement mode acknowledgeMode	string Optional	Choose automatic completion or explicit acknowledgement. Guarantees differ by connector and execution path; read Advanced before relying on rollback/redelivery.
Enable auto commit enableAutoCommit	boolean Optional	Native periodic offset commit toggle. For manual acknowledgement keep this disabled; otherwise offsets can advance before downstream work finishes.
Auto offset reset autoOffsetReset	string Optional	Policy used when no valid committed offset exists, commonly earliest or latest. It does not rewind a group that already has committed offsets.
Fetch minimum bytes fetchMinBytes	number Optional	Minimum fetch bytes requested from the broker. Larger values can add latency at low traffic.
Maximum poll records maxPollRecords	number Optional	Editor batch-size setting. The engine currently passes max.poll.records to its native consumer; this Java-style key may be rejected by the installed native client. Treat this as a known compatibility issue, not a guaranteed supported control.
Session timeout (ms) sessionTimeoutMs	number Optional	Consumer failure-detection timeout in milliseconds; must satisfy broker limits.
Heartbeat interval (ms) heartbeatIntervalMs	number Optional	Consumer heartbeat interval in milliseconds; coordinate with session timeout.
Additional consumer properties (JSON) additionalProperties	string Optional	JSON object of native client configuration values merged into the producer/consumer configuration. Native keys differ from Java client keys; invalid keys are rejected by the installed client.
Seek position seekPosition	string Optional	Explicit-assignment start offset. The engine maps beginning to -2 and custom to the supplied offset; other selections currently use -1 (end). This is not a reliable resume-current implementation.
Custom offset customOffset	number Optional	Absolute offset used with custom seek and explicit partition assignment.
Maximum messages maxMessages	number Optional	Maximum poll attempts/records requested by one Get call. Each poll can wait for the configured timeout; total latency can grow with this count.
Poll timeout (seconds) timeout	number Optional	Unit depends on the activity: Kafka Get uses seconds; SAP editor fields use milliseconds. Consult the runtime notes because SAP worker calls use timeoutSeconds separately.

Output

Illustrative runtime output

```
{"messages":[],"count":0,"ackId":null,"ackIds":[]}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Kafka records messages	array Optional	Array of received records with data, key, topic, partition, offset, timestamp, headers and optional ackId.
Record count count	integer Optional	Number of records returned by this receive attempt. A zero count means no message was returned, not successful business processing.

Field / key	Type / required	Details
Topic topic	string Optional	Broker topic name. Resolve it from the active environment, for example ${properties.messaging.topic}. Consumer topic lists use semicolons.
Partition partition	integer Optional	Partition number on each returned message. The engine receive result nests it under messages[].
Offset offset	long Optional	Kafka offset within one partition; it is not globally ordered across the topic.
Record timestamp timestamp	dateTime Optional	Timestamp reported by the operation. Kafka native record timestamps are epoch milliseconds; do not assume every connector returns the editor-labelled dateTime representation.
Record headers headers	object Optional	Header name/value object. The messaging runtime stringifies outgoing values; this is not a guarantee of arbitrary native binary headers or JMS header setter support.
Manual acknowledgement handle ackId	string Optional	Opaque, runtime-local confirmation token. Pass the original value to Confirm Message; it is not the business message ID and cannot be reconstructed after restart.

Advanced

Delivery, transactions and performance

Separate local enqueue, broker acknowledgement and downstream processing. Monitor background publisher errors and shutdown draining. Use stable business IDs and idempotent consumers when retries are possible. Producer reuse reduces warm-send overhead, but first connection, TLS, metadata and broker availability still affect latency; no fixed latency is guaranteed.

Implemented behavior and limitations

Empty results are normal when no record is available. Each poll can consume the timeout, so the upper wait grows with maxMessages. The engine reads partitionIds/customOffset; the displayed dynamic partition/offset aliases are not equivalent overrides. Current seek maps to end in explicit assignment; do not use it as a resume guarantee. Consumer native-option and Avro limitations are the same as Receive.

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"

Field / key	Type / required	Details
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Troubleshooting procedure

Connection/authentication failures: verify DNS, advertised listeners, TLS chain and ACLs on the runtime host. Serialization failures: compare actual payload type with the chosen serializer. Queue full/timeouts: inspect broker latency and producer buffer pressure before increasing limits. Consumer configuration errors may be caused by Java-style options passed to the native client. Capture the resolved non-secret configuration and the full runtime error.

Acceptance test

Acceptance: seed a test partition with known offsets; check beginning/custom/end behavior and the zero-message branch.

Fault	Description
KafkaConnectionException	The Kafka connection or broker is unavailable.
KafkaSerializationException	The record key or value could not be serialized/deserialized.
KafkaProducerException	The producer could not publish the record.
KafkaConsumerException	The consumer could not fetch or acknowledge records.
KafkaSchemaRegistryException	The Avro schema or registry subject could not be resolved.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs



Pub/Sub Subscriber

Activities / GCP Pub/Sub

Overview

Publishes, receives, or acknowledges Google Cloud Pub/Sub messages.

Project and credential defaults come from the shared connection. Message data, attributes, ordering keys, and acknowledgement handles remain available on the execution path.

Activity type: pubsub; operation: subscribe.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Google Pub/Sub connection resourceId	resource Required	Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment. Connection type: pubsub
Project ID override projectId	text Configuration setting	Google Cloud project containing the Pub/Sub topic or subscription. Supply the project ID, not its display name.
Subscription ID subscription	text Configuration setting	Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access.
Maximum message fetch count maxMessages	number Configuration setting	Bound on received/buffered messages or bytes, according to this field. Keep the bound within available memory and downstream processing capacity.
Acknowledge mode acknowledgeMode	select Configuration setting	Message acknowledgement policy. Manual/client modes require explicit confirmation after successful processing; automatic/provider modes follow connector behavior. Choices: Auto, Client
Receive timeout (seconds) receiveTimeout	number Configuration setting	Receive timeout (seconds): bounded wait/delay in the connector-defined time unit. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.
Sequence key expression sequenceKey	text Configuration setting	Stable ordering/sequence key used to group related messages where the selected provider supports ordered delivery.
Custom job ID expression customJobId	text Configuration setting	Optional business identifier for tracing the received event/job. Use a value that distinguishes separate business deliveries.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Project ID override projectId	string Optional	Google Cloud project containing the Pub/Sub topic or subscription. Supply the project ID, not its display name.
Subscription ID subscription	string Optional	Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access.
Maximum message fetch count maxMessages	number Optional	Bound on received/buffered messages or bytes, according to this field. Keep the bound within available memory and downstream processing capacity.
Acknowledge mode acknowledgeMode	string Optional	Message acknowledgement policy. Manual/client modes require explicit confirmation after successful processing; automatic/provider modes follow connector behavior.
Receive timeout (seconds) receiveTimeout	number Optional	Receive timeout (seconds): bounded wait/delay in the connector-defined time unit. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.
Sequence key expression sequenceKey	string Optional	Stable ordering/sequence key used to group related messages where the selected provider supports ordered delivery.
Custom job ID expression customJobId	string Optional	Optional business identifier for tracing the received event/job. Use a value that distinguishes separate business deliveries.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Google message ID MessageID	string Optional	Message identifier returned by the provider or runtime. Retain it for tracing; use a separate stable business key for duplicate detection.
Publish time PublishTime	dateTime Optional	Time reported by the operation/provider. Preserve the original representation when tracing; epoch values and formatted date/time strings need explicit conversion.
Message data Data	string Optional	Message or response content. Map the business document rather than a previous publisher receipt. Objects retain their structure; string values carry serialized text.
Message attributes Attributes	object Optional	Name/value metadata associated with the message. Keep business content in the body and map only properties understood by the receiving system.
Client acknowledgement handle AckID	string Optional	Acknowledgement/settlement handle from a received message. Confirm only after required downstream work succeeds; preserve the original handle.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
GooglePubSubPluginException	The subscriber could not receive or acknowledge the message.
GooglePubSubConnectionException	The Google Pub/Sub connection is unavailable.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



Pub/Sub Publisher

Activities / GCP Pub/Sub

Overview

Publishes, receives, or acknowledges Google Cloud Pub/Sub messages.

Project and credential defaults come from the shared connection. Message data, attributes, ordering keys, and acknowledgement handles remain available on the execution path.

Activity type: pubsub; operation: publish.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Google Pub/Sub connection resourceId	resource Required	Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment. Connection type: pubsub
Project ID override projectId	text Configuration setting	Google Cloud project containing the Pub/Sub topic or subscription. Supply the project ID, not its display name.
Topic ID topic	text Configuration setting	Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access.
Ordering key orderingKey	text Configuration setting	Stable ordering/sequence key used to group related messages where the selected provider supports ordered delivery.
Wait for Google confirmation waitForDelivery	boolean Configuration setting	Off queues through the persistent batched gRPC publisher for maximum throughput. Enable only when the flow requires Google's message ID.
Publish timeout (seconds) publishTimeout	number Configuration setting	Publish timeout (seconds): bounded wait/delay in the connector-defined time unit. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.
Batch maximum messages batchMaxMessages	number Configuration setting	Pub/Sub batching threshold: message count, total bytes or waiting time in milliseconds, according to the field name. Tune with payload sizes and acceptable delivery latency.
Batch maximum bytes batchMaxBytes	number Configuration setting	Pub/Sub batching threshold: message count, total bytes or waiting time in milliseconds, according to the field name. Tune with payload sizes and acceptable delivery latency.
Batch delay threshold (ms) batchDelayThresholdMilliseconds	number Configuration setting	Pub/Sub batching threshold: message count, total bytes or waiting time in milliseconds, according to the field name. Tune with payload sizes and acceptable delivery latency.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Message data data	string|binary Required	Data or at least one attribute is required.
Message attributes attributes	object Optional	Name/value metadata associated with the message. Keep business content in the body and map only properties understood by the receiving system.
Dynamic Topic ID topic	string Optional	Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access.
Project ID override projectId	string Optional	Google Cloud project containing the Pub/Sub topic or subscription. Supply the project ID, not its display name.
Ordering key orderingKey	string Optional	Stable ordering/sequence key used to group related messages where the selected provider supports ordered delivery.
Wait for Google confirmation waitForDelivery	boolean Optional	Off queues through the persistent batched gRPC publisher for maximum throughput. Enable only when the flow requires Google's message ID.
Publish timeout (seconds) publishTimeout	number Optional	Publish timeout (seconds): bounded wait/delay in the connector-defined time unit. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.
Batch maximum messages batchMaxMessages	number Optional	Pub/Sub batching threshold: message count, total bytes or waiting time in milliseconds, according to the field name. Tune with payload sizes and acceptable delivery latency.
Batch maximum bytes batchMaxBytes	number Optional	Pub/Sub batching threshold: message count, total bytes or waiting time in milliseconds, according to the field name. Tune with payload sizes and acceptable delivery latency.
Batch delay threshold (ms) batchDelayThresholdMilliseconds	number Optional	Pub/Sub batching threshold: message count, total bytes or waiting time in milliseconds, according to the field name. Tune with payload sizes and acceptable delivery latency.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Published topic name TopicName	string Optional	Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access.
Google or local queued message ID MessageID	string Optional	Message identifier returned by the provider or runtime. Retain it for tracing; use a separate stable business key for duplicate detection.
Google message ID when confirmation is enabled providerMessageId	string Optional	Message identifier returned by the provider or runtime. Retain it for tracing; use a separate stable business key for duplicate detection.
Accepted by local publisher queued	boolean Optional	Operation outcome returned by the runtime. Inspect error output as well; provider acceptance does not establish processing by a downstream consumer.
Confirmed by Google deliveryConfirmed	boolean Optional	Operation outcome returned by the runtime. Inspect error output as well; provider acceptance does not establish processing by a downstream consumer.

Field / key	Type / required	Details
Activity enqueue latency (ms) publishLatencyMs	number Optional	Measured publish latency in milliseconds. It measures the reported publish stage, not downstream consumer processing.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Fault	Description
GooglePublisherException	The message could not be published.
GooglePubSubConnectionException	The Google Pub/Sub connection is unavailable.
PublisherInvalidInputException	Publisher requires message data or at least one attribute.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



Dynamic Connection

Activities / SAP ECC

Overview

Open or close a stateful SAP session

Use when multiple RFC/BAPI activities must share one JCo context and an explicit commit/rollback boundary.

Prerequisites

Install licensed SAP JCo JAR and the OS/architecture-matched native library on the execution host; optional IDoc-library validation also needs its licensed JAR. Configure the SAP shared resource, test connectivity, fetch and save the exact IDoc/function metadata required by the activity. Network routing, gateway registration, SAP authorizations and partner profiles must be provisioned by the SAP team. MINA is not SAP-certified by virtue of using JCo.

Executes the selected SAP ECC operation.

The shared SAP connection supplies system and authentication settings. IDoc metadata selected from SAP defines listener, parser, renderer, and sender structures.

Activity type: sap; operation: dynamic_connection.

Configuration

Configuration procedure

Select the SAP resource, connection mode and transactional setting. Bind secrets through properties. On the success path call Dynamic Connection again with terminateConnection=true and commitOnTerminate=true; on the error path terminate without commit.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
SAP ECC shared connection resourceId	resource Configuration setting	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: sap
Connection type connectionType	select Configuration setting	Connection routing mode: direct application server, message-server logon group, SNC variants or WebSocket configuration. Supply the corresponding shared-connection fields. Choices: dedicated, logongroup, snc, sncwithlogongroup, websocket
Transactional transactional	boolean Configuration setting	Whether a stateful transactional context is requested. Only operations sharing the returned live sessionID participate in that context.
Terminate connection terminateConnection	boolean Configuration setting	When true, closes the session identified by sessionID instead of creating a new session. Terminating a transactional session rolls back unless commitOnTerminate is enabled.
Commit before termination commitOnTerminate	boolean Configuration setting	Commit the live transactional session before closing it. Enable only on the success path; use rollback termination on the error path. Shown when: (config) => !!config.terminateConnection

Field / key	Type / required	Details
Session idle timeout (ms) contextTimeout	number Configuration setting	Idle lifetime of a dynamic SAP session in milliseconds. Default is 600000, clamped to at least 1000; later calls renew the session lifetime.
Timeout (ms) timeout	number Configuration setting	Unit depends on the activity: Kafka Get uses seconds; SAP editor fields use milliseconds. Consult the runtime notes because SAP worker calls use timeoutSeconds separately.

Input

Mapping example

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
Open: transactional = true
Invoke RFC Input sessionID = ${SAP-Open.sessionID}
Close Input sessionID = ${SAP-Open.sessionID}
Close: terminateConnection = true
Close success only: commitOnTerminate = true
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
Session ID (required only when terminating) sessionID	string Optional	Map the sessionID returned by Dynamic Connection. It identifies a live, process-local JCo context; do not hard-code or reuse it after termination or restart.
Transaction context timeout in milliseconds contextTimeout	integer Optional	Idle lifetime of a dynamic SAP session in milliseconds. Default is 600000, clamped to at least 1000; later calls renew the session lifetime.
dedicated, logongroup, snc, sncwithlogongroup, or websocket connectionType	string Optional	Connection routing mode: direct application server, message-server logon group, SNC variants or WebSocket configuration. Supply the corresponding shared-connection fields.
Application server host applicationServerHost	string Optional	Application-server hostname for direct connection mode; must resolve from the runtime host.
System number systemNumber	string Optional	SAP instance/system number for a direct application-server connection, normally a two-character string.
Message server host messageServerHost	string Optional	Message-server hostname for logon-group mode.
SAP system ID systemId	string Optional	SAP system ID used with message-server/logon-group routing.
SAP logon group logonGroup	string Optional	SAP logon-group name configured by the SAP administrator.
SAP client client	string Optional	SAP client identifier. Keep leading zeros; provide it as a string.
SAP user username	string Optional	SAP logon user with authorization for the selected RFCs and metadata APIs.

Field / key	Type / required	Details
SAP password password	string Optional	SAP logon secret. Bind a secret/environment property; do not put credentials into examples, source control or payload logs.
Logon language language	string Optional	SAP logon language key.
SAProuter string sapRouter	string Optional	Optional SAProuter routing string supplied by the network/SAP administrator.
SNC partner name sncPartnerName	string Optional	SNC partner identity expected by the target SAP system.
SNC library path sncLibraryPath	string Optional	Native SNC library path on the runtime host, matching OS and architecture.
SNC own name sncMyName	string Optional	Optional client SNC identity.
SNC quality of protection sncQop	string Optional	SNC quality-of-protection setting accepted by the target system.
Transactional transactional	boolean Optional	Whether a stateful transactional context is requested. Only operations sharing the returned live sessionID participate in that context.
Terminate connection terminateConnection	boolean Optional	When true, closes the session identified by sessionID instead of creating a new session. Terminating a transactional session rolls back unless commitOnTerminate is enabled.
Commit before termination commitOnTerminate	boolean Optional	Commit the live transactional session before closing it. Enable only on the success path; use rollback termination on the error path.
Timeout (ms) timeout	number Optional	Unit depends on the activity: Kafka Get uses seconds; SAP editor fields use milliseconds. Consult the runtime notes because SAP worker calls use timeoutSeconds separately.

Output

Illustrative runtime output

```
{"sessionID":"<live-session>","connected":true,"transactional":true}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
Dynamic SAP session ID sessionID	string Optional	Map the sessionID returned by Dynamic Connection. It identifies a live, process-local JCo context; do not hard-code or reuse it after termination or restart.
Connection status connected	boolean Optional	True when Dynamic Connection successfully opens and pings its JCo worker; it does not guarantee later calls will succeed.
Transaction mode transactional	boolean Optional	Whether a stateful transactional context is requested. Only operations sharing the returned live sessionID participate in that context.

Advanced

Delivery, transactions and performance

SAP transport success and business success are different. Inspect BAPI RETURN/status data even when the RFC call completes. Use bounded payloads and batches; parsing/rendering materializes documents in memory. Persistent outbound workers reduce JVM startup overhead but do not eliminate provider/network latency. Use a live Dynamic Connection session only within its owning runtime; never persist its ID for later reuse.

Implemented behavior and limitations

Close returns terminated/committed rather than connected. Sessions are process-local and expire after idle timeout; default is 600000 ms. A failed transactional worker call attempts rollback and removes its session. Ensure every branch closes its session. Do not confuse this with a JDBC transaction group or distributed transaction.

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Troubleshooting procedure

SAP_CONNECTION: verify native-library loading, routing and logon credentials. SAP_RFC: inspect the function name, parameter metadata and SAP authorization/ABAP error. SAP_IDOC or SAP_SCHEMA: compare basic type, extension, release and saved field offsets with the actual payload. SAP_TRANSACTION: inspect TID/queue/context state before retrying. Some underlying paths raise generic runtime errors instead of the advertised editor fault code; retain the full message and trace.

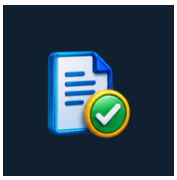
Acceptance test

Acceptance: perform two reversible test changes in one session, first rollback and then commit; verify both outcomes directly in SAP.

Fault	Description
SAP_CONNECTION	The SAP JCo/RFC connection could not be established.
SAP_RFC	SAP returned an RFC or BAPI exception.
SAP_IDOC	The IDoc could not be parsed, posted, confirmed, or acknowledged.
SAP_TRANSACTION	The tRFC, qRFC, bgRFC, or transactional context failed.
SAP_SCHEMA	The downloaded RFC, table, or IDoc metadata is invalid.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs



IDoc Acknowledgment

Activities / SAP ECC

Overview

Call an SAP acknowledgement function

Use only when the SAP team has defined an acknowledgement RFC and its required parameters.

Prerequisites

Install licensed SAP JCo JAR and the OS/architecture-matched native library on the execution host; optional IDoc-library validation also needs its licensed JAR. Configure the SAP shared resource, test connectivity, fetch and save the exact IDoc/function metadata required by the activity. Network routing, gateway registration, SAP authorizations and partner profiles must be provisioned by the SAP team. MINA is not SAP-certified by virtue of using JCo.

Executes the selected SAP ECC operation.

The shared SAP connection supplies system and authentication settings. IDoc metadata selected from SAP defines listener, parser, renderer, and sender structures.

Activity type: sap; operation: idoc_acknowledgment.

Configuration

Configuration procedure

Select the SAP resource and status intent. External mode requires functionName, but this activity editor does not currently expose that field. It must be present in resolved configuration/shared settings; otherwise execution fails. Status/successMessage/errorMessage alone do not implement an SAP acknowledgement protocol.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
SAP ECC shared connection resourceId	resource Configuration setting	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: sap
SAP IDoc status status	text Configuration setting	Desired acknowledgement status metadata. Actual SAP status changes depend on the configured acknowledgement RFC and its parameters.
Success message successMessage	text Configuration setting	Editor text for acknowledgement intent; not automatically translated into an SAP RFC parameter by the native adapter.
Error message errorMessage	text Configuration setting	Editor error text; explicitly map the required acknowledgement RFC arguments rather than assuming this field changes SAP status.

Input

Mapping example

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
Resolved functionName = <approved acknowledgement RFC>
payload = ${input.acknowledgementArguments}
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
SAP activity payload payload	object Required	Map the whole input document expected by this operation. Use the Data tree to select an earlier output; the examples below describe its required shape.
Dynamic connection session ID sessionID	string Optional	Map the sessionID returned by Dynamic Connection. It identifies a live, process-local JCo context; do not hard-code or reuse it after termination or restart.
SAP IDoc status status	string Optional	Desired acknowledgement status metadata. Actual SAP status changes depend on the configured acknowledgement RFC and its parameters.
Success message successMessage	string Optional	Editor text for acknowledgement intent; not automatically translated into an SAP RFC parameter by the native adapter.
Error message errorMessage	string Optional	Editor error text; explicitly map the required acknowledgement RFC arguments rather than assuming this field changes SAP status.

Output

Illustrative runtime output

```
{"acknowledged":true,"status":"<configured-status>","idocNumber":"<supplied-number>"}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
SAP response result	object Optional	Generic editor result placeholder. Native SAP calls return exports, changing parameters, tables and operation-specific metadata; inspect actual Job Data before mapping result.
Completion status successful	boolean Optional	Generic editor completion indicator. See the runtime output example: several native SAP operations return operation-specific fields instead of this generic field.

Advanced

Delivery, transactions and performance

SAP transport success and business success are different. Inspect BAPI RETURN/status data even when the RFC call completes. Use bounded payloads and batches; parsing/rendering materializes documents in memory. Persistent outbound workers reduce JVM startup overhead but do not eliminate provider/network latency. Use a live Dynamic Connection session only within its owning runtime; never persist its ID for later reuse.

Implemented behavior and limitations

The runtime invokes the configured RFC with mapped arguments and adds acknowledgement metadata. Check the RFC response before treating business acknowledgement as accepted. This is distinct from broker Confirm Message and JCo listener transport acknowledgement.

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Troubleshooting procedure

SAP_CONNECTION: verify native-library loading, routing and logon credentials. SAP_RFC: inspect the function name, parameter metadata and SAP authorization/ABAP error. SAP_IDOC or SAP_SCHEMA: compare basic type, extension, release and saved field offsets with the actual payload. SAP_TRANSACTION: inspect TID/queue/context state before retrying. Some underlying paths raise generic runtime errors instead of the advertised editor fault code; retain the full message and trace.

Acceptance test

Acceptance: verify the expected SAP status transition for a test IDoc and a deliberately rejected acknowledgement.

Fault	Description
SAP_CONNECTION	The SAP JCo/RFC connection could not be established.

Fault	Description
SAP_RFC	SAP returned an RFC or BAPI exception.
SAP_IDOC	The IDoc could not be parsed, posted, confirmed, or acknowledged.
SAP_TRANSACTION	The tRFC, qRFC, bgRFC, or transactional context failed.
SAP_SCHEMA	The downloaded RFC, table, or IDoc metadata is invalid.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs



IDoc Confirmation

Activities / SAP ECC

Overview

Confirm through a configured SAP RFC

Use for the confirmation contract explicitly provided by the target SAP system.

Prerequisites

Install licensed SAP JCo JAR and the OS/architecture-matched native library on the execution host; optional IDoc-library validation also needs its licensed JAR. Configure the SAP shared resource, test connectivity, fetch and save the exact IDoc/function metadata required by the activity. Network routing, gateway registration, SAP authorizations and partner profiles must be provisioned by the SAP team. MINA is not SAP-certified by virtue of using JCo.

Executes the selected SAP ECC operation.

The shared SAP connection supplies system and authentication settings. IDoc metadata selected from SAP defines listener, parser, renderer, and sender structures.

Activity type: sap; operation: idoc_confirmation.

Configuration

Configuration procedure

Set functionName to the approved confirmation RFC and map its actual parameters. Select the SAP connection. confirmationMode and destination are descriptive/editor settings, not a generic implementation of every confirmation protocol.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
SAP ECC shared connection resourceId	resource Configuration setting	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: sap
Confirmation mode confirmationMode	select Configuration setting	Editor setting; the current native adapter does not implement a separate generic confirmation-mode switch. Read the operation-specific limitations below.
Confirmation destination destination	text Configuration setting	Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default".
Confirmation RFC functionName	text Configuration setting	Exact remote-enabled function/BAPI name available in the target SAP repository. Its case and parameter names must match fetched metadata.

Input

Mapping example

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
functionName = <approved confirmation RFC>
payload = ${input.confirmationArguments}
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
SAP activity payload payload	object Required	Map the whole input document expected by this operation. Use the Data tree to select an earlier output; the examples below describe its required shape.
Dynamic connection session ID sessionID	string Optional	Map the sessionID returned by Dynamic Connection. It identifies a live, process-local JCo context; do not hard-code or reuse it after termination or restart.
Confirmation mode confirmationMode	string Optional	Editor setting; the current native adapter does not implement a separate generic confirmation-mode switch. Read the operation-specific limitations below.
Confirmation destination destination	string Optional	Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default".
Confirmation RFC functionName	string Optional	Exact remote-enabled function/BAPI name available in the target SAP repository. Its case and parameter names must match fetched metadata.

Output

Illustrative runtime output

```
{"acknowledged":true,"status":"<configured-status>","idocNumber":"<supplied-number>"}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
SAP response result	object Optional	Generic editor result placeholder. Native SAP calls return exports, changing parameters, tables and operation-specific metadata; inspect actual Job Data before mapping result.
Completion status successful	boolean Optional	Generic editor completion indicator. See the runtime output example: several native SAP operations return operation-specific fields instead of this generic field.

Advanced

Delivery, transactions and performance

SAP transport success and business success are different. Inspect BAPI RETURN/status data even when the RFC call completes. Use bounded payloads and batches; parsing/rendering materializes documents in memory. Persistent outbound workers reduce JVM startup overhead but do not eliminate provider/network latency. Use a live Dynamic Connection session only within its owning runtime; never persist its ID for later reuse.

Implemented behavior and limitations

No universal confirmation RFC name or parameter shape is assumed. The runtime requires functionName in external mode; a mock acknowledged result is not proof of a real SAP confirmation.

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Troubleshooting procedure

SAP_CONNECTION: verify native-library loading, routing and logon credentials. SAP_RFC: inspect the function name, parameter metadata and SAP authorization/ABAP error. SAP_IDOC or SAP_SCHEMA: compare basic type, extension, release and saved field offsets with the actual payload. SAP_TRANSACTION: inspect TID/queue/context state before retrying. Some underlying paths raise generic runtime errors instead of the advertised editor fault code; retain the full message and trace.

Acceptance test

Acceptance: inspect the configured RFC return and SAP document/transaction state, including duplicate confirmation behavior.

Fault	Description
SAP_CONNECTION	The SAP JCo/RFC connection could not be established.
SAP_RFC	SAP returned an RFC or BAPI exception.
SAP_IDOC	The IDoc could not be parsed, posted, confirmed, or acknowledged.
SAP_TRANSACTION	The tRFC, qRFC, bgRFC, or transactional context failed.
SAP_SCHEMA	The downloaded RFC, table, or IDoc metadata is invalid.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs



IDoc Converter

Activities / SAP ECC

Overview

Convert an IDoc representation

Use inside a task to transform supported IDoc source content into the selected representation without posting it.

Prerequisites

Install licensed SAP JCo JAR and the OS/architecture-matched native library on the execution host; optional IDoc-library validation also needs its licensed JAR. Configure the SAP shared resource, test connectivity, fetch and save the exact IDoc/function metadata required by the activity. Network routing, gateway registration, SAP authorizations and partner profiles must be provisioned by the SAP team. MINA is not SAP-certified by virtue of using JCo.

Executes the selected SAP ECC operation.

The shared SAP connection supplies system and authentication settings. IDoc metadata selected from SAP defines listener, parser, renderer, and sender structures.

Activity type: sap; operation: idoc_converter.

Configuration

Configuration procedure

Select the saved IDoc basic type/extension metadata. Map the full IDoc source. XML output returns serialized XML; Raw returns the original source. Use IDoc Renderer when transport-ready control/data records are required.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
SAP ECC shared connection resourceId	resource Configuration setting	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: sap
IDoc type fetched from SAP idocType	idoc Configuration setting	Select and save the basic IDoc type fetched from the chosen SAP connection. The saved catalog/schema drives segment layout, field offsets and mapping suggestions.
Extension / CIM type extensionType	text Configuration setting	Optional IDoc extension/CIM type. It must match the sender/receiver contract; changing it requires refreshing metadata.
SAP release release	text Configuration setting	SAP release used to resolve IDoc metadata. Use the release supported by the connected system, not an arbitrary version string.
IDoc output mode idocOutputMode	select Configuration setting	Parser: JSON or XML. Converter also exposes Raw; RAW returns the original source rather than manufacturing transport-ready EDI records. Choices: XML, Raw

Input

Mapping example

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
IDoc = ${SAP-Listener.payload}
SAPIDoc = ${SAP-Listener.controlRecord}
idocOutputMode = XML
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
SAP IDoc control record SAPIDoc	object Required	For parser input, control metadata or a structured IDoc. Parser output contains one canonical basic-type branch, for example SAPIDoc.ARTMAS05.IDOC. Legacy direct paths remain available for mapping compatibility but are not duplicated in serialized JSON.
Raw IDoc IDoc	string|object Required	Raw IDoc input: XML text, supported structured control/data records or flat IDoc bytes/text. Do not map only one segment when the operation needs the complete document.
Extension / CIM type extensionType	string Optional	Optional IDoc extension/CIM type. It must match the sender/receiver contract; changing it requires refreshing metadata.
SAP release release	string Optional	SAP release used to resolve IDoc metadata. Use the release supported by the connected system, not an arbitrary version string.
IDoc output mode idocOutputMode	string Optional	Parser: JSON or XML. Converter also exposes Raw; RAW returns the original source rather than manufacturing transport-ready EDI records.

Output

Illustrative runtime output

```
{"format":"XML","IDocXML":"<IDoc>...</IDoc>","payload":"<IDoc>...</IDoc>","SAPIDoc":{}}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
Parsed IDoc JSON SAPIDoc	object Optional	For parser input, control metadata or a structured IDoc. Parser output contains one canonical basic-type branch, for example SAPIDoc.ARTMAS05.IDOC. Legacy direct paths remain available for mapping compatibility but are not duplicated in serialized JSON.
Output format format	string Optional	Output representation selected by the parser/converter: JSON, XML or RAW. Map the accompanying content field appropriate for that format.

Advanced

Delivery, transactions and performance

SAP transport success and business success are different. Inspect BAPI RETURN/status data even when the RFC call completes. Use bounded payloads and batches; parsing/rendering materializes documents in memory. Persistent outbound

workers reduce JVM startup overhead but do not eliminate provider/network latency. Use a live Dynamic Connection sessiononly within its owning runtime; never persist its ID for later reuse.

Implemented behavior and limitations

The converter shares the parser implementation. Its generic editor output can differ from the selected mode; XML also includes structured views, RAW preserves the supplied value. This activity is not an outbound SAP call.

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Troubleshooting procedure

SAP_CONNECTION: verify native-library loading, routing and logon credentials. SAP_RFC: inspect the function name, parameter metadata and SAP authorization/ABAP error. SAP_IDOC or SAP_SCHEMA: compare basic type, extension, release and saved field offsets with the actual payload. SAP_TRANSACTION: inspect TID/queue/context state before retrying. Some underlying paths raise generic runtime errors instead of the advertised editor fault code; retain the full message and trace.

Acceptance test

Acceptance: convert a small fetched-type fixture, then parse it back and compare control fields, segment values and array cardinality.

Fault	Description
SAP_CONNECTION	The SAP JCo/RFC connection could not be established.
SAP_RFC	SAP returned an RFC or BAPI exception.
SAP_IDOC	The IDoc could not be parsed, posted, confirmed, or acknowledged.
SAP_TRANSACTION	The tRFC, qRFC, bgRFC, or transactional context failed.
SAP_SCHEMA	The downloaded RFC, table, or IDoc metadata is invalid.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs



IDoc Listener

Activities / SAP ECC

Overview

Start a flow when an IDoc arrives

Use as an event starter. Direct mode registers a JCo inbound server; broker mode consumes an already-published IDoc.

Prerequisites

Install licensed SAP JCo JAR and the OS/architecture-matched native library on the execution host; optional IDoc-library validation also needs its licensed JAR. Configure the SAP shared resource, test connectivity, fetch and save the exact IDoc/function metadata required by the activity. Network routing, gateway registration, SAP authorizations and partner profiles must be provisioned by the SAP team. MINA is not SAP-certified by virtue of using JCo.

Executes the selected SAP ECC operation.

The shared SAP connection supplies system and authentication settings. IDoc metadata selected from SAP defines listener, parser, renderer, and sender structures.

Activity type: sap; operation: idoc_listener.

Configuration

Configuration procedure

Configure SAP program ID/gateway and partner/RFC routing, fetch and save IDoc metadata, then run the task. NoMessaging uses direct JCo. For EMS/JMS/Kafka select the additional broker resource and destination; MINA does not automatically create the upstream SAP-to-broker bridge.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
SAP ECC shared connection resourceId	resource Configuration setting	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: sap
Messaging source messagingSource	select Configuration setting	NoMessaging receives directly through JCo. EMS/JMS/Kafka modes consume an IDoc carried by a broker; an upstream SAP-to-broker producer is still required. Choices: NoMessaging, EMS, JMS, Kafka
Messaging shared connection messagingResourceId	resource Configuration setting	Shared broker connection matching the chosen messaging source, in addition to the SAP metadata connection. Shown when: (config) => !["", "NoMessaging"].includes(config.messagingSource || "NoMessaging") Connection types:
Queue or topic messagingDestination	text Configuration setting	Queue or topic carrying the complete IDoc document. Shown when: (config) => !["", "NoMessaging"].includes(config.messagingSource || "NoMessaging")
SAP TID Manager resource tidManagerId	text Configuration setting	Shared SAP transaction-ID store selection. Durable TID state supports duplicate detection; it does not make downstream external writes atomic. Read-only

Field / key	Type / required	Details
IDoc type fetched from SAP idocType	idoc Configuration setting	Select and save the basic IDoc type fetched from the chosen SAP connection. The saved catalog/schema drives segment layout, field offsets and mapping suggestions.
Extension / CIM type extensionType	text Configuration setting	Optional IDoc extension/CIM type. It must match the sender/receiver contract; changing it requires refreshing metadata. Read-only
SAP release release	text Configuration setting	SAP release used to resolve IDoc metadata. Use the release supported by the connected system, not an arbitrary version string. Read-only
Program ID programId	text Configuration setting	Registered-server program ID. Must match the SAP RFC destination and gateway registration permissions. Read-only
Invocation protocol invocationProtocol	text Configuration setting	Editor protocol selection. Native bridge implements synchronous RFC and tRFC/qRFC dispatch; bgRFC is not a separately implemented native execution mode. Read-only

Input

Mapping example

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
messagingSource = NoMessaging
IDoc Parser IDoc = ${SAP-Listener.payload}
IDoc Parser SAPIDoc = ${SAP-Listener.controlRecord}
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
Messaging source messagingSource	string Optional	NoMessaging receives directly through JCo. EMS/JMS/Kafka modes consume an IDoc carried by a broker; an upstream SAP-to-broker producer is still required.
Queue or topic messagingDestination	string Optional	Queue or topic carrying the complete IDoc document.
SAP TID Manager resource tidManagerId	string Optional	Shared SAP transaction-ID store selection. Durable TID state supports duplicate detection; it does not make downstream external writes atomic.
Extension / CIM type extensionType	string Optional	Optional IDoc extension/CIM type. It must match the sender/receiver contract; changing it requires refreshing metadata.
SAP release release	string Optional	SAP release used to resolve IDoc metadata. Use the release supported by the connected system, not an arbitrary version string.
Program ID programId	string Optional	Registered-server program ID. Must match the SAP RFC destination and gateway registration permissions.
Invocation protocol invocationProtocol	string Optional	Editor protocol selection. Native bridge implements synchronous RFC and tRFC/qRFC dispatch; bgRFC is not a separately implemented native execution mode.

Output

Illustrative runtime output

```
{"SAPIDoc":{},"controlRecord":{"IDOCTYP":"<basic-type>"},"payload":"<IDoc>...</IDoc>","TID":"<transacti
on>"}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
IDoc/RFC metadata SAPIDoc	object Optional	For parser input, control metadata or a structured IDoc. Parser output contains one canonical basic-type branch, for example SAPIDoc.ARTMAS05.IDOC. Legacy direct paths remain available for mapping compatibility but are not duplicated in serialized JSON.
SAP IDoc control record controlRecord	object Optional	EDI_DC40 control metadata describing sender, receiver, message/basic type and extension. Keep it with the associated data records.
Inbound SAP payload (XML) payload	string Optional	Map the whole input document expected by this operation. Use the Data tree to select an earlier output; the examples below describe its required shape.
Transaction ID TID	string Optional	SAP transaction identifier when provided by the transport. It is not the IDoc business document number and is not a JMS acknowledgement token.

Advanced

Delivery, transactions and performance

SAP transport success and business success are different. Inspect BAPI RETURN/status data even when the RFC call completes. Use bounded payloads and batches; parsing/rendering materializes documents in memory. Persistent outbound workers reduce JVM startup overhead but do not eliminate provider/network latency. Use a live Dynamic Connection session only within its owning runtime; never persist its ID for later reuse.

Implemented behavior and limitations

Direct delivery uses durable TID handling and listener completion; broker modes inherit the broker path acknowledgement limitations. Metadata arrays follow the saved schema. Do not acknowledge transport success before irreversible downstream work unless your recovery design tolerates it. Debug breakpoints can hold requests until the listener timeout.

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Troubleshooting procedure

SAP_CONNECTION: verify native-library loading, routing and logon credentials. SAP_RFC: inspect the function name, parameter metadata and SAP authorization/ABAP error. SAP_IDOC or SAP_SCHEMA: compare basic type, extension, release and saved field offsets with the actual payload. SAP_TRANSACTION: inspect TID/queue/context state before retrying. Some underlying paths raise generic runtime errors instead of the advertised editor fault code; retain the full message and trace.

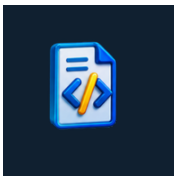
Acceptance test

Acceptance: send a test IDoc while Run is active, confirm automatic execution without Step Into, then test duplicate TID, downstream error, stop/restart and redelivery.

Fault	Description
SAP_CONNECTION	The SAP JCo/RFC connection could not be established.
SAP_RFC	SAP returned an RFC or BAPI exception.
SAP_IDOC	The IDoc could not be parsed, posted, confirmed, or acknowledged.
SAP_TRANSACTION	The tRFC, qRFC, bgRFC, or transactional context failed.
SAP_SCHEMA	The downloaded RFC, table, or IDoc metadata is invalid.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs



IDoc Parser

Activities / SAP ECC

Overview

Decode an IDoc into mapped fields

Use after a listener, file read or broker receive to expose control and segment fields in JSON/XML.

Prerequisites

Install licensed SAP JCo JAR and the OS/architecture-matched native library on the execution host; optional IDoc-library validation also needs its licensed JAR. Configure the SAP shared resource, test connectivity, fetch and save the exact IDoc/function metadata required by the activity. Network routing, gateway registration, SAP authorizations and partner profiles must be provisioned by the SAP team. MINA is not SAP-certified by virtue of using JCo.

Executes the selected SAP ECC operation.

The shared SAP connection supplies system and authentication settings. IDoc metadata selected from SAP defines listener, parser, renderer, and sender structures.

Activity type: sap; operation: idoc_parser.

Configuration

Configuration procedure

Fetch/save the matching basic type, extension and release. Choose JSON for field mappings or XML for serialized output. For flat data set the actual encoding/framing; keep strict invalid-byte handling unless replacement is intentional.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
SAP ECC shared connection resourceId	resource Configuration setting	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: sap
IDoc source destination sourceDestination	text Configuration setting	Editor metadata describing the IDoc source. The native parser/reader does not use this value as a separate network destination; select the actual shared resource.
IDoc type fetched from SAP idocType	idoc Configuration setting	Select and save the basic IDoc type fetched from the chosen SAP connection. The saved catalog/schema drives segment layout, field offsets and mapping suggestions.
Extension / CIM type extensionType	text Configuration setting	Optional IDoc extension/CIM type. It must match the sender/receiver contract; changing it requires refreshing metadata.
SAP release release	text Configuration setting	SAP release used to resolve IDoc metadata. Use the release supported by the connected system, not an arbitrary version string.
Flat IDoc encoding idocEncoding	text Configuration setting	Character set used to decode flat records and SDATA byte offsets, for example UTF-8 or windows-1252. Must match the actual sender encoding.

Field / key	Type / required	Details
Flat record delimiter idocRecordDelimiter	text Configuration setting	Explicit escaped record separator for flat input, such as \r\n. Use fixed-width/custom framing when segment data itself contains line breaks.
Invalid byte policy invalidCharacterPolicy	select Configuration setting	strict rejects invalid byte sequences; replace substitutes invalid characters and can lose data. Use strict when data fidelity is required. Choices: strict, replace
Parser engine parserEngine	select Configuration setting	Built-in parses XML, structured and supported flat records. The optional licensed IDoc library path validates XML; it does not implement flat-byte parsing. Choices: Built-in, SAP JIDocLib (when installed)
Reject mismatched IDoc type or extension validateIdocType	boolean Configuration setting	Checks available control-record basic type and configured extension against saved metadata. Missing metadata is not proof the payload was validated.
IDoc output format idocOutputMode	select Configuration setting	Parser: JSON or XML. Converter also exposes Raw; RAW returns the original source rather than manufacturing transport-ready EDI records. Choices: JSON, XML

Input

Mapping example

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
IDoc = ${SAP-Listener.payload}
SAPIDoc = ${SAP-Listener.controlRecord}
idocOutputMode = JSON
Next message = ${SAP-Parser.SAPIDoc}
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
SAP IDoc control record SAPIDoc	object Required	For parser input, control metadata or a structured IDoc. Parser output contains one canonical basic-type branch, for example SAPIDoc.ARTMAS05.IDOC. Legacy direct paths remain available for mapping compatibility but are not duplicated in serialized JSON.
Raw IDoc IDoc	string|object Required	Raw IDoc input: XML text, supported structured control/data records or flat IDoc bytes/text. Do not map only one segment when the operation needs the complete document.
IDoc source destination sourceDestination	string Optional	Editor metadata describing the IDoc source. The native parser/reader does not use this value as a separate network destination; select the actual shared resource.
Extension / CIM type extensionType	string Optional	Optional IDoc extension/CIM type. It must match the sender/receiver contract; changing it requires refreshing metadata.
SAP release release	string Optional	SAP release used to resolve IDoc metadata. Use the release supported by the connected system, not an arbitrary version string.

Field / key	Type / required	Details
Flat IDoc encoding idocEncoding	string Optional	Character set used to decode flat records and SDATA byte offsets, for example UTF-8 or windows-1252. Must match the actual sender encoding.
Flat record delimiter idocRecordDelimiter	string Optional	Explicit escaped record separator for flat input, such as \r\n. Use fixed-width/custom framing when segment data itself contains line breaks.
Invalid byte policy invalidCharacterPolicy	string Optional	strict rejects invalid byte sequences; replace substitutes invalid characters and can lose data. Use strict when data fidelity is required.
Parser engine parserEngine	string Optional	Built-in parses XML, structured and supported flat records. The optional licensed IDoc library path validates XML; it does not implement flat-byte parsing.
Reject mismatched IDoc type or extension validateIdocType	boolean Optional	Checks available control-record basic type and configured extension against saved metadata. Missing metadata is not proof the payload was validated.
IDoc output format idocOutputMode	string Optional	Parser: JSON or XML. Converter also exposes Raw; RAW returns the original source rather than manufacturing transport-ready EDI records.

Output

Illustrative runtime output

```
{"format":"JSON","idocType":"<basic-type>","SAPIDoc":{"IDOC":{"EDI_DC40":{},"<repeating-segment>":[{"<field>":"value"}]}}}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
Parsed IDoc JSON SAPIDoc	object Optional	For parser input, control metadata or a structured IDoc. Parser output contains one canonical basic-type branch, for example SAPIDoc.ARTMAS05.IDOC. Legacy direct paths remain available for mapping compatibility but are not duplicated in serialized JSON.
Output format format	string Optional	Output representation selected by the parser/converter: JSON, XML or RAW. Map the accompanying content field appropriate for that format.

Advanced

Delivery, transactions and performance

SAP transport success and business success are different. Inspect BAPI RETURN/status data even when the RFC call completes. Use bounded payloads and batches; parsing/rendering materializes documents in memory. Persistent outbound workers reduce JVM startup overhead but do not eliminate provider/network latency. Use a live Dynamic Connection session only within its owning runtime; never persist its ID for later reuse.

Implemented behavior and limitations

Illustrative shape only: actual paths come from your fetched schema. Repeating segments remain arrays even for one occurrence; optional absent fields are not minaated. The runtime also exposes the basic-type-named branch. The licensed IDoc-library option validates XML only; flat records require Built-in. Missing metadata can prevent SDATA expansion, and parser memory grows with document size.

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Troubleshooting procedure

SAP_CONNECTION: verify native-library loading, routing and logon credentials. SAP_RFC: inspect the function name, parameter metadata and SAP authorization/ABAP error. SAP_IDOC or SAP_SCHEMA: compare basic type, extension, release and saved field offsets with the actual payload. SAP_TRANSACTION: inspect TID/queue/context state before retrying. Some underlying paths raise generic runtime errors instead of the advertised editor fault code; retain the full message and trace.

Acceptance test

Acceptance: test one, multiple and absent repeating segments; malformed XML; mismatched type/extension; non-UTF-8 bytes; and segment data containing line breaks.

Fault	Description
SAP_CONNECTION	The SAP JCo/RFC connection could not be established.
SAP_RFC	SAP returned an RFC or BAPI exception.
SAP_IDOC	The IDoc could not be parsed, posted, confirmed, or acknowledged.
SAP_TRANSACTION	The tRFC, qRFC, bgRFC, or transactional context failed.
SAP_SCHEMA	The downloaded RFC, table, or IDoc metadata is invalid.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs



IDoc Reader

Activities / SAP ECC

Overview

Dispatch an IDoc using queued RFC

Despite its label, the current runtime converts input to control/data tables and calls the inbound-IDoc RFC. It does not fetch an existing IDoc by document number.

Prerequisites

Install licensed SAP JCo JAR and the OS/architecture-matched native library on the execution host; optional IDoc-library validation also needs its licensed JAR. Configure the SAP shared resource, test connectivity, fetch and save the exact IDoc/function metadata required by the activity. Network routing, gateway registration, SAP authorizations and partner profiles must be provisioned by the SAP team. MINA is not SAP-certified by virtue of using JCo.

Executes the selected SAP ECC operation.

The shared SAP connection supplies system and authentication settings. IDoc metadata selected from SAP defines listener, parser, renderer, and sender structures.

Activity type: sap; operation: idoc_reader.

Configuration

Configuration procedure

Select metadata and SAP connection, provide a complete input IDoc, and configure queueName in resolved settings for qRFC ordering. Reader defaults to qRFC; the editor does not expose every dispatch setting.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
SAP ECC shared connection resourceId	resource Configuration setting	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: sap
IDoc source destination sourceDestination	text Configuration setting	Editor metadata describing the IDoc source. The native parser/reader does not use this value as a separate network destination; select the actual shared resource.
IDoc type fetched from SAP idocType	idoc Configuration setting	Select and save the basic IDoc type fetched from the chosen SAP connection. The saved catalog/schema drives segment layout, field offsets and mapping suggestions.
Extension / CIM type extensionType	text Configuration setting	Optional IDoc extension/CIM type. It must match the sender/receiver contract; changing it requires refreshing metadata.
SAP release release	text Configuration setting	SAP release used to resolve IDoc metadata. Use the release supported by the connected system, not an arbitrary version string.
Confirmation mode confirmationMode	text Configuration setting	Editor setting; the current native adapter does not implement a separate generic confirmation-mode switch. Read the operation-specific limitations below.

Input

Mapping example

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
payload = ${SAP-Renderer.rawIDoc}
Resolved queueName = ${properties.sap.outboundQueue}
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
SAP activity payload payload	object Required	Map the whole input document expected by this operation. Use the Data tree to select an earlier output; the examples below describe its required shape.
Dynamic connection session ID sessionID	string Optional	Map the sessionID returned by Dynamic Connection. It identifies a live, process-local JCo context; do not hard-code or reuse it after termination or restart.
IDoc source destination sourceDestination	string Optional	Editor metadata describing the IDoc source. The native parser/reader does not use this value as a separate network destination; select the actual shared resource.
Extension / CIM type extensionType	string Optional	Optional IDoc extension/CIM type. It must match the sender/receiver contract; changing it requires refreshing metadata.
SAP release release	string Optional	SAP release used to resolve IDoc metadata. Use the release supported by the connected system, not an arbitrary version string.
Confirmation mode confirmationMode	string Optional	Editor setting; the current native adapter does not implement a separate generic confirmation-mode switch. Read the operation-specific limitations below.

Output

Illustrative runtime output

```
{"TID":"<transaction>","transactional":true,"transactionProtocol":"qrfc","confirmed":true}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
SAP response result	object Optional	Generic editor result placeholder. Native SAP calls return exports, changing parameters, tables and operation-specific metadata; inspect actual Job Data before mapping result.
Completion status successful	boolean Optional	Generic editor completion indicator. See the runtime output example: several native SAP operations return operation-specific fields instead of this generic field.

Advanced

Delivery, transactions and performance

SAP transport success and business success are different. Inspect BAPI RETURN/status data even when the RFC call completes. Use bounded payloads and batches; parsing/rendering materializes documents in memory. Persistent outbound workers reduce JVM startup overhead but do not eliminate provider/network latency. Use a live Dynamic Connection session only within its owning runtime; never persist its ID for later reuse.

Implemented behavior and limitations

Without queueName the bridge falls back to a transactional execution overload without the intended queue name. Treat this as a dispatch activity, not historical retrieval. Both EDI_DC40 control data and at least one EDI_DD40 segment are required.

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Troubleshooting procedure

SAP_CONNECTION: verify native-library loading, routing and logon credentials. SAP_RFC: inspect the function name, parameter metadata and SAP authorization/ABAP error. SAP_IDOC or SAP_SCHEMA: compare basic type, extension, release and saved field offsets with the actual payload. SAP_TRANSACTION: inspect TID/queue/context state before retrying. Some underlying paths raise generic runtime errors instead of the advertised editor fault code; retain the full message and trace.

Acceptance test

Acceptance: verify the queued outbound dispatch and ordering in SAP; do not test it against production expecting a read-only operation.

Fault	Description
SAP_CONNECTION	The SAP JCo/RFC connection could not be established.
SAP_RFC	SAP returned an RFC or BAPI exception.
SAP_IDOC	The IDoc could not be parsed, posted, confirmed, or acknowledged.
SAP_TRANSACTION	The tRFC, qRFC, bgRFC, or transactional context failed.
SAP_SCHEMA	The downloaded RFC, table, or IDoc metadata is invalid.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs



Post IDoc to SAP

Activities / SAP ECC

Overview

Post a business IDoc to SAP

Use inside a flow to dispatch a complete IDoc using the selected SAP metadata and transport protocol.

Prerequisites

Install licensed SAP JCo JAR and the OS/architecture-matched native library on the execution host; optional IDoc-library validation also needs its licensed JAR. Configure the SAP shared resource, test connectivity, fetch and save the exact IDoc/function metadata required by the activity. Network routing, gateway registration, SAP authorizations and partner profiles must be provisioned by the SAP team. MINA is not SAP-certified by virtue of using JCo.

Executes the selected SAP ECC operation.

The shared SAP connection supplies system and authentication settings. IDoc metadata selected from SAP defines listener, parser, renderer, and sender structures.

Activity type: sap; operation: post_idoc.

Configuration

Configuration procedure

Select SAP resource and saved basic type/extension. Map XML, structured IDoc or rendered control/data envelope. Select tRFC (default) or qRFC and supply queueName for ordered dispatch.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
SAP ECC shared connection resourceId	resource Configuration setting	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: sap
IDoc type fetched from SAP idocType	idoc Configuration setting	Select and save the basic IDoc type fetched from the chosen SAP connection. The saved catalog/schema drives segment layout, field offsets and mapping suggestions.
Extension / CIM type extensionType	text Configuration setting	Optional IDoc extension/CIM type. It must match the sender/receiver contract; changing it requires refreshing metadata.
SAP release release	text Configuration setting	SAP release used to resolve IDoc metadata. Use the release supported by the connected system, not an arbitrary version string.
IDoc delivery protocol idocInputMode	select Configuration setting	IDoc delivery protocol. Post defaults to tRFC; Reader defaults to qRFC. qRFC requires a real queue name for ordered dispatch. Choices: tRFC, qRFC
IDoc payload format inputFormat	select Configuration setting	Describes the intended XML or raw input for posting; the native adapter converts the actual input using saved metadata. Choices: XML, Raw

Field / key	Type / required	Details
Confirmation mode confirmationMode	text Configuration setting	Editor setting; the current native adapter does not implement a separate generic confirmation-mode switch. Read the operation-specific limitations below.
SAP queue name queueName	text Configuration setting	SAP outbound qRFC queue name. Without a queue name the bridge cannot establish the intended ordered queue execution.

Input

Mapping example

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
payload = ${SAP-Renderer.rawIDoc}
idocInputMode = tRFC
For qRFC: queueName = ${properties.sap.outboundQueue}
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
SAP activity payload payload	object Required	Map the whole input document expected by this operation. Use the Data tree to select an earlier output; the examples below describe its required shape.
Dynamic connection session ID sessionID	string Optional	Map the sessionID returned by Dynamic Connection. It identifies a live, process-local JCo context; do not hard-code or reuse it after termination or restart.
Extension / CIM type extensionType	string Optional	Optional IDoc extension/CIM type. It must match the sender/receiver contract; changing it requires refreshing metadata.
SAP release release	string Optional	SAP release used to resolve IDoc metadata. Use the release supported by the connected system, not an arbitrary version string.
IDoc delivery protocol idocInputMode	string Optional	IDoc delivery protocol. Post defaults to tRFC; Reader defaults to qRFC. qRFC requires a real queue name for ordered dispatch.
IDoc payload format inputFormat	string Optional	Describes the intended XML or raw input for posting; the native adapter converts the actual input using saved metadata.
Confirmation mode confirmationMode	string Optional	Editor setting; the current native adapter does not implement a separate generic confirmation-mode switch. Read the operation-specific limitations below.
SAP queue name queueName	string Optional	SAP outbound qRFC queue name. Without a queue name the bridge cannot establish the intended ordered queue execution.

Output

Illustrative runtime output

```
{"TID":"<transaction>","transactional":true,"transactionProtocol":"trfc","confirmed":true}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
SAP response result	object Optional	Generic editor result placeholder. Native SAP calls return exports, changing parameters, tables and operation-specific metadata; inspect actual Job Data before mapping result.
Completion status successful	boolean Optional	Generic editor completion indicator. See the runtime output example: several native SAP operations return operation-specific fields instead of this generic field.

Advanced

Delivery, transactions and performance

SAP transport success and business success are different. Inspect BAPI RETURN/status data even when the RFC call completes. Use bounded payloads and batches; parsing/rendering materializes documents in memory. Persistent outbound workers reduce JVM startup overhead but do not eliminate provider/network latency. Use a live Dynamic Connection session only within its owning runtime; never persist its ID for later reuse.

Implemented behavior and limitations

The adapter builds IDOC_CONTROL_REC_40 and IDOC_DATA_REC_40 table parameters and calls IDOC_INBOUND_ASYNCHRONOUS unless functionName overrides it. confirmed indicates transport confirmation, not final SAP business posting status. Retrying can create another transport transaction; use a business-level duplicate strategy.

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false

Field / key	Type / required	Details
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Troubleshooting procedure

SAP_CONNECTION: verify native-library loading, routing and logon credentials. SAP_RFC: inspect the function name, parameter metadata and SAP authorization/ABAP error. SAP_IDOC or SAP_SCHEMA: compare basic type, extension, release and saved field offsets with the actual payload. SAP_TRANSACTION: inspect TID/queue/context state before retrying. Some underlying paths raise generic runtime errors instead of the advertised editor fault code; retain the full message and trace.

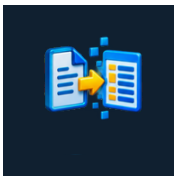
Acceptance test

Acceptance: inspect SAP inbound document status as well as transport success; test a rejected business document and ambiguous network failure.

Fault	Description
SAP_CONNECTION	The SAP JCo/RFC connection could not be established.
SAP_RFC	SAP returned an RFC or BAPI exception.
SAP_IDOC	The IDoc could not be parsed, posted, confirmed, or acknowledged.
SAP_TRANSACTION	The tRFC, qRFC, bgRFC, or transactional context failed.
SAP_SCHEMA	The downloaded RFC, table, or IDoc metadata is invalid.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs



IDoc Renderer

Activities / SAP ECC

Overview

Build transport-ready control/data records

Use to turn structured/XML IDoc data into the control/data envelope used by Post IDoc or Reader.

Prerequisites

Install licensed SAP JCo JAR and the OS/architecture-matched native library on the execution host; optional IDoc-library validation also needs its licensed JAR. Configure the SAP shared resource, test connectivity, fetch and save the exact IDoc/function metadata required by the activity. Network routing, gateway registration, SAP authorizations and partner profiles must be provisioned by the SAP team. MINA is not SAP-certified by virtue of using JCo.

Executes the selected SAP ECC operation.

The shared SAP connection supplies system and authentication settings. IDoc metadata selected from SAP defines listener, parser, renderer, and sender structures.

Activity type: sap; operation: idoc_renderer.

Configuration

Configuration procedure

Select the exact metadata and map the complete document. The saved field offsets, lengths and encoding govern SDATA construction. This operation renders locally; it does not post to SAP.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
SAP ECC shared connection resourceId	resource Configuration setting	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: sap
IDoc type fetched from SAP idocType	idoc Configuration setting	Select and save the basic IDoc type fetched from the chosen SAP connection. The saved catalog/schema drives segment layout, field offsets and mapping suggestions.
Extension / CIM type extensionType	text Configuration setting	Optional IDoc extension/CIM type. It must match the sender/receiver contract; changing it requires refreshing metadata.
SAP release release	text Configuration setting	SAP release used to resolve IDoc metadata. Use the release supported by the connected system, not an arbitrary version string.
IDoc delivery protocol idocInputMode	select Configuration setting	IDoc delivery protocol. Post defaults to tRFC; Reader defaults to qRFC. qRFC requires a real queue name for ordered dispatch. Choices: tRFC, qRFC

Input

Mapping example

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
payload = ${SAP-Parser.SAPIDoc}
Post IDoc payload = ${SAP-Renderer.rawIDoc}
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
SAP activity payload payload	object Required	Map the whole input document expected by this operation. Use the Data tree to select an earlier output; the examples below describe its required shape.
Dynamic connection session ID sessionID	string Optional	Map the sessionID returned by Dynamic Connection. It identifies a live, process-local JCo context; do not hard-code or reuse it after termination or restart.
Extension / CIM type extensionType	string Optional	Optional IDoc extension/CIM type. It must match the sender/receiver contract; changing it requires refreshing metadata.
SAP release release	string Optional	SAP release used to resolve IDoc metadata. Use the release supported by the connected system, not an arbitrary version string.
IDoc delivery protocol idocInputMode	string Optional	IDoc delivery protocol. Post defaults to tRFC; Reader defaults to qRFC. qRFC requires a real queue name for ordered dispatch.

Output

Illustrative runtime output

```
{"rawIDoc":{"control":{"IDOCTYP":"<basic-type>"},"data":[{"SEGNAM":"<segment>","SDATA":"<encoded-fields
>"}]},"format":"RAW"}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
SAP response result	object Optional	Generic editor result placeholder. Native SAP calls return exports, changing parameters, tables and operation-specific metadata; inspect actual Job Data before mapping result.
Completion status successful	boolean Optional	Generic editor completion indicator. See the runtime output example: several native SAP operations return operation-specific fields instead of this generic field.

Advanced

Delivery, transactions and performance

SAP transport success and business success are different. Inspect BAPI RETURN/status data even when the RFC call completes. Use bounded payloads and batches; parsing/rendering materializes documents in memory. Persistent outbound workers reduce JVM startup overhead but do not eliminate provider/network latency. Use a live Dynamic Connection session only within its owning runtime; never persist its ID for later reuse.

Implemented behavior and limitations

rawIDoc is a structured control/data envelope, not necessarily a flat text file. The base editor advertises generic result/successful fields; use actual rawIDoc output. Ensure input fits the fetched field lengths and test encoding before dispatch.

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Troubleshooting procedure

SAP_CONNECTION: verify native-library loading, routing and logon credentials. SAP_RFC: inspect the function name, parameter metadata and SAP authorization/ABAP error. SAP_IDOC or SAP_SCHEMA: compare basic type, extension, release and saved field offsets with the actual payload. SAP_TRANSACTION: inspect TID/queue/context state before retrying. Some underlying paths raise generic runtime errors instead of the advertised editor fault code; retain the full message and trace.

Acceptance test

Acceptance: render and parse back a fixture with repeating segments, blank fields and non-ASCII text; compare every business field.

Fault	Description
SAP_CONNECTION	The SAP JCo/RFC connection could not be established.
SAP_RFC	SAP returned an RFC or BAPI exception.
SAP_IDOC	The IDoc could not be parsed, posted, confirmed, or acknowledged.
SAP_TRANSACTION	The tRFC, qRFC, bgRFC, or transactional context failed.
SAP_SCHEMA	The downloaded RFC, table, or IDoc metadata is invalid.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs



RFC BAPI Listener

Activities / SAP ECC

Overview

Start a flow for an inbound RFC/BAPI call

Use as an event starter registered with the SAP gateway for a selected function.

Prerequisites

Install licensed SAP JCo JAR and the OS/architecture-matched native library on the execution host; optional IDoc-library validation also needs its licensed JAR. Configure the SAP shared resource, test connectivity, fetch and save the exact IDoc/function metadata required by the activity. Network routing, gateway registration, SAP authorizations and partner profiles must be provisioned by the SAP team. MINA is not SAP-certified by virtue of using JCo.

Executes the selected SAP ECC operation.

The shared SAP connection supplies system and authentication settings. IDoc metadata selected from SAP defines listener, parser, renderer, and sender structures.

Activity type: sap; operation: rfc_bapi_listener.

Configuration

Configuration procedure

Set functionName, programId and gateway settings. Provision the matching SAP destination and registration rules. Bound maximumConnections/maxPendingEvents and choose a reply timeout long enough for the flow. Fetch the function metadata.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
SAP ECC shared connection resourceId	resource Configuration setting	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: sap
RFC / BAPI name functionName	text Configuration setting	Exact remote-enabled function/BAPI name available in the target SAP repository. Its case and parameter names must match fetched metadata.
Invocation protocol invocationProtocol	select Configuration setting	Editor protocol selection. Native bridge implements synchronous RFC and tRFC/qRFC dispatch; bgRFC is not a separately implemented native execution mode. Choices: Request/Reply, tRFC, qRFC, bgRFC
Program ID programId	text Configuration setting	Registered-server program ID. Must match the SAP RFC destination and gateway registration permissions.
Gateway host override gatewayHost	text Configuration setting	SAP gateway hostname override for inbound RFC registration.
Gateway service override gatewayService	text Configuration setting	Gateway service/port override for inbound RFC registration.
Maximum concurrent requests maximumConnections	number Configuration setting	Listener connection/concurrency limit. Size it with SAP gateway capacity and downstream processing, not just CPU count.

Field / key	Type / required	Details
Maximum buffered requests maxPendingEvents	number Configuration setting	Bound on buffered listener events; prevents unbounded in-memory accumulation while downstream work is slow.
Reply / acknowledgement timeout (seconds) ackTimeoutSeconds	number Configuration setting	Maximum listener reply/acknowledgement wait in seconds. Set it above expected flow processing time and test timeout/redelivery behavior.
SAP TIDManager resource tidManagerId	resource Configuration setting	Shared SAP transaction-ID store selection. Durable TID state supports duplicate detection; it does not make downstream external writes atomic. Connection type: sap_tid

Input

Mapping example

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
Downstream input = ${SAP-RFC-Listener.imports}
Downstream tables = ${SAP-RFC-Listener.tables}
End response with Reply from RFC BAPI in SAP
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
RFC / BAPI name functionName	string Optional	Exact remote-enabled function/BAPI name available in the target SAP repository. Its case and parameter names must match fetched metadata.
Invocation protocol invocationProtocol	string Optional	Editor protocol selection. Native bridge implements synchronous RFC and tRFC/qRFC dispatch; bgRFC is not a separately implemented native execution mode.
Program ID programId	string Optional	Registered-server program ID. Must match the SAP RFC destination and gateway registration permissions.
Gateway host override gatewayHost	string Optional	SAP gateway hostname override for inbound RFC registration.
Gateway service override gatewayService	string Optional	Gateway service/port override for inbound RFC registration.
Maximum concurrent requests maximumConnections	number Optional	Listener connection/concurrency limit. Size it with SAP gateway capacity and downstream processing, not just CPU count.
Maximum buffered requests maxPendingEvents	number Optional	Bound on buffered listener events; prevents unbounded in-memory accumulation while downstream work is slow.
Reply / acknowledgement timeout (seconds) ackTimeoutSeconds	number Optional	Maximum listener reply/acknowledgement wait in seconds. Set it above expected flow processing time and test timeout/redelivery behavior.

Output

Illustrative runtime output

```
{"RfcRequest":{},"imports":{"<parameter>":"value"},"tables":{},"functionName":"<RFC>","TID":"<when-present>"}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
RFC/BAPI request RfcRequest	object Optional	Inbound RFC request envelope. Parameter shapes follow the selected SAP function metadata.
RFC import parameters imports	object Optional	Inbound RFC import parameter object.
RFC table parameters tables	object Optional	Inbound RFC table parameter arrays.
Invoked RFC/BAPI functionName	string Optional	Exact remote-enabled function/BAPI name available in the target SAP repository. Its case and parameter names must match fetched metadata.
Invocation protocol invocationProtocol	string Optional	Editor protocol selection. Native bridge implements synchronous RFC and tRFC/qRFC dispatch; bgRFC is not a separately implemented native execution mode.
Transaction ID TID	string Optional	SAP transaction identifier when provided by the transport. It is not the IDoc business document number and is not a JMS acknowledgement token.
SAP connection ID CPIC_ID	string Optional	Connection identifier supplied by SAP transport when available.

Advanced

Delivery, transactions and performance

SAP transport success and business success are different. Inspect BAPI RETURN/status data even when the RFC call completes. Use bounded payloads and batches; parsing/rendering materializes documents in memory. Persistent outbound workers reduce JVM startup overhead but do not eliminate provider/network latency. Use a live Dynamic Connection session only within its owning runtime; never persist its ID for later reuse.

Implemented behavior and limitations

A request/reply caller waits for the flow response. Reply must execute within the originating transport context. bgRFC appears in the editor but there is no dedicated bgRFC native dispatch implementation to claim as supported.

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Troubleshooting procedure

SAP_CONNECTION: verify native-library loading, routing and logon credentials. SAP_RFC: inspect the function name, parameter metadata and SAP authorization/ABAP error. SAP_IDOC or SAP_SCHEMA: compare basic type, extension,

release and saved field offsets with the actual payload. SAP_TRANSACTION: inspect TID/queue/context state beforeretrying. Some underlying paths raise generic runtime errors instead of the advertised editor fault code; retain the fullmessage and trace.

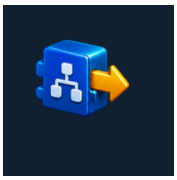
Acceptance test

Acceptance: invoke from a test SAP caller, inspect imports/tables, return a known value and test timeout/error propagation and concurrent callers.

Fault	Description
SAP_CONNECTION	The SAP JCo/RFC connection could not be established.
SAP_RFC	SAP returned an RFC or BAPI exception.
SAP_IDOC	The IDoc could not be parsed, posted, confirmed, or acknowledged.
SAP_TRANSACTION	The tRFC, qRFC, bgRFC, or transactional context failed.
SAP_SCHEMA	The downloaded RFC, table, or IDoc metadata is invalid.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs



Invoke RFC BAPI in SAP

Activities / SAP ECC

Overview

Call a remote-enabled SAP function

Use inside a flow to pass import/changing/table parameters to a remote-enabled function or BAPI.

Prerequisites

Install licensed SAP JCo JAR and the OS/architecture-matched native library on the execution host; optional IDoc-library validation also needs its licensed JAR. Configure the SAP shared resource, test connectivity, fetch and save the exact IDoc/function metadata required by the activity. Network routing, gateway registration, SAP authorizations and partner profiles must be provisioned by the SAP team. MINA is not SAP-certified by virtue of using JCo.

Executes the selected SAP ECC operation.

The shared SAP connection supplies system and authentication settings. IDoc metadata selected from SAP defines listener, parser, renderer, and sender structures.

Activity type: sap; operation: invoke_rfc_bapi.

Configuration

Configuration procedure

Select SAP resource and exact functionName. Match the fetched parameter types. For a multi-step logical unit of work map sessionID from Dynamic Connection and close the context explicitly.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
SAP ECC shared connection resourceId	resource Configuration setting	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: sap
RFC / BAPI name functionName	text Configuration setting	Exact remote-enabled function/BAPI name available in the target SAP repository. Its case and parameter names must match fetched metadata.
Invocation protocol invocationProtocol	select Configuration setting	Editor protocol selection. Native bridge implements synchronous RFC and tRFC/qRFC dispatch; bgRFC is not a separately implemented native execution mode. Choices: Request/Reply, tRFC, qRFC, bgRFC
SAP queue name queueName	text Configuration setting	SAP outbound qRFC queue name. Without a queue name the bridge cannot establish the intended ordered queue execution.
Transactional transactional	boolean Configuration setting	Whether a stateful transactional context is requested. Only operations sharing the returned live sessionID participate in that context.
Auto commit autoCommit	boolean Configuration setting	Requests BAPI_TRANSACTION_COMMIT for an eligible call. Calls inside an explicit dynamic context suppress per-activity commits until the context boundary.

Field / key	Type / required	Details
Context end contextEnd	boolean Configuration setting	Ends the dynamic context after this call; autoCommit controls whether that explicit boundary commits. Do not reuse the session afterwards.
Timeout (ms) timeout	number Configuration setting	Unit depends on the activity: Kafka Get uses seconds; SAP editor fields use milliseconds. Consult the runtime notes because SAP worker calls use timeoutSeconds separately.

Input

Mapping example

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
functionName = <authorized remote function>
importParameters = ${input.rfcImports}
changingParameters = ${input.rfcChanging}
tableParameters = ${input.rfcTables}
sessionID = ${SAP-Open.sessionID}
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
RFC import parameters importParameters	object Optional	Object keyed by exact RFC import parameter names; nested structures are flattened into named JCo parameters.
RFC changing parameters changingParameters	object Optional	Object keyed by RFC changing parameter names. Values are sent through the JCo changing-parameter list.
RFC table parameters tableParameters	object Optional	Object whose keys are RFC table parameter names and whose values are arrays of row objects, including single-row tables.
Dynamic connection session ID sessionID	string Optional	Map the sessionID returned by Dynamic Connection. It identifies a live, process-local JCo context; do not hard-code or reuse it after termination or restart.
RFC / BAPI name functionName	string Optional	Exact remote-enabled function/BAPI name available in the target SAP repository. Its case and parameter names must match fetched metadata.
Invocation protocol invocationProtocol	string Optional	Editor protocol selection. Native bridge implements synchronous RFC and tRFC/qRFC dispatch; bgRFC is not a separately implemented native execution mode.
SAP queue name queueName	string Optional	SAP outbound qRFC queue name. Without a queue name the bridge cannot establish the intended ordered queue execution.
Transactional transactional	boolean Optional	Whether a stateful transactional context is requested. Only operations sharing the returned live sessionID participate in that context.
Auto commit autoCommit	boolean Optional	Requests BAPI_TRANSACTION_COMMIT for an eligible call. Calls inside an explicit dynamic context suppress per-activity commits until the context boundary.

Field / key	Type / required	Details
Context end contextEnd	boolean Optional	Ends the dynamic context after this call; autoCommit controls whether that explicit boundary commits. Do not reuse the session afterwards.
Timeout (ms) timeout	number Optional	Unit depends on the activity: Kafka Get uses seconds; SAP editor fields use milliseconds. Consult the runtime notes because SAP worker calls use timeoutSeconds separately.

Output

Illustrative runtime output

```
{"exports":{"<parameter>":"value"},"changing":{},"tables":{"RETURN":[{"TYPE":"S","MESSAGE":"<message>"}]
}}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
SAP response result	object Optional	Generic editor result placeholder. Native SAP calls return exports, changing parameters, tables and operation-specific metadata; inspect actual Job Data before mapping result.
Completion status successful	boolean Optional	Generic editor completion indicator. See the runtime output example: several native SAP operations return operation-specific fields instead of this generic field.

Advanced

Delivery, transactions and performance

SAP transport success and business success are different. Inspect BAPI RETURN/status data even when the RFC call completes. Use bounded payloads and batches; parsing/rendering materializes documents in memory. Persistent outbound workers reduce JVM startup overhead but do not eliminate provider/network latency. Use a live Dynamic Connection session only within its owning runtime; never persist its ID for later reuse.

Implemented behavior and limitations

RETURN is illustrative: not every function has that table. The runtime returns native exports/changing/tables rather than wrapping every result in result. Check business error fields explicitly. bgRFC is not a dedicated supported native path; worker timeout uses timeoutSeconds, so the editor timeout field alone should not be treated as a verified per-call deadline.

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"

Field / key	Type / required	Details
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Troubleshooting procedure

SAP_CONNECTION: verify native-library loading, routing and logon credentials. SAP_RFC: inspect the function name, parameter metadata and SAP authorization/ABAP error. SAP_IDOC or SAP_SCHEMA: compare basic type, extension, release and saved field offsets with the actual payload. SAP_TRANSACTION: inspect TID/queue/context state before retrying. Some underlying paths raise generic runtime errors instead of the advertised editor fault code; retain the full message and trace.

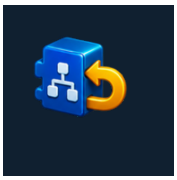
Acceptance test

Acceptance: test scalar, nested and table parameters, business error returns, rollback/commit boundaries and a disconnected server.

Fault	Description
SAP_CONNECTION	The SAP JCo/RFC connection could not be established.
SAP_RFC	SAP returned an RFC or BAPI exception.
SAP_IDOC	The IDoc could not be parsed, posted, confirmed, or acknowledged.
SAP_TRANSACTION	The tRFC, qRFC, bgRFC, or transactional context failed.
SAP_SCHEMA	The downloaded RFC, table, or IDoc metadata is invalid.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs



Reply from RFC BAPI in SAP

Activities / SAP ECC

Overview

Complete the originating RFC request

Use in a flow started by RFC BAPI Listener to send its output parameters back to the waiting SAP caller.

Prerequisites

Install licensed SAP JCo JAR and the OS/architecture-matched native library on the execution host; optional IDoc-library validation also needs its licensed JAR. Configure the SAP shared resource, test connectivity, fetch and save the exact IDoc/function metadata required by the activity. Network routing, gateway registration, SAP authorizations and partner profiles must be provisioned by the SAP team. MINA is not SAP-certified by virtue of using JCo.

Executes the selected SAP ECC operation.

The shared SAP connection supplies system and authentication settings. IDoc metadata selected from SAP defines listener, parser, renderer, and sender structures.

Activity type: sap; operation: reply_rfc_bapi.

Configuration

Configuration procedure

Map the response to the function output contract. The engine requires the original deliveryId/listenerKey transport context; functionName/listenerActivity alone cannot reply to a request from another job.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
SAP ECC shared connection resourceId	resource Configuration setting	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: sap
RFC / BAPI name functionName	text Configuration setting	Exact remote-enabled function/BAPI name available in the target SAP repository. Its case and parameter names must match fetched metadata.
RFC BAPI Listener activity listenerActivity	text Configuration setting	Design-time reference to the listener. Runtime reply routing uses the current listener transport context, not an arbitrary activity name.

Input

Mapping example

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
payload = ${input.rfcResponse}
Illustrative response: {"exports":{"RESULT":"OK"},"tables":{}}
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
SAP activity payload payload	object Required	Map the whole input document expected by this operation. Use the Data tree to select an earlier output; the examples below describe its required shape.
Dynamic connection session ID sessionID	string Optional	Map the sessionID returned by Dynamic Connection. It identifies a live, process-local JCo context; do not hard-code or reuse it after termination or restart.
RFC / BAPI name functionName	string Optional	Exact remote-enabled function/BAPI name available in the target SAP repository. Its case and parameter names must match fetched metadata.
RFC BAPI Listener activity listenerActivity	string Optional	Design-time reference to the listener. Runtime reply routing uses the current listener transport context, not an arbitrary activity name.

Output

Illustrative runtime output

```
{"replied":true,"functionName":"<RFC>","response":{"exports":{"RESULT":"OK"}}}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
SAP response result	object Optional	Generic editor result placeholder. Native SAP calls return exports, changing parameters, tables and operation-specific metadata; inspect actual Job Data before mapping result.
Completion status successful	boolean Optional	Generic editor completion indicator. See the runtime output example: several native SAP operations return operation-specific fields instead of this generic field.

Advanced

Delivery, transactions and performance

SAP transport success and business success are different. Inspect BAPI RETURN/status data even when the RFC call completes. Use bounded payloads and batches; parsing/rendering materializes documents in memory. Persistent outbound workers reduce JVM startup overhead but do not eliminate provider/network latency. Use a live Dynamic Connection session only within its owning runtime; never persist its ID for later reuse.

Implemented behavior and limitations

Running this activity alone in external mode fails without a live listener context. Mock replied=true does not establish an SAP response. Reply within the configured timeout and avoid sending a second response on another branch.

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Troubleshooting procedure

SAP_CONNECTION: verify native-library loading, routing and logon credentials. SAP_RFC: inspect the function name, parameter metadata and SAP authorization/ABAP error. SAP_IDOC or SAP_SCHEMA: compare basic type, extension, release and saved field offsets with the actual payload. SAP_TRANSACTION: inspect TID/queue/context state before retrying. Some underlying paths raise generic runtime errors instead of the advertised editor fault code; retain the full message and trace.

Acceptance test

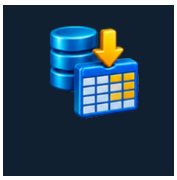
Acceptance: verify the exact output in the waiting SAP caller, then test a late reply and a reply outside a listener flow.

Fault	Description
SAP_CONNECTION	The SAP JCo/RFC connection could not be established.
SAP_RFC	SAP returned an RFC or BAPI exception.
SAP_IDOC	The IDoc could not be parsed, posted, confirmed, or acknowledged.

Fault	Description
SAP_TRANSACTION	The tRFC, qRFC, bgRFC, or transactional context failed.
SAP_SCHEMA	The downloaded RFC, table, or IDoc metadata is invalid.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs



Read Table

Activities / SAP ECC

Overview

Read bounded SAP table rows

Use for an authorized RFC_READ_TABLE query, not unrestricted SQL or a bulk replication engine.

Prerequisites

Install licensed SAP JCo JAR and the OS/architecture-matched native library on the execution host; optional IDoc-library validation also needs its licensed JAR. Configure the SAP shared resource, test connectivity, fetch and save the exact IDoc/function metadata required by the activity. Network routing, gateway registration, SAP authorizations and partner profiles must be provisioned by the SAP team. MINA is not SAP-certified by virtue of using JCo.

Executes the selected SAP ECC operation.

The shared SAP connection supplies system and authentication settings. IDoc metadata selected from SAP defines listener, parser, renderer, and sender structures.

Activity type: sap; operation: read_table.

Configuration

Configuration procedure

Select tableName, explicit fields, ABAP WHERE clauses, delimiter and a bounded rowCount. Use rowSkip for paging and avoid changing source data during paging when consistency matters.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
SAP ECC shared connection resourceId	resource Configuration setting	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: sap
SAP table / view tableName	text Configuration setting	Authorized SAP table or view read through RFC_READ_TABLE. This is not a general-purpose database connection.
Fields (comma separated) fields	text Configuration setting	For Read Table input, a field-name array or configured comma-separated list. Output fields contains SAP field metadata, not row data.
ABAP WHERE clauses where	text Configuration setting	ABAP filter clauses, not SQL. Dynamic input is an array of clause strings. Treat user-supplied values as untrusted and validate them before constructing filters.
Rows to skip rowSkip	number Configuration setting	Rows skipped by RFC_READ_TABLE, used with a bounded rowCount for paging.
Maximum rows rowCount	number Configuration setting	Maximum rows requested; zero delegates an unbounded read to SAP. Prefer an explicit limit for memory and operational safety.
Delimiter delimiter	text Configuration setting	Separator used in RFC_READ_TABLE text rows; choose one absent from column values because the adapter splits by this separator.

Input

Mapping example

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
tableName = <authorized-table>
fields = ["<key-field>", "<value-field>"]
where = ["<validated ABAP filter>"]
rowCount = 100
rowSkip = 0
delimiter = |
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
Dynamic filter clauses where	array Optional	ABAP filter clauses, not SQL. Dynamic input is an array of clause strings. Treat user-supplied values as untrusted and validate them before constructing filters.
Dynamic field list fields	array Optional	For Read Table input, a field-name array or configured comma-separated list. Output fields contains SAP field metadata, not row data.
SAP table / view tableName	string Optional	Authorized SAP table or view read through RFC_READ_TABLE. This is not a general-purpose database connection.
Rows to skip rowSkip	number Optional	Rows skipped by RFC_READ_TABLE, used with a bounded rowCount for paging.
Maximum rows rowCount	number Optional	Maximum rows requested; zero delegates an unbounded read to SAP. Prefer an explicit limit for memory and operational safety.
Delimiter delimiter	string Optional	Separator used in RFC_READ_TABLE text rows; choose one absent from column values because the adapter splits by this separator.

Output

Illustrative runtime output

```
{"rows":[{"<key-field>":"value","<value-field>":"text"}],"fields":[],"rowCount":1,"tableName":"<table>"
}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
SAP table rows rows	array Optional	Array of row objects. SAP Read Table returns decoded text values; convert numeric/date columns explicitly before arithmetic or comparison.

Field / key	Type / required	Details
Field metadata fields	array Optional	For Read Table input, a field-name array or configured comma-separated list. Output fields contains SAP field metadata, not row data.

Advanced

Delivery, transactions and performance

SAP transport success and business success are different. Inspect BAPI RETURN/status data even when the RFC call completes. Use bounded payloads and batches; parsing/rendering materializes documents in memory. Persistent outbound workers reduce JVM startup overhead but do not eliminate provider/network latency. Use a live Dynamic Connection session only within its owning runtime; never persist its ID for later reuse.

Implemented behavior and limitations

The adapter splits DATA.WA by the configured delimiter and returns strings. Embedded delimiters can split values incorrectly. SAP row-width/function restrictions still apply; rowCount=0 is potentially unbounded. A successful connection does not imply table authorization.

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Troubleshooting procedure

SAP_CONNECTION: verify native-library loading, routing and logon credentials. SAP_RFC: inspect the function name, parameter metadata and SAP authorization/ABAP error. SAP_IDOC or SAP_SCHEMA: compare basic type, extension, release and saved field offsets with the actual payload. SAP_TRANSACTION: inspect TID/queue/context state before retrying. Some underlying paths raise generic runtime errors instead of the advertised editor fault code; retain the full message and trace.

Acceptance test

Acceptance: test a small known result, no matches, paging, authorization failure and values containing the delimiter.

Fault	Description
SAP_CONNECTION	The SAP JCo/RFC connection could not be established.
SAP_RFC	SAP returned an RFC or BAPI exception.
SAP_IDOC	The IDoc could not be parsed, posted, confirmed, or acknowledged.
SAP_TRANSACTION	The tRFC, qRFC, bgRFC, or transactional context failed.
SAP_SCHEMA	The downloaded RFC, table, or IDoc metadata is invalid.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs



JDBC Create

Activities / JDBC

Overview

Executes a JDBC Query, Update, Call Procedure, or SQL Direct contract.

The JDBC designer runs and fetches SQL metadata, derives prepared parameters and result fields, supports dynamic SQL overrides, batching, subsets, NULL handling, maximum rows, transaction overrides, and database-specific drivers.

Activity type: jdbc; operation: insert.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
JDBC shared connection resourceId	resource Required	Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment. Connection type: jdbc
Timeout (seconds) timeout	number Configuration setting	SQL execution timeout in seconds passed to the selected database driver.
Override transaction behavior overrideTransactionBehavior	boolean Configuration setting	Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group.
Override JDBC connection overrideJdbcConnection	boolean Configuration setting	Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group.
Interpret empty string as NULL interpretEmptyStringAsNull	boolean Configuration setting	Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider.
Batch update batchUpdate	boolean Configuration setting	Batch execution size/policy. Bound batches to control memory and transaction duration; inspect per-batch failures before considering all records complete.
Allow mapped SQL override overrideSqlStatement	boolean Configuration setting	SQL statement executed by the database activity. Use bound parameters for values and match the statement to the selected operation/connection.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

Prepared SQL parameters and fetched output column metadata add typed fields to these base contracts. Dynamic SQL needs matching parameter names and values at runtime.

Field / key	Type / required	Details
Database server time zone ServerTimeZone	string Optional	Timezone used for database or scheduling date/time interpretation. Match the source timezone and explicitly convert timestamps crossing systems.

Field / key	Type / required	Details
Dynamic timeout timeout	number Optional	SQL execution timeout in seconds passed to the selected database driver.
Prepared statement parameters parameters	object Optional	SQL/procedure parameters or mapped columns. Match parameter names and types to metadata and use parameter binding rather than constructing SQL from values.
Batch parameter records records	array Optional	Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema.
Dynamic update SQL SqlUpdateStatement	string Optional	SQL statement executed by the database activity. Use bound parameters for values and match the statement to the selected operation/connection.
Override transaction behavior overrideTransactionBehavior	boolean Optional	Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group.
Override JDBC connection overrideJdbcConnection	boolean Optional	Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group.
Interpret empty string as NULL interpretEmptyStringAsNull	boolean Optional	Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider.
Batch update batchUpdate	boolean Optional	Batch execution size/policy. Bound batches to control memory and transaction duration; inspect per-batch failures before considering all records complete.
Allow mapped SQL override overrideSqlStatement	boolean Optional	SQL statement executed by the database activity. Use bound parameters for values and match the statement to the selected operation/connection.

Output

Prepared SQL parameters and fetched output column metadata add typed fields to these base contracts. Dynamic SQL needs matching parameter names and values at runtime.

Field / key	Type / required	Details
Number of updates noOfUpdates	integer Optional	Number of items returned or affected by this operation. Use zero-result handling where no matching data is a valid outcome.
Affected rows rowCount	integer Optional	Number of items returned or affected by this operation. Use zero-result handling where no matching data is a valid outcome.
Generated key lastInsertId	integer|string Optional	Generated identity reported after an insert when supported by the database driver. Do not assume every provider supplies it.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"

Field / key	Type / required	Details
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Fault	Description
JDBCConnectionNotFoundException	The selected JDBC shared connection is unavailable.
InvalidTimeZoneException	The database server time zone is invalid.
JDBCSQLException	The database rejected the SQL or procedure call.
LoginTimedOutException	The database login timed out.
InvalidSQLTypeException	A prepared parameter type does not match its database column.
DuplicatedFieldNameException	The query result contains an unsupported duplicate field name.
ActivityTimedOutException	The configured activity timeout was reached.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



JDBC Update

Activities / JDBC

Overview

Executes a JDBC Query, Update, Call Procedure, or SQL Direct contract.

The JDBC designer runs and fetches SQL metadata, derives prepared parameters and result fields, supports dynamic SQL overrides, batching, subsets, NULL handling, maximum rows, transaction overrides, and database-specific drivers.

Activity type: jdbc; operation: update.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
JDBC shared connection resourceId	resource Required	Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment. Connection type: jdbc
Timeout (seconds) timeout	number Configuration setting	SQL execution timeout in seconds passed to the selected database driver.
Override transaction behavior overrideTransactionBehavior	boolean Configuration setting	Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group.
Override JDBC connection overrideJdbcConnection	boolean Configuration setting	Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group.
Insert if record does not exist insertIfRecordDoesNotExist	boolean Configuration setting	Insert/merge policy. Identify the stable matching key columns for merge/update operations and verify uniqueness before loading records.
Interpret empty string as NULL interpretEmptyStringAsNull	boolean Configuration setting	Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider.
Batch update batchUpdate	boolean Configuration setting	Batch execution size/policy. Bound batches to control memory and transaction duration; inspect per-batch failures before considering all records complete.
Allow mapped SQL override overrideSqlStatement	boolean Configuration setting	SQL statement executed by the database activity. Use bound parameters for values and match the statement to the selected operation/connection.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

Prepared SQL parameters and fetched output column metadata add typed fields to these base contracts. Dynamic SQL needs matching parameter names and values at runtime.

Field / key	Type / required	Details
Database server time zone ServerTimeZone	string Optional	Timezone used for database or scheduling date/time interpretation. Match the source timezone and explicitly convert timestamps crossing systems.
Dynamic timeout timeout	number Optional	SQL execution timeout in seconds passed to the selected database driver.
Prepared statement parameters parameters	object Optional	SQL/procedure parameters or mapped columns. Match parameter names and types to metadata and use parameter binding rather than constructing SQL from values.
Batch parameter records records	array Optional	Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema.
Dynamic update SQL SqlUpdateStatement	string Optional	SQL statement executed by the database activity. Use bound parameters for values and match the statement to the selected operation/connection.
Insert-if-missing SQL InsertStatement	string Optional	SQL statement executed by the database activity. Use bound parameters for values and match the statement to the selected operation/connection.
Override transaction behavior overrideTransactionBehavior	boolean Optional	Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group.
Override JDBC connection overrideJdbcConnection	boolean Optional	Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group.
Insert if record does not exist insertIfRecordDoesNotExist	boolean Optional	Insert/merge policy. Identify the stable matching key columns for merge/update operations and verify uniqueness before loading records.
Interpret empty string as NULL interpretEmptyStringAsNull	boolean Optional	Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider.
Batch update batchUpdate	boolean Optional	Batch execution size/policy. Bound batches to control memory and transaction duration; inspect per-batch failures before considering all records complete.
Allow mapped SQL override overrideSqlStatement	boolean Optional	SQL statement executed by the database activity. Use bound parameters for values and match the statement to the selected operation/connection.

Output

Prepared SQL parameters and fetched output column metadata add typed fields to these base contracts. Dynamic SQL needs matching parameter names and values at runtime.

Field / key	Type / required	Details
Number of updates noOfUpdates	integer Optional	Number of items returned or affected by this operation. Use zero-result handling where no matching data is a valid outcome.
Affected rows rowCount	integer Optional	Number of items returned or affected by this operation. Use zero-result handling where no matching data is a valid outcome.
Generated key lastInsertId	integer|string Optional	Generated identity reported after an insert when supported by the database driver. Do not assume every provider supplies it.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Fault	Description
JDBCConnectionNotFoundException	The selected JDBC shared connection is unavailable.
InvalidTimeZoneException	The database server time zone is invalid.
JDBCSQLException	The database rejected the SQL or procedure call.
LoginTimedOutException	The database login timed out.
InvalidSQLTypeException	A prepared parameter type does not match its database column.
DuplicatedFieldNameException	The query result contains an unsupported duplicate field name.
ActivityTimedOutException	The configured activity timeout was reached.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



JDBC Query

Activities / JDBC

Overview

Executes a JDBC Query, Update, Call Procedure, or SQL Direct contract.

The JDBC designer runs and fetches SQL metadata, derives prepared parameters and result fields, supports dynamic SQL overrides, batching, subsets, NULL handling, maximum rows, transaction overrides, and database-specific drivers.

Activity type: jdbc; operation: query.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
JDBC shared connection resourceId	resource Required	Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment. Connection type: jdbc
Timeout (seconds) timeout	number Configuration setting	SQL execution timeout in seconds passed to the selected database driver.
Maximum rows (0 retrieves all) maxRows	number Configuration setting	Upper bound on returned rows/items. Use a positive bound for large datasets and page/process subsequent batches where available.
Override transaction behavior overrideTransactionBehavior	boolean Configuration setting	Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group.
Override JDBC connection overrideJdbcConnection	boolean Configuration setting	Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group.
Use nil for SQL NULL useNil	boolean Configuration setting	Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider.
Interpret empty string as NULL interpretEmptyStringAsNull	boolean Configuration setting	Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider.
Interpret invalid XML characters interpretInvalidXmlChars	boolean Configuration setting	Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider.
Process result in subsets processInSubsets	boolean Configuration setting	Enable result subsets and set the size of each page. The lastSubset output is true on the final page or when the maximum row limit is reached.
Subset size subsetSize	number Configuration setting	Enable result subsets and set the size of each page. The lastSubset output is true on the final page or when the maximum row limit is reached.
Allow mapped SQL override overrideSqlStatement	boolean Configuration setting	SQL statement executed by the database activity. Use bound parameters for values and match the statement to the selected operation/connection.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

Prepared SQL parameters and fetched output column metadata add typed fields to these base contracts. Dynamic SQL needs matching parameter names and values at runtime.

Field / key	Type / required	Details
Database server time zone ServerTimeZone	string Optional	Timezone used for database or scheduling date/time interpretation. Match the source timezone and explicitly convert timestamps crossing systems.
Dynamic timeout timeout	number Optional	SQL execution timeout in seconds passed to the selected database driver.
Dynamic maximum rows maxRows	number Optional	Upper bound on returned rows/items. Use a positive bound for large datasets and page/process subsequent batches where available.
Dynamic SQL override SqlStatement	string Optional	SQL statement executed by the database activity. Use bound parameters for values and match the statement to the selected operation/connection.
Prepared statement parameters parameters	object Optional	SQL/procedure parameters or mapped columns. Match parameter names and types to metadata and use parameter binding rather than constructing SQL from values.
Row offset when processing subsets rowOffset	integer Optional	Start at row offset zero. For subsequent pages, map the previous nextRowOffset to rowOffset. Use a stable ORDER BY and stop when lastSubset is true.
Override transaction behavior overrideTransactionBehavior	boolean Optional	Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group.
Override JDBC connection overrideJdbcConnection	boolean Optional	Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group.
Use nil for SQL NULL useNil	boolean Optional	Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider.
Interpret empty string as NULL interpretEmptyStringAsNull	boolean Optional	Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider.
Interpret invalid XML characters interpretInvalidXmlChars	boolean Optional	Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider.
Process result in subsets processInSubsets	boolean Optional	Enable result subsets and set the size of each page. The lastSubset output is true on the final page or when the maximum row limit is reached.
Subset size subsetSize	number Optional	Enable result subsets and set the size of each page. The lastSubset output is true on the final page or when the maximum row limit is reached.
Allow mapped SQL override overrideSqlStatement	boolean Optional	SQL statement executed by the database activity. Use bound parameters for values and match the statement to the selected operation/connection.

Output

Prepared SQL parameters and fetched output column metadata add typed fields to these base contracts. Dynamic SQL needs matching parameter names and values at runtime.

Field / key	Type / required	Details
JDBC result set resultSet	object Optional	Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema.
Result records resultSet.Record	array Optional	Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema.
Result records rows	array Optional	Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema.
Rows returned rowCount	integer Optional	Number of items returned or affected by this operation. Use zero-result handling where no matching data is a valid outcome.
Last result subset lastSubset	boolean Optional	Enable result subsets and set the size of each page. The lastSubset output is true on the final page or when the maximum row limit is reached.
Offset for the next subset, when available nextRowOffset	integer Optional	Start at row offset zero. For subsequent pages, map the previous nextRowOffset to rowOffset. Use a stable ORDER BY and stop when lastSubset is true.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false

Field / key	Type / required	Details
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Fault	Description
JDBCConnectionNotFoundException	The selected JDBC shared connection is unavailable.
InvalidTimeZoneException	The database server time zone is invalid.
JDBCSQLException	The database rejected the SQL or procedure call.
LoginTimedOutException	The database login timed out.
InvalidSQLTypeException	A prepared parameter type does not match its database column.
DuplicatedFieldNameException	The query result contains an unsupported duplicate field name.
ActivityTimedOutException	The configured activity timeout was reached.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



JDBC Truncate

Activities / JDBC

Overview

Executes a JDBC Query, Update, Call Procedure, or SQL Direct contract.

The JDBC designer runs and fetches SQL metadata, derives prepared parameters and result fields, supports dynamic SQL overrides, batching, subsets, NULL handling, maximum rows, transaction overrides, and database-specific drivers.

Activity type: jdbc; operation: truncate.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
JDBC shared connection resourceId	resource Required	Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment. Connection type: jdbc
Timeout (seconds) timeout	number Configuration setting	SQL execution timeout in seconds passed to the selected database driver.
Override transaction behavior overrideTransactionBehavior	boolean Configuration setting	Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group.
Override JDBC connection overrideJdbcConnection	boolean Configuration setting	Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

Prepared SQL parameters and fetched output column metadata add typed fields to these base contracts. Dynamic SQL needs matching parameter names and values at runtime.

Field / key	Type / required	Details
Database server time zone ServerTimeZone	string Optional	Timezone used for database or scheduling date/time interpretation. Match the source timezone and explicitly convert timestamps crossing systems.
Dynamic timeout timeout	number Optional	SQL execution timeout in seconds passed to the selected database driver.
Prepared statement parameters parameters	object Optional	SQL/procedure parameters or mapped columns. Match parameter names and types to metadata and use parameter binding rather than constructing SQL from values.
Batch parameter records records	array Optional	Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema.

Field / key	Type / required	Details
Dynamic update SQL SqlUpdateStatement	string Optional	SQL statement executed by the database activity. Use bound parameters for values and match the statement to the selected operation/connection.
Override transaction behavior overrideTransactionBehavior	boolean Optional	Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group.
Override JDBC connection overrideJdbcConnection	boolean Optional	Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group.

Output

Prepared SQL parameters and fetched output column metadata add typed fields to these base contracts. Dynamic SQL needs matching parameter names and values at runtime.

Field / key	Type / required	Details
Number of updates noOfUpdates	integer Optional	Number of items returned or affected by this operation. Use zero-result handling where no matching data is a valid outcome.
Affected rows rowCount	integer Optional	Number of items returned or affected by this operation. Use zero-result handling where no matching data is a valid outcome.
Generated key lastInsertId	integer|string Optional	Generated identity reported after an insert when supported by the database driver. Do not assume every provider supplies it.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0

Field / key	Type / required	Details
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Fault	Description
JDBCConnectionNotFoundException	The selected JDBC shared connection is unavailable.
InvalidTimeZoneException	The database server time zone is invalid.
JDBCSQLException	The database rejected the SQL or procedure call.
LoginTimedOutException	The database login timed out.
InvalidSQLTypeException	A prepared parameter type does not match its database column.
DuplicatedFieldNameException	The query result contains an unsupported duplicate field name.
ActivityTimedOutException	The configured activity timeout was reached.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



JDBC Delete

Activities / JDBC

Overview

Executes a JDBC Query, Update, Call Procedure, or SQL Direct contract.

The JDBC designer runs and fetches SQL metadata, derives prepared parameters and result fields, supports dynamic SQL overrides, batching, subsets, NULL handling, maximum rows, transaction overrides, and database-specific drivers.

Activity type: jdbc; operation: delete.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
JDBC shared connection resourceId	resource Required	Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment. Connection type: jdbc
Timeout (seconds) timeout	number Configuration setting	SQL execution timeout in seconds passed to the selected database driver.
Override transaction behavior overrideTransactionBehavior	boolean Configuration setting	Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group.
Override JDBC connection overrideJdbcConnection	boolean Configuration setting	Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group.
Interpret empty string as NULL interpretEmptyStringAsNull	boolean Configuration setting	Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider.
Batch update batchUpdate	boolean Configuration setting	Batch execution size/policy. Bound batches to control memory and transaction duration; inspect per-batch failures before considering all records complete.
Allow mapped SQL override overrideSqlStatement	boolean Configuration setting	SQL statement executed by the database activity. Use bound parameters for values and match the statement to the selected operation/connection.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

Prepared SQL parameters and fetched output column metadata add typed fields to these base contracts. Dynamic SQL needs matching parameter names and values at runtime.

Field / key	Type / required	Details
Database server time zone ServerTimeZone	string Optional	Timezone used for database or scheduling date/time interpretation. Match the source timezone and explicitly convert timestamps crossing systems.

Field / key	Type / required	Details
Dynamic timeout timeout	number Optional	SQL execution timeout in seconds passed to the selected database driver.
Prepared statement parameters parameters	object Optional	SQL/procedure parameters or mapped columns. Match parameter names and types to metadata and use parameter binding rather than constructing SQL from values.
Batch parameter records records	array Optional	Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema.
Dynamic update SQL SqlUpdateStatement	string Optional	SQL statement executed by the database activity. Use bound parameters for values and match the statement to the selected operation/connection.
Override transaction behavior overrideTransactionBehavior	boolean Optional	Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group.
Override JDBC connection overrideJdbcConnection	boolean Optional	Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group.
Interpret empty string as NULL interpretEmptyStringAsNull	boolean Optional	Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider.
Batch update batchUpdate	boolean Optional	Batch execution size/policy. Bound batches to control memory and transaction duration; inspect per-batch failures before considering all records complete.
Allow mapped SQL override overrideSqlStatement	boolean Optional	SQL statement executed by the database activity. Use bound parameters for values and match the statement to the selected operation/connection.

Output

Prepared SQL parameters and fetched output column metadata add typed fields to these base contracts. Dynamic SQL needs matching parameter names and values at runtime.

Field / key	Type / required	Details
Number of updates noOfUpdates	integer Optional	Number of items returned or affected by this operation. Use zero-result handling where no matching data is a valid outcome.
Affected rows rowCount	integer Optional	Number of items returned or affected by this operation. Use zero-result handling where no matching data is a valid outcome.
Generated key lastInsertId	integer|string Optional	Generated identity reported after an insert when supported by the database driver. Do not assume every provider supplies it.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"

Field / key	Type / required	Details
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Fault	Description
JDBCConnectionNotFoundException	The selected JDBC shared connection is unavailable.
InvalidTimeZoneException	The database server time zone is invalid.
JDBCSQLException	The database rejected the SQL or procedure call.
LoginTimedOutException	The database login timed out.
InvalidSQLTypeException	A prepared parameter type does not match its database column.
DuplicatedFieldNameException	The query result contains an unsupported duplicate field name.
ActivityTimedOutException	The configured activity timeout was reached.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



Stored Procedure

Activities / JDBC

Overview

Executes a JDBC Query, Update, Call Procedure, or SQL Direct contract.

The JDBC designer runs and fetches SQL metadata, derives prepared parameters and result fields, supports dynamic SQL overrides, batching, subsets, NULL handling, maximum rows, transaction overrides, and database-specific drivers.

Activity type: jdbc; operation: call.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
JDBC shared connection resourceId	resource Required	Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment. Connection type: jdbc
Procedure schema schema	text Configuration setting	Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions.
Catalog or package catalog	text Configuration setting	Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions.
Procedure / function procedure	text Configuration setting	Browse and refresh procedure metadata from the JDBC designer below.
Timeout (seconds) timeout	number Configuration setting	SQL execution timeout in seconds passed to the selected database driver.
Maximum rows (0 retrieves all) maxRows	number Configuration setting	Upper bound on returned rows/items. Use a positive bound for large datasets and page/process subsequent batches where available.
Override transaction behavior overrideTransactionBehavior	boolean Configuration setting	Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group.
Override JDBC connection overrideJdbcConnection	boolean Configuration setting	Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

Prepared SQL parameters and fetched output column metadata add typed fields to these base contracts. Dynamic SQL needs matching parameter names and values at runtime.

Field / key	Type / required	Details
Database server time zone ServerTimeZone	string Optional	Timezone used for database or scheduling date/time interpretation. Match the source timezone and explicitly convert timestamps crossing systems.
Dynamic timeout timeout	number Optional	SQL execution timeout in seconds passed to the selected database driver.
IN and IN/OUT procedure parameters parameters	object Optional	SQL/procedure parameters or mapped columns. Match parameter names and types to metadata and use parameter binding rather than constructing SQL from values.
Procedure schema schema	string Optional	Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions.
Catalog or package catalog	string Optional	Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions.
Procedure / function procedure	string Optional	Browse and refresh procedure metadata from the JDBC designer below.
Maximum rows (0 retrieves all) maxRows	number Optional	Upper bound on returned rows/items. Use a positive bound for large datasets and page/process subsequent batches where available.
Override transaction behavior overrideTransactionBehavior	boolean Optional	Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group.
Override JDBC connection overrideJdbcConnection	boolean Optional	Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group.

Output

Prepared SQL parameters and fetched output column metadata add typed fields to these base contracts. Dynamic SQL needs matching parameter names and values at runtime.

Field / key	Type / required	Details
Procedure result sets resultSets	array Optional	Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema.
OUT and IN/OUT parameters outParameters	object Optional	SQL/procedure parameters or mapped columns. Match parameter names and types to metadata and use parameter binding rather than constructing SQL from values.
Unresolved result sets UnresolvedResultSets	array Optional	Additional result sets whose row metadata was not resolved at design time. Inspect the actual procedure response before mapping child fields.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

Field / key	Type / required	Details
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Fault	Description
JDBCConnectionNotFoundException	The selected JDBC shared connection is unavailable.
InvalidTimeZoneException	The database server time zone is invalid.
JDBCSQLException	The database rejected the SQL or procedure call.
LoginTimedOutException	The database login timed out.
InvalidSQLTypeException	A prepared parameter type does not match its database column.
DuplicatedFieldNameException	The query result contains an unsupported duplicate field name.
ActivityTimedOutException	The configured activity timeout was reached.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



Dynamic SQL

Activities / JDBC

Overview

Executes a JDBC Query, Update, Call Procedure, or SQL Direct contract.

The JDBC designer runs and fetches SQL metadata, derives prepared parameters and result fields, supports dynamic SQL overrides, batching, subsets, NULL handling, maximum rows, transaction overrides, and database-specific drivers.

Activity type: jdbc; operation: dynamic.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
JDBC shared connection resourceId	resource Required	Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment. Connection type: jdbc
Timeout (seconds) timeout	number Configuration setting	SQL execution timeout in seconds passed to the selected database driver.
Maximum rows (0 retrieves all) maxRows	number Configuration setting	Upper bound on returned rows/items. Use a positive bound for large datasets and page/process subsequent batches where available.
Override transaction behavior overrideTransactionBehavior	boolean Configuration setting	Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group.
Override JDBC connection overrideJdbcConnection	boolean Configuration setting	Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group.
Use nil for SQL NULL useNil	boolean Configuration setting	Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider.
Interpret empty string as NULL interpretEmptyStringAsNull	boolean Configuration setting	Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider.
Interpret invalid XML characters interpretInvalidXmlChars	boolean Configuration setting	Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider.
Process result in subsets processInSubsets	boolean Configuration setting	Enable result subsets and set the size of each page. The lastSubset output is true on the final page or when the maximum row limit is reached.
Subset size subsetSize	number Configuration setting	Enable result subsets and set the size of each page. The lastSubset output is true on the final page or when the maximum row limit is reached.
Allow mapped SQL override overrideSqlStatement	boolean Configuration setting	SQL statement executed by the database activity. Use bound parameters for values and match the statement to the selected operation/connection.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

Prepared SQL parameters and fetched output column metadata add typed fields to these base contracts. Dynamic SQL needs matching parameter names and values at runtime.

Field / key	Type / required	Details
Database server time zone ServerTimeZone	string Optional	Timezone used for database or scheduling date/time interpretation. Match the source timezone and explicitly convert timestamps crossing systems.
Dynamic timeout timeout	number Optional	SQL execution timeout in seconds passed to the selected database driver.
Dynamic maximum rows maxRows	number Optional	Upper bound on returned rows/items. Use a positive bound for large datasets and page/process subsequent batches where available.
Dynamic SQL override SqlStatement	string Optional	SQL statement executed by the database activity. Use bound parameters for values and match the statement to the selected operation/connection.
Prepared statement parameters parameters	object Optional	SQL/procedure parameters or mapped columns. Match parameter names and types to metadata and use parameter binding rather than constructing SQL from values.
Row offset when processing subsets rowOffset	integer Optional	Start at row offset zero. For subsequent pages, map the previous nextRowOffset to rowOffset. Use a stable ORDER BY and stop when lastSubset is true.
Override transaction behavior overrideTransactionBehavior	boolean Optional	Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group.
Override JDBC connection overrideJdbcConnection	boolean Optional	Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group.
Use nil for SQL NULL useNil	boolean Optional	Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider.
Interpret empty string as NULL interpretEmptyStringAsNull	boolean Optional	Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider.
Interpret invalid XML characters interpretInvalidXmlChars	boolean Optional	Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider.
Process result in subsets processInSubsets	boolean Optional	Enable result subsets and set the size of each page. The lastSubset output is true on the final page or when the maximum row limit is reached.
Subset size subsetSize	number Optional	Enable result subsets and set the size of each page. The lastSubset output is true on the final page or when the maximum row limit is reached.
Allow mapped SQL override overrideSqlStatement	boolean Optional	SQL statement executed by the database activity. Use bound parameters for values and match the statement to the selected operation/connection.

Output

Prepared SQL parameters and fetched output column metadata add typed fields to these base contracts. Dynamic SQL needs matching parameter names and values at runtime.

Field / key	Type / required	Details
JDBC result set resultSet	object Optional	Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema.
Result records resultSet.Record	array Optional	Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema.
Result records rows	array Optional	Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema.
Rows returned rowCount	integer Optional	Number of items returned or affected by this operation. Use zero-result handling where no matching data is a valid outcome.
Last result subset lastSubset	boolean Optional	Enable result subsets and set the size of each page. The lastSubset output is true on the final page or when the maximum row limit is reached.
Offset for the next subset, when available nextRowOffset	integer Optional	Start at row offset zero. For subsequent pages, map the previous nextRowOffset to rowOffset. Use a stable ORDER BY and stop when lastSubset is true.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false

Field / key	Type / required	Details
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Fault	Description
JDBCConnectionNotFoundException	The selected JDBC shared connection is unavailable.
InvalidTimeZoneException	The database server time zone is invalid.
JDBCSQLException	The database rejected the SQL or procedure call.
LoginTimedOutException	The database login timed out.
InvalidSQLTypeException	A prepared parameter type does not match its database column.
DuplicatedFieldNameException	The query result contains an unsupported duplicate field name.
ActivityTimedOutException	The configured activity timeout was reached.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



Snowflake Insert

Activities / Snowflake

Overview

Runs Snowflake Insert, Query, Update, Delete, or staged Bulk Load operations.

The Snowflake JDBC shared connection supplies authentication, warehouse, database, schema, pooling, and downloaded table/view metadata. Activity inputs and outputs use MINA's schema-aware Snowflake contracts.

Activity type: snowflake; operation: insert.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Snowflake JDBC connection resourceId	resource Required	Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment. Connection type: snowflake
Entity entity	snowflake_entity Configuration setting	Tables and views retrieved in the Snowflake connection Schema section.
Activity timeout (seconds) timeout	number Configuration setting	Statement execution timeout in seconds; a positive value configures the Snowflake session statement timeout.
Override database name overrideDatabaseName	text Configuration setting	Per-operation database/schema override. The runtime identity must have access to the resolved namespace.
Override schema name overrideSchemaName	text Configuration setting	Per-operation database/schema override. The runtime identity must have access to the resolved namespace.
Interpret empty string as NULL interpretEmptyStringAsNull	boolean Configuration setting	Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider.
Create table from XSD createTableFromXsd	boolean Configuration setting	Allow the Snowflake activity to create a missing table from its configured contract. Requires creation privileges; review generated column types before production use.
Table XSD/schema schemaId	text Configuration setting	Selected schema/contract for the request, response or record structure. Refresh mappings after changing schema fields, types or cardinalities.
Table name tableName	text Configuration setting	Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions.
Values columns valueColumns	text Configuration setting	SQL/procedure parameters or mapped columns. Match parameter names and types to metadata and use parameter binding rather than constructing SQL from values.
Batch size batchSize	number Configuration setting	Batch execution size/policy. Bound batches to control memory and transaction duration; inspect per-batch failures before considering all records complete.

Field / key	Type / required	Details
Fault on batch failure faultOnBatchFailure	boolean Configuration setting	Batch failure policy/results. With faultOnBatchFailure enabled, failed batches fault the activity; inspect batchFailures for affected records and provider errors.
Merge existing records merge	boolean Configuration setting	Insert/merge policy. Identify the stable matching key columns for merge/update operations and verify uniqueness before loading records.
Merge on columns mergeOnColumns	text Configuration setting	Insert/merge policy. Identify the stable matching key columns for merge/update operations and verify uniqueness before loading records.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

Entity/table metadata adds actual input record and result row columns.

Field / key	Type / required	Details
Rows to insert records	array Required	Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema.
Activity timeout (seconds) timeout	number Optional	Statement execution timeout in seconds; a positive value configures the Snowflake session statement timeout.
Override database name overrideDatabaseName	string Optional	Per-operation database/schema override. The runtime identity must have access to the resolved namespace.
Override schema name overrideSchemaName	string Optional	Per-operation database/schema override. The runtime identity must have access to the resolved namespace.
Interpret empty string as NULL interpretEmptyStringAsNull	boolean Optional	Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider.
Create table from XSD createTableFromXsd	boolean Optional	Allow the Snowflake activity to create a missing table from its configured contract. Requires creation privileges; review generated column types before production use.
Table XSD/schema schemaId	string Optional	Selected schema/contract for the request, response or record structure. Refresh mappings after changing schema fields, types or cardinalities.
Table name tableName	string Optional	Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions.
Values columns valueColumns	string Optional	SQL/procedure parameters or mapped columns. Match parameter names and types to metadata and use parameter binding rather than constructing SQL from values.
Batch size batchSize	number Optional	Batch execution size/policy. Bound batches to control memory and transaction duration; inspect per-batch failures before considering all records complete.
Fault on batch failure faultOnBatchFailure	boolean Optional	Batch failure policy/results. With faultOnBatchFailure enabled, failed batches fault the activity; inspect batchFailures for affected records and provider errors.

Field / key	Type / required	Details
Merge existing records merge	boolean Optional	Insert/merge policy. Identify the stable matching key columns for merge/update operations and verify uniqueness before loading records.
Merge on columns mergeOnColumns	string Optional	Insert/merge policy. Identify the stable matching key columns for merge/update operations and verify uniqueness before loading records.

Output

Entity/table metadata adds actual input record and result row columns.

Field / key	Type / required	Details
Rows attempted rowsAttempted	integer Optional	Number of rows submitted for processing; compare with rowsAffected and batchFailures to detect partial success.
Rows affected rowsAffected	integer Optional	Number of items returned or affected by this operation. Use zero-result handling where no matching data is a valid outcome.
Batch failure details batchFailures	array Optional	Batch failure policy/results. With faultOnBatchFailure enabled, failed batches fault the activity; inspect batchFailures for affected records and provider errors.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false

Field / key	Type / required	Details
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Fault	Description
SNOWFLAKE_CONNECTION	The Snowflake shared resource or connection pool is unavailable.
SNOWFLAKE_DATABASE_JDBC-500005	Snowflake failed to create a prepared statement.
SNOWFLAKE_DATABASE_JDBC-500007	Snowflake failed to bind prepared-statement parameters.
SNOWFLAKE_DATABASE_JDBC-500009	Snowflake failed to execute the generated query.
SNOWFLAKE_DATABASE_JDBC-500013	The activity could not generate its structured output.
SNOWFLAKE_DATABASE_JDBC-500016	One or more insert batches failed.
SNOWFLAKE_DATABASE_JDBC-500022	Schema or table creation failed.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



Snowflake Query

Activities / Snowflake

Overview

Runs Snowflake Insert, Query, Update, Delete, or staged Bulk Load operations.

The Snowflake JDBC shared connection supplies authentication, warehouse, database, schema, pooling, and downloaded table/view metadata. Activity inputs and outputs use MINA's schema-aware Snowflake contracts.

Activity type: snowflake; operation: query.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Snowflake JDBC connection resourceId	resource Required	Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment. Connection type: snowflake
Entity entity	snowflake_entity Configuration setting	Tables and views retrieved in the Snowflake connection Schema section.
Activity timeout (seconds) timeout	number Configuration setting	Statement execution timeout in seconds; a positive value configures the Snowflake session statement timeout.
Override database name overrideDatabaseName	text Configuration setting	Per-operation database/schema override. The runtime identity must have access to the resolved namespace.
Override schema name overrideSchemaName	text Configuration setting	Per-operation database/schema override. The runtime identity must have access to the resolved namespace.
Interpret empty string as NULL interpretEmptyStringAsNull	boolean Configuration setting	Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider.
Snowflake SELECT statement statement	textarea Configuration setting	SQL statement executed by the database activity. Use bound parameters for values and match the statement to the selected operation/connection.
Prepared parameters [name:type] preparedParameters	text Configuration setting	Named or positional operation arguments. Match names, order and datatypes to the selected SQL statement, function or method contract.
Maximum rows (0 retrieves all) maximumRows	number Configuration setting	Upper bound on returned rows/items. Use a positive bound for large datasets and page/process subsequent batches where available.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

Entity/table metadata adds actual input record and result row columns.

Field / key	Type / required	Details
Prepared statement parameters parameters	object Optional	SQL/procedure parameters or mapped columns. Match parameter names and types to metadata and use parameter binding rather than constructing SQL from values.
Activity timeout (seconds) timeout	number Optional	Statement execution timeout in seconds; a positive value configures the Snowflake session statement timeout.
Override database name overrideDatabaseName	string Optional	Per-operation database/schema override. The runtime identity must have access to the resolved namespace.
Override schema name overrideSchemaName	string Optional	Per-operation database/schema override. The runtime identity must have access to the resolved namespace.
Interpret empty string as NULL interpretEmptyStringAsNull	boolean Optional	Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider.
Snowflake SELECT statement statement	string Optional	SQL statement executed by the database activity. Use bound parameters for values and match the statement to the selected operation/connection.
Prepared parameters [name:type] preparedParameters	string Optional	Named or positional operation arguments. Match names, order and datatypes to the selected SQL statement, function or method contract.
Maximum rows (0 retrieves all) maximumRows	number Optional	Upper bound on returned rows/items. Use a positive bound for large datasets and page/process subsequent batches where available.

Output

Entity/table metadata adds actual input record and result row columns.

Field / key	Type / required	Details
Query result records rows	array Optional	Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema.
Rows returned rowCount	integer Optional	Number of items returned or affected by this operation. Use zero-result handling where no matching data is a valid outcome.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"

Field / key	Type / required	Details
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Fault	Description
SNOWFLAKE_CONNECTION	The Snowflake shared resource or connection pool is unavailable.
SNOWFLAKE_DATABASE_JDBC-500005	Snowflake failed to create a prepared statement.
SNOWFLAKE_DATABASE_JDBC-500007	Snowflake failed to bind prepared-statement parameters.
SNOWFLAKE_DATABASE_JDBC-500009	Snowflake failed to execute the generated query.
SNOWFLAKE_DATABASE_JDBC-500013	The activity could not generate its structured output.
SNOWFLAKE_DATABASE_JDBC-500014	Maximum Rows is negative.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



Snowflake Update

Activities / Snowflake

Overview

Runs Snowflake Insert, Query, Update, Delete, or staged Bulk Load operations.

The Snowflake JDBC shared connection supplies authentication, warehouse, database, schema, pooling, and downloaded table/view metadata. Activity inputs and outputs use MINA's schema-aware Snowflake contracts.

Activity type: snowflake; operation: update.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Snowflake JDBC connection resourceId	resource Required	Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment. Connection type: snowflake
Entity entity	snowflake_entity Configuration setting	Tables and views retrieved in the Snowflake connection Schema section.
Activity timeout (seconds) timeout	number Configuration setting	Statement execution timeout in seconds; a positive value configures the Snowflake session statement timeout.
Override database name overrideDatabaseName	text Configuration setting	Per-operation database/schema override. The runtime identity must have access to the resolved namespace.
Override schema name overrideSchemaName	text Configuration setting	Per-operation database/schema override. The runtime identity must have access to the resolved namespace.
Interpret empty string as NULL interpretEmptyStringAsNull	boolean Configuration setting	Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider.
SET / Values columns valueColumns	text Configuration setting	SQL/procedure parameters or mapped columns. Match parameter names and types to metadata and use parameter binding rather than constructing SQL from values.
WHERE / Parameter columns parameterColumns	text Configuration setting	SQL/procedure parameters or mapped columns. Match parameter names and types to metadata and use parameter binding rather than constructing SQL from values.
Insert when record does not exist merge	boolean Configuration setting	Insert/merge policy. Identify the stable matching key columns for merge/update operations and verify uniqueness before loading records.
Merge on columns mergeOnColumns	text Configuration setting	Insert/merge policy. Identify the stable matching key columns for merge/update operations and verify uniqueness before loading records.
Create table if none exists createTableIfNoneExists	boolean Configuration setting	Allow the Snowflake activity to create a missing table from its configured contract. Requires creation privileges; review generated column types before production use.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

Entity/table metadata adds actual input record and result row columns.

Field / key	Type / required	Details
Rows containing values and parameters records	array Required	Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema.
Activity timeout (seconds) timeout	number Optional	Statement execution timeout in seconds; a positive value configures the Snowflake session statement timeout.
Override database name overrideDatabaseName	string Optional	Per-operation database/schema override. The runtime identity must have access to the resolved namespace.
Override schema name overrideSchemaName	string Optional	Per-operation database/schema override. The runtime identity must have access to the resolved namespace.
Interpret empty string as NULL interpretEmptyStringAsNull	boolean Optional	Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider.
SET / Values columns valueColumns	string Optional	SQL/procedure parameters or mapped columns. Match parameter names and types to metadata and use parameter binding rather than constructing SQL from values.
WHERE / Parameter columns parameterColumns	string Optional	SQL/procedure parameters or mapped columns. Match parameter names and types to metadata and use parameter binding rather than constructing SQL from values.
Insert when record does not exist merge	boolean Optional	Insert/merge policy. Identify the stable matching key columns for merge/update operations and verify uniqueness before loading records.
Merge on columns mergeOnColumns	string Optional	Insert/merge policy. Identify the stable matching key columns for merge/update operations and verify uniqueness before loading records.
Create table if none exists createTableIfNoneExists	boolean Optional	Allow the Snowflake activity to create a missing table from its configured contract. Requires creation privileges; review generated column types before production use.

Output

Entity/table metadata adds actual input record and result row columns.

Field / key	Type / required	Details
Rows affected rowsAffected	integer Optional	Number of items returned or affected by this operation. Use zero-result handling where no matching data is a valid outcome.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

Field / key	Type / required	Details
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Fault	Description
SNOWFLAKE_CONNECTION	The Snowflake shared resource or connection pool is unavailable.
SNOWFLAKE_DATABASE_JDBC-500005	Snowflake failed to create a prepared statement.
SNOWFLAKE_DATABASE_JDBC-500007	Snowflake failed to bind prepared-statement parameters.
SNOWFLAKE_DATABASE_JDBC-500009	Snowflake failed to execute the generated query.
SNOWFLAKE_DATABASE_JDBC-500013	The activity could not generate its structured output.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



Snowflake Delete

Activities / Snowflake

Overview

Runs Snowflake Insert, Query, Update, Delete, or staged Bulk Load operations.

The Snowflake JDBC shared connection supplies authentication, warehouse, database, schema, pooling, and downloaded table/view metadata. Activity inputs and outputs use MINA's schema-aware Snowflake contracts.

Activity type: snowflake; operation: delete.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Snowflake JDBC connection resourceId	resource Required	Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment. Connection type: snowflake
Entity entity	snowflake_entity Configuration setting	Tables and views retrieved in the Snowflake connection Schema section.
Activity timeout (seconds) timeout	number Configuration setting	Statement execution timeout in seconds; a positive value configures the Snowflake session statement timeout.
Override database name overrideDatabaseName	text Configuration setting	Per-operation database/schema override. The runtime identity must have access to the resolved namespace.
Override schema name overrideSchemaName	text Configuration setting	Per-operation database/schema override. The runtime identity must have access to the resolved namespace.
Interpret empty string as NULL interpretEmptyStringAsNull	boolean Configuration setting	Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider.
WHERE / Parameter columns parameterColumns	text Configuration setting	SQL/procedure parameters or mapped columns. Match parameter names and types to metadata and use parameter binding rather than constructing SQL from values.
Merge when record does not exist merge	boolean Configuration setting	Insert/merge policy. Identify the stable matching key columns for merge/update operations and verify uniqueness before loading records.
Merge on columns mergeOnColumns	text Configuration setting	Insert/merge policy. Identify the stable matching key columns for merge/update operations and verify uniqueness before loading records.
Create table if none exists createTableIfNoneExists	boolean Configuration setting	Allow the Snowflake activity to create a missing table from its configured contract. Requires creation privileges; review generated column types before production use.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

Entity/table metadata adds actual input record and result row columns.

Field / key	Type / required	Details
Rows containing delete parameters records	array Optional	Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema.
Activity timeout (seconds) timeout	number Optional	Statement execution timeout in seconds; a positive value configures the Snowflake session statement timeout.
Override database name overrideDatabaseName	string Optional	Per-operation database/schema override. The runtime identity must have access to the resolved namespace.
Override schema name overrideSchemaName	string Optional	Per-operation database/schema override. The runtime identity must have access to the resolved namespace.
Interpret empty string as NULL interpretEmptyStringAsNull	boolean Optional	Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider.
WHERE / Parameter columns parameterColumns	string Optional	SQL/procedure parameters or mapped columns. Match parameter names and types to metadata and use parameter binding rather than constructing SQL from values.
Merge when record does not exist merge	boolean Optional	Insert/merge policy. Identify the stable matching key columns for merge/update operations and verify uniqueness before loading records.
Merge on columns mergeOnColumns	string Optional	Insert/merge policy. Identify the stable matching key columns for merge/update operations and verify uniqueness before loading records.
Create table if none exists createTableIfNoneExists	boolean Optional	Allow the Snowflake activity to create a missing table from its configured contract. Requires creation privileges; review generated column types before production use.

Output

Entity/table metadata adds actual input record and result row columns.

Field / key	Type / required	Details
Rows affected rowsAffected	integer Optional	Number of items returned or affected by this operation. Use zero-result handling where no matching data is a valid outcome.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"

Field / key	Type / required	Details
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Fault	Description
SNOWFLAKE_CONNECTION	The Snowflake shared resource or connection pool is unavailable.
SNOWFLAKE_DATABASE_JDBC-500005	Snowflake failed to create a prepared statement.
SNOWFLAKE_DATABASE_JDBC-500007	Snowflake failed to bind prepared-statement parameters.
SNOWFLAKE_DATABASE_JDBC-500009	Snowflake failed to execute the generated query.
SNOWFLAKE_DATABASE_JDBC-500013	The activity could not generate its structured output.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



Snowflake Bulk Load

Activities / Snowflake

Overview

Runs Snowflake Insert, Query, Update, Delete, or staged Bulk Load operations.

The Snowflake JDBC shared connection supplies authentication, warehouse, database, schema, pooling, and downloaded table/view metadata. Activity inputs and outputs use MINA's schema-aware Snowflake contracts.

Activity type: snowflake; operation: bulk_load.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Snowflake JDBC connection resourceId	resource Required	Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment. Connection type: snowflake
Entity entity	snowflake_entity Configuration setting	Tables and views retrieved in the Snowflake connection Schema section.
Activity timeout (seconds) timeout	number Configuration setting	Statement execution timeout in seconds; a positive value configures the Snowflake session statement timeout.
Override database name overrideDatabaseName	text Configuration setting	Per-operation database/schema override. The runtime identity must have access to the resolved namespace.
Override schema name overrideSchemaName	text Configuration setting	Per-operation database/schema override. The runtime identity must have access to the resolved namespace.
Interpret empty string as NULL interpretEmptyStringAsNull	boolean Configuration setting	Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider.
Create table from XSD createTableFromXsd	boolean Configuration setting	Allow the Snowflake activity to create a missing table from its configured contract. Requires creation privileges; review generated column types before production use.
Table XSD/schema schemaId	text Configuration setting	Selected schema/contract for the request, response or record structure. Refresh mappings after changing schema fields, types or cardinalities.
Table name tableName	text Configuration setting	Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions.
Stage type stageType	select Configuration setting	Snowflake staging/COPY configuration. Select the stage, file format and load options matching the uploaded data and target table. Choices: UserStage, TableStage, NamedStage, AmazonS3
Named stage / storage integration stage namedStage	text Configuration setting	Snowflake staging/COPY configuration. Select the stage, file format and load options matching the uploaded data and target table.

Field / key	Type / required	Details
Amazon S3 file format fileFormat	select Configuration setting	Snowflake staging/COPY configuration. Select the stage, file format and load options matching the uploaded data and target table. Choices: DelimitedFiles, JSON, AVRO, ORC, PARQUET, XML
Validation mode validationMode	boolean Configuration setting	Snowflake COPY error/validation policy. Select abort, continuation or skip thresholds deliberately; skipped records/files are not successful business delivery.
Purge stage files purgeStageFiles	boolean Configuration setting	Remove staged files after a successful load where configured. Retain files when replay or investigation requires the original input.
Compress internal-stage data compressData	boolean Configuration setting	Compression selection for file or staging content. Match decompression settings at the receiving operation; compression changes the wire bytes.
On error onError	select Configuration setting	Snowflake COPY error/validation policy. Select abort, continuation or skip thresholds deliberately; skipped records/files are not successful business delivery. Choices: CONTINUE, SKIP_FILE, SKIP_FILE_<num>, SKIP_FILE_<num>%, ABORT_STATEMENT
Skip file at error count skipFileErrorCount	number Configuration setting	Snowflake COPY error/validation policy. Select abort, continuation or skip thresholds deliberately; skipped records/files are not successful business delivery.
Skip file at error percentage skipFileErrorPercentage	number Configuration setting	Snowflake COPY error/validation policy. Select abort, continuation or skip thresholds deliberately; skipped records/files are not successful business delivery.
Merge loaded records merge	boolean Configuration setting	Insert/merge policy. Identify the stable matching key columns for merge/update operations and verify uniqueness before loading records.
Merge on columns mergeOnColumns	text Configuration setting	Insert/merge policy. Identify the stable matching key columns for merge/update operations and verify uniqueness before loading records.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

Entity/table metadata adds actual input record and result row columns.

Field / key	Type / required	Details
Rows for an internal stage records	array Optional	Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema.
Local file for an internal stage filePath	string Optional	File path on the runtime host. Ensure the runtime account can read the file and that the path exists in the deployment environment.
Stage load options loadOptions	object Optional	Additional provider settings as name/value entries. Use exact supported provider option names; invalid options may be rejected at connection or execution time.
Activity timeout (seconds) timeout	number Optional	Statement execution timeout in seconds; a positive value configures the Snowflake session statement timeout.
Override database name overrideDatabaseName	string Optional	Per-operation database/schema override. The runtime identity must have access to the resolved namespace.

Field / key	Type / required	Details
Override schema name overrideSchemaName	string Optional	Per-operation database/schema override. The runtime identity must have access to the resolved namespace.
Interpret empty string as NULL interpretEmptyStringAsNull	boolean Optional	Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider.
Create table from XSD createTableFromXsd	boolean Optional	Allow the Snowflake activity to create a missing table from its configured contract. Requires creation privileges; review generated column types before production use.
Table XSD/schema schemaId	string Optional	Selected schema/contract for the request, response or record structure. Refresh mappings after changing schema fields, types or cardinalities.
Table name tableName	string Optional	Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions.
Stage type stageType	string Optional	Snowflake staging/COPY configuration. Select the stage, file format and load options matching the uploaded data and target table.
Named stage / storage integration stage namedStage	string Optional	Snowflake staging/COPY configuration. Select the stage, file format and load options matching the uploaded data and target table.
Amazon S3 file format fileFormat	string Optional	Snowflake staging/COPY configuration. Select the stage, file format and load options matching the uploaded data and target table.
Validation mode validationMode	boolean Optional	Snowflake COPY error/validation policy. Select abort, continuation or skip thresholds deliberately; skipped records/files are not successful business delivery.
Purge stage files purgeStageFiles	boolean Optional	Remove staged files after a successful load where configured. Retain files when replay or investigation requires the original input.
Compress internal-stage data compressData	boolean Optional	Compression selection for file or staging content. Match decompression settings at the receiving operation; compression changes the wire bytes.
On error onError	string Optional	Snowflake COPY error/validation policy. Select abort, continuation or skip thresholds deliberately; skipped records/files are not successful business delivery.
Skip file at error count skipFileErrorCount	number Optional	Snowflake COPY error/validation policy. Select abort, continuation or skip thresholds deliberately; skipped records/files are not successful business delivery.
Skip file at error percentage skipFileErrorPercentage	number Optional	Snowflake COPY error/validation policy. Select abort, continuation or skip thresholds deliberately; skipped records/files are not successful business delivery.
Merge loaded records merge	boolean Optional	Insert/merge policy. Identify the stable matching key columns for merge/update operations and verify uniqueness before loading records.
Merge on columns mergeOnColumns	string Optional	Insert/merge policy. Identify the stable matching key columns for merge/update operations and verify uniqueness before loading records.

Output

Entity/table metadata adds actual input record and result row columns.

Field / key	Type / required	Details
Per-file load results loadResults	array Optional	Snowflake COPY result rows. Inspect status, parsed/loaded row counts and first-error fields for each input file.
Source file loadResults.FILE	string Optional	Snowflake per-file load result: file/status, counts, or location/message of the first load error, according to this field. Use these values to identify failed or skipped data.
Load status loadResults.STATUS	string Optional	Snowflake per-file load result: file/status, counts, or location/message of the first load error, according to this field. Use these values to identify failed or skipped data.
Rows parsed loadResults.ROWS_PARSED	integer Optional	Snowflake per-file load result: file/status, counts, or location/message of the first load error, according to this field. Use these values to identify failed or skipped data.
Rows loaded loadResults.ROWS_LOADED	integer Optional	Snowflake per-file load result: file/status, counts, or location/message of the first load error, according to this field. Use these values to identify failed or skipped data.
Errors seen loadResults.ERRORS_SEEN	integer Optional	Snowflake per-file load result: file/status, counts, or location/message of the first load error, according to this field. Use these values to identify failed or skipped data.
First error loadResults.FIRST_ERROR	string Optional	Snowflake per-file load result: file/status, counts, or location/message of the first load error, according to this field. Use these values to identify failed or skipped data.
First error line loadResults.FIRST_ERROR_LINE	integer Optional	Snowflake per-file load result: file/status, counts, or location/message of the first load error, according to this field. Use these values to identify failed or skipped data.
First error character loadResults.FIRST_ERROR_CHARACTER	integer Optional	Snowflake per-file load result: file/status, counts, or location/message of the first load error, according to this field. Use these values to identify failed or skipped data.
First error column loadResults.FIRST_ERROR_COLUMN_NAME	string Optional	Snowflake per-file load result: file/status, counts, or location/message of the first load error, according to this field. Use these values to identify failed or skipped data.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"

Field / key	Type / required	Details
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Fault	Description
SNOWFLAKE_CONNECTION	The Snowflake shared resource or connection pool is unavailable.
SNOWFLAKE_DATABASE_JDBC-500005	Snowflake failed to create a prepared statement.
SNOWFLAKE_DATABASE_JDBC-500007	Snowflake failed to bind prepared-statement parameters.
SNOWFLAKE_DATABASE_JDBC-500009	Snowflake failed to execute the generated query.
SNOWFLAKE_DATABASE_JDBC-500013	The activity could not generate its structured output.
SNOWFLAKE_DATABASE_JDBC-500018	Uploading data to the Snowflake stage failed.
SNOWFLAKE_DATABASE_JDBC-500020	Bulk-load validation found invalid data.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



AMQP Receive Message

Activities / AMQP

Overview

Sends, receives, gets, or dead-letters AMQP messages.

The shared connection supports RabbitMQ AMQP 0.9.1/1.0 and AMQP 1.0 broker profiles. Destinations, subscriptions, typed messages, properties, settlement handles, recovery, SSL, sessions, and Azure dead-letter behavior are configured in MINA.

Activity type: amqp; operation: receive.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
AMQP connection resourceId	resource Required	Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment. Connection type: amqp
Queue name queueName	text Configuration setting	Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access.
Entity type entityType	select Configuration setting	AMQP routing configuration. entityType selects queue/topic where supported; exchanges route messages using the routing key and exchange type. Verify bindings before publishing. Choices: Queue, Topic
Entity name entityName	text Configuration setting	Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access.
Subscription name subscriptionName	text Configuration setting	Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access.
Durable subscription durableSubscription	boolean Configuration setting	Subscription/destination policy. Durable subscriptions retain eligible messages for an offline named subscriber; shared subscriptions distribute eligible deliveries where supported by the provider.
Shared subscription sharedSubscription	boolean Configuration setting	Subscription/destination policy. Durable subscriptions retain eligible messages for an offline named subscriber; shared subscriptions distribute eligible deliveries where supported by the provider.
Message type messageType	select Configuration setting	Message representation/media type. Match text, bytes or structured content to what the destination consumer expects. Choices: TextMessage, BytesMessage, Simple, Any
Maximum messages maxMessages	number Configuration setting	Bound on received/buffered messages or bytes, according to this field. Keep the bound within available memory and downstream processing capacity.

Field / key	Type / required	Details
Acknowledge mode acknowledgeMode	select Configuration setting	Message acknowledgement policy. Manual/client modes require explicit confirmation after successful processing; automatic/provider modes follow connector behavior. Choices: Auto, Client, Client or Dead Letter Queue
Azure receiver mode receiverMode	select Configuration setting	Provider/protocol or receive mode selection. Choose the broker implementation and delivery/settlement behavior supported by the configured connection. Choices: PeekLock, ReceiveAndDelete
Get correlation ID getCorrelationID	boolean Configuration setting	Include the provider message/correlation identifier in receive results when supported. These identifiers assist tracing and request/reply matching.
Azure session enabled sessionEnabled	boolean Configuration setting	Messaging/SAP session setting or identifier. Preserve the live session identifier for related operations; session semantics differ by provider.
Session receive type receiveType	select Configuration setting	Provider/protocol or receive mode selection. Choose the broker implementation and delivery/settlement behavior supported by the configured connection. Choices: FirstAvailableSession, SpecificSession, AllAvailableSessions
Maximum concurrent sessions maxConcurrentSessions	number Configuration setting	Provider/listener concurrency setting. Coordinate it with downstream capacity and the operation-specific runtime support; it is not a whole-workflow parallel-branch limit.
Specific session ID sessionId	text Configuration setting	Messaging/SAP session setting or identifier. Preserve the live session identifier for related operations; session semantics differ by provider.
Prefetch count / credits prefetchCount	number Configuration setting	Maximum number of messages prefetched before processing/settlement. Bound it to worker capacity to avoid excessive in-flight memory.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Dynamic queue queueName	string Optional	Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access.
Dynamic topic topicName	string Optional	Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access.
Dynamic subscription subscriptionName	string Optional	Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access.
Entity type entityType	string Optional	AMQP routing configuration. entityType selects queue/topic where supported; exchanges route messages using the routing key and exchange type. Verify bindings before publishing.
Entity name entityName	string Optional	Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access.

Field / key	Type / required	Details
Durable subscription durableSubscription	boolean Optional	Subscription/destination policy. Durable subscriptions retain eligible messages for an offline named subscriber; shared subscriptions distribute eligible deliveries where supported by the provider.
Shared subscription sharedSubscription	boolean Optional	Subscription/destination policy. Durable subscriptions retain eligible messages for an offline named subscriber; shared subscriptions distribute eligible deliveries where supported by the provider.
Message type messageType	string Optional	Message representation/media type. Match text, bytes or structured content to what the destination consumer expects.
Maximum messages maxMessages	number Optional	Bound on received/buffered messages or bytes, according to this field. Keep the bound within available memory and downstream processing capacity.
Acknowledge mode acknowledgeMode	string Optional	Message acknowledgement policy. Manual/client modes require explicit confirmation after successful processing; automatic/provider modes follow connector behavior.
Azure receiver mode receiverMode	string Optional	Provider/protocol or receive mode selection. Choose the broker implementation and delivery/settlement behavior supported by the configured connection.
Get correlation ID getCorrelationID	boolean Optional	Include the provider message/correlation identifier in receive results when supported. These identifiers assist tracing and request/reply matching.
Azure session enabled sessionEnabled	boolean Optional	Messaging/SAP session setting or identifier. Preserve the live session identifier for related operations; session semantics differ by provider.
Session receive type receiveType	string Optional	Provider/protocol or receive mode selection. Choose the broker implementation and delivery/settlement behavior supported by the configured connection.
Maximum concurrent sessions maxConcurrentSessions	number Optional	Provider/listener concurrency setting. Coordinate it with downstream capacity and the operation-specific runtime support; it is not a whole-workflow parallel-branch limit.
Specific session ID sessionId	string Optional	Messaging/SAP session setting or identifier. Preserve the live session identifier for related operations; session semantics differ by provider.
Prefetch count / credits prefetchCount	number Optional	Maximum number of messages prefetched before processing/settlement. Bound it to worker capacity to avoid excessive in-flight memory.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Message received received	boolean Optional	Operation outcome returned by the runtime. Inspect error output as well; provider acceptance does not establish processing by a downstream consumer.
User properties UserProperties	object Optional	Name/value metadata associated with the message. Keep business content in the body and map only properties understood by the receiving system.

Field / key	Type / required	Details
AMQP message properties MessageProperties	object Optional	Name/value metadata associated with the message. Keep business content in the body and map only properties understood by the receiving system.
Message body body	string|binary Optional	Message or response content. Map the business document rather than a previous publisher receipt. Objects retain their structure; string values carry serialized text.
Client/dead-letter settlement token settlementToken	string Optional	Acknowledgement/settlement handle from a received message. Confirm only after required downstream work succeeds; preserve the original handle.
Confirm Message handle ackId	string Optional	Acknowledgement/settlement handle from a received message. Confirm only after required downstream work succeeds; preserve the original handle.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
AMQPPluginException	The AMQP operation failed or its message configuration is invalid.
AMQPConnectionException	The broker connection failed or could not be recovered.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



AMQP Get Message

Activities / AMQP

Overview

Sends, receives, gets, or dead-letters AMQP messages.

The shared connection supports RabbitMQ AMQP 0.9.1/1.0 and AMQP 1.0 broker profiles. Destinations, subscriptions, typed messages, properties, settlement handles, recovery, SSL, sessions, and Azure dead-letter behavior are configured in MINA.

Activity type: amqp; operation: get.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
AMQP connection resourceId	resource Required	Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment. Connection type: amqp
Queue name queueName	text Configuration setting	Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access.
Entity type entityType	select Configuration setting	AMQP routing configuration. entityType selects queue/topic where supported; exchanges route messages using the routing key and exchange type. Verify bindings before publishing. Choices: Queue, Topic
Entity name entityName	text Configuration setting	Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access.
Subscription name subscriptionName	text Configuration setting	Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access.
Durable subscription durableSubscription	boolean Configuration setting	Subscription/destination policy. Durable subscriptions retain eligible messages for an offline named subscriber; shared subscriptions distribute eligible deliveries where supported by the provider.
Shared subscription sharedSubscription	boolean Configuration setting	Subscription/destination policy. Durable subscriptions retain eligible messages for an offline named subscriber; shared subscriptions distribute eligible deliveries where supported by the provider.
Message type messageType	select Configuration setting	Message representation/media type. Match text, bytes or structured content to what the destination consumer expects. Choices: TextMessage, BytesMessage, Simple
Maximum messages maxMessages	number Configuration setting	Bound on received/buffered messages or bytes, according to this field. Keep the bound within available memory and downstream processing capacity.

Field / key	Type / required	Details
Acknowledge mode acknowledgeMode	select Configuration setting	Message acknowledgement policy. Manual/client modes require explicit confirmation after successful processing; automatic/provider modes follow connector behavior. Choices: Auto, Client
Azure receiver mode receiverMode	select Configuration setting	Provider/protocol or receive mode selection. Choose the broker implementation and delivery/settlement behavior supported by the configured connection. Choices: PeekLock, ReceiveAndDelete
Get correlation ID getCorrelationID	boolean Configuration setting	Include the provider message/correlation identifier in receive results when supported. These identifiers assist tracing and request/reply matching.
Azure session enabled sessionEnabled	boolean Configuration setting	Messaging/SAP session setting or identifier. Preserve the live session identifier for related operations; session semantics differ by provider.
Session receive type receiveType	select Configuration setting	Provider/protocol or receive mode selection. Choose the broker implementation and delivery/settlement behavior supported by the configured connection. Choices: FirstAvailableSession, SpecificSession, AllAvailableSessions
Maximum concurrent sessions maxConcurrentSessions	number Configuration setting	Provider/listener concurrency setting. Coordinate it with downstream capacity and the operation-specific runtime support; it is not a whole-workflow parallel-branch limit.
Specific session ID sessionId	text Configuration setting	Messaging/SAP session setting or identifier. Preserve the live session identifier for related operations; session semantics differ by provider.
Prefetch count / credits prefetchCount	number Configuration setting	Maximum number of messages prefetched before processing/settlement. Bound it to worker capacity to avoid excessive in-flight memory.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Dynamic queue queueName	string Optional	Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access.
Dynamic topic topicName	string Optional	Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access.
Dynamic subscription subscriptionName	string Optional	Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access.
Entity type entityType	string Optional	AMQP routing configuration. entityType selects queue/topic where supported; exchanges route messages using the routing key and exchange type. Verify bindings before publishing.
Entity name entityName	string Optional	Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access.

Field / key	Type / required	Details
Durable subscription durableSubscription	boolean Optional	Subscription/destination policy. Durable subscriptions retain eligible messages for an offline named subscriber; shared subscriptions distribute eligible deliveries where supported by the provider.
Shared subscription sharedSubscription	boolean Optional	Subscription/destination policy. Durable subscriptions retain eligible messages for an offline named subscriber; shared subscriptions distribute eligible deliveries where supported by the provider.
Message type messageType	string Optional	Message representation/media type. Match text, bytes or structured content to what the destination consumer expects.
Maximum messages maxMessages	number Optional	Bound on received/buffered messages or bytes, according to this field. Keep the bound within available memory and downstream processing capacity.
Acknowledge mode acknowledgeMode	string Optional	Message acknowledgement policy. Manual/client modes require explicit confirmation after successful processing; automatic/provider modes follow connector behavior.
Azure receiver mode receiverMode	string Optional	Provider/protocol or receive mode selection. Choose the broker implementation and delivery/settlement behavior supported by the configured connection.
Get correlation ID getCorrelationID	boolean Optional	Include the provider message/correlation identifier in receive results when supported. These identifiers assist tracing and request/reply matching.
Azure session enabled sessionEnabled	boolean Optional	Messaging/SAP session setting or identifier. Preserve the live session identifier for related operations; session semantics differ by provider.
Session receive type receiveType	string Optional	Provider/protocol or receive mode selection. Choose the broker implementation and delivery/settlement behavior supported by the configured connection.
Maximum concurrent sessions maxConcurrentSessions	number Optional	Provider/listener concurrency setting. Coordinate it with downstream capacity and the operation-specific runtime support; it is not a whole-workflow parallel-branch limit.
Specific session ID sessionId	string Optional	Messaging/SAP session setting or identifier. Preserve the live session identifier for related operations; session semantics differ by provider.
Prefetch count / credits prefetchCount	number Optional	Maximum number of messages prefetched before processing/settlement. Bound it to worker capacity to avoid excessive in-flight memory.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Message received received	boolean Optional	Operation outcome returned by the runtime. Inspect error output as well; provider acceptance does not establish processing by a downstream consumer.
User properties UserProperties	object Optional	Name/value metadata associated with the message. Keep business content in the body and map only properties understood by the receiving system.

Field / key	Type / required	Details
AMQP message properties MessageProperties	object Optional	Name/value metadata associated with the message. Keep business content in the body and map only properties understood by the receiving system.
Message body body	string|binary Optional	Message or response content. Map the business document rather than a previous publisher receipt. Objects retain their structure; string values carry serialized text.
Client/dead-letter settlement token settlementToken	string Optional	Acknowledgement/settlement handle from a received message. Confirm only after required downstream work succeeds; preserve the original handle.
Confirm Message handle ackId	string Optional	Acknowledgement/settlement handle from a received message. Confirm only after required downstream work succeeds; preserve the original handle.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Fault	Description
AMQPPluginException	The AMQP operation failed or its message configuration is invalid.
AMQPConnectionException	The broker connection failed or could not be recovered.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



AMQP Send Message

Activities / AMQP

Overview

Sends, receives, gets, or dead-letters AMQP messages.

The shared connection supports RabbitMQ AMQP 0.9.1/1.0 and AMQP 1.0 broker profiles. Destinations, subscriptions, typed messages, properties, settlement handles, recovery, SSL, sessions, and Azure dead-letter behavior are configured in MINA.

Activity type: amqp; operation: send.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
AMQP connection resourceId	resource Required	Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment. Connection type: amqp
RabbitMQ destination type destinationType	select Configuration setting	AMQP routing configuration. entityType selects queue/topic where supported; exchanges route messages using the routing key and exchange type. Verify bindings before publishing. Choices: Queue, Exchange
Queue name queueName	text Configuration setting	Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access.
Entity type entityType	select Configuration setting	AMQP routing configuration. entityType selects queue/topic where supported; exchanges route messages using the routing key and exchange type. Verify bindings before publishing. Choices: Queue, Topic
Entity name entityName	text Configuration setting	Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access.
Exchange type exchangeType	select Configuration setting	AMQP routing configuration. entityType selects queue/topic where supported; exchanges route messages using the routing key and exchange type. Verify bindings before publishing. Choices: direct, topic, fanout, headers
Exchange name exchangeName	text Configuration setting	AMQP routing configuration. entityType selects queue/topic where supported; exchanges route messages using the routing key and exchange type. Verify bindings before publishing.
Routing key routingKey	text Configuration setting	AMQP routing configuration. entityType selects queue/topic where supported; exchanges route messages using the routing key and exchange type. Verify bindings before publishing.
Message type messageType	select Configuration setting	Message representation/media type. Match text, bytes or structured content to what the destination consumer expects. Choices: TextMessage, BytesMessage, Simple

Field / key	Type / required	Details
Get message ID getMessageID	boolean Configuration setting	Include the provider message/correlation identifier in receive results when supported. These identifiers assist tracing and request/reply matching.
Delivery mode deliveryMode	select Configuration setting	Message persistence policy. Persistent delivery asks the provider to retain the message according to its durability configuration; it does not guarantee consumer processing. Choices: Persistent, Non-Persistent
Expiration (msec) expiration	number Configuration setting	Message lifetime before expiry, in milliseconds for JMS/EMS. Zero requests no expiration; choose a TTL appropriate for delayed consumers.
Priority (0-9) priority	number Configuration setting	Provider message priority. JMS/EMS priorities range from 0 to 9; ordering effects depend on broker scheduling and destination policy.
Message type/property type	text Configuration setting	Type reported for this value or selected for this operation. Use the corresponding schema datatype and operation-specific choices.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
User properties userProperties	object Optional	Name/value metadata associated with the message. Keep business content in the body and map only properties understood by the receiving system.
Dynamic delivery mode deliveryMode	string Optional	Message persistence policy. Persistent delivery asks the provider to retain the message according to its durability configuration; it does not guarantee consumer processing.
Message ID messageID	string Optional	Message identifier returned by the provider or runtime. Retain it for tracing; use a separate stable business key for duplicate detection.
Dynamic expiration expiration	integer Optional	Message lifetime before expiry, in milliseconds for JMS/EMS. Zero requests no expiration; choose a TTL appropriate for delayed consumers.
Dynamic priority priority	integer Optional	Provider message priority. JMS/EMS priorities range from 0 to 9; ordering effects depend on broker scheduling and destination policy.
Dynamic type type	string Optional	Type reported for this value or selected for this operation. Use the corresponding schema datatype and operation-specific choices.
Content type contentType	string Optional	Message representation/media type. Match text, bytes or structured content to what the destination consumer expects.
Azure session ID sessionID	string Optional	Messaging/SAP session setting or identifier. Preserve the live session identifier for related operations; session semantics differ by provider.
Correlation ID correlationID	string Optional	Correlation identifier for tracing or request/reply matching. Preserve the request value in the reply rather than generating an unrelated identifier.

Field / key	Type / required	Details
Dynamic exchange exchangeName	string Optional	AMQP routing configuration. entityType selects queue/topic where supported; exchanges route messages using the routing key and exchange type. Verify bindings before publishing.
Dynamic routing key routingKey	string Optional	AMQP routing configuration. entityType selects queue/topic where supported; exchanges route messages using the routing key and exchange type. Verify bindings before publishing.
Dynamic topic topicName	string Optional	Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access.
Dynamic queue queueName	string Optional	Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access.
Message body body	string|binary Required	Message or response content. Map the business document rather than a previous publisher receipt. Objects retain their structure; string values carry serialized text.
RabbitMQ destination type destinationType	string Optional	AMQP routing configuration. entityType selects queue/topic where supported; exchanges route messages using the routing key and exchange type. Verify bindings before publishing.
Entity type entityType	string Optional	AMQP routing configuration. entityType selects queue/topic where supported; exchanges route messages using the routing key and exchange type. Verify bindings before publishing.
Entity name entityName	string Optional	Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access.
Exchange type exchangeType	string Optional	AMQP routing configuration. entityType selects queue/topic where supported; exchanges route messages using the routing key and exchange type. Verify bindings before publishing.
Message type messageType	string Optional	Message representation/media type. Match text, bytes or structured content to what the destination consumer expects.
Get message ID getMessageID	boolean Optional	Include the provider message/correlation identifier in receive results when supported. These identifiers assist tracing and request/reply matching.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Send succeeded sendResult	boolean Optional	Provider result of the send attempt. Inspect its status/identifier together with faults; it does not indicate downstream consumption.
Broker message ID MessageId	string Optional	Message identifier returned by the provider or runtime. Retain it for tracing; use a separate stable business key for duplicate detection.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

Field / key	Type / required	Details
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Fault	Description
AMQPPluginException	The AMQP operation failed or its message configuration is invalid.
AMQPConnectionException	The broker connection failed or could not be recovered.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



AMQP Dead Letter Message

Activities / AMQP

Overview

Sends, receives, gets, or dead-letters AMQP messages.

The shared connection supports RabbitMQ AMQP 0.9.1/1.0 and AMQP 1.0 broker profiles. Destinations, subscriptions, typed messages, properties, settlement handles, recovery, SSL, sessions, and Azure dead-letter behavior are configured in MINA.

Activity type: amqp; operation: dead_letter.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
AMQP connection resourceId	resource Required	Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment. Connection type: amqp
Use retry useRetry	boolean Configuration setting	Retry attempt policy. A retry can repeat an accepted external write; use stable identifiers and destination-side duplicate handling.
Total timeout (seconds) totalTimeoutSeconds	number Configuration setting	Total timeout (seconds): bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.
Maximum attempts maxAttempts	number Configuration setting	Retry attempt policy. A retry can repeat an accepted external write; use stable identifiers and destination-side duplicate handling.
Backoff time (msec) backoffTimeMsec	number Configuration setting	Backoff time (msec): bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Settlement token settlementToken	string Required	Acknowledgement/settlement handle from a received message. Confirm only after required downstream work succeeds; preserve the original handle.
Dead-letter reason deadLetterReason	string Optional	Reason and diagnostic description attached when rejecting a message to the provider dead-letter destination. Keep descriptions useful and free of credentials.
Dead-letter error description deadLetterErrorDescription	string Optional	Reason and diagnostic description attached when rejecting a message to the provider dead-letter destination. Keep descriptions useful and free of credentials.

Field / key	Type / required	Details
Dynamic retry useRetry	boolean Optional	Retry attempt policy. A retry can repeat an accepted external write; use stable identifiers and destination-side duplicate handling.
Dynamic total timeout totalTimeoutSeconds	integer Optional	Dynamic total timeout: bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.
Dynamic attempts maxAttempts	integer Optional	Retry attempt policy. A retry can repeat an accepted external write; use stable identifiers and destination-side duplicate handling.
Dynamic backoff backoffTimeMsec	integer Optional	Dynamic backoff: bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Dead-letter status status	string Optional	Operation outcome returned by the runtime. Inspect error output as well; provider acceptance does not establish processing by a downstream consumer.
Settlement token settlementToken	string Optional	Acknowledgement/settlement handle from a received message. Confirm only after required downstream work succeeds; preserve the original handle.
Processed message ID messageId	string Optional	Message identifier returned by the provider or runtime. Retain it for tracing; use a separate stable business key for duplicate detection.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Configuration setting	Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Configuration setting	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "${properties.advanced.retryIntervalSeconds}"
retryBackoff retryBackoff	string expression Configuration setting	Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval. Default: "fixed"
retryMaxIntervalSeconds retryMaxIntervalSeconds	number expression Configuration setting	Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater. Default: ""

Field / key	Type / required	Details
retryMaxElapsedSeconds retryMaxElapsedSeconds	number expression Configuration setting	Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted. Default: 0
retryJitter retryJitter	boolean expression Configuration setting	Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts. Default: false
retryFaultTypes retryFaultTypes	string expression Configuration setting	Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried. Default: ""

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

Errors

Fault	Description
AMQPPluginException	The AMQP operation failed or its message configuration is invalid.
AMQPConnectionException	The broker connection failed or could not be recovered.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



Parse XML

Activities / Data

Overview

Parses XML text into an XSD-defined tree or renders an XSD-defined tree as XML.

Parse XML accepts serialized XML in Input and defines its published tree in Output Editor. Render XML defines and maps its source tree in Input Editor, then emits xmlString (or binary output when selected).

Activity type: xml; operation: parse.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Input style inputStyle	select Configuration setting	Input/execution mode selected from the available choices. Match inline text, file or structured input to the actual source; connection modes such as mock/memory are for local testing. Choices: Text, Binary, Dynamic
Validate parsed output validateOutput	boolean Configuration setting	Validate the associated value against the configured contract. A mismatch raises a validation fault; this does not replace validation of business rules.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The selected XSD or inline schema determines actual parse output or render input.

Field / key	Type / required	Details
XML text xmlString	string Optional	Textual document/content. Supply valid XML/JSON for parsers, or map rendered text for a wire-format destination. Encoding controls conversion to and from bytes.
Validate parsed output formatOptions.validateOutput	boolean Optional	Validate the associated value against the configured contract. A mismatch raises a validation fault; this does not replace validation of business rules.
Fallback binary encoding formatOptions.forceEncoding	string Optional	Character encoding for text conversion, for example UTF-8. Use the sender encoding when parsing; a wrong encoding can corrupt non-ASCII characters.
Legacy binary encoding formatOptions.encoding	string Optional	Character encoding for text conversion, for example UTF-8. Use the sender encoding when parsing; a wrong encoding can corrupt non-ASCII characters.

Output

The selected XSD or inline schema determines actual parse output or render input.

Field / key	Type / required	Details
Parsed XML xml	xml Optional	Textual document/content. Supply valid XML/JSON for parsers, or map rendered text for a wire-format destination. Encoding controls conversion to and from bytes.
XML text xmlString	string Optional	Textual document/content. Supply valid XML/JSON for parsers, or map rendered text for a wire-format destination. Encoding controls conversion to and from bytes.
Root element root	string Optional	Qualified root element name of the parsed XML document. Its namespace URI is retained; use the parsed value/tree for element and attribute mappings.
Parsed tree value	object Optional	The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
XML_PARSE	The XML input cannot be parsed.
XML_RENDER	The value cannot be rendered as XML.
SCHEMA_VALIDATION	The data does not conform to the selected schema.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



Render XML

Activities / Data

Overview

Parses XML text into an XSD-defined tree or renders an XSD-defined tree as XML.

Parse XML accepts serialized XML in Input and defines its published tree in Output Editor. Render XML defines and maps its source tree in Input Editor, then emits xmlString (or binary output when selected).

Activity type: xml; operation: render.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Output style outputStyle	select Configuration setting	Representation used for reading, writing or returning the value. Select the mode matching the actual document and downstream consumer contract. Choices: Text, Binary
Validate input tree validateInput	boolean Configuration setting	Validate the associated value against the configured contract. A mismatch raises a validation fault; this does not replace validation of business rules.
Suppress XML declaration suppressXmlDeclaration	boolean Configuration setting	Omit the XML declaration from rendered XML when the receiving interface requires only the document element.
Pretty print prettyPrint	boolean Configuration setting	Output formatting. prettyPrint enables readable layout and indent selects indentation width; formatting does not change business values.
Use default namespace formatUsingDefaultNamespacePrefix	boolean Configuration setting	Render the root namespace as the default namespace while preserving the namespace of each child and attribute.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The selected XSD or inline schema determines actual parse output or render input.

Field / key	Type / required	Details
XML schema tree value	object Optional	The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes.
Existing XML document xmlString	string Optional	Map an XML document to validate and serialize it directly.
Validate input tree formatOptions.validateInput	boolean Optional	Validate the associated value against the configured contract. A mismatch raises a validation fault; this does not replace validation of business rules.

Field / key	Type / required	Details
Encoding formatOptions.encoding	string Optional	Character encoding for text conversion, for example UTF-8. Use the sender encoding when parsing; a wrong encoding can corrupt non-ASCII characters.
Suppress XML declaration formatOptions.suppressXmlDeclaration	boolean Optional	Omit the XML declaration from rendered XML when the receiving interface requires only the document element.
Pretty print formatOptions.prettyPrint	boolean Optional	Output formatting. prettyPrint enables readable layout and indent selects indentation width; formatting does not change business values.
Use default namespace formatOptions.formatUsingDefaultNamespacePrefix	boolean Optional	Render the root namespace as the default namespace while preserving the namespace of each child and attribute.
XSD root override formatOptions.rootElement	string Optional	Optional global element name. The first declared root is used by default.
CDATA element paths formatOptions.cdataElements	string Optional	Map a comma-separated list of schema paths, such as order.notes.

Output

The selected XSD or inline schema determines actual parse output or render input.

Field / key	Type / required	Details
Rendered XML text xmlString	string Optional	Textual document/content. Supply valid XML/JSON for parsers, or map rendered text for a wire-format destination. Encoding controls conversion to and from bytes.
Rendered content content	string Optional	Textual document/content. Supply valid XML/JSON for parsers, or map rendered text for a wire-format destination. Encoding controls conversion to and from bytes.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
XML_PARSE	The XML input cannot be parsed.
XML_RENDER	The value cannot be rendered as XML.
SCHEMA_VALIDATION	The data does not conform to the selected schema.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



Parse JSON

Activities / Data

Overview

Converts between serialized JSON and a schema-defined activity tree.

Parse JSON accepts jsonString and publishes the structure selected in Output Editor. Render JSON maps the structure selected in Input Editor and emits jsonString.

Activity type: json; operation: parse.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Schema type schemaType	select Configuration setting	Choose Native JSON for ordinary JSON data, Generic for XML and JSON conversion without a schema, or XSD for a schema-defined XML structure. Choices: Native JSON, Generic, XSD
Validate parsed output validateOutput	boolean Configuration setting	Validate the associated value against the configured contract. A mismatch raises a validation fault; this does not replace validation of business rules.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The selected JSON schema determines actual parse output or render input.

Field / key	Type / required	Details
JSON text jsonString	string Required	Textual document/content. Supply valid XML/JSON for parsers, or map rendered text for a wire-format destination. Encoding controls conversion to and from bytes.
Validate parsed output formatOptions.validateOutput	boolean Optional	Validate the associated value against the configured contract. A mismatch raises a validation fault; this does not replace validation of business rules.
Duplicate keys formatOptions.duplicateKeyPolicy	string Optional	Allowed values: Last wins, First wins, Error.

Output

The selected JSON schema determines actual parse output or render input.

Field / key	Type / required	Details
Parsed JSON value value	json Optional	The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
JSON_PARSE	The JSON input cannot be parsed.
JSON_RENDER	The value cannot be rendered as JSON.
SCHEMA_VALIDATION	The data does not conform to the selected schema.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



Render JSON

Activities / Data

Overview

Converts between serialized JSON and a schema-defined activity tree.

Parse JSON accepts jsonString and publishes the structure selected in Output Editor. Render JSON maps the structure selected in Input Editor and emits jsonString.

Activity type: json; operation: render.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Schema type schemaType	select Configuration setting	Choose Native JSON for ordinary JSON data, Generic for XML and JSON conversion without a schema, or XSD for a schema-defined XML structure. Choices: Native JSON, Generic, XSD
Validate input tree validateInput	boolean Configuration setting	Validate the associated value against the configured contract. A mismatch raises a validation fault; this does not replace validation of business rules.
Omit null fields omitNulls	boolean Configuration setting	Omit null-valued properties from rendered output. Confirm that absence and explicit null have the intended meaning in the target contract.
Pretty print prettyPrint	boolean Configuration setting	Output formatting. prettyPrint enables readable layout and indent selects indentation width; formatting does not change business values.
Escape non-ASCII characters asciiOnly	boolean Configuration setting	Escape non-ASCII characters in rendered JSON when enabled. Disable to retain readable Unicode characters in the output text.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The selected JSON schema determines actual parse output or render input.

Field / key	Type / required	Details
JSON value value	any Optional	The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes.
Validate input tree formatOptions.validateInput	boolean Optional	Validate the associated value against the configured contract. A mismatch raises a validation fault; this does not replace validation of business rules.
JSON root formatOptions.rootStyle	string Optional	Allowed values: With root, Anonymous.

Field / key	Type / required	Details
Omit null fields formatOptions.omitNulls	boolean Optional	Omit null-valued properties from rendered output. Confirm that absence and explicit null have the intended meaning in the target contract.
Pretty print formatOptions.prettyPrint	boolean Optional	Output formatting. prettyPrint enables readable layout and indent selects indentation width; formatting does not change business values.
Indent spaces formatOptions.indent	integer Optional	Output formatting. prettyPrint enables readable layout and indent selects indentation width; formatting does not change business values.
Escape non-ASCII characters formatOptions.asciiOnly	boolean Optional	Escape non-ASCII characters in rendered JSON when enabled. Disable to retain readable Unicode characters in the output text.

Output

The selected JSON schema determines actual parse output or render input.

Field / key	Type / required	Details
Rendered JSON text jsonString	string Optional	Textual document/content. Supply valid XML/JSON for parsers, or map rendered text for a wire-format destination. Encoding controls conversion to and from bytes.
Rendered content content	string Optional	Textual document/content. Supply valid XML/JSON for parsers, or map rendered text for a wire-format destination. Encoding controls conversion to and from bytes.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
JSON_PARSE	The JSON input cannot be parsed.
JSON_RENDER	The value cannot be rendered as JSON.
SCHEMA_VALIDATION	The data does not conform to the selected schema.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



Parse Data

Activities / Data

Overview

Parses delimited/fixed-width text into records or renders records as formatted text.

The schema contract defines record fields; Configuration controls separators, widths, headers, trimming, and line endings. Parse Data publishes records while Render Data accepts mapped records and emits text.

Activity type: flat; operation: parse.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Input source inputSource	select Configuration setting	Input/execution mode selected from the available choices. Match inline text, file or structured input to the actual source; connection modes such as mock/memory are for local testing. Choices: String, File path
Manually specify start record manualStartRecord	boolean Configuration setting	Expose the zero-based start offset in Input. Otherwise a batch size advances the offset within the task.
Continue on record errors continueOnError	boolean Configuration setting	Report rejected records in ErrorRows and continue with the remaining records.
Shared Data Format dataFormatId	dataFormat Configuration setting	Choose a project JSON schema asset containing a dataFormat object, or configure the format inline.
First row is header header	boolean Configuration setting	Header/starting-row selection for tabular data. Row-number fields use the activity-defined indexing; Excel headerRow/startRow are one-based. Shown when: (c) => !c.dataFormatId
Require exact column count strictColumns	boolean Configuration setting	Require record column counts to match the configured flat-data layout; inconsistent records fault instead of being silently accepted. Shown when: (c) => !c.dataFormatId
Trim parsed values trimValues	boolean Configuration setting	Trim leading/trailing whitespace from parsed values. Disable when spaces are significant in the source contract. Shown when: (c) => !c.dataFormatId
Append final line separator includeFinalLineSeparator	boolean Configuration setting	Line separator handling for text output. Choose the delimiter expected by the consumer; includeFinalLineSeparator adds a separator after the last record. Shown when: (c) => !c.dataFormatId

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The record schema determines child fields under records.

Field / key	Type / required	Details
Formatted data string text	string Optional	Textual document/content. Supply valid XML/JSON for parsers, or map rendered text for a wire-format destination. Encoding controls conversion to and from bytes.
Header characters to skip skipHeaderCharacters	integer Optional	Number of leading characters to ignore before reading the header and records.
Records to parse (-1 for all) noOfRecords	integer Optional	Maximum input records to read in this batch, including rejected records. Use -1 to read to the end.
Manually specify start record formatOptions.manualStartRecord	boolean Optional	Expose the zero-based start offset in Input. Otherwise a batch size advances the offset within the task.
Continue on record errors formatOptions.continueOnError	boolean Optional	Report rejected records in ErrorRows and continue with the remaining records.
Output XML root element formatOptions.rootElement	string Optional	XML element name used for the document root or repeating record. Match the target XSD and namespace requirements.
Output XML record element formatOptions.recordElement	string Optional	XML element name used for the document root or repeating record. Match the target XSD and namespace requirements.
Format formatOptions.format	string Optional	Allowed values: delimited, fixed.
Column separator formatOptions.delimiter	string Optional	Separator used to split/join record fields. Choose a separator consistent with the data contract and account for quoting or embedded delimiters.
Multi-character separator rule formatOptions.separatorRule	string Optional	Allowed values: single, any-character.
Line separator formatOptions.lineSeparator	string Optional	Allowed values: Auto, LF, CRLF, CR.
First row is header formatOptions.header	boolean Optional	Header/starting-row selection for tabular data. Row-number fields use the activity-defined indexing; Excel headerRow/startRow are one-based.
Field names formatOptions.fields	string Optional	Comma-separated names used when the input has no header. An attached schema supplies these automatically.
Field data types formatOptions.fieldTypes	string Optional	Optional comma-separated XML schema types such as string, integer, decimal, and boolean.
Fixed field widths formatOptions.widths	string Optional	Fixed-width layout: widths defines each field length and fillCharacter supplies padding. Ensure lengths match the sender/receiver record specification.
Fixed field offsets formatOptions.fieldOffsets	string Optional	Optional zero-based start:end offsets (end excluded), one per field. Overrides widths.
Fixed line length including separator formatOptions.lineLength	integer Optional	Total fixed record length, including the configured line separator. It must cover all field offsets.
Fixed-width fill character formatOptions.fillCharacter	string Optional	Fixed-width layout: widths defines each field length and fillCharacter supplies padding. Ensure lengths match the sender/receiver record specification.
Require exact column count formatOptions.strictColumns	boolean Optional	Require record column counts to match the configured flat-data layout; inconsistent records fault instead of being silently accepted.
Trim parsed values formatOptions.trimValues	boolean Optional	Trim leading/trailing whitespace from parsed values. Disable when spaces are significant in the source contract.

Field / key	Type / required	Details
Blank lines formatOptions.blankLinePolicy	string Optional	Allowed values: Skip, Stop, Keep.
Append final line separator formatOptions.includeFinalLineSeparator	boolean Optional	Line separator handling for text output. Choose the delimiter expected by the consumer; includeFinalLineSeparator adds a separator after the last record.
Custom line ending formatOptions.lineEnding	string Optional	Map a custom line ending when no line separator is selected.
Skip blank records formatOptions.skipBlankLines	boolean Optional	Ignore empty input lines/rows. Disable when blank records carry a business meaning or positional layout.

Output

The record schema determines child fields under records.

Field / key	Type / required	Details
Parsed data XML xml	xml Optional	Textual document/content. Supply valid XML/JSON for parsers, or map rendered text for a wire-format destination. Encoding controls conversion to and from bytes.
XML text xmlString	string Optional	Textual document/content. Supply valid XML/JSON for parsers, or map rendered text for a wire-format destination. Encoding controls conversion to and from bytes.
Root element root	string Optional	Qualified root element name of the parsed XML document. Its namespace URI is retained; use the parsed value/tree for element and attribute mappings.
Parsed tree value	object Optional	The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes.
Typed records records	array Optional	Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema.
Record count recordCount	integer Optional	Number of parsed or rendered data records, excluding the header row. Use this integer to validate expected row counts or report processing totals.
Field names fields	array Optional	Ordered field names from the parsed header, configured field list or selected flat-record schema. The order determines record column positions.
Parsed row structure Rows.schema	object Optional	Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions.
Rejected records ErrorRows.error	array Optional	Fault/debug metadata identifying the activity, error code, underlying cause, diagnostic details or stack trace. Use it to diagnose the failing operation.
Record number ErrorRows.error.recordNumber	integer Optional	One-based record number after the preamble and header, for the rejected record.
Error message ErrorRows.error.message	string Optional	Message or response content. Map the business document rather than a previous publisher receipt. Objects retain their structure; string values carry serialized text.
Raw record ErrorRows.error.raw	string Optional	Input values from the rejected record, before conversion to the configured field types.

Field / key	Type / required	Details
End of input done	boolean Optional	True when no more records are available from this input.
Next record offset nextRecord	integer Optional	Zero-based offset for the next batch when manually choosing the start record.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
FLAT_PARSE	The FLAT input cannot be parsed.
FLAT_RENDER	The value cannot be rendered as FLAT.
SCHEMA_VALIDATION	The data does not conform to the selected schema.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



Render Data

Activities / Data

Overview

Parses delimited/fixed-width text into records or renders records as formatted text.

The schema contract defines record fields; Configuration controls separators, widths, headers, trimming, and line endings. Parse Data publishes records while Render Data accepts mapped records and emits text.

Activity type: flat; operation: render.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Shared Data Format dataFormatId	dataFormat Configuration setting	Choose a project JSON schema asset containing a dataFormat object, or configure the format inline.
First row is header header	boolean Configuration setting	Header/starting-row selection for tabular data. Row-number fields use the activity-defined indexing; Excel headerRow/startRow are one-based. Shown when: (c) => !c.dataFormatId
Require exact column count strictColumns	boolean Configuration setting	Require record column counts to match the configured flat-data layout; inconsistent records fault instead of being silently accepted. Shown when: (c) => !c.dataFormatId
Trim parsed values trimValues	boolean Configuration setting	Trim leading/trailing whitespace from parsed values. Disable when spaces are significant in the source contract. Shown when: (c) => !c.dataFormatId
Append final line separator includeFinalLineSeparator	boolean Configuration setting	Line separator handling for text output. Choose the delimiter expected by the consumer; includeFinalLineSeparator adds a separator after the last record. Shown when: (c) => !c.dataFormatId

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The record schema determines child fields under records.

Field / key	Type / required	Details
Data records records	array Required	Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema.
Format formatOptions.format	string Optional	Allowed values: delimited, fixed.
Column separator formatOptions.delimiter	string Optional	Separator used to split/join record fields. Choose a separator consistent with the data contract and account for quoting or embedded delimiters.

Field / key	Type / required	Details
Multi-character separator rule formatOptions.separatorRule	string Optional	Allowed values: single, any-character.
Line separator formatOptions.lineSeparator	string Optional	Allowed values: Auto, LF, CRLF, CR.
First row is header formatOptions.header	boolean Optional	Header/starting-row selection for tabular data. Row-number fields use the activity-defined indexing; Excel headerRow/startRow are one-based.
Field names formatOptions.fields	string Optional	Comma-separated names used when the input has no header. An attached schema supplies these automatically.
Field data types formatOptions.fieldTypes	string Optional	Optional comma-separated XML schema types such as string, integer, decimal, and boolean.
Fixed field widths formatOptions.widths	string Optional	Fixed-width layout: widths defines each field length and fillCharacter supplies padding. Ensure lengths match the sender/receiver record specification.
Fixed field offsets formatOptions.fieldOffsets	string Optional	Optional zero-based start:end offsets (end excluded), one per field. Overrides widths.
Fixed line length including separator formatOptions.lineLength	integer Optional	Total fixed record length, including the configured line separator. It must cover all field offsets.
Fixed-width fill character formatOptions.fillCharacter	string Optional	Fixed-width layout: widths defines each field length and fillCharacter supplies padding. Ensure lengths match the sender/receiver record specification.
Require exact column count formatOptions.strictColumns	boolean Optional	Require record column counts to match the configured flat-data layout; inconsistent records fault instead of being silently accepted.
Trim parsed values formatOptions.trimValues	boolean Optional	Trim leading/trailing whitespace from parsed values. Disable when spaces are significant in the source contract.
Append final line separator formatOptions.includeFinalLineSeparator	boolean Optional	Line separator handling for text output. Choose the delimiter expected by the consumer; includeFinalLineSeparator adds a separator after the last record.
Custom line ending formatOptions.lineEnding	string Optional	Map a custom line ending when no line separator is selected.
Skip blank records formatOptions.skipBlankLines	boolean Optional	Ignore empty input lines/rows. Disable when blank records carry a business meaning or positional layout.

Output

The record schema determines child fields under records.

Field / key	Type / required	Details
Rendered FLAT text text	string Optional	Textual document/content. Supply valid XML/JSON for parsers, or map rendered text for a wire-format destination. Encoding controls conversion to and from bytes.
Rendered content content	string Optional	Textual document/content. Supply valid XML/JSON for parsers, or map rendered text for a wire-format destination. Encoding controls conversion to and from bytes.
Formatted data text Text	string Optional	Formatted records as text, including the configured column and line separators.

Field / key	Type / required	Details
Record count recordCount	integer Optional	Number of parsed or rendered data records, excluding the header row. Use this integer to validate expected row counts or report processing totals.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
FLAT_PARSE	The FLAT input cannot be parsed.
FLAT_RENDER	The value cannot be rendered as FLAT.
SCHEMA_VALIDATION	The data does not conform to the selected schema.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



Read Excel Workbook

Activities / Data

Overview

Reads an Excel workbook and publishes every selected worksheet as nested JSON.

Leave Worksheet blank to read every tab. The configured header row becomes field names; dotted headers such as customer.address.city create nested objects. Formula cells can return their cached calculated values.

Activity type: excel; operation: read.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Workbook file path filePath	text Configuration setting	Reads .xlsx, .xlsm, or legacy .xls workbooks at runtime.
Worksheet name sheetName	text Configuration setting	Leave blank to return every worksheet/tab.
Header row headerRow	number Configuration setting	Header/starting-row selection for tabular data. Row-number fields use the activity-defined indexing; Excel headerRow/startRow are one-based.
First data row startRow	number Configuration setting	Header/starting-row selection for tabular data. Row-number fields use the activity-defined indexing; Excel headerRow/startRow are one-based.
Maximum rows per sheet (0 = all) maximumRows	number Configuration setting	Upper bound on returned rows/items. Use a positive bound for large datasets and page/process subsequent batches where available.
Return calculated cell values dataOnly	boolean Configuration setting	Read stored Excel formula results instead of formula text. Results depend on the workbook having been calculated and saved by a spreadsheet application.
Convert dotted headers into nested JSON fields nestedHeaders	boolean Configuration setting	Interpret dotted Excel header names as nested object paths when enabled; otherwise retain literal column names.
Skip blank rows skipBlankRows	boolean Configuration setting	Ignore empty input lines/rows. Disable when blank records carry a business meaning or positional layout.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Dynamic workbook path filePath	string Optional	File path on the runtime host. Ensure the runtime account can read the file and that the path exists in the deployment environment.

Field / key	Type / required	Details
Dynamic worksheet/tab name sheetName	string Optional	Excel worksheet name. Supply an exact existing sheet name, or leave empty to read the workbook sheets supported by this operation.
Dynamic header row headerRow	integer Optional	Header/starting-row selection for tabular data. Row-number fields use the activity-defined indexing; Excel headerRow/startRow are one-based.
Dynamic first row startRow	integer Optional	Header/starting-row selection for tabular data. Row-number fields use the activity-defined indexing; Excel headerRow/startRow are one-based.
Dynamic maximum rows maximumRows	integer Optional	Upper bound on returned rows/items. Use a positive bound for large datasets and page/process subsequent batches where available.
Return calculated cell values dataOnly	boolean Optional	Read stored Excel formula results instead of formula text. Results depend on the workbook having been calculated and saved by a spreadsheet application.
Convert dotted headers into nested JSON fields nestedHeaders	boolean Optional	Interpret dotted Excel header names as nested object paths when enabled; otherwise retain literal column names.
Skip blank rows skipBlankRows	boolean Optional	Ignore empty input lines/rows. Disable when blank records carry a business meaning or positional layout.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Workbook result workbook	object Optional	Workbook result metadata and worksheet collection. Each sheet includes its name, headers and row objects.
Workbook file name workbook.fileName	string Optional	File metadata: name, full path, directory, modification time, access flags or size in bytes, according to this field. Values describe the runtime filesystem.
Worksheet count workbook.sheetCount	integer Optional	Workbook result metadata and worksheet collection. Each sheet includes its name, headers and row objects.
Worksheets/tabs workbook.sheets	array Optional	Workbook result metadata and worksheet collection. Each sheet includes its name, headers and row objects.
Worksheet name workbook.sheets.name	string Optional	Name identifying this entry. Keep it stable wherever downstream mappings or matching rules refer to it.
Column headers workbook.sheets.headers	array Optional	Header name/value object. Map only headers required by the target protocol; credentials belong in protected authentication settings.
Nested JSON row data workbook.sheets.rows	array Optional	Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

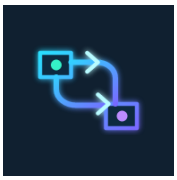
This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
EXCEL_READ	The workbook, requested worksheet, or cell data could not be read.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



Mapper

Activities / General

Overview

Maps execution-path data into a selected target schema.

In Input, drop a repeating complex source onto a repeating complex target. Copy-Of copies the whole collection and hides child bindings. For-Each creates editable child bindings for matching names; SmartMap also matches names that differ by underscores or hyphens. The loop shows the collection expression once and child expressions relative to the current item. Run Map & Test to check the output.

Activity type: mapper; operation: map.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Target schema targetSchemaId	schema selection Configuration setting	Select a schema from project Resources. Studio shows its name and derives the Input/Output trees, datatypes and cardinalities from its content. Select Inline schema to author a contract locally.
Inline target schema targetSchemaText	JSON Schema / XSD Configuration setting	Paste the target JSON Schema or XSD when Inline schema is selected. Update affected mapping paths after changing fields or repeating structures.
Mapping dialect language	select Configuration setting	Select the expression dialect presented by the editor. Use MINA-supported mapping expressions and functions; a compatibility selection does not supply a complete external XPath/XSLT processor. Choices: XPath 2.0 / functions, XPath 1.0 compatibility, JSONPath / functions, XSLT 2.0 compatibility Default: "XPath 2.0 / functions"
Null and missing values nullPolicy	select Configuration setting	Choose whether an unresolved/null mapped value omits its target, preserves null, emits an empty string or uses Default null value. Individual mapping rules can override the policy. Choices: omit, preserve, empty-string, default Default: "omit"
Default null value defaultValue	value Configuration setting	Fallback used for null/missing mapped values under the default policy. Supply a value compatible with the target datatype. Shown when: nullPolicy = default
Type coercion typeCoercion	select Configuration setting	Strict enforces target types; safe allows supported conversions; off leaves mapped values uncoerced. Target-output validation can still reject values that do not match the schema. Choices: strict, safe, off Default: "safe"
Mapping error behavior onMappingError	select Configuration setting	Choose whether a field expression/conversion error fails the activity, omits that field or emits null. Continuing can produce incomplete output, so retain target validation where required. Choices: fail, skip-field, use-null Default: "fail"

Field / key	Type / required	Details
Maximum output size (KB) maxOutputSizeKb	integer Configuration setting	Bound the generated output size in kilobytes. Zero disables this bound; use a positive limit to reject oversized mapped documents. Default: 0
Validate target output validateOutput	boolean Configuration setting	Check the completed mapped result against the selected target contract. A schema mismatch fails validation even when individual expression evaluation succeeded. Default: true
Trim mapped strings trimStrings	boolean Configuration setting	Remove leading/trailing whitespace from mapped strings before output. Leave disabled when whitespace carries business meaning. Default: false
Remove empty structures removeEmptyStructures	boolean Configuration setting	Clean empty containers from the completed mapped result. Check required schema structures after enabling this policy. Default: false
Copy nil semantics copyNil	boolean Configuration setting	Retain explicit nil/null semantics from mapped sources. Disable to omit nil values; combine with the null policy and target schema requirements. Default: true

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The selected target schema and mapping rules determine actual output fields.

Field / key	Type / required	Details
Source document source	object Required	The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes.

Output

The selected target schema and mapping rules determine actual output fields.

Field / key	Type / required	Details
Mapped document result	object Optional	The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
MAPPING	A mapping expression failed.
SCHEMA_VALIDATION	The mapped output violates its target schema.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



Transform

Activities / General

Overview

Transforms payloads with MINA's executable DataWeave-compatible language subset.

Scripts support payload/attributes/vars selectors, objects, arrays, defaults, conditionals, concatenation, common coercion/string/collection functions, map/filter/groupBy/orderBy/distinctBy, JSON, XML, and text output. The editor validates and runs the same engine used at runtime.

Activity type: dataweave; operation: transform.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

No additional fields declared for this section.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
DataWeave payload payload	object Required	The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes.
Input attributes attributes	object Optional	Name/value metadata associated with the message. Keep business content in the body and map only properties understood by the receiving system.
Transform variables variables	object Optional	Environment/variable values for this operation. Use defaults only where a missing value is acceptable; variable scope is distinct from shared resource configuration.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Transformed payload result	object Optional	The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

Field / key	Type / required	Details

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
DATAWEAVE_SYNTAX	The transform script could not be parsed.
DATAWEAVE_EXECUTION	A selector, function, coercion, or collection operation failed.
DATAWEAVE_OUTPUT	The transformed value cannot be rendered in the requested MIME type.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



Log

Activities / General

Overview

Writes a structured application log event.

Message, level, and payload accept constants or dynamic mappings. Advanced automatic payload logging can be used when a dedicated Log activity is unnecessary.

Activity type: log; operation: write.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Log level level	select Configuration setting	Logging configuration. Select an appropriate level; includePayload adds mapped content to logs and should be disabled for sensitive or large documents. Choices: DEBUG, INFO, WARN, ERROR
Message template message	text Configuration setting	Message or response content. Map the business document rather than a previous publisher receipt. Objects retain their structure; string values carry serialized text.
Include payload includePayload	boolean Configuration setting	Logging configuration. Select an appropriate level; includePayload adds mapped content to logs and should be disabled for sensitive or large documents.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Message expression message	string Optional	Message or response content. Map the business document rather than a previous publisher receipt. Objects retain their structure; string values carry serialized text.
Payload text payload	string Optional	Map a value or enter a text/expression; objects are rendered into the log as structured JSON.
Log level level	string Optional	Logging configuration. Select an appropriate level; includePayload adds mapped content to logs and should be disabled for sensitive or large documents.
Include payload includePayload	boolean Optional	Logging configuration. Select an appropriate level; includePayload adds mapped content to logs and should be disabled for sensitive or large documents.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Unchanged payload payload	object Optional	The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
LOGGING	The runtime logger rejected the event.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



Catch Exception

Activities / General

Overview

Runs Catch Exception on the task execution path.

Activity type: catch; operation: catch.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Catch all unhandled exceptions catchAll	boolean Configuration setting	Exception selection for Catch. catchAll handles any unhandled fault; otherwise select the fault type/code emitted by the relevant process activity.
Exception type errorType	text Configuration setting	Used when Catch All is false; for example XMLParseException.
Error code filter errorCode	text Configuration setting	Exception selection for Catch. catchAll handles any unhandled fault; otherwise select the fault type/code emitted by the relevant process activity.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Catch all unhandled exceptions catchAll	boolean Optional	Exception selection for Catch. catchAll handles any unhandled fault; otherwise select the fault type/code emitted by the relevant process activity.
Exception type errorType	string Optional	Used when Catch All is false; for example XMLParseException.
Error code filter errorCode	string Optional	Exception selection for Catch. catchAll handles any unhandled fault; otherwise select the fault type/code emitted by the relevant process activity.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Exception type type	string Optional	Type reported for this value or selected for this operation. Use the corresponding schema datatype and operation-specific choices.

Field / key	Type / required	Details
Error code code	string Optional	Fault/debug metadata identifying the activity, error code, underlying cause, diagnostic details or stack trace. Use it to diagnose the failing operation.
Error message message	string Optional	Message or response content. Map the business document rather than a previous publisher receipt. Objects retain their structure; string values carry serialized text.
Stack trace stackTrace	string Optional	Fault/debug metadata identifying the activity, error code, underlying cause, diagnostic details or stack trace. Use it to diagnose the failing operation.
Failed activity ID activityId	string Optional	Fault/debug metadata identifying the activity, error code, underlying cause, diagnostic details or stack trace. Use it to diagnose the failing operation.
Fault details details	object Optional	Fault/debug metadata identifying the activity, error code, underlying cause, diagnostic details or stack trace. Use it to diagnose the failing operation.
Original cause cause	object Optional	Fault/debug metadata identifying the activity, error code, underlying cause, diagnostic details or stack trace. Use it to diagnose the failing operation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

No additional fields declared for this section.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



Throw Exception

Activities / General

Overview

Runs Throw Exception on the task execution path.

Activity type: throw; operation: throw.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Exception type errorType	text Configuration setting	Exception selection for Catch. catchAll handles any unhandled fault; otherwise select the fault type/code emitted by the relevant process activity.
Error code code	text Configuration setting	Fault/debug metadata identifying the activity, error code, underlying cause, diagnostic details or stack trace. Use it to diagnose the failing operation.
Error message message	text Configuration setting	Message or response content. Map the business document rather than a previous publisher receipt. Objects retain their structure; string values carry serialized text.
Fault details (JSON) details	textarea Configuration setting	Fault/debug metadata identifying the activity, error code, underlying cause, diagnostic details or stack trace. Use it to diagnose the failing operation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Error message message	string Required	Message or response content. Map the business document rather than a previous publisher receipt. Objects retain their structure; string values carry serialized text.
Error code code	string Optional	Fault/debug metadata identifying the activity, error code, underlying cause, diagnostic details or stack trace. Use it to diagnose the failing operation.
Exception type type	string Optional	Type reported for this value or selected for this operation. Use the corresponding schema datatype and operation-specific choices.
Stack trace stackTrace	string Optional	Fault/debug metadata identifying the activity, error code, underlying cause, diagnostic details or stack trace. Use it to diagnose the failing operation.
Fault details details	object Optional	Fault/debug metadata identifying the activity, error code, underlying cause, diagnostic details or stack trace. Use it to diagnose the failing operation.

Field / key	Type / required	Details
Exception type errorType	string Optional	Exception selection for Catch. catchAll handles any unhandled fault; otherwise select the fault type/code emitted by the relevant process activity.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

No additional fields declared for this section.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
UserDefinedException	The configured business fault is raised to the nearest matching handler.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



Rethrow Exception

Activities / General

Overview

Runs Rethrow Exception on the task execution path.

Activity type: rethrow; operation: rethrow.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

No additional fields declared for this section.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Current caught exception error	object Optional	Fault/debug metadata identifying the activity, error code, underlying cause, diagnostic details or stack trace. Use it to diagnose the failing operation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

No additional fields declared for this section.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
RethrowException	The current caught fault is propagated to the next available handler or calling task.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



Confirm Message

Activities / General

Overview

Acknowledges one or more client-managed messages.

Map the acknowledgement handle emitted by an EMS, Kafka, Pub/Sub, or compatible future receiver. The runtime dispatches confirmation to the originating connector.

Activity type: confirm; operation: acknowledge.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Acknowledgement handle ackId	text Configuration setting	Map a broker acknowledgement handle, or place this activity after an SAP IDoc Parser to confirm the SAP tRFC early.
Fail when handle is missing failIfMissing	boolean Configuration setting	Fault confirmation when no acknowledgement handle is available. Keep enabled when missing confirmation would otherwise hide an incorrectly mapped receive path.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Acknowledgement handle ackId	string Optional	Optional for SAP IDoc early confirmation; otherwise map the client acknowledgement handle.
Multiple acknowledgement handles ackIds	array Optional	Acknowledgement/settlement handle from a received message. Confirm only after required downstream work succeeds; preserve the original handle.
Fail when handle is missing failIfMissing	boolean Optional	Fault confirmation when no acknowledgement handle is available. Keep enabled when missing confirmation would otherwise hide an incorrectly mapped receive path.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Confirmation status confirmed	boolean Optional	Operation outcome returned by the runtime. Inspect error output as well; provider acceptance does not establish processing by a downstream consumer.
Confirmed message count count	integer Optional	Number of items returned or affected by this operation. Use zero-result handling where no matching data is a valid outcome.

Field / key	Type / required	Details
Confirmed transports technologies	array Optional	Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
ACKNOWLEDGEMENT_NOT_FOUND	The acknowledgement handle is missing, expired, or was already confirmed.
SAP_EARLY_CONFIRM_FAILED	SAP rejected the early IDoc confirmation.
ACKNOWLEDGEMENT_FAILED	The transport rejected the message confirmation.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



Java Activity

Activities / General

Overview

Runs Java Activity on the task execution path.

Activity type: java; operation: invoke.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Source mode mode	select Configuration setting	Input/execution mode selected from the available choices. Match inline text, file or structured input to the actual source; connection modes such as mock/memory are for local testing. Choices: JAR / class, Inline source
JAR, .class, or .java artifactPath	artifact Configuration setting	Checkpoint identifier/content/output location. Process-state capture can include business payloads; protect generated artifacts and use stable checkpoint names.
Fully qualified Java class className	text Configuration setting	Java invocation settings/result. Provide the available class/method, select instance creation when required, and map the returned value using its actual datatype.
Method method	text Configuration setting	HTTP verb or invoked method name, according to this activity. For HTTP select the verb required by the API, such as GET, POST or PUT.
Instantiate class instantiate	boolean Configuration setting	Java invocation settings/result. Provide the available class/method, select instance creation when required, and map the returned value using its actual datatype.
Inline Java source sourceCode	textarea Configuration setting	Python module/function or inline source to invoke. Deploy the module and its dependencies, and match the function signature to configured input arguments.
Timeout seconds timeout	number Configuration setting	Maximum time allowed for the selected operation. Configure the provider-specific wait and whole-activity timeout separately where both are exposed.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Method parameters parameters	array Optional	SQL/procedure parameters or mapped columns. Match parameter names and types to metadata and use parameter binding rather than constructing SQL from values.

Field / key	Type / required	Details
JSON input payload	object Optional	The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes.
Source mode mode	string Optional	Input/execution mode selected from the available choices. Match inline text, file or structured input to the actual source; connection modes such as mock/memory are for local testing.
JAR, .class, or .java artifactPath	string Optional	Checkpoint identifier/content/output location. Process-state capture can include business payloads; protect generated artifacts and use stable checkpoint names.
Fully qualified Java class className	string Optional	Java invocation settings/result. Provide the available class/method, select instance creation when required, and map the returned value using its actual datatype.
Method method	string Optional	HTTP verb or invoked method name, according to this activity. For HTTP select the verb required by the API, such as GET, POST or PUT.
Instantiate class instantiate	boolean Optional	Java invocation settings/result. Provide the available class/method, select instance creation when required, and map the returned value using its actual datatype.
Inline Java source sourceCode	string Optional	Python module/function or inline source to invoke. Deploy the module and its dependencies, and match the function signature to configured input arguments.
Timeout seconds timeout	number Optional	Maximum time allowed for the selected operation. Configure the provider-specific wait and whole-activity timeout separately where both are exposed.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Method return value methodReturnValue	object Optional	Java invocation settings/result. Provide the available class/method, select instance creation when required, and map the returned value using its actual datatype.
Invoked class className	string Optional	Java invocation settings/result. Provide the available class/method, select instance creation when required, and map the returned value using its actual datatype.
Invoked method method	string Optional	HTTP verb or invoked method name, according to this activity. For HTTP select the verb required by the API, such as GET, POST or PUT.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
JAVA_CLASS_NOT_FOUND	The configured class or JAR cannot be loaded.
JAVA_INVOCATION	The Java method threw an exception.
JAVA_TIMEOUT	The worker exceeded its timeout.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



Python Invoke

Activities / General

Overview

Runs Python Invoke on the task execution path.

Activity type: python; operation: invoke.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Source mode mode	select Configuration setting	Input/execution mode selected from the available choices. Match inline text, file or structured input to the actual source; connection modes such as mock/memory are for local testing. Choices: Python file / package, Inline source
.py, .zip, or .whl artifactPath	artifact Configuration setting	Checkpoint identifier/content/output location. Process-state capture can include business payloads; protect generated artifacts and use stable checkpoint names.
Module name moduleName	text Configuration setting	Python module/function or inline source to invoke. Deploy the module and its dependencies, and match the function signature to configured input arguments.
Function function	text Configuration setting	Python module/function or inline source to invoke. Deploy the module and its dependencies, and match the function signature to configured input arguments.
Inline Python source sourceCode	textarea Configuration setting	Python module/function or inline source to invoke. Deploy the module and its dependencies, and match the function signature to configured input arguments.
Timeout seconds timeout	number Configuration setting	Maximum time allowed for the selected operation. Configure the provider-specific wait and whole-activity timeout separately where both are exposed.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Function parameters parameters	array Optional	SQL/procedure parameters or mapped columns. Match parameter names and types to metadata and use parameter binding rather than constructing SQL from values.
JSON input payload	object Optional	The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes.

Field / key	Type / required	Details
Source mode mode	string Optional	Input/execution mode selected from the available choices. Match inline text, file or structured input to the actual source; connection modes such as mock/memory are for local testing.
.py, .zip, or .whl artifactPath	string Optional	Checkpoint identifier/content/output location. Process-state capture can include business payloads; protect generated artifacts and use stable checkpoint names.
Module name moduleName	string Optional	Python module/function or inline source to invoke. Deploy the module and its dependencies, and match the function signature to configured input arguments.
Function function	string Optional	Python module/function or inline source to invoke. Deploy the module and its dependencies, and match the function signature to configured input arguments.
Inline Python source sourceCode	string Optional	Python module/function or inline source to invoke. Deploy the module and its dependencies, and match the function signature to configured input arguments.
Timeout seconds timeout	number Optional	Maximum time allowed for the selected operation. Configure the provider-specific wait and whole-activity timeout separately where both are exposed.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Python result result	object Optional	The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes.
Loaded module module	string Optional	Python module/function or inline source to invoke. Deploy the module and its dependencies, and match the function signature to configured input arguments.
Invoked function function	string Optional	Python module/function or inline source to invoke. Deploy the module and its dependencies, and match the function signature to configured input arguments.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
PYTHON_IMPORT	The module or package could not be imported.
PYTHON_INVOCATION	The selected Python function raised an exception.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



External Command

Activities / General

Overview

Executes the selected general-purpose activity.

External Command runs an executable without a shell, accepts stdin/environment/working-directory overrides, can capture or persist stdout and stderr, applies configured line splitting, and publishes the process return code.

Activity type: basic; operation: external_command.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Command to execute command	textarea Configuration setting	Executable and arguments. Shell pipes are intentionally not evaluated.
Provide command output provideCommandOutput	boolean Configuration setting	External command execution/output policy. Supply the executable and arguments, choose captured/file output and environment handling, and retain quoting required by the executable.
Remove parameter quotes removeParameterQuotes	boolean Configuration setting	External command execution/output policy. Supply the executable and arguments, choose captured/file output and environment handling, and retain quoting required by the executable.
Output filename outputFile	text Configuration setting	External command execution/output policy. Supply the executable and arguments, choose captured/file output and environment handling, and retain quoting required by the executable.
Output line splitting outputLineSplitting	select Configuration setting	External command execution/output policy. Supply the executable and arguments, choose captured/file output and environment handling, and retain quoting required by the executable. Choices: None, AtOperatingSystemLineEnd, AtSpecifiedToken
Specified split token splitToken	text Configuration setting	Token used to split command/text output into values. Choose a delimiter consistent with the producer output.
Working directory workingDirectory	text Configuration setting	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Environment variables [NAME=value,...] environment	text Configuration setting	Environment/variable values for this operation. Use defaults only where a missing value is acceptable; variable scope is distinct from shared resource configuration.
Replace inherited environment replaceEnvironment	boolean Configuration setting	External command execution/output policy. Supply the executable and arguments, choose captured/file output and environment handling, and retain quoting required by the executable.
Timeout seconds timeoutSeconds	number Configuration setting	Timeout seconds: bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.
Output encoding encoding	text Configuration setting	Character encoding for text conversion, for example UTF-8. Use the sender encoding when parsing; a wrong encoding can corrupt non-ASCII characters.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Dynamic command command	string Optional	External command execution/output policy. Supply the executable and arguments, choose captured/file output and environment handling, and retain quoting required by the executable.
Standard input input	string Optional	The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes.
Dynamic output file outputFile	string Optional	External command execution/output policy. Supply the executable and arguments, choose captured/file output and environment handling, and retain quoting required by the executable.
Dynamic environment environment	object|string Optional	Environment/variable values for this operation. Use defaults only where a missing value is acceptable; variable scope is distinct from shared resource configuration.
Dynamic working directory workingDirectory	string Optional	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.
Dynamic split token splitToken	string Optional	Token used to split command/text output into values. Choose a delimiter consistent with the producer output.
Provide command output provideCommandOutput	boolean Optional	External command execution/output policy. Supply the executable and arguments, choose captured/file output and environment handling, and retain quoting required by the executable.
Remove parameter quotes removeParameterQuotes	boolean Optional	External command execution/output policy. Supply the executable and arguments, choose captured/file output and environment handling, and retain quoting required by the executable.
Output line splitting outputLineSplitting	string Optional	External command execution/output policy. Supply the executable and arguments, choose captured/file output and environment handling, and retain quoting required by the executable.
Replace inherited environment replaceEnvironment	boolean Optional	External command execution/output policy. Supply the executable and arguments, choose captured/file output and environment handling, and retain quoting required by the executable.
Timeout seconds timeoutSeconds	number Optional	Timeout seconds: bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.
Output encoding encoding	string Optional	Character encoding for text conversion, for example UTF-8. Use the sender encoding when parsing; a wrong encoding can corrupt non-ASCII characters.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Process return code returnCode	integer Optional	External process exit code. Zero conventionally indicates success; inspect command output and the executable-specific exit-code contract.

Field / key	Type / required	Details
Standard output output	string|array Optional	Post-operation action or returned output. Use the activity-specific operation/contract to determine whether the value is content, metadata or a completion result.
Standard error error	string|array Optional	Fault/debug metadata identifying the activity, error code, underlying cause, diagnostic details or stack trace. Use it to diagnose the failing operation.
Written output file outputFile	string Optional	External command execution/output policy. Supply the executable and arguments, choose captured/file output and environment handling, and retain quoting required by the executable.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
CommandExecutionError	The operating system command could not start or timed out.
FileIOError	Command output could not be written to the configured file.
InvalidInputException	The command or line-splitting configuration is invalid.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



Assign

Activities / General

Overview

Executes the selected general-purpose activity.

External Command runs an executable without a shell, accepts stdin/environment/working-directory overrides, can capture or persist stdout and stderr, applies configured line splitting, and publishes the process return code.

Activity type: basic; operation: assign.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Process variable variable	text Configuration setting	Name stored in process variables.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Assigned value value	object Required	The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes.
Process variable variable	string Optional	Name stored in process variables.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Variable name name	string Optional	Name identifying this entry. Keep it stable wherever downstream mappings or matching rules refer to it.
Assigned value value	object Optional	The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.

Field / key	Type / required	Details
		Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
VALIDATION	Required input or mapped data is invalid.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



Checkpoint

Activities / General

Overview

Executes the selected general-purpose activity.

External Command runs an executable without a shell, accepts stdin/environment/working-directory overrides, can capture or persist stdout and stderr, applies configured line splitting, and publishes the process return code.

Activity type: basic; operation: checkpoint.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Checkpoint name checkpointName	text Configuration setting	Optional logical name stored in runtime execution metadata.
Include process state includeProcessState	boolean Configuration setting	Checkpoint identifier/content/output location. Process-state capture can include business payloads; protect generated artifacts and use stable checkpoint names.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Checkpoint name checkpointName	string Optional	Optional logical name stored in runtime execution metadata.
Include process state includeProcessState	boolean Optional	Checkpoint identifier/content/output location. Process-state capture can include business payloads; protect generated artifacts and use stable checkpoint names.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Checkpoint ID checkpointId	string Optional	Checkpoint identifier/content/output location. Process-state capture can include business payloads; protect generated artifacts and use stable checkpoint names.
Checkpoint timestamp timestamp	dateTime Optional	Time reported by the operation/provider. Preserve the original representation when tracing; epoch values and formatted date/time strings need explicit conversion.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
CheckpointException	The process state could not be persisted.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx

Deduplicate Message

Activities / General

Overview

Executes the selected general-purpose activity.

External Command runs an executable without a shell, accepts stdin/environment/working-directory overrides, can capture or persist stdout and stderr, applies configured line splitting, and publishes the process return code.

Activity type: basic; operation: deduplicate.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Durable state file stateFile	text Configuration setting	Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing.
Message namespace namespace	text Required	Use the same namespace for claims, receipts and replay.
State capacity (MB) maxStateSizeMb	number Configuration setting	Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing.
Claim lease (seconds) leaseSeconds	number Configuration setting	Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Stable message key messageKey	string Optional	Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing.
Message payload payload	any Optional	The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes.

Field / key	Type / required	Details
Durable state file stateFile	string Optional	Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing.
Message namespace namespace	string Required	Use the same namespace for claims, receipts and replay.
State capacity (MB) maxStateSizeMb	number Optional	Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing.
Claim lease (seconds) leaseSeconds	number Optional	Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Claim accepted accepted	boolean Optional	Operation outcome returned by the runtime. Inspect error output as well; provider acceptance does not establish processing by a downstream consumer.
Duplicate delivery duplicate	boolean Optional	Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing.
Message key messageKey	string Optional	Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing.
Claim token claimToken	string Optional	Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing.
Claim status status	string Optional	Operation outcome returned by the runtime. Inspect error output as well; provider acceptance does not establish processing by a downstream consumer.
Message payload payload	any Optional	The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
MessageStateException	Durable state could not be read or written.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx

Message Receipt

Activities / General

Overview

Executes the selected general-purpose activity.

External Command runs an executable without a shell, accepts stdin/environment/working-directory overrides, can capture or persist stdout and stderr, applies configured line splitting, and publishes the process return code.

Activity type: basic; operation: message_receipt.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Durable state file stateFile	text Configuration setting	Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing.
Message namespace namespace	text Required	Use the same namespace for claims, receipts and replay.
State capacity (MB) maxStateSizeMb	number Configuration setting	Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing.
Receipt action action	select Configuration setting	Complete after downstream work; fail retains the message for explicit replay. Choices: complete, fail
Completed-key retention (seconds) retentionSeconds	number Configuration setting	Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Message key messageKey	string Optional	Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing.

Field / key	Type / required	Details
Claim token claimToken	string Optional	Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing.
Failure reason error	string Optional	Fault/debug metadata identifying the activity, error code, underlying cause, diagnostic details or stack trace. Use it to diagnose the failing operation.
Durable state file stateFile	string Optional	Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing.
Message namespace namespace	string Required	Use the same namespace for claims, receipts and replay.
State capacity (MB) maxStateSizeMb	number Optional	Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing.
Receipt action action	string Optional	Complete after downstream work; fail retains the message for explicit replay.
Completed-key retention (seconds) retentionSeconds	number Optional	Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Message key messageKey	string Optional	Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing.
Receipt status status	string Optional	Operation outcome returned by the runtime. Inspect error output as well; provider acceptance does not establish processing by a downstream consumer.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
MessageStateException	Claim token is missing, stale or expired.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



Replay Messages

Activities / General

Overview

Executes the selected general-purpose activity.

External Command runs an executable without a shell, accepts stdin/environment/working-directory overrides, can capture or persist stdout and stderr, applies configured line splitting, and publishes the process return code.

Activity type: basic; operation: replay_messages.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Durable state file stateFile	text Configuration setting	Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing.
Message namespace namespace	text Required	Use the same namespace for claims, receipts and replay.
State capacity (MB) maxStateSizeMb	number Configuration setting	Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing.
Replay action action	select Configuration setting	List failed/expired messages or explicitly requeue one key. No downstream work runs automatically. Choices: list, requeue, purge
Maximum replay records limit	number Configuration setting	Upper bound on returned rows/items. Use a positive bound for large datasets and page/process subsequent batches where available.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Key to requeue messageKey	string Optional	Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing.

Field / key	Type / required	Details
Durable state file stateFile	string Optional	Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing.
Message namespace namespace	string Required	Use the same namespace for claims, receipts and replay.
State capacity (MB) maxStateSizeMb	number Optional	Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing.
Replay action action	string Optional	List failed/expired messages or explicitly requeue one key. No downstream work runs automatically.
Maximum replay records limit	number Optional	Upper bound on returned rows/items. Use a positive bound for large datasets and page/process subsequent batches where available.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Replay candidates messages	array Optional	Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema.
Candidate count count	integer Optional	Number of items returned or affected by this operation. Use zero-result handling where no matching data is a valid outcome.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
MessageStateException	Message is not eligible for replay or state is unavailable.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx

Aggregate Messages

Activities / General

Overview

Executes the selected general-purpose activity.

External Command runs an executable without a shell, accepts stdin/environment/working-directory overrides, can capture or persist stdout and stderr, applies configured line splitting, and publishes the process return code.

Activity type: basic; operation: aggregate_messages.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Durable state file stateFile	text Configuration setting	Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing.
Message namespace namespace	text Required	Use the same namespace for claims, receipts and replay.
State capacity (MB) maxStateSizeMb	number Configuration setting	Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing.
Aggregation action action	select Configuration setting	Append a message; flush emits a partial group only after its timeout. Choices: append, flush
Completion count expectedCount	number Configuration setting	Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing.
Maximum group records maxGroupItems	number Configuration setting	Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing.
Group timeout (seconds) timeoutSeconds	number Configuration setting	Group timeout (seconds): bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Correlation key correlationKey	string Optional	Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing.
Stable message ID messageId	string Optional	Message identifier returned by the provider or runtime. Retain it for tracing; use a separate stable business key for duplicate detection.
Message payload payload	any Optional	The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes.
Durable state file stateFile	string Optional	Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing.
Message namespace namespace	string Required	Use the same namespace for claims, receipts and replay.
State capacity (MB) maxStateSizeMb	number Optional	Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing.
Aggregation action action	string Optional	Append a message; flush emits a partial group only after its timeout.
Completion count expectedCount	number Optional	Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing.
Maximum group records maxGroupItems	number Optional	Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing.
Group timeout (seconds) timeoutSeconds	number Optional	Group timeout (seconds): bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Group ready for delivery ready	boolean Optional	Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing.
Completed group records records	array Optional	Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema.

Field / key	Type / required	Details
Group count count	integer Optional	Number of items returned or affected by this operation. Use zero-result handling where no matching data is a valid outcome.
Group timed out timedOut	boolean Optional	Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing.
Group status status	string Optional	Operation outcome returned by the runtime. Inspect error output as well; provider acceptance does not establish processing by a downstream consumer.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
MessageStateException	Correlation, completion settings or capacity are invalid.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx

Split Records

Activities / General

Overview

Executes the selected general-purpose activity.

External Command runs an executable without a shell, accepts stdin/environment/working-directory overrides, can capture or persist stdout and stderr, applies configured line splitting, and publishes the process return code.

Activity type: basic; operation: split_records.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Records per batch batchSize	number Configuration setting	Batch execution size/policy. Bound batches to control memory and transaction duration; inspect per-batch failures before considering all records complete.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Records payload	array Optional	The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes.
Records per batch batchSize	number Optional	Batch execution size/policy. Bound batches to control memory and transaction duration; inspect per-batch failures before considering all records complete.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Record batches batches	array Optional	Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing.
Record count count	integer Optional	Number of items returned or affected by this operation. Use zero-result handling where no matching data is a valid outcome.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
InvalidInputException	Records or batch size are invalid.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



Sleep

Activities / General

Overview

Executes the selected general-purpose activity.

External Command runs an executable without a shell, accepts stdin/environment/working-directory overrides, can capture or persist stdout and stderr, applies configured line splitting, and publishes the process return code.

Activity type: basic; operation: sleep.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Duration duration	number Configuration setting	Sleep duration and its selected unit, or actual reported sleep in milliseconds. Sleeping pauses this branch, not unrelated parallel branches.
Unit unit	select Configuration setting	Sleep duration and its selected unit, or actual reported sleep in milliseconds. Sleeping pauses this branch, not unrelated parallel branches. Choices: milliseconds, seconds, minutes

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Dynamic duration duration	number Optional	Sleep duration and its selected unit, or actual reported sleep in milliseconds. Sleeping pauses this branch, not unrelated parallel branches.
Unit unit	string Optional	Sleep duration and its selected unit, or actual reported sleep in milliseconds. Sleeping pauses this branch, not unrelated parallel branches.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Elapsed delay sleptMilliseconds	integer Optional	Sleep duration and its selected unit, or actual reported sleep in milliseconds. Sleeping pauses this branch, not unrelated parallel branches.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
INTERRUPTED	The wait was interrupted or cancelled.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



Get Shared Variable

Activities / General

Overview

Executes the selected general-purpose activity.

External Command runs an executable without a shell, accepts stdin/environment/working-directory overrides, can capture or persist stdout and stderr, applies configured line splitting, and publishes the process return code.

Activity type: basic; operation: get_shared_variable.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Shared variable name name	text Configuration setting	Name identifying this entry. Keep it stable wherever downstream mappings or matching rules refer to it.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Default value default	object Optional	Environment/variable values for this operation. Use defaults only where a missing value is acceptable; variable scope is distinct from shared resource configuration.
Shared variable name name	string Optional	Name identifying this entry. Keep it stable wherever downstream mappings or matching rules refer to it.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Variable name name	string Optional	Name identifying this entry. Keep it stable wherever downstream mappings or matching rules refer to it.
Shared value value	object Optional	The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.

Field / key	Type / required	Details
		Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

No additional fields declared for this section.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



Set Shared Variable

Activities / General

Overview

Executes the selected general-purpose activity.

External Command runs an executable without a shell, accepts stdin/environment/working-directory overrides, can capture or persist stdout and stderr, applies configured line splitting, and publishes the process return code.

Activity type: basic; operation: set_shared_variable.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Shared variable name name	text Configuration setting	Name identifying this entry. Keep it stable wherever downstream mappings or matching rules refer to it.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Shared value value	object Required	The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes.
Shared variable name name	string Optional	Name identifying this entry. Keep it stable wherever downstream mappings or matching rules refer to it.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Variable name name	string Optional	Name identifying this entry. Keep it stable wherever downstream mappings or matching rules refer to it.
Shared value value	object Optional	The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Configuration setting	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.

Field / key	Type / required	Details
		Default: "${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

No additional fields declared for this section.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx

If

Groups

Configuration

Evaluates condition before entry; false skips the group.

Set Group name, Parent group and direct activity membership. A member has one direct owner; use parent_group_id for nesting. Normal groups require a connected single-entry, single-exit graph. Pick First has specialized branch selection.

Field / key	Type / required	Details
condition condition	expression / setting Configuration setting	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed.

Input

Conditions, collections, limits and retry intervals accept mapped earlier output, initial task input, properties, functions or constants. While/Repeat can inspect prior-iteration output. An initial condition cannot depend on output that has never executed.

```
${input.enabled} == true
```

Output

Member activities publish their normal outputs. Groups do not have a separate universal output schema.

Advanced

Iteration limits and retry counts provide runaway protection. Enclosing retry policies can repeat external writes. Debug uses the group scheduler and exposes group iteration context; task archives preserve membership and nesting.

Errors

Missing members, cyclic parents, invalid expressions and invalid boundaries fail validation. Starter/event, End and Catch are task boundaries, not ordinary direct members. Parallel fan-out in non-Pick-First groups remains explicitly restricted. Escaping exceptions unwind group resources before propagating to task or caller handlers.

Reference source: backend/app/runtime.py; frontend/src/main.tsx

While True

Groups

Configuration

Evaluates condition before each iteration, including the first.

Set Group name, Parent group and direct activity membership. A member has one direct owner; use parent_group_id for nesting. Normal groups require a connected single-entry, single-exit graph. Pick First has specialized branch selection.

Field / key	Type / required	Details
condition condition	expression / setting Configuration setting	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed.
maxIterations maxIterations	expression / setting Configuration setting	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. Default: "${properties.advanced.groupMaxIterations}"
indexVariable indexVariable	expression / setting Configuration setting	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. Default: "index"

Input

Conditions, collections, limits and retry intervals accept mapped earlier output, initial task input, properties, functions or constants. While/Repeat can inspect prior-iteration output. An initial condition cannot depend on output that has never executed.

```
${input.enabled} == true
```

Output

Member activities publish their normal outputs. Groups do not have a separate universal output schema.

Advanced

Iteration limits and retry counts provide runaway protection. Enclosing retry policies can repeat external writes. Debug uses the group scheduler and exposes group iteration context; task archives preserve membership and nesting.

Errors

Missing members, cyclic parents, invalid expressions and invalid boundaries fail validation. Starter/event, End and Catch are task boundaries, not ordinary direct members. Parallel fan-out in non-Pick-First groups remains explicitly restricted. Escaping exceptions unwind group resources before propagating to task or caller handlers.

Reference source: backend/app/runtime.py; frontend/src/main.tsx

For Each

Groups

Configuration

Resolves the collection at entry and runs once per item. Range mode uses inclusive start/end with a nonzero increment.

Set Group name, Parent group and direct activity membership. A member has one direct owner; use parent_group_id for nesting. Normal groups require a connected single-entry, single-exit graph. Pick First has specialized branch selection.

Field / key	Type / required	Details
iterationMode iterationMode	expression / setting Configuration setting	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. Default: "collection"
source source	expression / setting Configuration setting	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed.
start start	expression / setting Configuration setting	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed.
end end	expression / setting Configuration setting	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed.
increment increment	expression / setting Configuration setting	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed.
currentElementName currentElementName	expression / setting Configuration setting	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. Default: "currentElement"
itemVariable itemVariable	expression / setting Configuration setting	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. Default: "currentElement"
indexVariable indexVariable	expression / setting Configuration setting	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. Default: "index"
maxIterations maxIterations	expression / setting Configuration setting	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. Default: "${properties.advanced.groupMaxIterations}"
accumulateOutput accumulateOutput	expression / setting Configuration setting	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed.
accumulatorVariable accumulatorVariable	expression / setting Configuration setting	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed.

Input

Conditions, collections, limits and retry intervals accept mapped earlier output, initial task input, properties, functions or constants. While/Repeat can inspect prior-iteration output. An initial condition cannot depend on output that has never executed.

```
${input.records}
${vars.currentElement}
${vars.currentIndex}
```

Output

The configured current element and one-based index are visible inside the group. Optional accumulation records each successful group-exit result under accumulatorVariable.

Advanced

Iteration limits and retry counts provide runaway protection. Enclosing retry policies can repeat external writes. Debug uses the group scheduler and exposes group iteration context; task archives preserve membership and nesting.

Errors

Missing members, cyclic parents, invalid expressions and invalid boundaries fail validation. Starter/event, End and Catch are task boundaries, not ordinary direct members. Parallel fan-out in non-Pick-First groups remains explicitly restricted. Escaping exceptions unwind group resources before propagating to task or caller handlers.

Reference source: backend/app/runtime.py; frontend/src/main.tsx

Iterate

Groups

Configuration

Resolves source once at entry and iterates its elements.

Set Group name, Parent group and direct activity membership. A member has one direct owner; use parent_group_id for nesting. Normal groups require a connected single-entry, single-exit graph. Pick First has specialized branch selection.

Field / key	Type / required	Details
source source	expression / setting Configuration setting	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed.
currentElementName currentElementName	expression / setting Configuration setting	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. Default: "currentElement"
itemVariable itemVariable	expression / setting Configuration setting	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. Default: "currentElement"
indexVariable indexVariable	expression / setting Configuration setting	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. Default: "index"
maxIterations maxIterations	expression / setting Configuration setting	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. Default: "${properties.advanced.groupMaxIterations}"
accumulateOutput accumulateOutput	expression / setting Configuration setting	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed.
accumulatorVariable accumulatorVariable	expression / setting Configuration setting	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed.

Input

Conditions, collections, limits and retry intervals accept mapped earlier output, initial task input, properties, functions or constants. While/Repeat can inspect prior-iteration output. An initial condition cannot depend on output that has never executed.

```
${input.records}
${vars.currentElement}
${vars.currentIndex}
```

Output

The configured current element and one-based index are visible inside the group. Optional accumulation records each successful group-exit result under accumulatorVariable.

Advanced

Iteration limits and retry counts provide runaway protection. Enclosing retry policies can repeat external writes. Debug uses the group scheduler and exposes group iteration context; task archives preserve membership and nesting.

Errors

Missing members, cyclic parents, invalid expressions and invalid boundaries fail validation. Starter/event, End and Catch are task boundaries, not ordinary direct members. Parallel fan-out in non-Pick-First groups remains explicitly restricted. Escaping exceptions unwind group resources before propagating to task or caller handlers.

Reference source: backend/app/runtime.py; frontend/src/main.tsx

Repeat Until True

Groups

Configuration

Runs once, checks condition at exit and repeats until true.

Set Group name, Parent group and direct activity membership. A member has one direct owner; use parent_group_id for nesting. Normal groups require a connected single-entry, single-exit graph. Pick First has specialized branch selection.

Field / key	Type / required	Details
condition condition	expression / setting Configuration setting	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed.
maxIterations maxIterations	expression / setting Configuration setting	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. Default: "${properties.advanced.groupMaxIterations}"
indexVariable indexVariable	expression / setting Configuration setting	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. Default: "index"

Input

Conditions, collections, limits and retry intervals accept mapped earlier output, initial task input, properties, functions or constants. While/Repeat can inspect prior-iteration output. An initial condition cannot depend on output that has never executed.

```
${input.enabled} == true
```

Output

Member activities publish their normal outputs. Groups do not have a separate universal output schema.

Advanced

Iteration limits and retry counts provide runaway protection. Enclosing retry policies can repeat external writes. Debug uses the group scheduler and exposes group iteration context; task archives preserve membership and nesting.

Errors

Missing members, cyclic parents, invalid expressions and invalid boundaries fail validation. Starter/event, End and Catch are task boundaries, not ordinary direct members. Parallel fan-out in non-Pick-First groups remains explicitly restricted. Escaping exceptions unwind group resources before propagating to task or caller handlers.

Reference source: backend/app/runtime.py; frontend/src/main.tsx

Repeat on Error Until True

Groups

Configuration

Retries the whole group after an unhandled fault while stopCondition is false. A true stop condition or exhausted retry count propagates the fault.

Set Group name, Parent group and direct activity membership. A member has one direct owner; use parent_group_id for nesting. Normal groups require a connected single-entry, single-exit graph. Pick First has specialized branch selection.

Field / key	Type / required	Details
stopCondition stopCondition	expression / setting Configuration setting	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed.
retryCount retryCount	expression / setting Configuration setting	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. Default: "${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	expression / setting Configuration setting	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. Default: "${properties.advanced.retryIntervalSeconds}"
indexVariable indexVariable	expression / setting Configuration setting	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. Default: "index"

Input

Conditions, collections, limits and retry intervals accept mapped earlier output, initial task input, properties, functions or constants. While/Repeat can inspect prior-iteration output. An initial condition cannot depend on output that has never executed.

```
${context.error}
${properties.advanced.retryCount}
${properties.advanced.retryIntervalSeconds}
```

Output

Member activities publish their normal outputs. Groups do not have a separate universal output schema.

Advanced

Iteration limits and retry counts provide runaway protection. Enclosing retry policies can repeat external writes. Debug uses the group scheduler and exposes group iteration context; task archives preserve membership and nesting.

Errors

Missing members, cyclic parents, invalid expressions and invalid boundaries fail validation. Starter/event, End and Catch are task boundaries, not ordinary direct members. Parallel fan-out in non-Pick-First groups remains explicitly restricted. Escaping exceptions unwind group resources before propagating to task or caller handlers.

Reference source: backend/app/runtime.py; frontend/src/main.tsx

Scope

Groups

Configuration

Runs the enclosed graph once. Nested groups retain their own semantics.

Set Group name, Parent group and direct activity membership. A member has one direct owner; use parent_group_id for nesting. Normal groups require a connected single-entry, single-exit graph. Pick First has specialized branch selection.

No additional fields declared for this section.

Input

Conditions, collections, limits and retry intervals accept mapped earlier output, initial task input, properties, functions or constants. While/Repeat can inspect prior-iteration output. An initial condition cannot depend on output that has never executed.

```
${input.enabled} == true
```

Output

Member activities publish their normal outputs. Groups do not have a separate universal output schema.

Advanced

Iteration limits and retry counts provide runaway protection. Enclosing retry policies can repeat external writes. Debug uses the group scheduler and exposes group iteration context; task archives preserve membership and nesting.

Errors

Missing members, cyclic parents, invalid expressions and invalid boundaries fail validation. Starter/event, End and Catch are task boundaries, not ordinary direct members. Parallel fan-out in non-Pick-First groups remains explicitly restricted. Escaping exceptions unwind group resources before propagating to task or caller handlers.

Reference source: backend/app/runtime.py; frontend/src/main.tsx

None

Groups

Configuration

Runs the enclosed graph once; this is an execution boundary, not a separate transaction.

Set Group name, Parent group and direct activity membership. A member has one direct owner; use parent_group_id for nesting. Normal groups require a connected single-entry, single-exit graph. Pick First has specialized branch selection.

No additional fields declared for this section.

Input

Conditions, collections, limits and retry intervals accept mapped earlier output, initial task input, properties, functions or constants. While/Repeat can inspect prior-iteration output. An initial condition cannot depend on output that has never executed.

```
${input.enabled} == true
```

Output

Member activities publish their normal outputs. Groups do not have a separate universal output schema.

Advanced

Iteration limits and retry counts provide runaway protection. Enclosing retry policies can repeat external writes. Debug uses the group scheduler and exposes group iteration context; task archives preserve membership and nesting.

Errors

Missing members, cyclic parents, invalid expressions and invalid boundaries fail validation. Starter/event, End and Catch are task boundaries, not ordinary direct members. Parallel fan-out in non-Pick-First groups remains explicitly restricted. Escaping exceptions unwind group resources before propagating to task or caller handlers.

Reference source: backend/app/runtime.py; frontend/src/main.tsx

Critical Section

Groups

Configuration

Holds a runtime-local named lock until completion, failure or cancellation. This is not a distributed lock across machines or separate runtime processes.

Set Group name, Parent group and direct activity membership. A member has one direct owner; use parent_group_id for nesting. Normal groups require a connected single-entry, single-exit graph. Pick First has specialized branch selection.

Field / key	Type / required	Details
lockName lockName	expression / setting Configuration setting	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed.

Input

Conditions, collections, limits and retry intervals accept mapped earlier output, initial task input, properties, functions or constants. While/Repeat can inspect prior-iteration output. An initial condition cannot depend on output that has never executed.

```
${input.enabled} == true
```

Output

Member activities publish their normal outputs. Groups do not have a separate universal output schema.

Advanced

Iteration limits and retry counts provide runaway protection. Enclosing retry policies can repeat external writes. Debug uses the group scheduler and exposes group iteration context; task archives preserve membership and nesting.

Errors

Missing members, cyclic parents, invalid expressions and invalid boundaries fail validation. Starter/event, End and Catch are task boundaries, not ordinary direct members. Parallel fan-out in non-Pick-First groups remains explicitly restricted. Escaping exceptions unwind group resources before propagating to task or caller handlers.

Reference source: backend/app/runtime.py; frontend/src/main.tsx

JDBC Transaction

Groups

Configuration

Uses one database connection for matching JDBC operations, commits on successful exit and rolls back on an escaping fault or cancellation. External non-JDBC side effects are not rolled back.

Set Group name, Parent group and direct activity membership. A member has one direct owner; use parent_group_id for nesting. Normal groups require a connected single-entry, single-exit graph. Pick First has specialized branch selection.

Field / key	Type / required	Details
resourceId resourceId	expression / setting Configuration setting	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed.

Input

Conditions, collections, limits and retry intervals accept mapped earlier output, initial task input, properties, functions or constants. While/Repeat can inspect prior-iteration output. An initial condition cannot depend on output that has never executed.

```
${input.enabled} == true
```

Output

Member activities publish their normal outputs. Groups do not have a separate universal output schema.

Advanced

Iteration limits and retry counts provide runaway protection. Enclosing retry policies can repeat external writes. Debug uses the group scheduler and exposes group iteration context; task archives preserve membership and nesting.

Errors

Missing members, cyclic parents, invalid expressions and invalid boundaries fail validation. Starter/event, End and Catch are task boundaries, not ordinary direct members. Parallel fan-out in non-Pick-First groups remains explicitly restricted. Escaping exceptions unwind group resources before propagating to task or caller handlers.

Reference source: backend/app/runtime.py; frontend/src/main.tsx

Pick First

Groups

Configuration

Chooses the first eligible transition in graph order. This is first-match routing, not concurrent event racing or fastest-response selection.

Set Group name, Parent group and direct activity membership. A member has one direct owner; use parent_group_id for nesting. Normal groups require a connected single-entry, single-exit graph. Pick First has specialized branch selection.

No additional fields declared for this section.

Input

Conditions, collections, limits and retry intervals accept mapped earlier output, initial task input, properties, functions or constants. While/Repeat can inspect prior-iteration output. An initial condition cannot depend on output that has never executed.

```
${input.enabled} == true
```

Output

Member activities publish their normal outputs. Groups do not have a separate universal output schema.

Advanced

Iteration limits and retry counts provide runaway protection. Enclosing retry policies can repeat external writes. Debug uses the group scheduler and exposes group iteration context; task archives preserve membership and nesting.

Errors

Missing members, cyclic parents, invalid expressions and invalid boundaries fail validation. Starter/event, End and Catch are task boundaries, not ordinary direct members. Parallel fan-out in non-Pick-First groups remains explicitly restricted. Escaping exceptions unwind group resources before propagating to task or caller handlers.

Reference source: backend/app/runtime.py; frontend/src/main.tsx

concat

Functions / String

Overview

Concatenate two or more values.

Signature

```
concat(value, ...values)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
concat($value, "")
```

Output

Concatenate two or more values.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

trim

Functions / String

Overview

Remove leading and trailing whitespace.

Signature

```
trim(value)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
trim($value)
```

Output

Remove leading and trailing whitespace.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

normalizeSpace

Functions / String

Overview

Trim and collapse repeated whitespace.

Signature

```
normalizeSpace(value)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
normalizeSpace($value)
```

Output

Trim and collapse repeated whitespace.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

upperCase

Functions / String

Overview

Convert text to uppercase.

Signature

```
upperCase(value)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
upperCase($value)
```

Output

Convert text to uppercase.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

lowerCase

Functions / String

Overview

Convert text to lowercase.

Signature

```
lowerCase(value)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
lowerCase($value)
```

Output

Convert text to lowercase.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

capitalize

Functions / String

Overview

Uppercase the first character.

Signature

```
capitalize(value)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
capitalize($value)
```

Output

Uppercase the first character.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

stringLength

Functions / String

Overview

Return the number of characters.

Signature

```
stringLength(value)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
stringLength($value)
```

Output

Return the number of characters.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

substring

Functions / String

Overview

Extract text using one-based positions.

Signature

```
substring(value, start, length?)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
substring($value, 1, 10)
```

Output

Extract text using one-based positions.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

substringBefore

Functions / String

Overview

Return text before the first token.

Signature

```
substringBefore(value, token)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
substringBefore($value, "-")
```

Output

Return text before the first token.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

substringAfter

Functions / String

Overview

Return text after the first token.

Signature

```
substringAfter(value, token)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
substringAfter($value, "-")
```

Output

Return text after the first token.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

replace

Functions / String

Overview

Replace text using a regular expression.

Signature

```
replace(value, pattern, replacement)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
replace($value, "old", "new")
```

Output

Replace text using a regular expression.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

matches

Functions / String

Overview

Test text against a regular expression.

Signature

```
matches(value, pattern)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
matches($value, "^[A-Z]+$")
```

Output

Test text against a regular expression.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

startsWith

Functions / String

Overview

Test whether text starts with a prefix.

Signature

```
startsWith(value, prefix)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
startsWith($value, "PRE")
```

Output

Test whether text starts with a prefix.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

endsWith

Functions / String

Overview

Test whether text ends with a suffix.

Signature

```
endsWith(value, suffix)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
endsWith($value, "END")
```

Output

Test whether text ends with a suffix.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

contains

Functions / String

Overview

Test whether text contains a token.

Signature

```
contains(value, token)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
contains($value, "token")
```

Output

Test whether text contains a token.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

compare

Functions / String

Overview

Compare two strings and return -1, 0, or 1.

Signature

```
compare(value, other)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
compare($value, "other")
```

Output

Compare two strings and return -1, 0, or 1.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

translate

Functions / String

Overview

Translate characters by position.

Signature

```
translate(value, from, to)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
translate($value, "abc", "ABC")
```

Output

Translate characters by position.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

padLeft

Functions / String

Overview

Left-pad text to a fixed width.

Signature

```
padLeft(value, width, fill?)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
padLeft($value, 10, "0")
```

Output

Left-pad text to a fixed width.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

padRight

Functions / String

Overview

Right-pad text to a fixed width.

Signature

```
padRight(value, width, fill?)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
padRight($value, 10, " ")
```

Output

Right-pad text to a fixed width.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

tokenize

Functions / String

Overview

Split text using a regular expression.

Signature

```
tokenize(value, pattern)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
tokenize($value, ",")
```

Output

Split text using a regular expression.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

currentDate

Functions / Date & Time

Overview

Current UTC date in ISO format.

Signature

```
currentDate()
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
currentDate()
```

Output

Current UTC date in ISO format.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

currentTime

Functions / Date & Time

Overview

Current UTC time in ISO format.

Signature

```
currentTime()
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
currentTime()
```

Output

Current UTC time in ISO format.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

currentDateTime

Functions / Date & Time

Overview

Current UTC timestamp in ISO format.

Signature

```
currentDateTime()
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
currentDateTime()
```

Output

Current UTC timestamp in ISO format.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

parseDate

Functions / Date & Time

Overview

Parse an ISO date or timestamp.

Signature

```
parseDate(value)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
parseDate($value)
```

Output

Parse an ISO date or timestamp.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

formatDate

Functions / Date & Time

Overview

Format a date with yyyy/MM/dd-style patterns.

Signature

```
formatDate(value, pattern)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
formatDate($value, "yyyy-MM-dd")
```

Output

Format a date with yyyy/MM/dd-style patterns.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

formatDateTime

Functions / Date & Time

Overview

Format a timestamp.

Signature

```
formatDateTime(value, pattern)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
formatDateTime($value, "yyyy-MM-dd HH:mm:ss")
```

Output

Format a timestamp.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

addDays

Functions / Date & Time

Overview

Add days to a date or timestamp.

Signature

```
addDays(value, days)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
addDays($value, 1)
```

Output

Add days to a date or timestamp.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

addMonths

Functions / Date & Time

Overview

Add calendar months safely.

Signature

```
addMonths(value, months)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
addMonths($value, 1)
```

Output

Add calendar months safely.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

addHours

Functions / Date & Time

Overview

Add hours to a timestamp.

Signature

```
addHours(value, hours)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
addHours($value, 1)
```

Output

Add hours to a timestamp.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

addMinutes

Functions / Date & Time

Overview

Add minutes to a timestamp.

Signature

```
addMinutes(value, minutes)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
addMinutes($value, 30)
```

Output

Add minutes to a timestamp.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

addSeconds

Functions / Date & Time

Overview

Add seconds to a timestamp.

Signature

```
addSeconds(value, seconds)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
addSeconds($value, 30)
```

Output

Add seconds to a timestamp.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

dateDifference

Functions / Date & Time

Overview

Return the difference in days.

Signature

```
dateDifference(from, to)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
dateDifference($value, currentDateTime())
```

Output

Return the difference in days.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

year

Functions / Date & Time

Overview

Extract the year component.

Signature

```
year(value)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
year($value)
```

Output

Extract the year component.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

month

Functions / Date & Time

Overview

Extract the month component.

Signature

```
month(value)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
month($value)
```

Output

Extract the month component.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

day

Functions / Date & Time

Overview

Extract the day component.

Signature

```
day(value)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
day($value)
```

Output

Extract the day component.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

hour

Functions / Date & Time

Overview

Extract the hour component.

Signature

```
hour(value)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
hour($value)
```

Output

Extract the hour component.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

minute

Functions / Date & Time

Overview

Extract the minute component.

Signature

```
minute(value)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
minute($value)
```

Output

Extract the minute component.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

second

Functions / Date & Time

Overview

Extract the second component.

Signature

```
second(value)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
second($value)
```

Output

Extract the second component.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

timezoneFromDateTime

Functions / Date & Time

Overview

Return the timestamp timezone offset.

Signature

```
timezoneFromDateTime(value)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
timezoneFromDateTime($value)
```

Output

Return the timestamp timezone offset.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

equal

Functions / Comparison

Overview

Compare values using =.

Signature

```
equal(value, other)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
equal($value, 0)
```

Output

Compare values using =.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

notEqual

Functions / Comparison

Overview

Compare values using !=.

Signature

```
notEqual(value, other)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
notEqual($value, 0)
```

Output

Compare values using !=.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

greaterThan

Functions / Comparison

Overview

Compare values using >.

Signature

```
greaterThan(value, other)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
greaterThan($value, 0)
```

Output

Compare values using >.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

lessThan

Functions / Comparison

Overview

Compare values using <.

Signature

```
lessThan(value, other)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
lessThan($value, 0)
```

Output

Compare values using <.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

greaterOrEqual

Functions / Comparison

Overview

Compare values using >=.

Signature

```
greaterOrEqual(value, other)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
greaterOrEqual($value, 0)
```

Output

Compare values using >=.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

lessOrEqual

Functions / Comparison

Overview

Compare values using <=.

Signature

```
lessOrEqual(value, other)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
lessOrEqual($value, 0)
```

Output

Compare values using <=.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

add

Functions / Number

Overview

Add numeric values.

Signature

```
add(value, other)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
add($value, 2)
```

Output

Add numeric values.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

subtract

Functions / Number

Overview

Subtract numeric values.

Signature

```
subtract(value, other)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
subtract($value, 2)
```

Output

Subtract numeric values.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

multiply

Functions / Number

Overview

Multiply numeric values.

Signature

```
multiply(value, other)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
multiply($value, 2)
```

Output

Multiply numeric values.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

divide

Functions / Number

Overview

Divide numeric values.

Signature

```
divide(value, other)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
divide($value, 2)
```

Output

Divide numeric values.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

mod

Functions / Number

Overview

Mod numeric values.

Signature

```
mod(value, other)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
mod($value, 2)
```

Output

Mod numeric values.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

number

Functions / Number

Overview

Convert a value to a number.

Signature

```
number(value)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
number($value)
```

Output

Convert a value to a number.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

integer

Functions / Number

Overview

Convert a value to an integer.

Signature

```
integer(value)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
integer($value)
```

Output

Convert a value to an integer.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

round

Functions / Number

Overview

Round to an optional decimal precision.

Signature

```
round(value, precision?)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
round($value, 2)
```

Output

Round to an optional decimal precision.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

roundHalfToEven

Functions / Number

Overview

Banker's rounding for financial values.

Signature

```
roundHalfToEven(value, precision?)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
roundHalfToEven($value, 2)
```

Output

Banker's rounding for financial values.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

floor

Functions / Number

Overview

Round down to an integer.

Signature

```
floor(value)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
floor($value)
```

Output

Round down to an integer.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

ceiling

Functions / Number

Overview

Round up to an integer.

Signature

```
ceiling(value)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
ceiling($value)
```

Output

Round up to an integer.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

abs

Functions / Number

Overview

Return the absolute value.

Signature

```
abs(value)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
abs($value)
```

Output

Return the absolute value.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

sqrt

Functions / Number

Overview

Return the square root.

Signature

```
sqrt(value)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
sqrt($value)
```

Output

Return the square root.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

power

Functions / Number

Overview

Raise a value to a power.

Signature

```
power(value, exponent)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
power($value, 2)
```

Output

Raise a value to a power.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

modulo

Functions / Number

Overview

Return the division remainder.

Signature

```
modulo(value, divisor)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
modulo($value, 2)
```

Output

Return the division remainder.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

clamp

Functions / Number

Overview

Constrain a number to a range.

Signature

```
clamp(value, minimum, maximum)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
clamp($value, 0, 100)
```

Output

Constrain a number to a range.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

min

Functions / Number

Overview

Min collection values.

Signature

```
min(values)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
min($value)
```

Output

Min collection values.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

max

Functions / Number

Overview

Max collection values.

Signature

```
max(values)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
max($value)
```

Output

Max collection values.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

sum

Functions / Number

Overview

Sum collection values.

Signature

```
sum(values)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
sum($value)
```

Output

Sum collection values.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

average

Functions / Number

Overview

Average collection values.

Signature

```
average(values)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
average($value)
```

Output

Average collection values.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

count

Functions / Number

Overview

Count collection values.

Signature

```
count(values)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
count($value)
```

Output

Count collection values.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

lookup

Functions / Collection

Overview

Look up a key in an object table.

Signature

```
lookup(value, table, default?)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
lookup($value, ${input.table}, "Unknown")
```

Output

Look up a key in an object table.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

lookupTable

Functions / Collection

Overview

Look up a key in a named table saved with this Mapper.

Signature

```
lookupTable(value, name, default?)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
lookupTable($value, "countries", "Unknown")
```

Output

Look up a key in a named table saved with this Mapper.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

enrich

Functions / Collection

Overview

Merge enrichment fields into a record; enrichment replaces matching fields.

Signature

```
enrich(record, fields)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
enrich($value, ${input.fields})
```

Output

Merge enrichment fields into a record; enrichment replaces matching fields.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

chunks

Functions / Collection

Overview

Split records into bounded batches.

Signature

```
chunks(records, size)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
chunks($value, 100)
```

Output

Split records into bounded batches.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

joinBy

Functions / Collection

Overview

Join records by matching keys; every match produces a merged record.

Signature

```
joinBy(left, right, leftKey, rightKey)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
joinBy($value, ${input.right}, "id", "id")
```

Output

Join records by matching keys; every match produces a merged record.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

group

Functions / Collection

Overview

Group records by a field path, or group equal values. Returns an array of groups.

Signature

```
group(values, path?)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
group($value, "product")
```

Output

Group records by a field path, or group equal values. Returns an array of groups.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

top

Functions / Collection

Overview

Return the highest values in descending order (default: one).

Signature

```
top(values, count?)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
top($value, 10)
```

Output

Return the highest values in descending order (default: one).

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

distinctValues

Functions / Collection

Overview

Remove duplicate values while preserving order.

Signature

```
distinctValues(values)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
distinctValues($value)
```

Output

Remove duplicate values while preserving order.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

sort

Functions / Collection

Overview

Sort collection values.

Signature

```
sort(values, descending?)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
sort($value)
```

Output

Sort collection values.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

reverse

Functions / Collection

Overview

Reverse a collection or string.

Signature

```
reverse(values)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
reverse($value)
```

Output

Reverse a collection or string.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

first

Functions / Collection

Overview

Return the first value.

Signature

```
first(values)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
first($value)
```

Output

Return the first value.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

last

Functions / Collection

Overview

Return the last value.

Signature

```
last(values)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
last($value)
```

Output

Return the last value.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

indexOf

Functions / Collection

Overview

Return the zero-based position of an item.

Signature

```
indexOf(values, item)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
indexOf($value, "item")
```

Output

Return the zero-based position of an item.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

subsequence

Functions / Collection

Overview

Return part of a collection.

Signature

```
subsequence(values, start, end?)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
subsequence($value, 0, 10)
```

Output

Return part of a collection.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

flatten

Functions / Collection

Overview

Flatten one level of nested arrays.

Signature

```
flatten(values)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
flatten($value)
```

Output

Flatten one level of nested arrays.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

join

Functions / Collection

Overview

Join collection values into text.

Signature

```
join(values, separator)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
join($value, ",")
```

Output

Join collection values into text.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

split

Functions / Collection

Overview

Split text into a collection.

Signature

```
split(value, pattern)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
split($value, ",")
```

Output

Split text into a collection.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

deepEqual

Functions / Collection

Overview

Compare nested values structurally.

Signature

```
deepEqual(value, other)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
deepEqual($value, $input)
```

Output

Compare nested values structurally.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

string

Functions / Conversion

Overview

Convert a value to text.

Signature

```
string(value)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
string($value)
```

Output

Convert a value to text.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

boolean

Functions / Conversion

Overview

Convert common true/false representations.

Signature

```
boolean(value)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
boolean($value)
```

Output

Convert common true/false representations.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

jsonParse

Functions / Conversion

Overview

Parse JSON text.

Signature

```
jsonParse(value)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
jsonParse($value)
```

Output

Parse JSON text.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

jsonRender

Functions / Conversion

Overview

Serialize a value as JSON.

Signature

```
jsonRender(value)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
jsonRender($value)
```

Output

Serialize a value as JSON.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

render-xml

Functions / Conversion

Overview

Convert an XML object (including a parsed XML value) into an XML string. Use @name for attributes and #text for element text.

Signature

```
render-xml(value, rootName?, pretty?)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
render-xml($value)
```

Output

Convert an XML object (including a parsed XML value) into an XML string. Use @name for attributes and #text for element text.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

coalesce

Functions / General

Overview

Return the first non-null/non-empty value.

Signature

```
coalesce(value, ...fallbacks)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
coalesce($value, "fallback")
```

Output

Return the first non-null/non-empty value.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

default

Functions / General

Overview

Apply a fallback to null or empty values.

Signature

```
default(value, fallback)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
default($value, "fallback")
```

Output

Apply a fallback to null or empty values.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

exists

Functions / General

Overview

Test whether a value exists.

Signature

```
exists(value)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
exists($value)
```

Output

Test whether a value exists.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

empty

Functions / General

Overview

Test whether a value is empty.

Signature

```
empty(value)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
empty($value)
```

Output

Test whether a value is empty.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

not

Functions / General

Overview

Negate a boolean value.

Signature

```
not(value)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
not($value)
```

Output

Negate a boolean value.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

ifThenElse

Functions / General

Overview

Choose between two values.

Signature

```
ifThenElse(condition, trueValue, falseValue)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
ifThenElse($value, "yes", "no")
```

Output

Choose between two values.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

uuid

Functions / General

Overview

Generate a UUID.

Signature

```
uuid()
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
uuid()
```

Output

Generate a UUID.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

base64Encode

Functions / Encoding

Overview

Encode UTF-8 text as Base64.

Signature

```
base64Encode(value)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
base64Encode($value)
```

Output

Encode UTF-8 text as Base64.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

base64Decode

Functions / Encoding

Overview

Decode Base64 into UTF-8 text.

Signature

```
base64Decode(value)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
base64Decode($value)
```

Output

Decode Base64 into UTF-8 text.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

hexEncode

Functions / Encoding

Overview

Encode UTF-8 text as hexadecimal.

Signature

```
hexEncode(value)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
hexEncode($value)
```

Output

Encode UTF-8 text as hexadecimal.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

hexDecode

Functions / Encoding

Overview

Decode hexadecimal into UTF-8 text.

Signature

```
hexDecode(value)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
hexDecode($value)
```

Output

Decode hexadecimal into UTF-8 text.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

urlEncode

Functions / Encoding

Overview

Percent-encode text for a URL.

Signature

```
urlEncode(value)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
urlEncode($value)
```

Output

Percent-encode text for a URL.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

urlDecode

Functions / Encoding

Overview

Decode URL-encoded text.

Signature

```
urlDecode(value)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
urlDecode($value)
```

Output

Decode URL-encoded text.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

hash

Functions / Encoding

Overview

Hash text with SHA-256 or another supported algorithm.

Signature

```
hash(value, algorithm?)
```

Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
hash($value, "sha256")
```

Output

Hash text with SHA-256 or another supported algorithm.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

JDBC shared connection

Shared connections

Configuration

Create a shared resource in the project Resources tree, select it on activities, and use Test connection after resolving the active environment properties. Generated IDs are managed by Studio. Conditional fields below are shown only when their mode applies.

Field / key	Type / required	Details
Database driver	select Configuration setting	Database/JMS driver implementation or driver directory. Install the matching provider library on the runtime host; Java drivers require the bridge and compatible Java runtime. Choices: sqlite, postgresql, mysql, mariadb, sqlserver, oracle, db2, snowflake, databricks Default: "postgresql"
Connection runtime connectionMode	select Configuration setting	Connection lifecycle policy. Pooling/persistent connections reduce connection setup; recovery and keepalive apply to provider connections, not transaction-level replay. Choices: jdbc, python Default: "python" Shown when: (config) => ["sqlserver", "mssql", " oracle"].includes(config.driver)
JDBC driver JAR directory driverDirectory	text Configuration setting	Database/JMS driver implementation or driver directory. Install the matching provider library on the runtime host; Java drivers require the bridge and compatible Java runtime. Default: "" Shown when: (config) => ["sqlserver", "mssql", " oracle"].includes(config.driver) && config.connectionMode !== " python" Example / hint: Blank uses the MINA driver directory
JDBC driver class (blank = vendor default) driverClass	text Configuration setting	Database/JMS driver implementation or driver directory. Install the matching provider library on the runtime host; Java drivers require the bridge and compatible Java runtime. Default: "" Shown when: (config) => ["sqlserver", "mssql", " oracle"].includes(config.driver) && config.connectionMode !== " python"
JDBC URL url	text Configuration setting	Endpoint or filesystem path for this operation. For HTTP use the service URI/path; for files use a path accessible to the runtime host, not only the designer machine. Default: "jdbc:postgresql://localhost:5432/integration"
Host host	text Configuration setting	Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https. Default: "localhost"
Port port	text Configuration setting	Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https. Default: 5432
Database name database	text Configuration setting	Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions. Default: "integration"

Field / key	Type / required	Details
Schema schema	text Configuration setting	Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions. Default: "public"
PostgreSQL TLS mode postgresSslMode	select Configuration setting	PostgreSQL libpq TLS mode. Leave blank to use the connection URL or libpq default. Verify-full verifies the certificate chain and hostname. Choices: , disable, allow, prefer, require, verify-ca, verify-full Default: "" Shown when: (config) => config.driver === "postgresql"
PostgreSQL trusted root certificate postgresSslRootCert	text Configuration setting	Path to the PostgreSQL root certificate on the runtime host for TLS verification. Default: "" Shown when: (config) => config.driver === "postgresql"
Username username	text Configuration setting	Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions. Default: ""
Password password	password Configuration setting	Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs. Default: ""
SQL Server ODBC driver (legacy Python mode) odbcDriver	text Configuration setting	Database/JMS driver implementation or driver directory. Install the matching provider library on the runtime host; Java drivers require the bridge and compatible Java runtime. Default: "" Shown when: (config) => ["sqlserver", "mssql"].includes(config.driver) && config.connectionMode === "python"
SQL Server authentication authentication	select Configuration setting	Authentication mechanism for this connection. Supply the corresponding credentials/token/key settings and verify the receiving endpoint accepts that mechanism. Choices: SQL Server Authentication, Windows Integrated Default: "SQL Server Authentication" Shown when: (config) => ["sqlserver", "mssql"].includes(config.driver)
Encrypt SQL Server connection encrypt	select Configuration setting	Transport security setting. Enable TLS for confidential transport and verify server identity/certificates; disabling verification or trusting an unverified server weakens connection authentication. Choices: true, false Default: "true" Shown when: (config) => ["sqlserver", "mssql"].includes(config.driver)
Trust SQL Server certificate trustServerCertificate	select Configuration setting	Transport security setting. Enable TLS for confidential transport and verify server identity/certificates; disabling verification or trusting an unverified server weakens connection authentication. Choices: false, true Default: "false" Shown when: (config) => ["sqlserver", "mssql"].includes(config.driver)
Oracle service name serviceName	text Configuration setting	SOAP contract/operation settings. Select the service and operation from the WSDL; soapAction identifies the operation and envelope contains the SOAP XML request. Default: "" Shown when: (config) => config.driver === "oracle"

Field / key	Type / required	Details
Oracle SID (when service name is not used) sid	text Configuration setting	Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions. Default: "" Shown when: (config) => config.driver === "oracle"
MySQL/MariaDB CA certificate file sslCaFile	text Configuration setting	Trusted CA certificate location on the runtime host. A certificate folder can contain root certificates and optional chain certificates; configure the resolved deployment path. Default: "" Shown when: (config) => ["mysql", "mariadb"].includes(config.driver)
Databricks server hostname serverHostname	text Configuration setting	Provider endpoint override. Supply the reachable hostname/port or HTTP path expected by the chosen transport and the configured certificate identity. Default: "" Shown when: (config) => config.driver === "databricks"
Databricks HTTP path httpPath	text Configuration setting	Provider endpoint override. Supply the reachable hostname/port or HTTP path expected by the chosen transport and the configured certificate identity. Default: "" Shown when: (config) => config.driver === "databricks"
Databricks authentication authentication	select Configuration setting	Authentication mechanism for this connection. Supply the corresponding credentials/token/key settings and verify the receiving endpoint accepts that mechanism. Choices: Personal Access Token, OAuth M2M, OAuth U2M Default: "SQL Server Authentication" Shown when: (config) => config.driver === "databricks"
Databricks access token accessToken	password Configuration setting	Token or identity-provider endpoint for the selected authentication flow. Tokens expire; use the supported acquisition/refresh settings where required. Default: "" Shown when: (config) => config.driver === "databricks" && config.authentication === "Personal Access Token"
Databricks OAuth client ID clientId	text Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. Default: "" Shown when: (config) => config.driver === "databricks" && config.authentication === "OAuth M2M"
Databricks OAuth client secret clientSecret	password Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. Default: "" Shown when: (config) => config.driver === "databricks" && config.authentication === "OAuth M2M"
Databricks catalog catalog	text Configuration setting	Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions. Default: "" Shown when: (config) => config.driver === "databricks"

Field / key	Type / required	Details
Databricks CloudFetch useCloudFetch	select Configuration setting	Provider-specific connection feature. Enable only when the selected driver/server supports it; check the connector procedure and vendor prerequisites for that mode. Choices: true, false

Field / key	Type / required	Details
		Default: true Shown when: (config) => config.driver === "databricks"
Connection timeout (seconds) timeoutSeconds	text Configuration setting	Connection timeout (seconds): bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 30
Minimum pool size minimumPoolSize	text Configuration setting	Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity. Default: 1
Maximum pool size maximumPoolSize	text Configuration setting	Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity. Default: 10

Properties and credentials

Browse an environment property to bind a field; inspect the resolved value in Studio. Provide real credentials locally or through deployment secrets; never put secrets in documentation, source control or AI prompts. A password-style field is a UI indication, not a guarantee that all exports are redacted.

Testing and prerequisites

A connection test checks connectivity, not full application behavior, permissions for every operation, throughput or failover. Java-based connectors require their licensed driver JARs and bridge; SAP also requires a matching platform-native library. See Vendor Drivers and the connector guides.

Authentication and mandatory fields depend on the selected provider/mode; validate in Studio and on the target runtime.

Reference source: frontend/src/main.tsx

SNOWFLAKE shared connection

Shared connections

Configuration

Create a shared resource in the project Resources tree, select it on activities, and use Test connection after resolving the active environment properties. Generated IDs are managed by Studio. Conditional fields below are shown only when their mode applies.

Field / key	Type / required	Details
Runtime mode mode	select Configuration setting	Input/execution mode selected from the available choices. Match inline text, file or structured input to the actual source; connection modes such as mock/memory are for local testing. Choices: external, mock Default: "external"
Authentication type authenticationType	select Configuration setting	Authentication mechanism for this connection. Supply the corresponding credentials/token/key settings and verify the receiving endpoint accepts that mechanism. Choices: Username/Password, Federated Authentication and SSO, OAuth, Key Pair Authentication Default: "Username/Password"
Authentication provider provider	select Configuration setting	Provider/protocol or receive mode selection. Choose the broker implementation and delivery/settlement behavior supported by the configured connection. Choices: Snowflake, Okta Default: "Snowflake"
Account [account.region.platform] account	text Configuration setting	Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions. Default: ""
Snowflake username username	text Configuration setting	Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions. Default: ""
Snowflake password password	password Configuration setting	Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs. Default: "" Shown when: (config) => config.authenticationType === "Username/Password"
Okta token endpoint oktaTokenEndpoint	text Configuration setting	Token or identity-provider endpoint for the selected authentication flow. Tokens expire; use the supported acquisition/refresh settings where required. Default: "" Shown when: (config) => config.authenticationType === "OAuth" && config.provider === "Okta"
Okta username oktaUsername	text Configuration setting	Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions. Default: "" Shown when: (config) => config.authenticationType === "Federated Authentication and SSO" && config.provider === "Okta"
Okta password oktaPassword	password Configuration setting	Identity-provider password. Bind protected deployment configuration rather than embedding it in the project or logs.

Field / key	Type / required	Details
		Default: "" Shown when: (config) => config.authenticationType === " Federated Authentication and SSO" && config.provider === "Okta"
Okta endpoint URL oktaEndpointUrl	text Configuration setting	Token or identity-provider endpoint for the selected authentication flow. Tokens expire; use the supported acquisition/refresh settings where required. Default: "" Shown when: (config) => config.authenticationType === " Federated Authentication and SSO" && config.provider === "Okta"
OAuth client ID clientId	text Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. Default: "" Shown when: (config) => config.authenticationType === "OAuth"
OAuth client secret clientSecret	password Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. Default: "" Shown when: (config) => config.authenticationType === "OAuth"
OAuth scope [session:role:<role>] scope	text Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. Default: "" Shown when: (config) => config.authenticationType === "OAuth"
Authorization code authorizationCode	password Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. Default: "" Shown when: (config) => config.authenticationType === "OAuth" && config.provider === "Snowflake"
OAuth redirect URI redirectUri	text Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. Default: "" Shown when: (config) => config.authenticationType === "OAuth" && config.provider === "Snowflake"
OAuth access token accessToken	password Configuration setting	Token or identity-provider endpoint for the selected authentication flow. Tokens expire; use the supported acquisition/refresh settings where required. Default: "" Shown when: (config) => config.authenticationType === "OAuth"
Private key file privateKeyFile	text Configuration setting	Client/server certificate or private-key location/selection. Deploy matching certificate and key material and grant read access only to the runtime identity. Default: "" Shown when: (config) => config.authenticationType === "Key Pair Authentication"
Field / key	Type / required	Details
Private key passphrase privateKeyPassphrase	password Configuration setting	Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs.

Field / key	Type / required	Details
		Default: "" Shown when: (config) => config.authenticationType === "Key Pair Authentication"
Login timeout (seconds) loginTimeoutSeconds	text Configuration setting	Login timeout (seconds): bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 60
Warehouse warehouse	text Configuration setting	Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions. Default: ""
Default database database	text Configuration setting	Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions. Default: ""
Default schema schema	text Configuration setting	Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions. Default: "PUBLIC"
Default role role	text Configuration setting	Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions. Default: ""
Other properties [Name=Value;] otherProperties	text Configuration setting	Additional provider settings as name/value entries. Use exact supported provider option names; invalid options may be rejected at connection or execution time. Default: ""
Minimum connections minimumConnections	text Configuration setting	Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity. Default: 2
Maximum connections maximumConnections	text Configuration setting	Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity. Default: 8
Maximum connection wait (seconds) maximumConnectionWaitSeconds	text Configuration setting	Provider lifetime/wait setting in seconds, as named by this field. Set it above normal work/latency while retaining a bounded recovery time. Default: 300
Service number of threads serviceThreads	text Configuration setting	Provider/listener concurrency setting. Coordinate it with downstream capacity and the operation-specific runtime support; it is not a whole-workflow parallel-branch limit. Default: 8

Properties and credentials

Browse an environment property to bind a field; inspect the resolved value in Studio. Provide real credentials locally or through deployment secrets; never put secrets in documentation, source control or AI prompts. A password-style field is a UI indication, not a guarantee that all exports are redacted.

Testing and prerequisites

A connection test checks connectivity, not full application behavior, permissions for every operation, throughput or failover. Java-based connectors require their licensed driver JARs and bridge; SAP also requires a matching platform-native library. See Vendor Drivers and the connector guides.

Authentication and mandatory fields depend on the selected provider/mode; validate in Studio and on the target runtime.

Reference source: frontend/src/main.tsx

AMQP shared connection

Shared connections

Configuration

Create a shared resource in the project Resources tree, select it on activities, and use Test connection after resolving the active environment properties. Generated IDs are managed by Studio. Conditional fields below are shown only when their mode applies.

Field / key	Type / required	Details
Broker type brokerType	select Configuration setting	Provider/protocol or receive mode selection. Choose the broker implementation and delivery/settlement behavior supported by the configured connection. Choices: Qpid-1-0, RabbitMQ, ActiveMQ-1-0, AzureSB-1-0, AMQ-1-0, ActiveMQ-Artemis-1-0 Default: "RabbitMQ"
AMQP version amqpVersion	select Configuration setting	Provider/protocol or receive mode selection. Choose the broker implementation and delivery/settlement behavior supported by the configured connection. Choices: AMQP-0-9-1, AMQP-1-0 Default: "AMQP-0-9-1" Shown when: (config) => config.brokerType === "RabbitMQ"
Host:Port (comma-separated for failover) hostPort	text Required	Provider endpoint override. Supply the reachable hostname/port or HTTP path expected by the chosen transport and the configured certificate identity. Default: "localhost:5672" Shown when: (config) => config.brokerType !== "AzureSB-1-0"
Virtual host virtualHost	text Configuration setting	Provider connection address/namespace. Use the provider URI/connection-string syntax; Kafka bootstrapServers accepts broker host:port endpoints and AMQP virtualHost selects the broker namespace. Default: "/" Shown when: (config) => ["RabbitMQ", " Qpid-1-0"].includes(config.brokerType)
Username username	text Configuration setting	Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions. Default: "guest" Shown when: (config) => config.brokerType !== "AzureSB-1-0"
Password password	password Configuration setting	Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs. Default: "guest" Shown when: (config) => config.brokerType !== "AzureSB-1-0"
Client ID clientId	text Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. Default: "" Shown when: (config) => !["RabbitMQ", " AzureSB-1-0"].includes(config.brokerType)
Azure authentication authenticationType	select Configuration setting	Authentication mechanism for this connection. Supply the corresponding credentials/token/key settings and verify the receiving endpoint accepts that mechanism. Choices: SAS, OAuth, ManagedIdentity Default: "SAS" Shown when: (config) => config.brokerType === "AzureSB-1-0"

Field / key	Type / required	Details
Azure Service Bus endpoint / connection string connectionString	text Required	Provider connection address/namespace. Use the provider URI/connection-string syntax; Kafka bootstrapServers accepts broker host:port endpoints and AMQP virtualHost selects the broker namespace. Default: "" Shown when: (config) => config.brokerType === "AzureSB-1-0"
Azure tenant ID tenantId	text Required	Provider credential/identity material. Use the mechanism supported by the selected connection and runtime; keep private keys, access keys and credential files outside shared source packages. Default: "" Shown when: (config) => config.brokerType === "AzureSB-1-0" && config.authenticationType === "OAuth"
Azure application/client ID azureClientId	text Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. Default: "" Shown when: (config) => config.brokerType === "AzureSB-1-0" && ["OAuth", "ManagedIdentity"].includes(config.authenticationType)
Azure client secret clientSecret	password Required	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. Default: "" Shown when: (config) => config.brokerType === "AzureSB-1-0" && config.authenticationType === "OAuth"
Shared access key name sharedAccessKeyName	text Configuration setting	Provider credential/identity material. Use the mechanism supported by the selected connection and runtime; keep private keys, access keys and credential files outside shared source packages. Default: "" Shown when: (config) => config.brokerType === "AzureSB-1-0" && config.authenticationType === "SAS"
Shared access key sharedAccessKey	password Configuration setting	Provider credential/identity material. Use the mechanism supported by the selected connection and runtime; keep private keys, access keys and credential files outside shared source packages. Default: "" Shown when: (config) => config.brokerType === "AzureSB-1-0" && config.authenticationType === "SAS"
Default entity type entityType	select Configuration setting	AMQP routing configuration. entityType selects queue/topic where supported; exchanges route messages using the routing key and exchange type. Verify bindings before publishing. Choices: Queue, Topic Default: "Queue" Shown when: (config) => config.brokerType === "AzureSB-1-0"
Default entity name entityName	text Configuration setting	Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access. Default: "" Shown when: (config) => config.brokerType === "AzureSB-1-0"

Field / key	Type / required	Details
Default subscription entitySubscriberName	text Configuration setting	Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access.

Field / key	Type / required	Details
		Default: "" Shown when: (config) => config.brokerType === "AzureSB-1-0" && config.entityType === "Topic"
Connection timeout (msec) connectionTimeoutMsec	text Configuration setting	Connection timeout (msec): bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 30000
RabbitMQ session count (1-20) sessionCount	text Configuration setting	Provider/listener concurrency setting. Coordinate it with downstream capacity and the operation-specific runtime support; it is not a whole-workflow parallel-branch limit. Default: 1 Shown when: (config) => config.brokerType === "RabbitMQ"
Idle timeout (msec) idleTimeoutMsec	text Configuration setting	Idle timeout (msec): bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 0 Shown when: (config) => config.brokerType === "RabbitMQ" && config.amqpVersion === "AMQP-1-0"
Connection recovery connectionRecovery	select Configuration setting	Connection lifecycle policy. Pooling/persistent connections reduce connection setup; recovery and keepalive apply to provider connections, not transaction-level replay. Choices: true, false Default: "true"
Retry interval (msec) retryIntervalMsec	text Configuration setting	Retry interval (msec): bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 3000 Shown when: (config) => config.brokerType !== "RabbitMQ"
Retry attempts retryAttempts	text Configuration setting	Retry attempt policy. A retry can repeat an accepted external write; use stable identifiers and destination-side duplicate handling. Default: 20 Shown when: (config) => config.brokerType !== "RabbitMQ"
RabbitMQ recovery interval (msec) networkRecoveryIntervalMsec	text Configuration setting	RabbitMQ recovery interval (msec): bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 5000 Shown when: (config) => config.brokerType === "RabbitMQ"
SSL confidentiality sslEnabled	select Configuration setting	Transport security setting. Enable TLS for confidential transport and verify server identity/certificates; disabling verification or trusting an unverified server weakens connection authentication. Choices: false, true Default: "false"
Trusted CA file caFile	text Configuration setting	Trusted CA certificate location on the runtime host. A certificate folder can contain root certificates and optional chain certificates; configure the resolved deployment path. Default: "" Shown when: (config) => String(config.sslEnabled) === "true"
Client certificate clientCertificateFile	text Configuration setting	Client/server certificate or private-key location/selection. Deploy matching certificate and key material and grant read access only to the runtime identity. Default: "" Shown when: (config) => String(config.sslEnabled) === "true"

Field / key	Type / required	Details
Client private key clientKeyFile	text Configuration setting	Client/server certificate or private-key location/selection. Deploy matching certificate and key material and grant read access only to the runtime identity. Default: "" Shown when: (config) => String(config.sslEnabled) === "true"
Private key password clientKeyPassword	password Configuration setting	Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs. Default: "" Shown when: (config) => String(config.sslEnabled) === "true"

Properties and credentials

Browse an environment property to bind a field; inspect the resolved value in Studio. Provide real credentials locally or through deployment secrets; never put secrets in documentation, source control or AI prompts. A password-style field is a UI indication, not a guarantee that all exports are redacted.

Testing and prerequisites

A connection test checks connectivity, not full application behavior, permissions for every operation, throughput or failover. Java-based connectors require their licensed driver JARs and bridge; SAP also requires a matching platform-native library. See Vendor Drivers and the connector guides.

Authentication and mandatory fields depend on the selected provider/mode; validate in Studio and on the target runtime.

Reference source: frontend/src/main.tsx

HTTP shared connection

Shared connections

Configuration

Create a shared resource in the project Resources tree, select it on activities, and use Test connection after resolving the active environment properties. Generated IDs are managed by Studio. Conditional fields below are shown only when their mode applies.

Field / key	Type / required	Details
Connection role connectorMode	select Configuration setting	Connection lifecycle policy. Pooling/persistent connections reduce connection setup; recovery and keepalive apply to provider connections, not transaction-level replay. Choices: client, server, both Default: "both"
MINA API Blueprint contract (JSON / YAML) openApiDocument	text Configuration setting	API Blueprint stores one validated contract on this dedicated server connection. Operation tasks select their operation IDs. Re-import when adding operations; editing a contract does not replace existing business mappings. Default: "" Shown when: (config) => !!config.apiBlueprintId
Protocol scheme	select Configuration setting	Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https. Choices: http, https Default: "http"
Default host host	text Configuration setting	Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https. Default: "localhost"
Default port port	text Configuration setting	Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https. Default: 8080
Base path basePath	text Configuration setting	Endpoint or filesystem path for this operation. For HTTP use the service URI/path; for files use a path accessible to the runtime host, not only the designer machine. Default: ""
Host Swagger documentation apiDocsEnabled	select Configuration setting	Serve browser API documentation at /_mina/docs, the OpenAPI document at /_mina/openapi.json, and a downloadable Postman collection at /_mina/postman.json. Enabled by default for HTTP and REST listeners; /_mina is reserved while enabled. Choices: true, false Default: "" Shown when: (config) => config.connectorMode !== "client"
API documentation access apiDocsAccess	select Configuration setting	Authenticated documentation uses the listener's configured authentication for its OpenAPI document and Postman download. Public exposes those documents without credentials. API operations always retain their own authentication. Choices: authenticated, public Default: "" Shown when: (config) => config.connectorMode !== "client"

Field / key	Type / required	Details
Public API base URL apiPublicBaseUrl	text Configuration setting	Optional externally reachable HTTP or HTTPS URL, including a gateway prefix. Used by generated API clients. Run and Debug

Field / key	Type / required	Details
		show the live listener address separately. Do not include credentials, query strings, or fragments. Default: "" Shown when: (config) => config.connectorMode !== "client"
Outbound base URL baseUrl	text Configuration setting	Endpoint or filesystem path for this operation. For HTTP use the service URI/path; for files use a path accessible to the runtime host, not only the designer machine. Default: "https://api.example.com" Shown when: (config) => config.connectorMode !== "server"
HTTP protocol version httpVersion	select Configuration setting	HTTP protocol version reported for the response, for example HTTP/1.1. Choices: 1.1, 2, 3 Default: "1.1"
Authentication authentication	select Configuration setting	Authentication mechanism for this connection. Supply the corresponding credentials/token/key settings and verify the receiving endpoint accepts that mechanism. Choices: None, Basic, Digest, NTLM, Bearer, OAuth2, JWT, HMAC, LDAP, Certificate Default: "None"
Username username	text Configuration setting	Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions. Default: "" Shown when: (config) => ["Basic", "Digest", " NTLM"].includes(config.authentication)
Password password	password Configuration setting	Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs. Default: "" Shown when: (config) => ["Basic", "Digest", " NTLM"].includes(config.authentication)
Non-preemptive Basic authentication nonPreemptiveAuthentication	select Configuration setting	Outbound proxy/authentication policy. Configure the proxy address and credentials when the runtime network requires it; non-preemptive authentication waits for a challenge where supported. Choices: false, true Default: "" Shown when: (config) => config.authentication === "Basic" && config.connectorMode !== "server"
Bearer access token bearerToken	password Configuration setting	Token or identity-provider endpoint for the selected authentication flow. Tokens expire; use the supported acquisition/refresh settings where required. Default: "" Shown when: (config) => ["Bearer"].includes(config.authentication)
Default confidentiality / system trust defaultConfidentiality	select Configuration setting	Require HTTPS, verify the provider certificate and hostname, and use system trust plus any configured trusted roots. A separate client certificate is needed only when the provider requires mutual TLS. Choices: false, true Default: "false" Shown when: (config) => config.connectorMode !== "server"
Custom confidentiality confidentiality	select Configuration setting	Transport security setting. Enable TLS for confidential transport and verify server identity/certificates; disabling verification or trusting an unverified server weakens connection authentication. Choices: false, true Default: ""

Field / key	Type / required	Details
Enable TLS tlsEnabled	select Configuration setting	Transport security setting. Enable TLS for confidential transport and verify server identity/certificates; disabling verification or trusting an unverified server weakens connection authentication. Choices: false, true Default: "false"
Verify server certificate and hostname verifyTls	select Configuration setting	Transport security setting. Enable TLS for confidential transport and verify server identity/certificates; disabling verification or trusting an unverified server weakens connection authentication. Choices: true, false Default: "true" Shown when: (config) => config.connectorMode !== "server"
Minimum TLS version tlsVersion	select Configuration setting	TLS negotiation/client-certificate/revocation configuration. Select protocols and ciphers compatible with the peer and provide a CRL or client identity only where supported by the chosen provider. Choices: TLSv1.2, TLSv1.3 Default: "TLSv1.2"
Maximum TLS version maximumTlsVersion	select Configuration setting	TLS negotiation/client-certificate/revocation configuration. Select protocols and ciphers compatible with the peer and provide a CRL or client identity only where supported by the chosen provider. Choices: , TLSv1.2, TLSv1.3 Default: ""
TLS 1.2 cipher list cipherSuites	text Configuration setting	TLS negotiation/client-certificate/revocation configuration. Select protocols and ciphers compatible with the peer and provide a CRL or client identity only where supported by the chosen provider. Default: ""
Trusted root certificate folder (chain certificates optional) trustedCertificateFolder	text Configuration setting	Place trusted root CA certificates in this folder. Intermediates are optional: the provider normally sends them during TLS negotiation. Do not copy the provider leaf certificate or private keys into the folder. A provider that omits an intermediate must correct its chain or supply the optional intermediate. Default: "" Example / hint: Folder path or ${properties.connections.http.trustedCertificateFolder}
Trust store format trustStoreType	select Configuration setting	Trust-store file and format used by the selected provider. Supply trusted issuers in the supported store format; a certificate folder is an alternative where the connector supports it. Choices: PEM, PKCS12, JKS Default: ""
Trust store / CA certificate path trustStoreFile	text Configuration setting	Trust-store file and format used by the selected provider. Supply trusted issuers in the supported store format; a certificate folder is an alternative where the connector supports it. Default: ""
Trust store password trustStorePassword	password Configuration setting	Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs. Default: ""
Identity store format identityStoreType	select Configuration setting	Identity store containing the local certificate/private key. Supply its runtime path, password and supported format; this is distinct from the peer CA trust store. Choices: PKCS12, JKS Default: ""
Identity store path identityStoreFile	text Configuration setting	Identity store containing the local certificate/private key. Supply its runtime path, password and supported format; this is distinct from the peer CA trust store. Default: ""

Field / key	Type / required	Details
Identity store password identityStorePassword	password Configuration setting	Identity store containing the local certificate/private key. Supply its runtime path, password and supported format; this is distinct from the peer CA trust store. Default: ""
Identity key alias (JKS) keyAlias	text Configuration setting	Client/server certificate or private-key location/selection. Deploy matching certificate and key material and grant read access only to the runtime identity. Default: ""
PEM certificate chain path certificateFile	text Configuration setting	Client/server certificate or private-key location/selection. Deploy matching certificate and key material and grant read access only to the runtime identity. Default: ""
PEM private key path privateKeyFile	text Configuration setting	Client/server certificate or private-key location/selection. Deploy matching certificate and key material and grant read access only to the runtime identity. Default: ""
PEM private key password privateKeyPassword	password Configuration setting	Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs. Default: ""
Inbound client certificates clientAuthentication	select Configuration setting	TLS negotiation/client-certificate/revocation configuration. Select protocols and ciphers compatible with the peer and provide a CRL or client identity only where supported by the chosen provider. Choices: none, optional, required Default: "none" Shown when: (config) => config.connectorMode !== "client"
PEM certificate revocation list crlFile	text Configuration setting	TLS negotiation/client-certificate/revocation configuration. Select protocols and ciphers compatible with the peer and provide a CRL or client identity only where supported by the chosen provider. Default: ""
Connection timeout (ms; 0 unlimited) connectionTimeoutMs	text Configuration setting	Connection timeout (ms; 0 unlimited): bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 30000 Shown when: (config) => config.connectorMode !== "server"
Socket / read timeout (ms; 0 unlimited) socketTimeoutMs	text Configuration setting	Socket / read timeout (ms; 0 unlimited): bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 60000 Shown when: (config) => config.connectorMode !== "server"
Write timeout (ms; 0 unlimited) writeTimeoutMs	text Configuration setting	Write timeout (ms; 0 unlimited): bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 60000 Shown when: (config) => config.connectorMode !== "server"
Connection pool wait timeout (ms; 0 unlimited) poolTimeoutMs	text Configuration setting	Connection pool wait timeout (ms; 0 unlimited): bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 30000 Shown when: (config) => config.connectorMode !== "server"

Field / key	Type / required	Details
Maximum total client connections maximumTotalConnections	text Configuration setting	Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity. Default: 200 Shown when: (config) => config.connectorMode !== "server"
Maximum client connections per host maximumConnectionsPerHost	text Configuration setting	Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity. Default: 20 Shown when: (config) => config.connectorMode !== "server"
Disable client pooling disableConnectionPooling	select Configuration setting	Connection lifecycle policy. Pooling/persistent connections reduce connection setup; recovery and keepalive apply to provider connections, not transaction-level replay. Choices: false, true Default: "" Shown when: (config) => config.connectorMode !== "server"
Persistent connections usePersistentConnections	select Configuration setting	Connection lifecycle policy. Pooling/persistent connections reduce connection setup; recovery and keepalive apply to provider connections, not transaction-level replay. Choices: true, false Default: ""
Idle connection timeout (ms) idleConnectionTimeoutMs	text Configuration setting	Idle connection timeout (ms): bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 40000
Follow redirects followRedirects	select Configuration setting	Follow HTTP redirect responses. Disable when the flow must inspect or reject redirects explicitly. Choices: false, true Default: "" Shown when: (config) => config.connectorMode !== "server"
Connection retry count retryCount	text Configuration setting	Retry attempt policy. A retry can repeat an accepted external write; use stable identifiers and destination-side duplicate handling. Default: "" Shown when: (config) => config.connectorMode !== "server"
Connection retry delay (ms) retryDelayMs	text Configuration setting	Connection retry delay (ms): bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: "" Shown when: (config) => config.connectorMode !== "server"
Retry response status codes retryStatusCodes	text Configuration setting	Comma-separated HTTP statuses eligible for bounded retries. Disabled when empty. Non-idempotent methods require an Idempotency-Key or the explicit retry override. Default: "" Shown when: (config) => config.connectorMode !== "server" Example / hint: 429,502,503,504
Maximum retry delay (ms) maxRetryDelayMs	text Configuration setting	Caps exponential backoff and Retry-After waits. All attempts share the activity timeout budget. Default: "" Shown when: (config) => config.connectorMode !== "server"
Field / key	Type / required	Details
Randomize retry backoff retryJitter	select Configuration setting	Spread simultaneous retries using bounded random backoff to reduce load spikes. Choices: true, false

Field / key	Type / required	Details
		Default: "" Shown when: (config) => config.connectorMode !== "server"
Respect Retry-After respectRetryAfter	select Configuration setting	Honor a server retry delay or HTTP date, bounded by Maximum retry delay and the remaining activity budget. Choices: true, false Default: "" Shown when: (config) => config.connectorMode !== "server"
Allow retries of non-idempotent methods retryNonIdempotent	select Configuration setting	Enable only when the downstream operation safely deduplicates repeated POST/PATCH calls. Prefer a mapped Idempotency-Key. Choices: false, true Default: "" Shown when: (config) => config.connectorMode !== "server"
Circuit breaker failures (0 disabled) circuitFailureThreshold	text Configuration setting	Consecutive connectivity/server failures before opening this runtime's outbound circuit. A half-open probe tests recovery. Default: "" Shown when: (config) => config.connectorMode !== "server"
Circuit recovery delay (seconds) circuitResetSeconds	text Configuration setting	Wait before permitting one half-open probe. Success closes the circuit; failure reopens it. Default: "" Shown when: (config) => config.connectorMode !== "server"
Allowed browser origins corsAllowedOrigins	text Configuration setting	Space/comma-separated exact origins. Empty disables CORS. Wildcard cannot be combined with credentialed requests. Default: "" Shown when: (config) => config.connectorMode !== "client"
Allow credentialed CORS corsAllowCredentials	select Configuration setting	Permit browser credentials only for explicitly listed origins; never combine with a wildcard origin. Choices: false, true Default: "" Shown when: (config) => config.connectorMode !== "client"
Allowed CORS request headers corsAllowedHeaders	text Configuration setting	Comma-separated header names permitted by preflight. Defaults to content-type and authorization. Default: "" Shown when: (config) => config.connectorMode !== "client"
Response headers exposed to browsers corsExposeHeaders	text Configuration setting	Comma-separated response header names that browser code may read. Default: "" Shown when: (config) => config.connectorMode !== "client"
CORS preflight cache (seconds) corsMaxAgeSeconds	text Configuration setting	Browser preflight cache duration, bounded to 86400 seconds. Default 600. Default: "" Shown when: (config) => config.connectorMode !== "client"
Requests per client/window (0 disabled) rateLimitRequests	text Configuration setting	Fixed-window limit keyed by the actual remote IP and listener route. Excess requests return 429 with Retry-After. Default: "" Shown when: (config) => config.connectorMode !== "client"
Rate limit window (seconds) rateLimitWindowSeconds	text Configuration setting	Window duration for per-client throttling. Use Redis policy storage when limits must span replicas. Default: "" Shown when: (config) => config.connectorMode !== "client"
Field / key	Type / required	Details
Replay protection / rate limit storage policyStore	select Configuration setting	Memory applies within one runtime. Redis uses atomic counters and nonce claims across replicas and fails closed if unavailable. Choices: memory, redis

Field / key	Type / required	Details
		Default: "" Shown when: (config) => config.connectorMode !== "client"
Redis policy connection URL policyRedisUrl	password Configuration setting	Property-bound redis:// or rediss:// connection URL. Use rediss:// for encrypted connections and protect credentials. Default: "" Shown when: (config) => config.policyStore === "redis"
Redis policy timeout (seconds) policyTimeoutSeconds	text Configuration setting	Bound Redis connection and socket waits. Default 3 seconds; failures reject protected requests with 503. Default: "" Shown when: (config) => config.policyStore === "redis"
Inbound body storage requestBodyMode	select Configuration setting	File mode streams uploads to a temporary file; mappings receive fileName/size. The file is removed after the request flow finishes. Schema body validation requires memory mode. Choices: memory, file Default: "" Shown when: (config) => config.connectorMode !== "client"
Temporary upload directory uploadDirectory	text Configuration setting	Existing writable runtime directory for file-backed request bodies. Uploaded files are automatically cleaned on completion, failure and disconnect. Default: "" Shown when: (config) => config.requestBodyMode === "file"
Accept and resend cookies enableCookies	select Configuration setting	Connection lifecycle policy. Pooling/persistent connections reduce connection setup; recovery and keepalive apply to provider connections, not transaction-level replay. Choices: false, true Default: "" Shown when: (config) => config.connectorMode !== "server"
Default headers JSON headers	text Configuration setting	Header name/value object. Map only headers required by the target protocol; credentials belong in protected authentication settings. Default: "" Shown when: (config) => config.connectorMode !== "server"
Default query parameters JSON query	text Configuration setting	HTTP query parameters. Use a name/value object for structured parameters or QueryString for an already composed query; encode values rather than concatenating untrusted text. Default: "" Shown when: (config) => config.connectorMode !== "server"
Proxy URL proxyUrl	text Configuration setting	Outbound proxy/authentication policy. Configure the proxy address and credentials when the runtime network requires it; non-preemptive authentication waits for a challenge where supported. Default: "" Shown when: (config) => config.connectorMode !== "server"
Proxy host proxyHost	text Configuration setting	Outbound proxy/authentication policy. Configure the proxy address and credentials when the runtime network requires it; non-preemptive authentication waits for a challenge where supported. Default: "" Shown when: (config) => config.connectorMode !== "server"
Proxy port proxyPort	text Configuration setting	Outbound proxy/authentication policy. Configure the proxy address and credentials when the runtime network requires it; non-preemptive authentication waits for a challenge where supported. Default: 8080 Shown when: (config) => config.connectorMode !== "server"

Field / key	Type / required	Details
Proxy username proxyUsername	text Configuration setting	Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions. Default: "" Shown when: (config) => config.connectorMode !== "server"
Proxy password proxyPassword	password Configuration setting	Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs. Default: "" Shown when: (config) => config.connectorMode !== "server"
Minimum QTP-equivalent authentication workers minimumQtpThreads	text Configuration setting	Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity. Default: 10 Shown when: (config) => config.connectorMode !== "client"
Maximum HTTP workers / concurrent executions maximumQtpThreads	text Configuration setting	Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity. Default: 75 Shown when: (config) => config.connectorMode !== "client"
Maximum queued HTTP connections maxQueueSize	text Configuration setting	Size/capacity bound for the named request, response, header or event queue. Byte limits measure encoded content; queue limits bound buffered work. Default: 100 Shown when: (config) => config.connectorMode !== "client"
Execution queue timeout (ms) queueTimeoutMs	text Configuration setting	Execution queue timeout (ms): bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: "" Shown when: (config) => config.connectorMode !== "client"
Socket accept backlog acceptQueueSize	text Configuration setting	Size/capacity bound for the named request, response, header or event queue. Byte limits measure encoded content; queue limits bound buffered work. Default: 128 Shown when: (config) => config.connectorMode !== "client"
Inbound read timeout (ms) readTimeoutMs	text Configuration setting	Inbound read timeout (ms): bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 30000 Shown when: (config) => config.connectorMode !== "client"
Maximum request header bytes maxHeaderBytes	text Configuration setting	Size/capacity bound for the named request, response, header or event queue. Byte limits measure encoded content; queue limits bound buffered work. Default: 65536 Shown when: (config) => config.connectorMode !== "client"
Maximum request body bytes maxRequestBodyBytes	text Configuration setting	Size/capacity bound for the named request, response, header or event queue. Byte limits measure encoded content; queue limits bound buffered work. Default: 67108864
Maximum response body bytes maxResponseBodyBytes	text Configuration setting	Size/capacity bound for the named request, response, header or event queue. Byte limits measure encoded content; queue limits bound buffered work. Default: 67108864

Field / key	Type / required	Details
Allowed client IPs / CIDRs allowedIps	text Configuration setting	Inbound client IP allowlist for the hosted service. Configure the addresses visible to the runtime, including proxy/network translation where applicable. Default: "" Shown when: (config) => config.connectorMode !== "client"
OpenAPI API key secrets openApiApiKeySecrets	password Configuration setting	Bind a protected property or enter a JSON object that maps OpenAPI security scheme names to API key secrets. Archived projects require the secrets at deployment. Default: "" Shown when: (config) => config.connectorMode !== "client"
Trusted proxy IPs / CIDRs trustedProxyCidrs	text Configuration setting	Proxy IPs or CIDRs allowed to supply X-Forwarded-For. Leave empty to use the transport peer for all requests. Default: "" Shown when: (config) => config.connectorMode !== "client"
Listener error response format httpErrorFormat	select Configuration setting	Use problem to return RFC 9457 Problem Details for listener-generated errors. The json option keeps the original error response shape. Choices: json, problem Default: "" Shown when: (config) => config.connectorMode !== "client"
Conditional GET and HEAD conditionalResponses	select Configuration setting	Return 304 for matching GET or HEAD validators using mapped ETag or Last-Modified response headers. The workflow still runs. Choices: false, true Default: "" Shown when: (config) => config.connectorMode !== "client"
Propagate HTTP trace context propagateTraceContext	select Configuration setting	Enable W3C version 00 trace context for this connection. Enable it on both the listener and outbound connection to follow a request across calls. Choices: false, true Default: ""
Shared public response cache sharedHttpCache	select Configuration setting	Share fresh, explicitly public GET responses within the runtime process. Credentialed and conditional requests bypass this cache. Choices: false, true Default: "" Shown when: (config) => config.connectorMode !== "server"
Allowed callback HTTPS origins callbackAllowedOrigins	text Configuration setting	Explicit HTTPS origins trusted to receive callback requests. Separate origins with commas or spaces. Default: "" Shown when: (config) => config.connectorMode !== "server"
Total activity / listener timeout (seconds) activityTimeoutSeconds	text Configuration setting	Total activity / listener timeout (seconds): bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 180
OAuth2 grant oauthGrantType	select Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. Choices: client_credentials, authorization_code, refresh_token Default: "" Shown when: (config) => ["OAuth2"].includes(config.authentication)
Field / key	Type / required	Details
OAuth2 token URL oauthTokenUrl	text Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding

Field / key	Type / required	Details
		grant. Default: "" Shown when: (config) => ["OAuth2"].includes(config.authentication)
OAuth2 client ID oauthClientId	text Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. Default: "" Shown when: (config) => ["OAuth2"].includes(config.authentication)
OAuth2 client secret oauthClientSecret	password Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. Default: "" Shown when: (config) => config.authentication === "OAuth2" && config.oauthClientAuthentication !== "private_key_jwt"
OAuth2 client authentication oauthClientAuthentication	select Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. Choices: basic, body, private_key_jwt Default: "" Shown when: (config) => config.authentication === "OAuth2"
OAuth client signing key file oauthClientPrivateKeyFile	text Required	Path on the runtime host to the PEM signing key registered for the OAuth client. Provide the key file separately from the archive. Default: "" Shown when: (config) => config.authentication === "OAuth2" && config.oauthClientAuthentication === "private_key_jwt"
OAuth client assertion algorithm oauthClientAssertionAlgorithm	select Configuration setting	Asymmetric algorithm registered for the OAuth client. Defaults to RS256. Choices: RS256, RS384, RS512, ES256, ES384, ES512 Default: "" Shown when: (config) => config.authentication === "OAuth2" && config.oauthClientAuthentication === "private_key_jwt"
OAuth client assertion key ID oauthClientAssertionKeyId	text Configuration setting	Optional kid included in the signed OAuth client assertion so the provider can select a verification key. Default: "" Shown when: (config) => config.authentication === "OAuth2" && config.oauthClientAuthentication === "private_key_jwt"
OAuth2 scopes oauthScope	text Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. Default: "" Shown when: (config) => ["OAuth2"].includes(config.authentication)
OAuth2 authorization code oauthAuthorizationCode	password Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. Default: "" Shown when: (config) => ["OAuth2"].includes(config.authentication)
Field / key	Type / required	Details
OAuth2 redirect URI oauthRedirectUri	text Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.

Field / key	Type / required	Details
		Default: "" Shown when: (config) => ["OAuth2"].includes(config.authentication)
OAuth2 PKCE verifier oauthCodeVerifier	password Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. Default: "" Shown when: (config) => ["OAuth2"].includes(config.authentication)
OAuth2 refresh token oauthRefreshToken	password Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. Default: "" Shown when: (config) => ["OAuth2"].includes(config.authentication)
OAuth2 endpoint timeout (seconds) oauthTimeoutSeconds	text Configuration setting	OAuth2 endpoint timeout (seconds): bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 30 Shown when: (config) => ["OAuth2"].includes(config.authentication)
OAuth2 inbound token introspection URL oauthIntrospectionUrl	text Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. Default: "" Shown when: (config) => ["OAuth2"].includes(config.authentication)
Required inbound OAuth2 scopes oauthRequiredScopes	text Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. Default: "" Shown when: (config) => ["OAuth2"].includes(config.authentication)
Outbound preissued JWT jwtToken	password Configuration setting	Token or identity-provider endpoint for the selected authentication flow. Tokens expire; use the supported acquisition/refresh settings where required. Default: "" Shown when: (config) => ["JWT"].includes(config.authentication)
JWT signature algorithm jwtAlgorithm	select Configuration setting	JWT signing/validation setting. Match algorithm, issuer/audience and signing/verification key to the counterpart; lifetime is in seconds and secrets/private keys require protected storage. Choices: HS256, HS384, HS512, RS256, RS384, RS512, ES256, ES384 Default: "" Shown when: (config) => ["JWT"].includes(config.authentication)
JWT HMAC secret jwtSecret	password Configuration setting	JWT signing/validation setting. Match algorithm, issuer/audience and signing/verification key to the counterpart; lifetime is in seconds and secrets/private keys require protected storage. Default: "" Shown when: (config) => ["JWT"].includes(config.authentication)
JWT signing private key path jwtPrivateKeyFile	text Configuration setting	JWT signing/validation setting. Match algorithm, issuer/audience and signing/verification key to the counterpart; lifetime is in seconds and secrets/private keys require protected storage. Default: "" Shown when: (config) => ["JWT"].includes(config.authentication)

Field / key	Type / required	Details
JWT verification public key path jwtPublicKeyFile	text Configuration setting	JWT signing/validation setting. Match algorithm, issuer/audience and signing/verification key to the counterpart; lifetime is in seconds and secrets/private keys require protected storage. Default: "" Shown when: (config) => ["JWT"].includes(config.authentication)
JWT verification JWKS URL (HTTPS) jwtJwksUrl	text Configuration setting	Trusted issuer's explicit HTTPS JWKS endpoint for RSA/EC verification. Tokens never select the URL. Configure the expected issuer, audience and asymmetric algorithm. Default: "" Shown when: (config) => config.authentication === "JWT" && config.connectorMode !== "client"
JWKS cache lifetime (seconds) jwtJwksCacheSeconds	text Configuration setting	Lifetime of verified discovery keys before refresh. Default 300 seconds. Expired cache does not bypass verification on discovery failure. Default: "" Shown when: (config) => config.authentication === "JWT" && !!config.jwtJwksUrl
JWKS refresh cooldown (seconds) jwtJwksRefreshSeconds	text Configuration setting	Minimum interval between unknown-key refreshes; limits discovery traffic from attacker-controlled key IDs. Default 10 seconds. Default: "" Shown when: (config) => config.authentication === "JWT" && !!config.jwtJwksUrl
JWKS fetch timeout (seconds) jwtJwksTimeoutSeconds	text Configuration setting	Maximum wait for key discovery. Default 5 seconds. Responses are bounded to 1 MiB and 128 keys. Default: "" Shown when: (config) => config.authentication === "JWT" && !!config.jwtJwksUrl
JWT issuer jwtIssuer	text Configuration setting	JWT signing/validation setting. Match algorithm, issuer/audience and signing/verification key to the counterpart; lifetime is in seconds and secrets/private keys require protected storage. Default: "" Shown when: (config) => ["JWT"].includes(config.authentication)
JWT audience jwtAudience	text Configuration setting	JWT signing/validation setting. Match algorithm, issuer/audience and signing/verification key to the counterpart; lifetime is in seconds and secrets/private keys require protected storage. Default: "" Shown when: (config) => ["JWT"].includes(config.authentication)
Outbound JWT claims JSON jwtClaims	text Configuration setting	Additional JWT claims as an object. Keep issuer/audience/expiry consistent with verification settings and avoid placing secrets in readable token claims. Default: "" Shown when: (config) => ["JWT"].includes(config.authentication)
JWT lifetime (seconds) jwtLifetimeSeconds	text Configuration setting	JWT signing/validation setting. Match algorithm, issuer/audience and signing/verification key to the counterpart; lifetime is in seconds and secrets/private keys require protected storage. Default: 300 Shown when: (config) => ["JWT"].includes(config.authentication)
JWT clock skew (seconds) jwtClockSkewSeconds	text Configuration setting	Allowed time difference in seconds when checking signed request/token timestamps. Use a small tolerance and synchronize runtime clocks. Default: 30 Shown when: (config) => ["JWT"].includes(config.authentication)
Field / key	Type / required	Details
Request HMAC algorithm hmacAlgorithm	select Configuration setting	HMAC request signing/verification setting. Both endpoints must use the same signing algorithm, protected secret and signature

Field / key	Type / required	Details
		header convention. Choices: SHA256, SHA384, SHA512 Default: "" Shown when: (config) => ["HMAC"].includes(config.authentication)
Request HMAC key ID hmacKeyId	text Configuration setting	Identifier selecting the HMAC signing key. It identifies the key without exposing the shared secret. Default: "default" Shown when: (config) => ["HMAC"].includes(config.authentication)
Request HMAC secret hmacSecret	password Configuration setting	HMAC request signing/verification setting. Both endpoints must use the same signing algorithm, protected secret and signature header convention. Default: "" Shown when: (config) => ["HMAC"].includes(config.authentication)
Request HMAC clock skew (seconds) hmacClockSkewSeconds	text Configuration setting	Allowed time difference in seconds when checking signed request/token timestamps. Use a small tolerance and synchronize runtime clocks. Default: 300 Shown when: (config) => ["HMAC"].includes(config.authentication)
LDAP / LDAPS URL ldapUrl	text Configuration setting	LDAP authentication setting. Configure the directory URL, search base/filter or user DN template, bind identity and required group; use TLS/StartTLS and the directory trust certificate where applicable. Default: "" Shown when: (config) => ["LDAP"].includes(config.authentication)
Require LDAP StartTLS ldapStartTls	select Configuration setting	LDAP authentication setting. Configure the directory URL, search base/filter or user DN template, bind identity and required group; use TLS/StartTLS and the directory trust certificate where applicable. Choices: true, false Default: "" Shown when: (config) => ["LDAP"].includes(config.authentication)
LDAP trusted CA PEM path ldapCaFile	text Configuration setting	Trusted CA certificate location on the runtime host. A certificate folder can contain root certificates and optional chain certificates; configure the resolved deployment path. Default: "" Shown when: (config) => ["LDAP"].includes(config.authentication)
LDAP operation timeout (seconds) ldapTimeoutSeconds	text Configuration setting	LDAP operation timeout (seconds): bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 10 Shown when: (config) => ["LDAP"].includes(config.authentication)
LDAP user DN template ({username}) ldapUserDnTemplate	text Configuration setting	LDAP authentication setting. Configure the directory URL, search base/filter or user DN template, bind identity and required group; use TLS/StartTLS and the directory trust certificate where applicable. Default: "" Shown when: (config) => ["LDAP"].includes(config.authentication)
LDAP search bind DN ldapBindDn	text Configuration setting	LDAP authentication setting. Configure the directory URL, search base/filter or user DN template, bind identity and required group; use TLS/StartTLS and the directory trust certificate where applicable. Default: "" Shown when: (config) => ["LDAP"].includes(config.authentication)

Field / key	Type / required	Details
LDAP search bind password ldapBindPassword	password Configuration setting	LDAP authentication setting. Configure the directory URL, search base/filter or user DN template, bind identity and required group; use TLS/StartTLS and the directory trust certificate where applicable. Default: "" Shown when: (config) => ["LDAP"].includes(config.authentication)
LDAP user search base DN ldapBaseDn	text Configuration setting	LDAP authentication setting. Configure the directory URL, search base/filter or user DN template, bind identity and required group; use TLS/StartTLS and the directory trust certificate where applicable. Default: "" Shown when: (config) => ["LDAP"].includes(config.authentication)
LDAP user filter ({username}) ldapSearchFilter	text Configuration setting	LDAP authentication setting. Configure the directory URL, search base/filter or user DN template, bind identity and required group; use TLS/StartTLS and the directory trust certificate where applicable. Default: "(uid={username})" Shown when: (config) => ["LDAP"].includes(config.authentication)
Required LDAP memberOf group DN ldapRequiredGroup	text Configuration setting	LDAP authentication setting. Configure the directory URL, search base/filter or user DN template, bind identity and required group; use TLS/StartTLS and the directory trust certificate where applicable. Default: "" Shown when: (config) => ["LDAP"].includes(config.authentication)

Properties and credentials

Browse an environment property to bind a field; inspect the resolved value in Studio. Provide real credentials locally or through deployment secrets; never put secrets in documentation, source control or AI prompts. A password-style field is a UI indication, not a guarantee that all exports are redacted.

Testing and prerequisites

A connection test checks connectivity, not full application behavior, permissions for every operation, throughput or failover. Java-based connectors require their licensed driver JARs and bridge; SAP also requires a matching platform-native library. See Vendor Drivers and the connector guides.

Authentication and mandatory fields depend on the selected provider/mode; validate in Studio and on the target runtime.

Reference source: frontend/src/main.tsx

FTP shared connection

Shared connections

Configuration

Create a shared resource in the project Resources tree, select it on activities, and use Test connection after resolving the active environment properties. Generated IDs are managed by Studio. Conditional fields below are shown only when their mode applies.

Field / key	Type / required	Details
Host host	text Configuration setting	Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https. Default: "ftp.example.com"
Port port	text Configuration setting	Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https. Default: 21
Username username	text Configuration setting	Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions. Default: ""
Password password	password Configuration setting	Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs. Default: ""
Working directory workingDirectory	text Configuration setting	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. Default: "/"
Passive mode passiveMode	select Configuration setting	Provider-specific connection feature. Enable only when the selected driver/server supports it; check the connector procedure and vendor prerequisites for that mode. Choices: true, false Default: true
Timeout (seconds) timeoutSeconds	text Configuration setting	Timeout (seconds): bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 60

Properties and credentials

Browse an environment property to bind a field; inspect the resolved value in Studio. Provide real credentials locally or through deployment secrets; never put secrets in documentation, source control or AI prompts. A password-style field is a UI indication, not a guarantee that all exports are redacted.

Testing and prerequisites

A connection test checks connectivity, not full application behavior, permissions for every operation, throughput or failover. Java-based connectors require their licensed driver JARs and bridge; SAP also requires a matching platform-native library. See Vendor Drivers and the connector guides.

Authentication and mandatory fields depend on the selected provider/mode; validate in Studio and on the target runtime.

Reference source: frontend/src/main.tsx

SFTP shared connection

Shared connections

Configuration

Create a shared resource in the project Resources tree, select it on activities, and use Test connection after resolving the active environment properties. Generated IDs are managed by Studio. Conditional fields below are shown only when their mode applies.

Field / key	Type / required	Details
Host host	text Configuration setting	Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https. Default: "sftp.example.com"
Port port	text Configuration setting	Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https. Default: 22
Username username	text Configuration setting	Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions. Default: ""
Password password	password Configuration setting	Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs. Default: ""
Working directory workingDirectory	text Configuration setting	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. Default: "/"
Private key file privateKeyFile	text Configuration setting	Client/server certificate or private-key location/selection. Deploy matching certificate and key material and grant read access only to the runtime identity. Default: ""
Private key passphrase privateKeyPassphrase	password Configuration setting	Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs. Default: ""
Known hosts file knownHostsFile	text Configuration setting	SSH host identity verification. Supply a trusted known-hosts file for SFTP; disabling verification permits connecting to an unverified server. Default: ""
Strict host-key checking strictHostKeyChecking	select Configuration setting	SSH host identity verification. Supply a trusted known-hosts file for SFTP; disabling verification permits connecting to an unverified server. Choices: true, false Default: true
Timeout (seconds) timeoutSeconds	text Configuration setting	Timeout (seconds): bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 60

Properties and credentials

Browse an environment property to bind a field; inspect the resolved value in Studio. Provide real credentials locally or through deployment secrets; never put secrets in documentation, source control or AI prompts. A password-style field is a UI indication, not a guarantee that all exports are redacted.

Testing and prerequisites

A connection test checks connectivity, not full application behavior, permissions for every operation, throughput or failover. Java-based connectors require their licensed driver JARs and bridge; SAP also requires a matching platform-native library. See Vendor Drivers and the connector guides.

Authentication and mandatory fields depend on the selected provider/mode; validate in Studio and on the target runtime.

Reference source: frontend/src/main.tsx

EMS shared connection

Shared connections

Configuration

Create a shared resource in the project Resources tree, select it on activities, and use Test connection after resolving the active environment properties. Generated IDs are managed by Studio. Conditional fields below are shown only when their mode applies.

Field / key	Type / required	Details
JMS connection URL serverUrl	text Required	Provider connection address/namespace. Use the provider URI/connection-string syntax; Kafka bootstrapServers accepts broker host:port endpoints and AMQP virtualHost selects the broker namespace. Default: "tcp://localhost:7222" Example / hint: tcp://ems-host:7222
EMS/JMS driver JAR directory driverDirectory	text Configuration setting	Database/JMS driver implementation or driver directory. Install the matching provider library on the runtime host; Java drivers require the bridge and compatible Java runtime. Default: "" Example / hint: Blank uses C:\ProgramData\MINA Studio\drivers\jms
Native connection factory class connectionFactoryClass	text Configuration setting	Database/JMS driver implementation or driver directory. Install the matching provider library on the runtime host; Java drivers require the bridge and compatible Java runtime. Default: "" Example / hint: Fully qualified class name from your driver documentation
Connection factory type connectionFactoryType	select Configuration setting	JMS connection-factory selection. Direct and JNDI modes require different provider configuration; JNDI uses the context factory/provider URL and registered factory name. Choices: Direct, JNDI Default: "Direct"
Messaging style messagingStyle	select Configuration setting	Queue/topic design hint. The current engine primarily chooses destination kind from the operation; do not use this to turn a queue sender into a topic publisher. Choices: Generic, Queue/Topic Default: "Generic"
Username username	text Required	SAP logon user with authorization for the selected RFCs and metadata APIs. Default: ""
Password password	password Required	SAP logon secret. Bind a secret/environment property; do not put credentials into examples, source control or payload logs. Default: ""
Client ID clientId	text Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. Default: "" Example / hint: Generated automatically when blank

Field / key	Type / required	Details
JNDI connection factory connectionFactory	select Required	JMS connection-factory selection. Direct and JNDI modes require different provider configuration; JNDI uses the context factory/provider URL and registered factory name. Choices: ConnectionFactory, QueueConnectionFactory,

Field / key	Type / required	Details
		TopicConnectionFactory Default: "ConnectionFactory" Shown when: (config) => config.connectionFactoryType === "JNDI"
Queue connection factory queueConnectionFactory	text Configuration setting	JMS connection-factory selection. Direct and JNDI modes require different provider configuration; JNDI uses the context factory/provider URL and registered factory name. Default: "QueueConnectionFactory" Shown when: (config) => config.connectionFactoryType === "JNDI" && config.messagingStyle === "Queue/Topic"
Topic connection factory topicConnectionFactory	text Configuration setting	JMS connection-factory selection. Direct and JNDI modes require different provider configuration; JNDI uses the context factory/provider URL and registered factory name. Default: "TopicConnectionFactory" Shown when: (config) => config.connectionFactoryType === "JNDI" && config.messagingStyle === "Queue/Topic"
JNDI context factory jndiContextFactory	text Configuration setting	JMS connection-factory selection. Direct and JNDI modes require different provider configuration; JNDI uses the context factory/provider URL and registered factory name. Default: "" Shown when: (config) => config.connectionFactoryType === "JNDI" Example / hint: Leave blank to use the connector default
JNDI provider URL jndiProviderUrl	text Required	JMS connection-factory selection. Direct and JNDI modes require different provider configuration; JNDI uses the context factory/provider URL and registered factory name. Default: "tcp://localhost:7222" Shown when: (config) => config.connectionFactoryType === "JNDI"
JNDI username jndiUsername	text Required	Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions. Default: "" Shown when: (config) => config.connectionFactoryType === "JNDI"
JNDI password jndiPassword	password Required	Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs. Default: "" Shown when: (config) => config.connectionFactoryType === "JNDI"
Use XA connection factory useXa	select Configuration setting	Provider-specific connection feature. Enable only when the selected driver/server supports it; check the connector procedure and vendor prerequisites for that mode. Choices: false, true Default: false
Use EMS unshared failover useUfo	select Configuration setting	Provider-specific connection feature. Enable only when the selected driver/server supports it; check the connector procedure and vendor prerequisites for that mode. Choices: false, true Default: false
Enable SSL sslEnabled	select Configuration setting	Transport security setting. Enable TLS for confidential transport and verify server identity/certificates; disabling verification or trusting an unverified server weakens connection authentication. Choices: false, true Default: false

Field / key	Type / required	Details
SSL trusted certificates sslTrustedCertificates	text Configuration setting	Trusted CA certificate location on the runtime host. A certificate folder can contain root certificates and optional chain certificates; configure the resolved deployment path. Default: ""
Reconnect attempts reconnectAttempts	text Configuration setting	Retry attempt policy. A retry can repeat an accepted external write; use stable identifiers and destination-side duplicate handling. Default: 3
Reconnect delay (ms) reconnectDelayMs	text Configuration setting	Reconnect delay (ms): bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 5000
Outgoing heartbeat (ms) heartbeatOutgoingMs	text Configuration setting	STOMP outgoing heartbeat interval in milliseconds. Zero disables that direction; the broker and client negotiate the effective heartbeat interval. Default: 0
Incoming heartbeat (ms) heartbeatIncomingMs	text Configuration setting	STOMP incoming heartbeat interval in milliseconds. Zero disables that direction; the broker and client negotiate the effective heartbeat interval. Default: 0
Connection timeout (seconds) connectionTimeoutSeconds	text Configuration setting	Connection timeout (seconds): bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 30

Properties and credentials

Browse an environment property to bind a field; inspect the resolved value in Studio. Provide real credentials locally or through deployment secrets; never put secrets in documentation, source control or AI prompts. A password-style field is a UI indication, not a guarantee that all exports are redacted.

Testing and prerequisites

A connection test checks connectivity, not full application behavior, permissions for every operation, throughput or failover. Java-based connectors require their licensed driver JARs and bridge; SAP also requires a matching platform-native library. See Vendor Drivers and the connector guides.

Direct connections use JMS URL, username and password. JNDI settings become required only when JNDI is enabled. Optional connection IDs are generated by the implementation.

Reference source: frontend/src/main.tsx

JMS shared connection

Shared connections

Configuration

Create a shared resource in the project Resources tree, select it on activities, and use Test connection after resolving the active environment properties. Generated IDs are managed by Studio. Conditional fields below are shown only when their mode applies.

Field / key	Type / required	Details
JMS provider provider	text Configuration setting	Provider/protocol or receive mode selection. Choose the broker implementation and delivery/settlement behavior supported by the configured connection. Default: "Generic JMS 2.0"
JMS connection URL serverUrl	text Required	Provider connection address/namespace. Use the provider URI/connection-string syntax; Kafka bootstrapServers accepts broker host:port endpoints and AMQP virtualHost selects the broker namespace. Default: "tcp://localhost:61613" Example / hint: tcp://jms-host:61613
JMS provider JAR directory driverDirectory	text Configuration setting	Database/JMS driver implementation or driver directory. Install the matching provider library on the runtime host; Java drivers require the bridge and compatible Java runtime. Default: "" Example / hint: Blank uses C:\ProgramData\MINA Studio\drivers\jms
Connection factory class connectionFactoryClass	text Configuration setting	Database/JMS driver implementation or driver directory. Install the matching provider library on the runtime host; Java drivers require the bridge and compatible Java runtime. Default: ""
Connection factory type connectionFactoryType	select Configuration setting	JMS connection-factory selection. Direct and JNDI modes require different provider configuration; JNDI uses the context factory/provider URL and registered factory name. Choices: Direct, JNDI Default: "Direct"
Username username	text Required	SAP logon user with authorization for the selected RFCs and metadata APIs. Default: ""
Password password	password Required	SAP logon secret. Bind a secret/environment property; do not put credentials into examples, source control or payload logs. Default: ""
Client ID clientId	text Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. Default: "" Example / hint: Generated automatically when blank
JNDI connection factory connectionFactory	text Required	JMS connection-factory selection. Direct and JNDI modes require different provider configuration; JNDI uses the context factory/provider URL and registered factory name. Default: "ConnectionFactory" Shown when: (config) => config.connectionFactoryType === "JNDI"

Field / key	Type / required	Details
JNDI initial context factory jndiContextFactory	text Required	JMS connection-factory selection. Direct and JNDI modes require different provider configuration; JNDI uses the context factory/provider URL and registered factory name. Default: "" Shown when: (config) => config.connectionFactoryType === "JNDI"
JNDI provider URL jndiProviderUrl	text Required	JMS connection-factory selection. Direct and JNDI modes require different provider configuration; JNDI uses the context factory/provider URL and registered factory name. Default: "" Shown when: (config) => config.connectionFactoryType === "JNDI"
JNDI username jndiUsername	text Required	Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions. Default: "" Shown when: (config) => config.connectionFactoryType === "JNDI"
JNDI password jndiPassword	password Required	Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs. Default: "" Shown when: (config) => config.connectionFactoryType === "JNDI"
Enable SSL sslEnabled	select Configuration setting	Transport security setting. Enable TLS for confidential transport and verify server identity/certificates; disabling verification or trusting an unverified server weakens connection authentication. Choices: false, true Default: false
Trusted certificates sslTrustedCertificates	text Configuration setting	Trusted CA certificate location on the runtime host. A certificate folder can contain root certificates and optional chain certificates; configure the resolved deployment path. Default: ""
Reconnect attempts reconnectAttempts	text Configuration setting	Retry attempt policy. A retry can repeat an accepted external write; use stable identifiers and destination-side duplicate handling. Default: 3
Reconnect delay (ms) reconnectDelayMs	text Configuration setting	Reconnect delay (ms): bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 5000
Connection timeout (seconds) connectionTimeoutSeconds	text Configuration setting	Connection timeout (seconds): bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 30

Properties and credentials

Browse an environment property to bind a field; inspect the resolved value in Studio. Provide real credentials locally or through deployment secrets; never put secrets in documentation, source control or AI prompts. A password-style field is a UI indication, not a guarantee that all exports are redacted.

Testing and prerequisites

A connection test checks connectivity, not full application behavior, permissions for every operation, throughput or failover. Java-based connectors require their licensed driver JARs and bridge; SAP also requires a matching platform-native library. See Vendor Drivers and the connector guides.

Direct connections use JMS URL, username and password. JNDI settings become required only when JNDI is enabled. Optional connection IDs are generated by the implementation.

Reference source: frontend/src/main.tsx

KAFKA shared connection

Shared connections

Configuration

Create a shared resource in the project Resources tree, select it on activities, and use Test connection after resolving the active environment properties. Generated IDs are managed by Studio. Conditional fields below are shown only when their mode applies.

Field / key	Type / required	Details
Bootstrap servers bootstrapServers	text Required	Provider connection address/namespace. Use the provider URI/connection-string syntax; Kafka bootstrapServers accepts broker host:port endpoints and AMQP virtualHost selects the broker namespace. Default: "localhost:9092"
Client ID clientId	text Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. Default: ""
Default consumer group groupId	text Configuration setting	Consumer group used for assignment and stored offsets. Separate applications requiring independent consumption need separate groups. Default: "mina"
Security protocol securityProtocol	select Configuration setting	Kafka transport/authentication selection. SASL_SSL combines TLS and SASL; configure credentials and CA verification for the selected SASL mechanism. Choices: PLAINTEXT, SASL_PLAINTEXT, SASL_SSL, SSL Default: "PLAINTEXT"
Authentication type authenticationType	select Configuration setting	Authentication mechanism for this connection. Supply the corresponding credentials/token/key settings and verify the receiving endpoint accepts that mechanism. Choices: None, API Key / Secret, PLAIN, SCRAM-SHA-256, SCRAM-SHA-512, GSSAPI, OAUTHBEARER Default: "None"
SASL mechanism saslMechanism	select Configuration setting	Kafka transport/authentication selection. SASL_SSL combines TLS and SASL; configure credentials and CA verification for the selected SASL mechanism. Choices: PLAIN, SCRAM-SHA-256, SCRAM-SHA-512, GSSAPI, OAUTHBEARER Default: "PLAIN" Shown when: (config) => String(config.securityProtocol || "").includes("SASL") && config.authenticationType !== "None"
Username / API key username	text Configuration setting	SAP logon user with authorization for the selected RFCs and metadata APIs. Default: "" Shown when: (config) => ["API Key / Secret", "PLAIN", "SCRAM-SHA-256", "SCRAM-SHA-512"].includes(config.authenticationType) && String(config.securityProtocol || "").includes("SASL")
Password / API secret password	password Configuration setting	SAP logon secret. Bind a secret/environment property; do not put credentials into examples, source control or payload logs. Default: "" Shown when: (config) => ["API Key / Secret", "PLAIN", "SCRAM-SHA-256", "SCRAM-SHA-512"].includes(config.authenticationType) && String(config.securityProtocol || "").includes("SASL")

Field / key	Type / required	Details
SSL CA location sslCaLocation	text Configuration setting	Trusted CA certificate location on the runtime host. A certificate folder can contain root certificates and optional chain certificates; configure the resolved deployment path. Default: "" Shown when: (config) => ["SSL", "SASL_SSL"].includes(config.securityProtocol)
SSL certificate location sslCertificateLocation	text Configuration setting	Client/server certificate or private-key location/selection. Deploy matching certificate and key material and grant read access only to the runtime identity. Default: "" Shown when: (config) => ["SSL", "SASL_SSL"].includes(config.securityProtocol)
SSL key location sslKeyLocation	text Configuration setting	Client/server certificate or private-key location/selection. Deploy matching certificate and key material and grant read access only to the runtime identity. Default: "" Shown when: (config) => ["SSL", "SASL_SSL"].includes(config.securityProtocol)
SSL key password sslKeyPassword	password Configuration setting	Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs. Default: "" Shown when: (config) => ["SSL", "SASL_SSL"].includes(config.securityProtocol)
Schema Registry URL schemaRegistryUrl	text Configuration setting	Registry endpoint metadata. This engine does not implement registry-backed Avro conversion merely by setting the URL. Default: ""
Schema Registry username schemaRegistryUsername	text Configuration setting	Kafka schema-registry configuration. Select the registry endpoint/vendor and authentication; the Avro runtime limitations are documented on the Kafka operation pages. Default: ""
Schema Registry password schemaRegistryPassword	password Configuration setting	Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs. Default: ""
Schema Registry vendor schemaRegistryVendor	select Configuration setting	Kafka schema-registry configuration. Select the registry endpoint/vendor and authentication; the Avro runtime limitations are documented on the Kafka operation pages. Choices: Confluent, Other, Apicurio Default: "Confluent"
Use Kerberos ticket cache useTicketCache	select Configuration setting	Provider credential/identity material. Use the mechanism supported by the selected connection and runtime; keep private keys, access keys and credential files outside shared source packages. Choices: false, true Default: false Shown when: (config) => config.authenticationType === "GSSAPI"
Kerberos keytab file keytabFile	text Configuration setting	Provider credential/identity material. Use the mechanism supported by the selected connection and runtime; keep private keys, access keys and credential files outside shared source packages. Default: "" Shown when: (config) => config.authenticationType === "GSSAPI"

Field / key	Type / required	Details
Kerberos principal principalName	text Configuration setting	Provider credential/identity material. Use the mechanism supported by the selected connection and runtime; keep private keys, access keys and credential files outside shared source packages. Default: "" Shown when: (config) => config.authenticationType === "GSSAPI"
OAuth JAAS config file jaasConfigFile	text Configuration setting	JAAS login configuration file on the runtime host for the selected authentication mechanism. Protect credentials contained in the file. Default: "" Shown when: (config) => config.authenticationType === "OAUTHBEARER"
OAuth login callback handler loginCallbackHandler	text Configuration setting	Provider credential/identity material. Use the mechanism supported by the selected connection and runtime; keep private keys, access keys and credential files outside shared source packages. Default: "" Shown when: (config) => config.authenticationType === "OAUTHBEARER"
Request timeout (ms) requestTimeoutMilliseconds	text Configuration setting	Request timeout (ms): bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 30000
Connection timeout (ms) connectionTimeoutMilliseconds	text Configuration setting	Connection timeout (ms): bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 10000
Reconnect backoff (ms) reconnectBackoffMilliseconds	text Configuration setting	Reconnect backoff (ms): bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 50
Retry backoff (ms) retryBackoffMilliseconds	text Configuration setting	Retry backoff (ms): bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 100
Advanced client properties (JSON) clientProperties	text Configuration setting	Additional provider settings as name/value entries. Use exact supported provider option names; invalid options may be rejected at connection or execution time. Default: "{}"

Properties and credentials

Browse an environment property to bind a field; inspect the resolved value in Studio. Provide real credentials locally or through deployment secrets; never put secrets in documentation, source control or AI prompts. A password-style field is a UI indication, not a guarantee that all exports are redacted.

Testing and prerequisites

A connection test checks connectivity, not full application behavior, permissions for every operation, throughput or failover. Java-based connectors require their licensed driver JARs and bridge; SAP also requires a matching platform-native library. See Vendor Drivers and the connector guides.

Authentication and mandatory fields depend on the selected provider/mode; validate in Studio and on the target runtime.

Reference source: frontend/src/main.tsx

PUBSUB shared connection

Shared connections

Configuration

Create a shared resource in the project Resources tree, select it on activities, and use Test connection after resolving the active environment properties. Generated IDs are managed by Studio. Conditional fields below are shown only when their mode applies.

Field / key	Type / required	Details
Authentication authenticationType	select Required	Authentication mechanism for this connection. Supply the corresponding credentials/token/key settings and verify the receiving endpoint accepts that mechanism. Choices: Service Account JSON, Application Default Credentials, Emulator Default: "Service Account JSON"
Service account JSON (inline) serviceAccountJson	text Configuration setting	Provider credential/identity material. Use the mechanism supported by the selected connection and runtime; keep private keys, access keys and credential files outside shared source packages. Default: "" Shown when: (config) => config.authenticationType === "Service Account JSON" && !String(config.credentialsFile || "").trim()
Service account JSON file credentialsFile	text Configuration setting	Provider credential/identity material. Use the mechanism supported by the selected connection and runtime; keep private keys, access keys and credential files outside shared source packages. Default: "" Shown when: (config) => config.authenticationType === "Service Account JSON" && !String(config.serviceAccountJson || "").trim() Example / hint: Environment property resolving to the complete .json file path
GCP project ID projectId	text Configuration setting	Google Cloud project containing the Pub/Sub topic or subscription. Supply the project ID, not its display name. Default: "" Shown when: (config) => config.authenticationType !== "Service Account JSON" Example / hint: Derived automatically from service-account JSON
Service endpoint endpoint	text Configuration setting	Endpoint or filesystem path for this operation. For HTTP use the service URI/path; for files use a path accessible to the runtime host, not only the designer machine. Default: "pubsub.googleapis.com:443" Shown when: (config) => config.authenticationType === " Application Default Credentials"
Corporate CA certificate file (optional) caCertificateFile	text Configuration setting	Trusted CA certificate location on the runtime host. A certificate folder can contain root certificates and optional chain certificates; configure the resolved deployment path. Default: "" Shown when: (config) => config.authenticationType !== "Emulator" Example / hint: Only required when TLS inspection uses an internal CA
Emulator host emulatorHost	text Configuration setting	Local Pub/Sub emulator address for testing. Leave empty for Google Cloud provider access and keep emulator configuration out of production environments. Default: "" Shown when: (config) => config.authenticationType === "Emulator" Example / hint: localhost:8085

Field / key	Type / required	Details
Ack deadline (seconds) ackDeadlineSeconds	text Configuration setting	Provider lifetime/wait setting in seconds, as named by this field. Set it above normal work/latency while retaining a bounded recovery time. Default: 60
Connection timeout (seconds) connectionTimeoutSeconds	text Configuration setting	Connection timeout (seconds): bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 30
Maximum inbound message bytes maxInboundMessageBytes	text Configuration setting	Size/capacity bound for the named request, response, header or event queue. Byte limits measure encoded content; queue limits bound buffered work. Default: 20971520
Keep-alive time (seconds) keepAliveSeconds	text Configuration setting	Connection lifecycle policy. Pooling/persistent connections reduce connection setup; recovery and keepalive apply to provider connections, not transaction-level replay. Default: 60
Publisher batch maximum messages batchMaxMessages	text Configuration setting	Pub/Sub batching threshold: message count, total bytes or waiting time in milliseconds, according to the field name. Tune with payload sizes and acceptable delivery latency. Default: 100
Publisher batch maximum bytes batchMaxBytes	text Configuration setting	Pub/Sub batching threshold: message count, total bytes or waiting time in milliseconds, according to the field name. Tune with payload sizes and acceptable delivery latency. Default: 1048576
Publisher batch delay threshold (ms) batchDelayThresholdMilliseconds	text Configuration setting	Pub/Sub batching threshold: message count, total bytes or waiting time in milliseconds, according to the field name. Tune with payload sizes and acceptable delivery latency. Default: 10
Publisher flow-control message limit flowControlMaxMessages	text Configuration setting	Bound on received/buffered messages or bytes, according to this field. Keep the bound within available memory and downstream processing capacity. Default: 1000
Publisher flow-control byte limit flowControlMaxBytes	text Configuration setting	Bound on received/buffered messages or bytes, according to this field. Keep the bound within available memory and downstream processing capacity. Default: 10485760

Properties and credentials

Browse an environment property to bind a field; inspect the resolved value in Studio. Provide real credentials locally or through deployment secrets; never put secrets in documentation, source control or AI prompts. A password-style field is a UI indication, not a guarantee that all exports are redacted.

Testing and prerequisites

A connection test checks connectivity, not full application behavior, permissions for every operation, throughput or failover. Java-based connectors require their licensed driver JARs and bridge; SAP also requires a matching platform-native library. See Vendor Drivers and the connector guides.

Service Account JSON supplies Google authentication. Verify project/topic/subscription access and IAM permissions on the target environment.

Reference source: frontend/src/main.tsx

SAP shared connection

Shared connections

Configuration

Create a shared resource in the project Resources tree, select it on activities, and use Test connection after resolving the active environment properties. Generated IDs are managed by Studio. Conditional fields below are shown only when their mode applies.

Field / key	Type / required	Details
Runtime adapter mode	select Configuration setting	Input/execution mode selected from the available choices. Match inline text, file or structured input to the actual source; connection modes such as mock/memory are for local testing. Choices: mock, external Default: "external"
SAP JCo driver directory driverDirectory	text Configuration setting	Database/JMS driver implementation or driver directory. Install the matching provider library on the runtime host; Java drivers require the bridge and compatible Java runtime. Default: "" Example / hint: Blank uses C:\ProgramData\MINA Studio\drivers\sap
JCo destination name destinationName	text Configuration setting	Connection routing mode or named provider destination. Select the mode that matches the server configuration and supply its corresponding connection fields. Default: "" Example / hint: mina-sap
SAP release release	select Configuration setting	SAP release used to resolve IDoc metadata. Use the release supported by the connected system, not an arbitrary version string. Choices: current, 720, 730 Default: "current"
Connection type connectionType	select Configuration setting	Connection routing mode: direct application server, message-server logon group, SNC variants or WebSocket configuration. Supply the corresponding shared-connection fields. Choices: dedicated, logongroup, snc, sncwithlogongroup, websocket Default: "dedicated"
Application server host applicationServerHost	text Configuration setting	Application-server hostname for direct connection mode; must resolve from the runtime host. Default: "sap-ecc.example.com"
System number systemNumber	text Configuration setting	SAP instance/system number for a direct application-server connection, normally a two-character string. Default: "00"
Client number client	text Configuration setting	SAP client identifier. Keep leading zeros; provide it as a string. Default: "100"
Language language	text Configuration setting	SAP logon language key. Default: "EN"
Username username	text Configuration setting	SAP logon user with authorization for the selected RFCs and metadata APIs. Default: ""
Password password	password Configuration setting	SAP logon secret. Bind a secret/environment property; do not put credentials into examples, source control or payload logs. Default: ""

Field / key	Type / required	Details
Message server host messageServerHost	text Configuration setting	Message-server hostname for logon-group mode. Default: ""
System ID systemId	text Configuration setting	SAP system ID used with message-server/logon-group routing. Default: ""
Logon group logonGroup	text Configuration setting	SAP logon-group name configured by the SAP administrator. Default: "PUBLIC"
SAP Router sapRouter	text Configuration setting	Optional SAProuter routing string supplied by the network/SAP administrator. Default: ""
SNC partner name sncPartnerName	text Configuration setting	SNC partner identity expected by the target SAP system. Default: "" Shown when: (config) => ["snc", "sncwithlogongroup"].includes(config.connectionType)
SNC library path sncLibraryPath	text Configuration setting	Native SNC library path on the runtime host, matching OS and architecture. Default: "" Shown when: (config) => ["snc", "sncwithlogongroup"].includes(config.connectionType)
SNC own name sncMyName	text Configuration setting	Optional client SNC identity. Default: "" Shown when: (config) => ["snc", "sncwithlogongroup"].includes(config.connectionType)
SNC quality of protection sncQop	select Configuration setting	SNC quality-of-protection setting accepted by the target system. Choices: , 1, 2, 3, 8, 9 Default: "" Shown when: (config) => ["snc", "sncwithlogongroup"].includes(config.connectionType)
Program ID (inbound) programId	text Configuration setting	Registered-server program ID. Must match the SAP RFC destination and gateway registration permissions. Default: ""
Gateway host gatewayHost	text Configuration setting	SAP gateway hostname override for inbound RFC registration. Default: ""
Gateway service gatewayService	text Configuration setting	Gateway service/port override for inbound RFC registration. Default: ""
Maximum connections maximumConnections	text Configuration setting	Listener connection/concurrency limit. Size it with SAP gateway capacity and downstream processing, not just CPU count. Default: 8
Persistent outbound JVM workers outboundWorkerCount	text Configuration setting	Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity. Default: 2
JCo destination pool capacity poolCapacity	text Configuration setting	Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity. Default: 1
JCo destination peak limit peakLimit	text Configuration setting	Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity. Default: 1

Field / key	Type / required	Details
Idle connection expiration (ms) connectionExpirationMilliseconds	text Configuration setting	Lifetime of an idle SAP pooled connection in milliseconds before it is eligible for expiration. Match this to the SAP server connection policy. Default: 600000
Outbound pool idle timeout (seconds) outboundPoolIdleSeconds	text Configuration setting	Provider lifetime/wait setting in seconds, as named by this field. Set it above normal work/latency while retaining a bounded recovery time. Default: 1800
Maximum distinct outbound pools maximumOutboundPools	text Configuration setting	Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity. Default: 16
Maximum buffered SAP requests maxPendingEvents	text Configuration setting	Bound on buffered listener events; prevents unbounded in-memory accumulation while downstream work is slow. Default: 16
JCo bridge initial heap (MB) jvmInitialHeapMb	text Configuration setting	Java bridge heap size in megabytes. Size it for concurrent payloads and driver overhead; keep the initial heap at or below the maximum. Default: 64
JCo bridge maximum heap (MB) jvmMaximumHeapMb	text Configuration setting	Java bridge heap size in megabytes. Size it for concurrent payloads and driver overhead; keep the initial heap at or below the maximum. Default: 512
Startup timeout (ms) timeoutMilliseconds	text Configuration setting	Startup timeout (ms): bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 30000
Inbound processing/reply timeout (seconds) ackTimeoutSeconds	text Configuration setting	Maximum listener reply/acknowledgement wait in seconds. Set it above expected flow processing time and test timeout/redelivery behavior. Default: 300
Maximum dynamic sessions maximumDynamicSessions	text Configuration setting	Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity. Default: 1024

Properties and credentials

Browse an environment property to bind a field; inspect the resolved value in Studio. Provide real credentials locally or through deployment secrets; never put secrets in documentation, source control or AI prompts. A password-style field is a UI indication, not a guarantee that all exports are redacted.

Testing and prerequisites

A connection test checks connectivity, not full application behavior, permissions for every operation, throughput or failover. Java-based connectors require their licensed driver JARs and bridge; SAP also requires a matching platform-native library. See Vendor Drivers and the connector guides.

Authentication and mandatory fields depend on the selected provider/mode; validate in Studio and on the target runtime.

Reference source: frontend/src/main.tsx

SAP_TID shared connection

Shared connections

Configuration

Create a shared resource in the project Resources tree, select it on activities, and use Test connection after resolving the active environment properties. Generated IDs are managed by Studio. Conditional fields below are shown only when their mode applies.

Field / key	Type / required	Details
TID management mode	select Configuration setting	Input/execution mode selected from the available choices. Match inline text, file or structured input to the actual source; connection modes such as mock/memory are for local testing. Choices: active, disabled Default: "active"
Transaction ID storage file storageFile	text Configuration setting	Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing. Default: "data/sap-tids.properties" Example / hint: Durable path shared by the deployed runtime

Properties and credentials

Browse an environment property to bind a field; inspect the resolved value in Studio. Provide real credentials locally or through deployment secrets; never put secrets in documentation, source control or AI prompts. A password-style field is a UI indication, not a guarantee that all exports are redacted.

Testing and prerequisites

A connection test checks connectivity, not full application behavior, permissions for every operation, throughput or failover. Java-based connectors require their licensed driver JARs and bridge; SAP also requires a matching platform-native library. See Vendor Drivers and the connector guides.

Authentication and mandatory fields depend on the selected provider/mode; validate in Studio and on the target runtime.

Reference source: frontend/src/main.tsx

Environment property defaults

Shared connections

Defaults

New projects start with core runtime properties. Adding a shared connection or a protocol activity automatically creates the missing required and optional protocol properties in each environment. Existing values are preserved. Use the property picker to bind connection fields to those properties.

The reference below lists the Studio templates. New connection settings supply non-secret values for new properties in the active environment. Other environments receive template defaults. Set password values separately for each environment.

Field / key	Type / required	Details
runtime.logDirectory runtime.logDirectory	string Configuration setting	Directory on the runtime host for execution logs. Ensure write permissions and configure retention to bound disk usage. Default: ""
advanced.logPayload advanced.logPayload	boolean Configuration setting	Environment-controlled payload logging. Disable for confidential or large messages; log business identifiers and execution timings instead. Default: false
advanced.retryEnabled advanced.retryEnabled	boolean Configuration setting	Enable outbound activity retries in this environment. Enable only when repeated external operations are safe or deduplicated. Default: false
advanced.retryCount advanced.retryCount	integer Configuration setting	Retry attempt policy. A retry can repeat an accepted external write; use stable identifiers and destination-side duplicate handling. Default: 3
advanced.retryIntervalSeconds advanced.retryIntervalSeconds	integer Configuration setting	advanced.retryIntervalSeconds: bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 60
advanced.groupMaxIterations advanced.groupMaxIterations	integer Configuration setting	Maximum iterations allowed for a repeating group. Bounds accidental endless loops; choose a limit appropriate for the expected collection size. Default: 10000
connections.jdbc.driver connections.jdbc.driver	string Configuration setting	Database/JMS driver implementation or driver directory. Install the matching provider library on the runtime host; Java drivers require the bridge and compatible Java runtime. Default: "postgresql"
connections.jdbc.url connections.jdbc.url	string Configuration setting	Endpoint or filesystem path for this operation. For HTTP use the service URI/path; for files use a path accessible to the runtime host, not only the designer machine. Default: "jdbc:postgresql://localhost:5432/integration"
connections.jdbc.host connections.jdbc.host	string Configuration setting	Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https. Default: "localhost"
connections.jdbc.port connections.jdbc.port	integer Configuration setting	Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https. Default: 5432

Field / key	Type / required	Details
connections.jdbc.database connections.jdbc.database	string Configuration setting	Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions. Default: "integration"
connections.jdbc.schema connections.jdbc.schema	string Configuration setting	Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions. Default: "public"
connections.jdbc.username connections.jdbc.username	string Configuration setting	Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions. Default: ""
connections.jdbc.password connections.jdbc.password	password Configuration setting	Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs. Default: "[supply securely]"
connections.jdbc.connectionMode connections.jdbc.connectionMode	string Configuration setting	Connection lifecycle policy. Pooling/persistent connections reduce connection setup; recovery and keepalive apply to provider connections, not transaction-level replay. Default: "python"
connections.jdbc.driverDirectory connections.jdbc.driverDirectory	string Configuration setting	Database/JMS driver implementation or driver directory. Install the matching provider library on the runtime host; Java drivers require the bridge and compatible Java runtime. Default: ""
connections.jdbc.driverClass connections.jdbc.driverClass	string Configuration setting	Database/JMS driver implementation or driver directory. Install the matching provider library on the runtime host; Java drivers require the bridge and compatible Java runtime. Default: ""
connections.jdbc.odbcDriver connections.jdbc.odbcDriver	string Configuration setting	Database/JMS driver implementation or driver directory. Install the matching provider library on the runtime host; Java drivers require the bridge and compatible Java runtime. Default: ""
connections.jdbc.authentication connections.jdbc.authentication	string Configuration setting	Authentication mechanism for this connection. Supply the corresponding credentials/token/key settings and verify the receiving endpoint accepts that mechanism. Default: "SQL Server Authentication"
connections.jdbc.encrypt connections.jdbc.encrypt	boolean Configuration setting	Transport security setting. Enable TLS for confidential transport and verify server identity/certificates; disabling verification or trusting an unverified server weakens connection authentication. Default: true
connections.jdbc.trustServerCertificate connections.jdbc.trustServerCertificate	boolean Configuration setting	Transport security setting. Enable TLS for confidential transport and verify server identity/certificates; disabling verification or trusting an unverified server weakens connection authentication. Default: false
connections.jdbc.serverHostname connections.jdbc.serverHostname	string Configuration setting	Provider endpoint override. Supply the reachable hostname/port or HTTP path expected by the chosen transport and the configured certificate identity. Default: ""
connections.jdbc.httpPath connections.jdbc.httpPath	string Configuration setting	Provider endpoint override. Supply the reachable hostname/port or HTTP path expected by the chosen transport and the configured certificate identity. Default: ""

Field / key	Type / required	Details
connections.jdbc.accessToken connections.jdbc.accessToken	password Configuration setting	Token or identity-provider endpoint for the selected authentication flow. Tokens expire; use the supported acquisition/refresh settings where required. Default: "[supply securely]"
connections.jdbc.clientId connections.jdbc.clientId	string Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. Default: ""
connections.jdbc.clientSecret connections.jdbc.clientSecret	password Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. Default: "[supply securely]"
connections.jdbc.catalog connections.jdbc.catalog	string Configuration setting	Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions. Default: ""
connections.jdbc.useCloudFetch connections.jdbc.useCloudFetch	boolean Configuration setting	Provider-specific connection feature. Enable only when the selected driver/server supports it; check the connector procedure and vendor prerequisites for that mode. Default: true
connections.jdbc.serviceName connections.jdbc.serviceName	string Configuration setting	SOAP contract/operation settings. Select the service and operation from the WSDL; soapAction identifies the operation and envelope contains the SOAP XML request. Default: ""
connections.jdbc.sid connections.jdbc.sid	string Configuration setting	Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions. Default: ""
connections.jdbc.sslCaFile connections.jdbc.sslCaFile	string Configuration setting	Trusted CA certificate location on the runtime host. A certificate folder can contain root certificates and optional chain certificates; configure the resolved deployment path. Default: ""
connections.jdbc.timeoutSeconds connections.jdbc.timeoutSeconds	integer Configuration setting	connections.jdbc.timeoutSeconds: bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 30
connections.jdbc.minimumPoolSize connections.jdbc.minimumPoolSize	integer Configuration setting	Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity. Default: 1
connections.jdbc.maximumPoolSize connections.jdbc.maximumPoolSize	integer Configuration setting	Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity. Default: 10
connections.jdbc.postgresSslMode connections.jdbc.postgresSslMode	string Configuration setting	PostgreSQL libpq TLS mode. Leave blank to use the connection URL or libpq default. Verify-full verifies the certificate chain and hostname. Default: ""

Field / key	Type / required	Details
connections.jdbc.postgresSslRootCert connections.jdbc.postgresSslRootCert	string Configuration setting	Path to the PostgreSQL root certificate on the runtime host for TLS verification. Default: ""
connections.snowflake.mode connections.snowflake.mode	string Configuration setting	Input/execution mode selected from the available choices. Match inline text, file or structured input to the actual source; connection modes such as mock/memory are for local testing. Default: "external"
connections.snowflake.authenticationType connections.snowflake.authenticationType	string Configuration setting	Authentication mechanism for this connection. Supply the corresponding credentials/token/key settings and verify the receiving endpoint accepts that mechanism. Default: "Username/Password"
connections.snowflake.provider connections.snowflake.provider	string Configuration setting	Provider/protocol or receive mode selection. Choose the broker implementation and delivery/settlement behavior supported by the configured connection. Default: "Snowflake"
connections.snowflake.account connections.snowflake.account	string Configuration setting	Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions. Default: ""
connections.snowflake.username connections.snowflake.username	string Configuration setting	Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions. Default: ""
connections.snowflake.password connections.snowflake.password	password Configuration setting	Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs. Default: "[supply securely]"
connections.snowflake.warehouse connections.snowflake.warehouse	string Configuration setting	Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions. Default: ""
connections.snowflake.database connections.snowflake.database	string Configuration setting	Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions. Default: ""
connections.snowflake.schema connections.snowflake.schema	string Configuration setting	Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions. Default: "PUBLIC"
connections.snowflake.role connections.snowflake.role	string Configuration setting	Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions. Default: ""
connections.snowflake.loginTimeoutSeconds connections.snowflake.loginTimeoutSeconds	integer Configuration setting	connections.snowflake.loginTimeoutSeconds: bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 60
connections.snowflake.minimumConnections connections.snowflake.minimumConnections	integer Configuration setting	Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity. Default: 2

Field / key	Type / required	Details
connections.snowflake.maximumConnections connections.snowflake.maximumConnections	integer Configuration setting	Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity. Default: 8
connections.snowflake.maximumConnectionWaitSeconds connections.snowflake.maximumConnectionWaitSeconds	integer Configuration setting	Provider lifetime/wait setting in seconds, as named by this field. Set it above normal work/latency while retaining a bounded recovery time. Default: 300
connections.snowflake.serviceThreads connections.snowflake.serviceThreads	integer Configuration setting	Provider/listener concurrency setting. Coordinate it with downstream capacity and the operation-specific runtime support; it is not a whole-workflow parallel-branch limit. Default: 8
connections.snowflake.oktaTokenEndpoint connections.snowflake.oktaTokenEndpoint	string Configuration setting	Token or identity-provider endpoint for the selected authentication flow. Tokens expire; use the supported acquisition/refresh settings where required. Default: ""
connections.snowflake.oktaUsername connections.snowflake.oktaUsername	string Configuration setting	Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions. Default: ""
connections.snowflake.oktaPassword connections.snowflake.oktaPassword	password Configuration setting	Identity-provider password. Bind protected deployment configuration rather than embedding it in the project or logs. Default: "[supply securely]"
connections.snowflake.oktaEndpointUrl connections.snowflake.oktaEndpointUrl	string Configuration setting	Token or identity-provider endpoint for the selected authentication flow. Tokens expire; use the supported acquisition/refresh settings where required. Default: ""
connections.snowflake.clientId connections.snowflake.clientId	string Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. Default: ""
connections.snowflake.clientSecret connections.snowflake.clientSecret	password Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. Default: "[supply securely]"
connections.snowflake.scope connections.snowflake.scope	string Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. Default: ""
connections.snowflake.authorizationCode connections.snowflake.authorizationCode	password Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. Default: "[supply securely]"
connections.snowflake.redirectUri connections.snowflake.redirectUri	string Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. Default: ""

Field / key	Type / required	Details
connections.snowflake.accessToken connections.snowflake.accessToken	password Configuration setting	Token or identity-provider endpoint for the selected authentication flow. Tokens expire; use the supported acquisition/refresh settings where required. Default: "[supply securely]"
connections.snowflake.privateKeyFile connections.snowflake.privateKeyFile	string Configuration setting	Client/server certificate or private-key location/selection. Deploy matching certificate and key material and grant read access only to the runtime identity. Default: ""
connections.snowflake.privateKeyPassphrase connections.snowflake.privateKeyPassphrase	password Configuration setting	Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs. Default: "[supply securely]"
connections.snowflake.otherProperties connections.snowflake.otherProperties	string Configuration setting	Additional provider settings as name/value entries. Use exact supported provider option names; invalid options may be rejected at connection or execution time. Default: ""
connections.amqp.brokerType connections.amqp.brokerType	string Configuration setting	Provider/protocol or receive mode selection. Choose the broker implementation and delivery/settlement behavior supported by the configured connection. Default: "RabbitMQ"
connections.amqp.amqpVersion connections.amqp.amqpVersion	string Configuration setting	Provider/protocol or receive mode selection. Choose the broker implementation and delivery/settlement behavior supported by the configured connection. Default: "AMQP-0-9-1"
connections.amqp.hostPort connections.amqp.hostPort	string Configuration setting	Provider endpoint override. Supply the reachable hostname/port or HTTP path expected by the chosen transport and the configured certificate identity. Default: "localhost:5672"
connections.amqp.virtualHost connections.amqp.virtualHost	string Configuration setting	Provider connection address/namespace. Use the provider URI/connection-string syntax; Kafka bootstrapServers accepts broker host:port endpoints and AMQP virtualHost selects the broker namespace. Default: "/"
connections.amqp.username connections.amqp.username	string Configuration setting	Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions. Default: "guest"
connections.amqp.password connections.amqp.password	password Configuration setting	Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs. Default: "[supply securely]"
connections.amqp.clientId connections.amqp.clientId	string Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. Default: ""
connections.amqp.authenticationType connections.amqp.authenticationType	string Configuration setting	Authentication mechanism for this connection. Supply the corresponding credentials/token/key settings and verify the receiving endpoint accepts that mechanism. Default: "SAS"

Field / key	Type / required	Details
connections.amqp.connectionString connections.amqp.connectionString	password Configuration setting	Provider connection address/namespace. Use the provider URI/connection-string syntax; Kafka bootstrapServers accepts

Field / key	Type / required	Details
		broker host:port endpoints and AMQP virtualHost selects the broker namespace. Default: "[supply securely]"
connections.amqp.tenantId connections.amqp.tenantId	string Configuration setting	Provider credential/identity material. Use the mechanism supported by the selected connection and runtime; keep private keys, access keys and credential files outside shared source packages. Default: ""
connections.amqp.azureClientId connections.amqp.azureClientId	string Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. Default: ""
connections.amqp.clientSecret connections.amqp.clientSecret	password Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. Default: "[supply securely]"
connections.amqp.sharedAccessKeyName connections.amqp.sharedAccessKeyName	string Configuration setting	Provider credential/identity material. Use the mechanism supported by the selected connection and runtime; keep private keys, access keys and credential files outside shared source packages. Default: ""
connections.amqp.sharedAccessKey connections.amqp.sharedAccessKey	password Configuration setting	Provider credential/identity material. Use the mechanism supported by the selected connection and runtime; keep private keys, access keys and credential files outside shared source packages. Default: "[supply securely]"
connections.amqp.entityType connections.amqp.entityType	string Configuration setting	AMQP routing configuration. entityType selects queue/topic where supported; exchanges route messages using the routing key and exchange type. Verify bindings before publishing. Default: "Queue"
connections.amqp.entityName connections.amqp.entityName	string Configuration setting	Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access. Default: ""
connections.amqp.entitySubscriberName connections.amqp.entitySubscriberName	string Configuration setting	Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access. Default: ""
connections.amqp.connectionTimeoutMsec connections.amqp.connectionTimeoutMsec	integer Configuration setting	connections.amqp.connectionTimeoutMsec: bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 30000
connections.amqp.sessionCount connections.amqp.sessionCount	integer Configuration setting	Provider/listener concurrency setting. Coordinate it with downstream capacity and the operation-specific runtime support; it is not a whole-workflow parallel-branch limit. Default: 1
connections.amqp.idleTimeoutMsec connections.amqp.idleTimeoutMsec	integer Configuration setting	connections.amqp.idleTimeoutMsec: bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 0

Field / key	Type / required	Details
connections.amqp.connectionRecovery connections.amqp.connectionRecovery	boolean Configuration setting	Connection lifecycle policy. Pooling/persistent connections reduce connection setup; recovery and keepalive apply to provider connections, not transaction-level replay. Default: true
connections.amqp.retryIntervalMsec connections.amqp.retryIntervalMsec	integer Configuration setting	connections.amqp.retryIntervalMsec: bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 3000
connections.amqp.retryAttempts connections.amqp.retryAttempts	integer Configuration setting	Retry attempt policy. A retry can repeat an accepted external write; use stable identifiers and destination-side duplicate handling. Default: 20
connections.amqp.networkRecoveryIntervalMsec connections.amqp.networkRecoveryIntervalMsec	integer Configuration setting	connections.amqp.networkRecoveryIntervalMsec: bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 5000
connections.amqp.sslEnabled connections.amqp.sslEnabled	boolean Configuration setting	Transport security setting. Enable TLS for confidential transport and verify server identity/certificates; disabling verification or trusting an unverified server weakens connection authentication. Default: false
connections.amqp.caFile connections.amqp.caFile	string Configuration setting	Trusted CA certificate location on the runtime host. A certificate folder can contain root certificates and optional chain certificates; configure the resolved deployment path. Default: ""
connections.amqp.clientCertificateFile connections.amqp.clientCertificateFile	string Configuration setting	Client/server certificate or private-key location/selection. Deploy matching certificate and key material and grant read access only to the runtime identity. Default: ""
connections.amqp.clientKeyFile connections.amqp.clientKeyFile	string Configuration setting	Client/server certificate or private-key location/selection. Deploy matching certificate and key material and grant read access only to the runtime identity. Default: ""
connections.amqp.clientKeyPassword connections.amqp.clientKeyPassword	password Configuration setting	Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs. Default: "[supply securely]"
connections.http.baseUrl connections.http.baseUrl	string Configuration setting	Endpoint or filesystem path for this operation. For HTTP use the service URI/path; for files use a path accessible to the runtime host, not only the designer machine. Default: "https://api.example.com"
connections.http.host connections.http.host	string Configuration setting	Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https. Default: "localhost"
connections.http.port connections.http.port	integer Configuration setting	Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https. Default: 8080
connections.http.basePath connections.http.basePath	string Configuration setting	Endpoint or filesystem path for this operation. For HTTP use the service URI/path; for files use a path accessible to the runtime host, not only the designer machine. Default: ""

Field / key	Type / required	Details
connections.http.scheme connections.http.scheme	string Configuration setting	Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https. Default: "http"
connections.http.connectorMode connections.http.connectorMode	string Configuration setting	Connection lifecycle policy. Pooling/persistent connections reduce connection setup; recovery and keepalive apply to provider connections, not transaction-level replay. Default: "both"
connections.http.authentication connections.http.authentication	string Configuration setting	Authentication mechanism for this connection. Supply the corresponding credentials/token/key settings and verify the receiving endpoint accepts that mechanism. Default: "None"
connections.http.bearerToken connections.http.bearerToken	password Configuration setting	Token or identity-provider endpoint for the selected authentication flow. Tokens expire; use the supported acquisition/refresh settings where required. Default: "[supply securely]"
connections.http.tlsEnabled connections.http.tlsEnabled	boolean Configuration setting	Transport security setting. Enable TLS for confidential transport and verify server identity/certificates; disabling verification or trusting an unverified server weakens connection authentication. Default: false
connections.http.defaultConfidentiality connections.http.defaultConfidentiality	boolean Configuration setting	Transport security setting. Enable TLS for confidential transport and verify server identity/certificates; disabling verification or trusting an unverified server weakens connection authentication. Default: false
connections.http.certificateFile connections.http.certificateFile	string Configuration setting	Client/server certificate or private-key location/selection. Deploy matching certificate and key material and grant read access only to the runtime identity. Default: ""
connections.http.privateKeyFile connections.http.privateKeyFile	string Configuration setting	Client/server certificate or private-key location/selection. Deploy matching certificate and key material and grant read access only to the runtime identity. Default: ""
connections.http.privateKeyPassword connections.http.privateKeyPassword	password Configuration setting	Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs. Default: "[supply securely]"
connections.http.trustedCertificateFolder connections.http.trustedCertificateFolder	string Configuration setting	Trusted CA certificate location on the runtime host. A certificate folder can contain root certificates and optional chain certificates; configure the resolved deployment path. Default: ""
connections.http.certificateAuthorityFile connections.http.certificateAuthorityFile	string Configuration setting	Environment property connections.http.certificateAuthorityFile. Bind it with ${properties.connections.http.certificateAuthorityFile}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it. Default: ""
connections.http.clientAuthentication connections.http.clientAuthentication	string Configuration setting	TLS negotiation/client-certificate/revocation configuration. Select protocols and ciphers compatible with the peer and provide a CRL or client identity only where supported by the chosen provider. Default: "none"
Field / key	Type / required	Details
connections.http.tlsVersion connections.http.tlsVersion	string Configuration setting	TLS negotiation/client-certificate/revocation configuration. Select protocols and ciphers compatible with the peer and provide a CRL

Field / key	Type / required	Details
		or client identity only where supported by the chosen provider. Default: "TLSv1.2"
connections.http.connectTimeoutSeconds connections.http.connectTimeoutSeconds	integer Configuration setting	connections.http.connectTimeoutSeconds: bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 30
connections.http.timeoutSeconds connections.http.timeoutSeconds	integer Configuration setting	connections.http.timeoutSeconds: bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 60
connections.http.username connections.http.username	string Configuration setting	Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions. Default: ""
connections.http.password connections.http.password	password Configuration setting	Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs. Default: "[supply securely]"
connections.http.proxyHost connections.http.proxyHost	string Configuration setting	Outbound proxy/authentication policy. Configure the proxy address and credentials when the runtime network requires it; non-preemptive authentication waits for a challenge where supported. Default: ""
connections.http.proxyPort connections.http.proxyPort	integer Configuration setting	Outbound proxy/authentication policy. Configure the proxy address and credentials when the runtime network requires it; non-preemptive authentication waits for a challenge where supported. Default: 8080
connections.http.verifyTls connections.http.verifyTls	boolean Configuration setting	Transport security setting. Enable TLS for confidential transport and verify server identity/certificates; disabling verification or trusting an unverified server weakens connection authentication. Default: true
connections.http.openApiDocument connections.http.openApiDocument	string Configuration setting	OpenAPI definition and selected operation identifier. The operation supplies method/path and request/response contracts; reselect it after replacing the specification. Default: ""
connections.http.apiDocsEnabled connections.http.apiDocsEnabled	string Configuration setting	Environment property connections.http.apiDocsEnabled. Bind it with ${properties.connections.http.apiDocsEnabled}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it. Default: ""
connections.http.apiDocsAccess connections.http.apiDocsAccess	string Configuration setting	Environment property connections.http.apiDocsAccess. Bind it with ${properties.connections.http.apiDocsAccess}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it. Default: ""
connections.http.apiPublicBaseUrl connections.http.apiPublicBaseUrl	string Configuration setting	Environment property connections.http.apiPublicBaseUrl. Bind it with ${properties.connections.http.apiPublicBaseUrl}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it. Default: ""

Field / key	Type / required	Details
connections.http.httpVersion connections.http.httpVersion	string Configuration setting	HTTP protocol version reported for the response, for example HTTP/1.1. Default: "1.1"
connections.http.nonPreemptiveAuthentication connections.http.nonPreemptiveAuthentication	string Configuration setting	Outbound proxy/authentication policy. Configure the proxy address and credentials when the runtime network requires it; non-preemptive authentication waits for a challenge where supported. Default: ""
connections.http.confidentiality connections.http.confidentiality	string Configuration setting	Transport security setting. Enable TLS for confidential transport and verify server identity/certificates; disabling verification or trusting an unverified server weakens connection authentication. Default: ""
connections.http.maximumTlsVersion connections.http.maximumTlsVersion	string Configuration setting	TLS negotiation/client-certificate/revocation configuration. Select protocols and ciphers compatible with the peer and provide a CRL or client identity only where supported by the chosen provider. Default: ""
connections.http.cipherSuites connections.http.cipherSuites	string Configuration setting	TLS negotiation/client-certificate/revocation configuration. Select protocols and ciphers compatible with the peer and provide a CRL or client identity only where supported by the chosen provider. Default: ""
connections.http.trustStoreType connections.http.trustStoreType	string Configuration setting	Trust-store file and format used by the selected provider. Supply trusted issuers in the supported store format; a certificate folder is an alternative where the connector supports it. Default: ""
connections.http.trustStoreFile connections.http.trustStoreFile	string Configuration setting	Trust-store file and format used by the selected provider. Supply trusted issuers in the supported store format; a certificate folder is an alternative where the connector supports it. Default: ""
connections.http.trustStorePassword connections.http.trustStorePassword	password Configuration setting	Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs. Default: "[supply securely]"
connections.http.identityStoreType connections.http.identityStoreType	string Configuration setting	Identity store containing the local certificate/private key. Supply its runtime path, password and supported format; this is distinct from the peer CA trust store. Default: ""
connections.http.identityStoreFile connections.http.identityStoreFile	string Configuration setting	Identity store containing the local certificate/private key. Supply its runtime path, password and supported format; this is distinct from the peer CA trust store. Default: ""
connections.http.identityStorePassword connections.http.identityStorePassword	password Configuration setting	Identity store containing the local certificate/private key. Supply its runtime path, password and supported format; this is distinct from the peer CA trust store. Default: "[supply securely]"
connections.http.keyAlias connections.http.keyAlias	string Configuration setting	Client/server certificate or private-key location/selection. Deploy matching certificate and key material and grant read access only to the runtime identity. Default: ""
connections.http.crlFile connections.http.crlFile	string Configuration setting	TLS negotiation/client-certificate/revocation configuration. Select protocols and ciphers compatible with the peer and provide a CRL or client identity only where supported by the chosen provider. Default: ""

Field / key	Type / required	Details
connections.http.connectionTimeoutMs connections.http.connectionTimeoutMs	integer Configuration setting	connections.http.connectionTimeoutMs: bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 30000
connections.http.socketTimeoutMs connections.http.socketTimeoutMs	integer Configuration setting	connections.http.socketTimeoutMs: bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 60000
connections.http.writeTimeoutMs connections.http.writeTimeoutMs	integer Configuration setting	connections.http.writeTimeoutMs: bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 60000
connections.http.poolTimeoutMs connections.http.poolTimeoutMs	integer Configuration setting	connections.http.poolTimeoutMs: bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 30000
connections.http.maximumTotalConnections connections.http.maximumTotalConnections	integer Configuration setting	Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity. Default: 200
connections.http.maximumConnectionsPerHost connections.http.maximumConnectionsPerHost	integer Configuration setting	Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity. Default: 20
connections.http.disableConnectionPooling connections.http.disableConnectionPooling	string Configuration setting	Connection lifecycle policy. Pooling/persistent connections reduce connection setup; recovery and keepalive apply to provider connections, not transaction-level replay. Default: ""
connections.http.usePersistentConnections connections.http.usePersistentConnections	string Configuration setting	Connection lifecycle policy. Pooling/persistent connections reduce connection setup; recovery and keepalive apply to provider connections, not transaction-level replay. Default: ""
connections.http.idleConnectionTimeoutMs connections.http.idleConnectionTimeoutMs	integer Configuration setting	connections.http.idleConnectionTimeoutMs: bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 40000
connections.http.followRedirects connections.http.followRedirects	string Configuration setting	Follow HTTP redirect responses. Disable when the flow must inspect or reject redirects explicitly. Default: ""
connections.http.retryCount connections.http.retryCount	string Configuration setting	Retry attempt policy. A retry can repeat an accepted external write; use stable identifiers and destination-side duplicate handling. Default: ""
connections.http.retryDelayMs connections.http.retryDelayMs	string Configuration setting	connections.http.retryDelayMs: bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: ""

Field / key	Type / required	Details
connections.http.retryStatusCodes connections.http.retryStatusCodes	string Configuration setting	Environment property connections.http.retryStatusCodes. Bind it with ${properties.connections.http.retryStatusCodes}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it. Default: ""
connections.http.maxRetryDelayMs connections.http.maxRetryDelayMs	string Configuration setting	connections.http.maxRetryDelayMs: bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: ""
connections.http.retryJitter connections.http.retryJitter	string Configuration setting	Environment property connections.http.retryJitter. Bind it with ${properties.connections.http.retryJitter}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it. Default: ""
connections.http.respectRetryAfter connections.http.respectRetryAfter	string Configuration setting	Environment property connections.http.respectRetryAfter. Bind it with ${properties.connections.http.respectRetryAfter}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it. Default: ""
connections.http.retryNonIdempotent connections.http.retryNonIdempotent	string Configuration setting	Environment property connections.http.retryNonIdempotent. Bind it with ${properties.connections.http.retryNonIdempotent}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it. Default: ""
connections.http.circuitFailureThreshold connections.http.circuitFailureThreshold	string Configuration setting	Environment property connections.http.circuitFailureThreshold. Bind it with ${properties.connections.http.circuitFailureThreshold}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it. Default: ""
connections.http.circuitResetSeconds connections.http.circuitResetSeconds	string Configuration setting	Environment property connections.http.circuitResetSeconds. Bind it with ${properties.connections.http.circuitResetSeconds}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it. Default: ""
connections.http.corsAllowedOrigins connections.http.corsAllowedOrigins	string Configuration setting	Environment property connections.http.corsAllowedOrigins. Bind it with ${properties.connections.http.corsAllowedOrigins}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it. Default: ""
connections.http.corsAllowCredentials connections.http.corsAllowCredentials	string Configuration setting	Environment property connections.http.corsAllowCredentials. Bind it with ${properties.connections.http.corsAllowCredentials}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it. Default: ""
connections.http.corsAllowedHeaders connections.http.corsAllowedHeaders	string Configuration setting	Environment property connections.http.corsAllowedHeaders. Bind it with ${properties.connections.http.corsAllowedHeaders}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it. Default: ""
Field / key	Type / required	Details
connections.http.corsExposeHeaders connections.http.corsExposeHeaders	string Configuration setting	Environment property connections.http.corsExposeHeaders. Bind it with ${properties.connections.http.corsExposeHeaders}; the active environment supplies its value. Keep its datatype consistent

Field / key	Type / required	Details
		with the activity or shared-connection field that references it. Default: ""
connections.http.corsMaxAgeSeconds connections.http.corsMaxAgeSeconds	string Configuration setting	Environment property connections.http.corsMaxAgeSeconds. Bind it with ${properties.connections.http.corsMaxAgeSeconds}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it. Default: ""
connections.http.rateLimitRequests connections.http.rateLimitRequests	string Configuration setting	Environment property connections.http.rateLimitRequests. Bind it with ${properties.connections.http.rateLimitRequests}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it. Default: ""
connections.http.rateLimitWindowSeconds connections.http.rateLimitWindowSeconds	string Configuration setting	Environment property connections.http.rateLimitWindowSeconds. Bind it with ${properties.connections.http.rateLimitWindowSeconds}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it. Default: ""
connections.http.policyStore connections.http.policyStore	string Configuration setting	Environment property connections.http.policyStore. Bind it with ${properties.connections.http.policyStore}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it. Default: ""
connections.http.policyRedisUrl connections.http.policyRedisUrl	password Configuration setting	Environment property connections.http.policyRedisUrl. Bind it with ${properties.connections.http.policyRedisUrl}; the active environment supplies its value. Supply the secret through protected deployment configuration. Default: "[supply securely]"
connections.http.policyTimeoutSeconds connections.http.policyTimeoutSeconds	string Configuration setting	connections.http.policyTimeoutSeconds: bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: ""
connections.http.requestBodyMode connections.http.requestBodyMode	string Configuration setting	Environment property connections.http.requestBodyMode. Bind it with ${properties.connections.http.requestBodyMode}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it. Default: ""
connections.http.uploadDirectory connections.http.uploadDirectory	string Configuration setting	Environment property connections.http.uploadDirectory. Bind it with ${properties.connections.http.uploadDirectory}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it. Default: ""
connections.http.enableCookies connections.http.enableCookies	string Configuration setting	Connection lifecycle policy. Pooling/persistent connections reduce connection setup; recovery and keepalive apply to provider connections, not transaction-level replay. Default: ""
connections.http.headers connections.http.headers	string Configuration setting	Header name/value object. Map only headers required by the target protocol; credentials belong in protected authentication settings. Default: ""
Field / key	Type / required	Details
connections.http.query connections.http.query	string Configuration setting	HTTP query parameters. Use a name/value object for structured parameters or QueryString for an already composed query;

Field / key	Type / required	Details
		encode values rather than concatenating untrusted text. Default: ""
connections.http.proxyUrl connections.http.proxyUrl	string Configuration setting	Outbound proxy/authentication policy. Configure the proxy address and credentials when the runtime network requires it; non-preemptive authentication waits for a challenge where supported. Default: ""
connections.http.proxyUsername connections.http.proxyUsername	string Configuration setting	Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions. Default: ""
connections.http.proxyPassword connections.http.proxyPassword	password Configuration setting	Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs. Default: "[supply securely]"
connections.http.minimumQtpThreads connections.http.minimumQtpThreads	integer Configuration setting	Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity. Default: 10
connections.http.maximumQtpThreads connections.http.maximumQtpThreads	integer Configuration setting	Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity. Default: 75
connections.http.maxQueueSize connections.http.maxQueueSize	integer Configuration setting	Size/capacity bound for the named request, response, header or event queue. Byte limits measure encoded content; queue limits bound buffered work. Default: 100
connections.http.queueTimeoutMs connections.http.queueTimeoutMs	string Configuration setting	connections.http.queueTimeoutMs: bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: ""
connections.http.acceptQueueSize connections.http.acceptQueueSize	integer Configuration setting	Size/capacity bound for the named request, response, header or event queue. Byte limits measure encoded content; queue limits bound buffered work. Default: 128
connections.http.readTimeoutMs connections.http.readTimeoutMs	integer Configuration setting	connections.http.readTimeoutMs: bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 30000
connections.http.maxHeaderBytes connections.http.maxHeaderBytes	integer Configuration setting	Size/capacity bound for the named request, response, header or event queue. Byte limits measure encoded content; queue limits bound buffered work. Default: 65536
connections.http.maxRequestBodyBytes connections.http.maxRequestBodyBytes	integer Configuration setting	Size/capacity bound for the named request, response, header or event queue. Byte limits measure encoded content; queue limits bound buffered work. Default: 67108864
connections.http.maxResponseBodyBytes connections.http.maxResponseBodyBytes	integer Configuration setting	Size/capacity bound for the named request, response, header or event queue. Byte limits measure encoded content; queue limits bound buffered work. Default: 67108864

Field / key	Type / required	Details
connections.http.allowedIps connections.http.allowedIps	string Configuration setting	Inbound client IP allowlist for the hosted service. Configure the addresses visible to the runtime, including proxy/network translation where applicable. Default: ""
connections.http.openApiApiKeySecrets connections.http.openApiApiKeySecrets	password Configuration setting	Bind a protected property or enter a JSON object that maps OpenAPI security scheme names to API key secrets. Archived projects require the secrets at deployment. Default: "[supply securely]"
connections.http.trustedProxyCidrs connections.http.trustedProxyCidrs	string Configuration setting	Proxy IPs or CIDRs allowed to supply X-Forwarded-For. Leave empty to use the transport peer for all requests. Default: ""
connections.http.httpErrorFormat connections.http.httpErrorFormat	string Configuration setting	Use problem to return RFC 9457 Problem Details for listener-generated errors. The json option keeps the original error response shape. Default: ""
connections.http.conditionalResponses connections.http.conditionalResponses	string Configuration setting	Return 304 for matching GET or HEAD validators using mapped ETag or Last-Modified response headers. The workflow still runs. Default: ""
connections.http.propagateTraceContext connections.http.propagateTraceContext	string Configuration setting	Enable W3C version 00 trace context for this connection. Enable it on both the listener and outbound connection to follow a request across calls. Default: ""
connections.http.sharedHttpCache connections.http.sharedHttpCache	string Configuration setting	Share fresh, explicitly public GET responses within the runtime process. Credentialed and conditional requests bypass this cache. Default: ""
connections.http.callbackAllowedOrigins connections.http.callbackAllowedOrigins	string Configuration setting	Explicit HTTPS origins trusted to receive callback requests. Separate origins with commas or spaces. Default: ""
connections.http.activityTimeoutSeconds connections.http.activityTimeoutSeconds	integer Configuration setting	connections.http.activityTimeoutSeconds: bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 180
connections.http.oauthGrantType connections.http.oauthGrantType	string Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. Default: ""
connections.http.oauthTokenUrl connections.http.oauthTokenUrl	string Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. Default: ""
connections.http.oauthClientId connections.http.oauthClientId	string Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. Default: ""
Field / key	Type / required	Details
connections.http.oauthClientSecret connections.http.oauthClientSecret	password Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use

Field / key	Type / required	Details
		code/verifier or refresh-token fields only for the corresponding grant. Default: "[supply securely]"
connections.http.oauthClientAuthentication connections.http.oauthClientAuthentication	string Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. Default: ""
connections.http.oauthClientPrivateKeyFile connections.http.oauthClientPrivateKeyFile	string Configuration setting	Path on the runtime host to the PEM signing key registered for the OAuth client. Provide the key file separately from the archive. Default: ""
connections.http.oauthClientAssertionAlgorithm connections.http.oauthClientAssertionAlgorithm	string Configuration setting	Asymmetric algorithm registered for the OAuth client. Defaults to RS256. Default: ""
connections.http.oauthClientAssertionKeyId connections.http.oauthClientAssertionKeyId	string Configuration setting	Optional kid included in the signed OAuth client assertion so the provider can select a verification key. Default: ""
connections.http.oauthScope connections.http.oauthScope	string Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. Default: ""
connections.http.oauthAuthorizationCode connections.http.oauthAuthorizationCode	password Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. Default: "[supply securely]"
connections.http.oauthRedirectUri connections.http.oauthRedirectUri	string Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. Default: ""
connections.http.oauthCodeVerifier connections.http.oauthCodeVerifier	password Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. Default: "[supply securely]"
connections.http.oauthRefreshToken connections.http.oauthRefreshToken	password Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. Default: "[supply securely]"
connections.http.oauthTimeoutSeconds connections.http.oauthTimeoutSeconds	integer Configuration setting	connections.http.oauthTimeoutSeconds: bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 30
Field / key	Type / required	Details
connections.http.oauthIntrospectionUrl connections.http.oauthIntrospectionUrl	string Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding

Field / key	Type / required	Details
		grant. Default: ""
connections.http.oauthRequiredScopes connections.http.oauthRequiredScopes	string Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. Default: ""
connections.http.jwtToken connections.http.jwtToken	password Configuration setting	Token or identity-provider endpoint for the selected authentication flow. Tokens expire; use the supported acquisition/refresh settings where required. Default: "[supply securely]"
connections.http.jwtAlgorithm connections.http.jwtAlgorithm	string Configuration setting	JWT signing/validation setting. Match algorithm, issuer/audience and signing/verification key to the counterpart; lifetime is in seconds and secrets/private keys require protected storage. Default: ""
connections.http.jwtSecret connections.http.jwtSecret	password Configuration setting	JWT signing/validation setting. Match algorithm, issuer/audience and signing/verification key to the counterpart; lifetime is in seconds and secrets/private keys require protected storage. Default: "[supply securely]"
connections.http.jwtPrivateKeyFile connections.http.jwtPrivateKeyFile	string Configuration setting	JWT signing/validation setting. Match algorithm, issuer/audience and signing/verification key to the counterpart; lifetime is in seconds and secrets/private keys require protected storage. Default: ""
connections.http.jwtPublicKeyFile connections.http.jwtPublicKeyFile	string Configuration setting	JWT signing/validation setting. Match algorithm, issuer/audience and signing/verification key to the counterpart; lifetime is in seconds and secrets/private keys require protected storage. Default: ""
connections.http.jwtJwksUrl connections.http.jwtJwksUrl	string Configuration setting	Environment property connections.http.jwtJwksUrl. Bind it with ${properties.connections.http.jwtJwksUrl}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it. Default: ""
connections.http.jwtJwksCacheSeconds connections.http.jwtJwksCacheSeconds	string Configuration setting	Environment property connections.http.jwtJwksCacheSeconds. Bind it with ${properties.connections.http.jwtJwksCacheSeconds}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it. Default: ""
connections.http.jwtJwksRefreshSeconds connections.http.jwtJwksRefreshSeconds	string Configuration setting	Environment property connections.http.jwtJwksRefreshSeconds. Bind it with ${properties.connections.http.jwtJwksRefreshSeconds}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it. Default: ""
connections.http.jwtJwksTimeoutSeconds connections.http.jwtJwksTimeoutSeconds	string Configuration setting	connections.http.jwtJwksTimeoutSeconds: bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: ""
connections.http.jwtIssuer connections.http.jwtIssuer	string Configuration setting	JWT signing/validation setting. Match algorithm, issuer/audience and signing/verification key to the counterpart; lifetime is in seconds and secrets/private keys require protected storage. Default: ""

Field / key	Type / required	Details
connections.http.jwtAudience connections.http.jwtAudience	string Configuration setting	JWT signing/validation setting. Match algorithm, issuer/audience and signing/verification key to the counterpart; lifetime is in seconds and secrets/private keys require protected storage. Default: ""
connections.http.jwtClaims connections.http.jwtClaims	string Configuration setting	Additional JWT claims as an object. Keep issuer/audience/expiry consistent with verification settings and avoid placing secrets in readable token claims. Default: ""
connections.http.jwtLifetimeSeconds connections.http.jwtLifetimeSeconds	integer Configuration setting	JWT signing/validation setting. Match algorithm, issuer/audience and signing/verification key to the counterpart; lifetime is in seconds and secrets/private keys require protected storage. Default: 300
connections.http.jwtClockSkewSeconds connections.http.jwtClockSkewSeconds	integer Configuration setting	Allowed time difference in seconds when checking signed request/token timestamps. Use a small tolerance and synchronize runtime clocks. Default: 30
connections.http.hmacAlgorithm connections.http.hmacAlgorithm	string Configuration setting	HMAC request signing/verification setting. Both endpoints must use the same signing algorithm, protected secret and signature header convention. Default: ""
connections.http.hmacKeyId connections.http.hmacKeyId	string Configuration setting	Identifier selecting the HMAC signing key. It identifies the key without exposing the shared secret. Default: "default"
connections.http.hmacSecret connections.http.hmacSecret	password Configuration setting	HMAC request signing/verification setting. Both endpoints must use the same signing algorithm, protected secret and signature header convention. Default: "[supply securely]"
connections.http.hmacClockSkewSeconds connections.http.hmacClockSkewSeconds	integer Configuration setting	Allowed time difference in seconds when checking signed request/token timestamps. Use a small tolerance and synchronize runtime clocks. Default: 300
connections.http.ldapUrl connections.http.ldapUrl	string Configuration setting	LDAP authentication setting. Configure the directory URL, search base/filter or user DN template, bind identity and required group; use TLS/StartTLS and the directory trust certificate where applicable. Default: ""
connections.http.ldapStartTls connections.http.ldapStartTls	string Configuration setting	LDAP authentication setting. Configure the directory URL, search base/filter or user DN template, bind identity and required group; use TLS/StartTLS and the directory trust certificate where applicable. Default: ""
connections.http.ldapCaFile connections.http.ldapCaFile	string Configuration setting	Trusted CA certificate location on the runtime host. A certificate folder can contain root certificates and optional chain certificates; configure the resolved deployment path. Default: ""
connections.http.ldapTimeoutSeconds connections.http.ldapTimeoutSeconds	integer Configuration setting	connections.http.ldapTimeoutSeconds: bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 10
Field / key	Type / required	Details
connections.http.ldapUserDnTemplate connections.http.ldapUserDnTemplate	string Configuration setting	LDAP authentication setting. Configure the directory URL, search base/filter or user DN template, bind identity and required group;

Field / key	Type / required	Details
		use TLS/StartTLS and the directory trust certificate where applicable. Default: ""
connections.http.ldapBindDn connections.http.ldapBindDn	string Configuration setting	LDAP authentication setting. Configure the directory URL, search base/filter or user DN template, bind identity and required group; use TLS/StartTLS and the directory trust certificate where applicable. Default: ""
connections.http.ldapBindPassword connections.http.ldapBindPassword	password Configuration setting	LDAP authentication setting. Configure the directory URL, search base/filter or user DN template, bind identity and required group; use TLS/StartTLS and the directory trust certificate where applicable. Default: "[supply securely]"
connections.http.ldapBaseDn connections.http.ldapBaseDn	string Configuration setting	LDAP authentication setting. Configure the directory URL, search base/filter or user DN template, bind identity and required group; use TLS/StartTLS and the directory trust certificate where applicable. Default: ""
connections.http.ldapSearchFilter connections.http.ldapSearchFilter	string Configuration setting	LDAP authentication setting. Configure the directory URL, search base/filter or user DN template, bind identity and required group; use TLS/StartTLS and the directory trust certificate where applicable. Default: "(uid={username})"
connections.http.ldapRequiredGroup connections.http.ldapRequiredGroup	string Configuration setting	LDAP authentication setting. Configure the directory URL, search base/filter or user DN template, bind identity and required group; use TLS/StartTLS and the directory trust certificate where applicable. Default: ""
connections.ftp.host connections.ftp.host	string Configuration setting	Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https. Default: "ftp.example.com"
connections.ftp.port connections.ftp.port	integer Configuration setting	Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https. Default: 21
connections.ftp.username connections.ftp.username	string Configuration setting	Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions. Default: ""
connections.ftp.password connections.ftp.password	password Configuration setting	Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs. Default: "[supply securely]"
connections.ftp.workingDirectory connections.ftp.workingDirectory	string Configuration setting	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. Default: "/"
connections.ftp.passiveMode connections.ftp.passiveMode	boolean Configuration setting	Provider-specific connection feature. Enable only when the selected driver/server supports it; check the connector procedure and vendor prerequisites for that mode. Default: true

Field / key	Type / required	Details
connections.ftp.timeoutSeconds connections.ftp.timeoutSeconds	integer Configuration setting	connections.ftp.timeoutSeconds: bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 60
connections.sftp.host connections.sftp.host	string Configuration setting	Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https. Default: "sftp.example.com"
connections.sftp.port connections.sftp.port	integer Configuration setting	Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https. Default: 22
connections.sftp.username connections.sftp.username	string Configuration setting	Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions. Default: ""
connections.sftp.password connections.sftp.password	password Configuration setting	Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs. Default: "[supply securely]"
connections.sftp.workingDirectory connections.sftp.workingDirectory	string Configuration setting	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. Default: "/"
connections.sftp.privateKeyFile connections.sftp.privateKeyFile	string Configuration setting	Client/server certificate or private-key location/selection. Deploy matching certificate and key material and grant read access only to the runtime identity. Default: ""
connections.sftp.privateKeyPassphrase connections.sftp.privateKeyPassphrase	password Configuration setting	Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs. Default: "[supply securely]"
connections.sftp.knownHostsFile connections.sftp.knownHostsFile	string Configuration setting	SSH host identity verification. Supply a trusted known-hosts file for SFTP; disabling verification permits connecting to an unverified server. Default: ""
connections.sftp.strictHostKeyChecking connections.sftp.strictHostKeyChecking	boolean Configuration setting	SSH host identity verification. Supply a trusted known-hosts file for SFTP; disabling verification permits connecting to an unverified server. Default: true
connections.sftp.timeoutSeconds connections.sftp.timeoutSeconds	integer Configuration setting	connections.sftp.timeoutSeconds: bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 60
connections.ems.host connections.ems.host	string Configuration setting	Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https. Default: "localhost"
Field / key	Type / required	Details
connections.ems.port connections.ems.port	integer Configuration setting	Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or

Field / key	Type / required	Details
		https. Default: 7222
connections.ems.serverUrl connections.ems.serverUrl	string Configuration setting	Provider connection address/namespace. Use the provider URI/connection-string syntax; Kafka bootstrapServers accepts broker host:port endpoints and AMQP virtualHost selects the broker namespace. Default: "tcp://localhost:7222"
connections.ems.driverDirectory connections.ems.driverDirectory	string Configuration setting	Database/JMS driver implementation or driver directory. Install the matching provider library on the runtime host; Java drivers require the bridge and compatible Java runtime. Default: ""
connections.ems.connectionFactoryClass connections.ems.connectionFactoryClass	string Configuration setting	Database/JMS driver implementation or driver directory. Install the matching provider library on the runtime host; Java drivers require the bridge and compatible Java runtime. Default: ""
connections.ems.connectionTimeoutSeconds connections.ems.connectionTimeoutSeconds	integer Configuration setting	connections.ems.connectionTimeoutSeconds: bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 30
connections.ems.destination connections.ems.destination	string Configuration setting	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. Default: "orders"
connections.ems.sessionCount connections.ems.sessionCount	integer Configuration setting	Provider/listener concurrency setting. Coordinate it with downstream capacity and the operation-specific runtime support; it is not a whole-workflow parallel-branch limit. Default: 1
connections.ems.flowLimit connections.ems.flowLimit	integer Configuration setting	Provider/listener concurrency setting. Coordinate it with downstream capacity and the operation-specific runtime support; it is not a whole-workflow parallel-branch limit. Default: 0
connections.ems.receiveTimeoutMs connections.ems.receiveTimeoutMs	integer Configuration setting	connections.ems.receiveTimeoutMs: bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 30000
connections.ems.username connections.ems.username	string Configuration setting	Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions. Default: ""
connections.ems.password connections.ems.password	password Configuration setting	Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs. Default: "[supply securely]"
connections.ems.clientId connections.ems.clientId	string Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. Default: ""
Field / key	Type / required	Details
connections.ems.connectionFactory connections.ems.connectionFactory	string Configuration setting	JMS connection-factory selection. Direct and JNDI modes require different provider configuration; JNDI uses the context

Field / key	Type / required	Details
		factory/provider URL and registered factory name. Default: "ConnectionFactory"
connections.ems.reconnectAttempts connections.ems.reconnectAttempts	integer Configuration setting	Retry attempt policy. A retry can repeat an accepted external write; use stable identifiers and destination-side duplicate handling. Default: 3
connections.ems.connectionFactoryType connections.ems.connectionFactoryType	string Configuration setting	JMS connection-factory selection. Direct and JNDI modes require different provider configuration; JNDI uses the context factory/provider URL and registered factory name. Default: "Direct"
connections.ems.messagingStyle connections.ems.messagingStyle	string Configuration setting	Subscription/destination policy. Durable subscriptions retain eligible messages for an offline named subscriber; shared subscriptions distribute eligible deliveries where supported by the provider. Default: "Generic"
connections.ems.queueConnectionFactory connections.ems.queueConnectionFactory	string Configuration setting	JMS connection-factory selection. Direct and JNDI modes require different provider configuration; JNDI uses the context factory/provider URL and registered factory name. Default: "QueueConnectionFactory"
connections.ems.topicConnectionFactory connections.ems.topicConnectionFactory	string Configuration setting	JMS connection-factory selection. Direct and JNDI modes require different provider configuration; JNDI uses the context factory/provider URL and registered factory name. Default: "TopicConnectionFactory"
connections.ems.jndiContextFactory connections.ems.jndiContextFactory	string Configuration setting	JMS connection-factory selection. Direct and JNDI modes require different provider configuration; JNDI uses the context factory/provider URL and registered factory name. Default: ""
connections.ems.jndiProviderUrl connections.ems.jndiProviderUrl	string Configuration setting	JMS connection-factory selection. Direct and JNDI modes require different provider configuration; JNDI uses the context factory/provider URL and registered factory name. Default: "tcp://localhost:7222"
connections.ems.jndiUsername connections.ems.jndiUsername	string Configuration setting	Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions. Default: ""
connections.ems.jndiPassword connections.ems.jndiPassword	password Configuration setting	Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs. Default: "[supply securely]"
connections.ems.useXa connections.ems.useXa	boolean Configuration setting	Provider-specific connection feature. Enable only when the selected driver/server supports it; check the connector procedure and vendor prerequisites for that mode. Default: false
connections.ems.useUfo connections.ems.useUfo	boolean Configuration setting	Provider-specific connection feature. Enable only when the selected driver/server supports it; check the connector procedure and vendor prerequisites for that mode. Default: false
connections.ems.sslEnabled connections.ems.sslEnabled	boolean Configuration setting	Transport security setting. Enable TLS for confidential transport and verify server identity/certificates; disabling verification or trusting an unverified server weakens connection authentication. Default: false

Field / key	Type / required	Details
connections.ems.sslTrustedCertificates connections.ems.sslTrustedCertificates	string Configuration setting	Trusted CA certificate location on the runtime host. A certificate folder can contain root certificates and optional chain certificates; configure the resolved deployment path. Default: ""
connections.ems.reconnectDelayMs connections.ems.reconnectDelayMs	integer Configuration setting	connections.ems.reconnectDelayMs: bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 5000
connections.ems.heartbeatOutgoingMs connections.ems.heartbeatOutgoingMs	integer Configuration setting	STOMP outgoing heartbeat interval in milliseconds. Zero disables that direction; the broker and client negotiate the effective heartbeat interval. Default: 0
connections.ems.heartbeatIncomingMs connections.ems.heartbeatIncomingMs	integer Configuration setting	STOMP incoming heartbeat interval in milliseconds. Zero disables that direction; the broker and client negotiate the effective heartbeat interval. Default: 0
connections.jms.provider connections.jms.provider	string Configuration setting	Provider/protocol or receive mode selection. Choose the broker implementation and delivery/settlement behavior supported by the configured connection. Default: "Generic JMS 2.0"
connections.jms.serverUrl connections.jms.serverUrl	string Configuration setting	Provider connection address/namespace. Use the provider URI/connection-string syntax; Kafka bootstrapServers accepts broker host:port endpoints and AMQP virtualHost selects the broker namespace. Default: "tcp://localhost:61613"
connections.jms.driverDirectory connections.jms.driverDirectory	string Configuration setting	Database/JMS driver implementation or driver directory. Install the matching provider library on the runtime host; Java drivers require the bridge and compatible Java runtime. Default: ""
connections.jms.connectionFactoryClass connections.jms.connectionFactoryClass	string Configuration setting	Database/JMS driver implementation or driver directory. Install the matching provider library on the runtime host; Java drivers require the bridge and compatible Java runtime. Default: ""
connections.jms.connectionTimeoutSeconds connections.jms.connectionTimeoutSeconds	integer Configuration setting	connections.jms.connectionTimeoutSeconds: bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 30
connections.jms.host connections.jms.host	string Configuration setting	Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https. Default: "localhost"
connections.jms.port connections.jms.port	integer Configuration setting	Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https. Default: 61613
connections.jms.username connections.jms.username	string Configuration setting	Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions. Default: ""

Field / key	Type / required	Details
connections.jms.password connections.jms.password	password Configuration setting	Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs. Default: "[supply securely]"
connections.jms.clientId connections.jms.clientId	string Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. Default: ""
connections.jms.connectionFactoryType connections.jms.connectionFactoryType	string Configuration setting	JMS connection-factory selection. Direct and JNDI modes require different provider configuration; JNDI uses the context factory/provider URL and registered factory name. Default: "Direct"
connections.jms.connectionFactory connections.jms.connectionFactory	string Configuration setting	JMS connection-factory selection. Direct and JNDI modes require different provider configuration; JNDI uses the context factory/provider URL and registered factory name. Default: "ConnectionFactory"
connections.jms.jndiContextFactory connections.jms.jndiContextFactory	string Configuration setting	JMS connection-factory selection. Direct and JNDI modes require different provider configuration; JNDI uses the context factory/provider URL and registered factory name. Default: ""
connections.jms.jndiProviderUrl connections.jms.jndiProviderUrl	string Configuration setting	JMS connection-factory selection. Direct and JNDI modes require different provider configuration; JNDI uses the context factory/provider URL and registered factory name. Default: ""
connections.jms.jndiUsername connections.jms.jndiUsername	string Configuration setting	Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions. Default: ""
connections.jms.jndiPassword connections.jms.jndiPassword	password Configuration setting	Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs. Default: "[supply securely]"
connections.jms.sslEnabled connections.jms.sslEnabled	boolean Configuration setting	Transport security setting. Enable TLS for confidential transport and verify server identity/certificates; disabling verification or trusting an unverified server weakens connection authentication. Default: false
connections.jms.sslTrustedCertificates connections.jms.sslTrustedCertificates	string Configuration setting	Trusted CA certificate location on the runtime host. A certificate folder can contain root certificates and optional chain certificates; configure the resolved deployment path. Default: ""
connections.jms.reconnectAttempts connections.jms.reconnectAttempts	integer Configuration setting	Retry attempt policy. A retry can repeat an accepted external write; use stable identifiers and destination-side duplicate handling. Default: 3
connections.jms.reconnectDelayMs connections.jms.reconnectDelayMs	integer Configuration setting	connections.jms.reconnectDelayMs: bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 5000
Field / key	Type / required	Details
connections.kafka.bootstrapServers connections.kafka.bootstrapServers	string Configuration setting	Provider connection address/namespace. Use the provider URI/connection-string syntax; Kafka bootstrapServers accepts broker host:port endpoints and AMQP virtualHost selects the

Field / key	Type / required	Details
		broker namespace. Default: "localhost:9092"
connections.kafka.topic connections.kafka.topic	string Configuration setting	Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access. Default: ""
connections.kafka.clientId connections.kafka.clientId	string Configuration setting	OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. Default: ""
connections.kafka.groupId connections.kafka.groupId	string Configuration setting	Kafka consumer-group identifier. Consumers in the same group share partitions and offsets; use a separate test group to avoid changing production consumption state. Default: "mina"
connections.kafka.securityProtocol connections.kafka.securityProtocol	string Configuration setting	Kafka transport/authentication selection. SASL_SSL combines TLS and SASL; configure credentials and CA verification for the selected SASL mechanism. Default: "PLAINTEXT"
connections.kafka.saslMechanism connections.kafka.saslMechanism	string Configuration setting	Kafka transport/authentication selection. SASL_SSL combines TLS and SASL; configure credentials and CA verification for the selected SASL mechanism. Default: "PLAIN"
connections.kafka.username connections.kafka.username	string Configuration setting	Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions. Default: ""
connections.kafka.password connections.kafka.password	password Configuration setting	Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs. Default: "[supply securely]"
connections.kafka.requestTimeoutMilliseconds connections.kafka.requestTimeoutMilliseconds	integer Configuration setting	connections.kafka.requestTimeoutMilliseconds: bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 30000
connections.kafka.connectionTimeoutMilliseconds connections.kafka.connectionTimeoutMilliseconds	integer Configuration setting	connections.kafka.connectionTimeoutMilliseconds: bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 10000
connections.kafka.sslCaLocation connections.kafka.sslCaLocation	string Configuration setting	Trusted CA certificate location on the runtime host. A certificate folder can contain root certificates and optional chain certificates; configure the resolved deployment path. Default: ""
connections.kafka.sslCertificateLocation connections.kafka.sslCertificateLocation	string Configuration setting	Client/server certificate or private-key location/selection. Deploy matching certificate and key material and grant read access only to the runtime identity. Default: ""
connections.kafka.sslKeyLocation connections.kafka.sslKeyLocation	string Configuration setting	Client/server certificate or private-key location/selection. Deploy matching certificate and key material and grant read access only to the runtime identity. Default: ""

Field / key	Type / required	Details
connections.kafka.sslKeyPassword connections.kafka.sslKeyPassword	password Configuration setting	Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs. Default: "[supply securely]"
connections.kafka.schemaRegistryUrl connections.kafka.schemaRegistryUrl	string Configuration setting	Kafka schema-registry configuration. Select the registry endpoint/vendor and authentication; the Avro runtime limitations are documented on the Kafka operation pages. Default: ""
connections.kafka.schemaRegistryUsername connections.kafka.schemaRegistryUsername	string Configuration setting	Kafka schema-registry configuration. Select the registry endpoint/vendor and authentication; the Avro runtime limitations are documented on the Kafka operation pages. Default: ""
connections.kafka.schemaRegistryPassword connections.kafka.schemaRegistryPassword	password Configuration setting	Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs. Default: "[supply securely]"
connections.kafka.reconnectBackoffMilliseconds connections.kafka.reconnectBackoffMilliseconds	integer Configuration setting	connections.kafka.reconnectBackoffMilliseconds: bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 50
connections.kafka.retryBackoffMilliseconds connections.kafka.retryBackoffMilliseconds	integer Configuration setting	connections.kafka.retryBackoffMilliseconds: bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 100
connections.kafka.authenticationType connections.kafka.authenticationType	string Configuration setting	Authentication mechanism for this connection. Supply the corresponding credentials/token/key settings and verify the receiving endpoint accepts that mechanism. Default: "None"
connections.kafka.useTicketCache connections.kafka.useTicketCache	boolean Configuration setting	Provider credential/identity material. Use the mechanism supported by the selected connection and runtime; keep private keys, access keys and credential files outside shared source packages. Default: false
connections.kafka.keytabFile connections.kafka.keytabFile	string Configuration setting	Provider credential/identity material. Use the mechanism supported by the selected connection and runtime; keep private keys, access keys and credential files outside shared source packages. Default: ""
connections.kafka.principalName connections.kafka.principalName	string Configuration setting	Provider credential/identity material. Use the mechanism supported by the selected connection and runtime; keep private keys, access keys and credential files outside shared source packages. Default: ""
connections.kafka.jaasConfigFile connections.kafka.jaasConfigFile	string Configuration setting	JAAS login configuration file on the runtime host for the selected authentication mechanism. Protect credentials contained in the file. Default: ""
connections.kafka.loginCallbackHandler connections.kafka.loginCallbackHandler	string Configuration setting	Provider credential/identity material. Use the mechanism supported by the selected connection and runtime; keep private keys, access keys and credential files outside shared source packages. Default: ""

Field / key	Type / required	Details
connections.kafka.schemaRegistryVendor connections.kafka.schemaRegistryVendor	string Configuration setting	Kafka schema-registry configuration. Select the registry endpoint/vendor and authentication; the Avro runtime limitations are documented on the Kafka operation pages. Default: "Confluent"
connections.kafka.clientProperties connections.kafka.clientProperties	json Configuration setting	Additional provider settings as name/value entries. Use exact supported provider option names; invalid options may be rejected at connection or execution time. Default: "{}"
connections.pubsub.projectId connections.pubsub.projectId	string Configuration setting	Google Cloud project containing the Pub/Sub topic or subscription. Supply the project ID, not its display name. Default: "my-gcp-project"
connections.pubsub.authenticationType connections.pubsub.authenticationType	string Configuration setting	Authentication mechanism for this connection. Supply the corresponding credentials/token/key settings and verify the receiving endpoint accepts that mechanism. Default: "Service Account JSON"
connections.pubsub.serviceAccountJson connections.pubsub.serviceAccountJson	password Configuration setting	Provider credential/identity material. Use the mechanism supported by the selected connection and runtime; keep private keys, access keys and credential files outside shared source packages. Default: "[supply securely]"
connections.pubsub.credentialsFile connections.pubsub.credentialsFile	string Configuration setting	Provider credential/identity material. Use the mechanism supported by the selected connection and runtime; keep private keys, access keys and credential files outside shared source packages. Default: ""
connections.pubsub.endpoint connections.pubsub.endpoint	string Configuration setting	Endpoint or filesystem path for this operation. For HTTP use the service URI/path; for files use a path accessible to the runtime host, not only the designer machine. Default: "pubsub.googleapis.com:443"
connections.pubsub.emulatorHost connections.pubsub.emulatorHost	string Configuration setting	Local Pub/Sub emulator address for testing. Leave empty for Google Cloud provider access and keep emulator configuration out of production environments. Default: ""
connections.pubsub.ackDeadlineSeconds connections.pubsub.ackDeadlineSeconds	integer Configuration setting	Provider lifetime/wait setting in seconds, as named by this field. Set it above normal work/latency while retaining a bounded recovery time. Default: 60
connections.pubsub.connectionTimeoutSeconds connections.pubsub.connectionTimeoutSeconds	integer Configuration setting	connections.pubsub.connectionTimeoutSeconds: bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 30
connections.pubsub.maxInboundMessageBytes connections.pubsub.maxInboundMessageBytes	integer Configuration setting	Size/capacity bound for the named request, response, header or event queue. Byte limits measure encoded content; queue limits bound buffered work. Default: 20971520
connections.pubsub.keepAliveSeconds connections.pubsub.keepAliveSeconds	integer Configuration setting	Connection lifecycle policy. Pooling/persistent connections reduce connection setup; recovery and keepalive apply to provider connections, not transaction-level replay. Default: 60
Field / key	Type / required	Details
connections.pubsub.batchMaxMessages	integer Configuration setting	Pub/Sub batching threshold: message count, total bytes or waiting time in milliseconds, according to the field name. Tune with

Field / key	Type / required	Details
connections.pubsub.batchMaxMessages		payload sizes and acceptable delivery latency. Default: 100
connections.pubsub.batchMaxBytes connections.pubsub.batchMaxBytes	integer Configuration setting	Pub/Sub batching threshold: message count, total bytes or waiting time in milliseconds, according to the field name. Tune with payload sizes and acceptable delivery latency. Default: 1048576
connections.pubsub.batchDelayThresholdMilliseconds connections.pubsub.batchDelayThresholdMilliseconds	integer Configuration setting	Pub/Sub batching threshold: message count, total bytes or waiting time in milliseconds, according to the field name. Tune with payload sizes and acceptable delivery latency. Default: 10
connections.pubsub.flowControlMaxMessages connections.pubsub.flowControlMaxMessages	integer Configuration setting	Bound on received/buffered messages or bytes, according to this field. Keep the bound within available memory and downstream processing capacity. Default: 1000
connections.pubsub.flowControlMaxBytes connections.pubsub.flowControlMaxBytes	integer Configuration setting	Bound on received/buffered messages or bytes, according to this field. Keep the bound within available memory and downstream processing capacity. Default: 10485760
connections.pubsub.caCertificateFile connections.pubsub.caCertificateFile	string Configuration setting	Trusted CA certificate location on the runtime host. A certificate folder can contain root certificates and optional chain certificates; configure the resolved deployment path. Default: ""
connections.sap.applicationServerHost connections.sap.applicationServerHost	string Configuration setting	Environment property connections.sap.applicationServerHost. Bind it with ${properties.connections.sap.applicationServerHost}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it. Default: "sap-ecc.example.com"
connections.sap.release connections.sap.release	string Configuration setting	Environment property connections.sap.release. Bind it with ${properties.connections.sap.release}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it. Default: "current"
connections.sap.systemNumber connections.sap.systemNumber	string Configuration setting	Environment property connections.sap.systemNumber. Bind it with ${properties.connections.sap.systemNumber}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it. Default: "00"
connections.sap.client connections.sap.client	string Configuration setting	Environment property connections.sap.client. Bind it with ${properties.connections.sap.client}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it. Default: "100"
connections.sap.language connections.sap.language	string Configuration setting	Environment property connections.sap.language. Bind it with ${properties.connections.sap.language}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it. Default: "EN"
connections.sap.username connections.sap.username	string Configuration setting	Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions. Default: ""

Field / key	Type / required	Details
connections.sap.password connections.sap.password	password Configuration setting	Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs. Default: "[supply securely]"
connections.sap.messageServerHost connections.sap.messageServerHost	string Configuration setting	Environment property connections.sap.messageServerHost. Bind it with ${properties.connections.sap.messageServerHost}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it. Default: ""
connections.sap.systemId connections.sap.systemId	string Configuration setting	Environment property connections.sap.systemId. Bind it with ${properties.connections.sap.systemId}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it. Default: ""
connections.sap.logonGroup connections.sap.logonGroup	string Configuration setting	Environment property connections.sap.logonGroup. Bind it with ${properties.connections.sap.logonGroup}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it. Default: "PUBLIC"
connections.sap.sapRouter connections.sap.sapRouter	string Configuration setting	Environment property connections.sap.sapRouter. Bind it with ${properties.connections.sap.sapRouter}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it. Default: ""
connections.sap.programId connections.sap.programId	string Configuration setting	Environment property connections.sap.programId. Bind it with ${properties.connections.sap.programId}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it. Default: ""
connections.sap.gatewayHost connections.sap.gatewayHost	string Configuration setting	Environment property connections.sap.gatewayHost. Bind it with ${properties.connections.sap.gatewayHost}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it. Default: ""
connections.sap.gatewayService connections.sap.gatewayService	string Configuration setting	Environment property connections.sap.gatewayService. Bind it with ${properties.connections.sap.gatewayService}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it. Default: ""
connections.sap.maximumConnections connections.sap.maximumConnections	integer Configuration setting	Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity. Default: 8
connections.sap.outboundWorkerCount connections.sap.outboundWorkerCount	integer Configuration setting	Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity. Default: 2
connections.sap.poolCapacity connections.sap.poolCapacity	integer Configuration setting	Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity. Default: 1
Field / key	Type / required	Details
connections.sap.peakLimit connections.sap.peakLimit	integer Configuration setting	Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider

Field / key	Type / required	Details
		limits and downstream processing capacity. Default: 1
connections.sap.connectionExpirationMilliseconds connections.sap.connectionExpirationMilliseconds	integer Configuration setting	Lifetime of an idle SAP pooled connection in milliseconds before it is eligible for expiration. Match this to the SAP server connection policy. Default: 600000
connections.sap.outboundPoolIdleSeconds connections.sap.outboundPoolIdleSeconds	integer Configuration setting	Provider lifetime/wait setting in seconds, as named by this field. Set it above normal work/latency while retaining a bounded recovery time. Default: 1800
connections.sap.maximumOutboundPools connections.sap.maximumOutboundPools	integer Configuration setting	Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity. Default: 16
connections.sap.maxPendingEvents connections.sap.maxPendingEvents	integer Configuration setting	Bound on received/buffered messages or bytes, according to this field. Keep the bound within available memory and downstream processing capacity. Default: 16
connections.sap.jvmInitialHeapMb connections.sap.jvmInitialHeapMb	integer Configuration setting	Java bridge heap size in megabytes. Size it for concurrent payloads and driver overhead; keep the initial heap at or below the maximum. Default: 64
connections.sap.jvmMaximumHeapMb connections.sap.jvmMaximumHeapMb	integer Configuration setting	Java bridge heap size in megabytes. Size it for concurrent payloads and driver overhead; keep the initial heap at or below the maximum. Default: 512
connections.sap.timeoutMilliseconds connections.sap.timeoutMilliseconds	integer Configuration setting	connections.sap.timeoutMilliseconds: bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 30000
connections.sap.ackTimeoutSeconds connections.sap.ackTimeoutSeconds	integer Configuration setting	connections.sap.ackTimeoutSeconds: bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. Default: 300
connections.sap.maximumDynamicSessions connections.sap.maximumDynamicSessions	integer Configuration setting	Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity. Default: 1024
connections.sap.mode connections.sap.mode	string Configuration setting	Input/execution mode selected from the available choices. Match inline text, file or structured input to the actual source; connection modes such as mock/memory are for local testing. Default: "external"
connections.sap.driverDirectory connections.sap.driverDirectory	string Configuration setting	Database/JMS driver implementation or driver directory. Install the matching provider library on the runtime host; Java drivers require the bridge and compatible Java runtime. Default: ""
connections.sap.destinationName connections.sap.destinationName	string Configuration setting	Connection routing mode or named provider destination. Select the mode that matches the server configuration and supply its corresponding connection fields. Default: ""

Field / key	Type / required	Details
connections.sap.connectionType connections.sap.connectionType	string Configuration setting	Connection routing mode or named provider destination. Select the mode that matches the server configuration and supply its corresponding connection fields. Default: "dedicated"
connections.sap.sncPartnerName connections.sap.sncPartnerName	string Configuration setting	Environment property connections.sap.sncPartnerName. Bind it with ${properties.connections.sap.sncPartnerName}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it. Default: ""
connections.sap.sncLibraryPath connections.sap.sncLibraryPath	string Configuration setting	Environment property connections.sap.sncLibraryPath. Bind it with ${properties.connections.sap.sncLibraryPath}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it. Default: ""
connections.sap.sncMyName connections.sap.sncMyName	string Configuration setting	Environment property connections.sap.sncMyName. Bind it with ${properties.connections.sap.sncMyName}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it. Default: ""
connections.sap.sncQop connections.sap.sncQop	string Configuration setting	Environment property connections.sap.sncQop. Bind it with ${properties.connections.sap.sncQop}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it. Default: ""
connections.sapTid.storageFile connections.sapTid.storageFile	string Configuration setting	Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing. Default: "data/sap-tids.json"
connections.sapTid.mode connections.sapTid.mode	string Configuration setting	Input/execution mode selected from the available choices. Match inline text, file or structured input to the actual source; connection modes such as mock/memory are for local testing. Default: "active"
connections.file.path connections.file.path	string Configuration setting	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. Default: ""
connections.file.destination connections.file.destination	string Configuration setting	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. Default: ""
connections.file.encoding connections.file.encoding	string Configuration setting	Character encoding for text conversion, for example UTF-8. Use the sender encoding when parsing; a wrong encoding can corrupt non-ASCII characters. Default: "utf-8"
connections.file.pattern connections.file.pattern	string Configuration setting	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. Default: "*"
connections.file.pollInterval connections.file.pollInterval	integer Configuration setting	Polling/scheduling interval for repeated execution. Size the interval to expected arrival rate and processing duration; select the unit where the activity exposes one. Default: 5

Field / key	Type / required	Details
connections.file.recursive connections.file.recursive	boolean Configuration setting	Include subdirectories when traversing/listing matching files. Restrict the starting directory to avoid scanning unrelated trees. Default: false
connections.file.includeTimestamp connections.file.includeTimestamp	boolean Configuration setting	Include timestamp metadata where the operation supports it, to distinguish or trace generated/polled content. Default: false
connections.file.readAs connections.file.readAs	string Configuration setting	Representation used for reading, writing or returning the value. Select the mode matching the actual document and downstream consumer contract. Default: "Text"
connections.file.excludeFileContent connections.file.excludeFileContent	boolean Configuration setting	Omit file content from polling output when only file metadata is needed. Downstream Read File can fetch content explicitly. Default: false
connections.file.writeAs connections.file.writeAs	string Configuration setting	Representation used for reading, writing or returning the value. Select the mode matching the actual document and downstream consumer contract. Default: "Text"
connections.file.append connections.file.append	boolean Configuration setting	Existing-file policy. Append adds content to the existing file; overwrite replaces existing content. Disable replacement where existing output must be preserved. Default: false
connections.file.overwrite connections.file.overwrite	boolean Configuration setting	Existing-file policy. Append adds content to the existing file; overwrite replaces existing content. Disable replacement where existing output must be preserved. Default: true
connections.file.createDirectories connections.file.createDirectories	boolean Configuration setting	Create missing parent directories for the output path. The runtime account still needs directory creation/write permission. Default: false
connections.file.addLineSeparator connections.file.addLineSeparator	boolean Configuration setting	Line separator handling for text output. Choose the delimiter expected by the consumer; includeFinalLineSeparator adds a separator after the last record. Default: false
connections.file.compression connections.file.compression	string Configuration setting	Compression selection for file or staging content. Match decompression settings at the receiving operation; compression changes the wire bytes. Default: "None"
connections.file.listType connections.file.listType	string Configuration setting	File listing selection/order. sortBy chooses the metadata key, sortOrder its direction, and listType limits the kinds of entries returned. Default: "Files and Directories"
connections.file.sortBy connections.file.sortBy	string Configuration setting	File listing selection/order. sortBy chooses the metadata key, sortOrder its direction, and listType limits the kinds of entries returned. Default: "Name"
connections.file.sortOrder connections.file.sortOrder	string Configuration setting	File listing selection/order. sortBy chooses the metadata key, sortOrder its direction, and listType limits the kinds of entries returned. Default: "Ascending"
connections.file.ignoreMissing connections.file.ignoreMissing	boolean Configuration setting	Allow a missing file/path where supported instead of faulting. Use only when absence is an expected business outcome. Default: false

Field / key	Type / required	Details
connections.file.preserveAttributes connections.file.preserveAttributes	boolean Configuration setting	Request preservation of supported file attributes during the operation. Availability depends on filesystem and runtime permissions. Default: false
connections.file.eventType connections.file.eventType	string Configuration setting	Type of file event selected/reported by the poller, such as a created or changed file. Default: "Created"
connections.file.includeExisting connections.file.includeExisting	boolean Configuration setting	Include files already present when polling begins, rather than only later arrivals. Test restart behavior to avoid unintended reprocessing. Default: false
connections.file.postAction connections.file.postAction	string Configuration setting	Post-operation action or returned output. Use the activity-specific operation/contract to determine whether the value is content, metadata or a completion result. Default: "None"
connections.file.moveTo connections.file.moveTo	string Configuration setting	Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. Default: ""

MINA documentation site and PDF

Getting started

Guide

MINA documentation site and PDF

Open Help > MINA Documentation in Studio, or visit `/help/index.html` on the same host and port as Studio. No separate documentation service is required. Installed desktop Studio uses its existing local runtime address; do not assume a fixed port.

The documentation includes a searchable, expandable index and deep links to individual topics and tabs. Use Download PDF for the complete offline manual, or Print topic for the selected topic, including all its tabs. Light and dark display modes are supported.

Coverage and accuracy

Activity Configuration, Input, Output, Advanced and Errors sections are generated from Studio's base contracts and documentation definitions. Tasks, groups, mapping, functions, shared connections, deployment, debugging, administration and security guides are included. Project-selected schemas, provider modes and JDBC parameters can change the fields shown in Studio: the reference does not substitute for those project-specific schemas. Provider prerequisites and production qualification still apply.

Only explicitly selected product source files and guides are read. The generator does not inspect project data, environment credentials or user payloads. Existing implementation limitations are documented, not claimed as implemented.

Project Explorer and task canvas

New projects start with a Sub Task containing Start and End. Use a Starter Task when an external event, such as HTTP, a timer, or a message, starts the workflow. Use Call Sub Task to invoke reusable logic. Starter and Sub Task icons distinguish the two in Project Explorer.

Subtasks and Call Sub Task use the same process-network-and-bent-arrow symbol in Project Explorer, task tabs, creation menus, the activity picker, and the canvas. Starter tasks use the process network with a play symbol.

The canvas, activity palette, and activity picker share one activity icon catalog. Shared connections use the same protocol or vendor artwork in Project Explorer and connection creation menus. Imported legacy Mapper activities retain the Mapper symbol; an unrecognized activity uses a neutral process symbol instead of a Start or End icon.

Tasks, shared connections, and schemas belong to packages. Right-click their category or an existing package and choose New Package to create a top-level or nested folder. Choose the package when creating an item. Default is available for each category. Right-click an existing item and choose Move to Package to organize it without changing its references. Studio replaces spaces with hyphens in object names.

To rename a package, right-click the folder under Tasks, Resources -> Connections, or Schemas and choose Rename Package. Its nested folders and items move with it, keeping task, connection, and schema references intact. Under Packaging, right-click the deployment artifact and choose Rename Package to update its name for future exports while keeping its version and settings. Save the project to keep these changes.

Task, connection, and schema names must be unique within the same package. Studio ignores case and treats spaces as hyphens when checking a name. You can reuse a name in a different package. For dynamic Call Sub Task routing, use the task ID or the qualified name (for example, `Orders/Validate`) if more than one subtask has the same name. Validate Project reports conflicts already present in older projects.

Package contents appear below their folder. If a name is shortened in a narrow Project Explorer, hover over it to see the full name or drag the pane divider to widen it.

When you first add a protocol activity or shared connection, Studio adds the relevant missing environment properties. Connection setup also offers those properties for binding. Review the generated defaults in the active environment before running the task.

Use Magnify orchestrator on the canvas and hover to see a 2× preview. Press Escape to turn it off. Select a transition to set its Line color or choose Reset color to restore the default. Close task tabs individually or use Close All to clear the canvas.

Use the Theme selector to change how Studio looks and feels. Compact Workbench uses a shorter ribbon and denser explorer rows. Blueprint Board uses a drafting grid and framed canvas activities. Reading Room uses larger labels, wider spacing, and a paper-like canvas. Existing themes are still available, and Studio remembers your choice.

The bottom-right Runtime status shows startup stages, current Studio operations, and the activities running in the active project. Click it for the designer and runtime details, including parallel activities and the latest detected slow UI operation on supported browsers. Startup progress counts completed stages; an animated bar means the operation has no reliable percentage. The Windows startup screen also shows the current stage before the designer is ready.

The footer and details use the same live runtime health check. If an older runtime does not provide activity-progress details, Studio reports that limitation separately. It does not mark a healthy runtime as unavailable.

The welcome screen shows packets moving along the integration routes. Use Pause route animation to stop them or Play route animation to start them. Until you choose a setting, the packets follow the Windows or browser reduced-motion preference. Studio remembers your choice.

JDBC query and writes

Use `:name` parameters for Query, Update, and Insert, and map each value under Input -> parameters. To execute multiple records together, enable Batch update and map records. An empty list writes zero rows. For Insert if record does not exist, map an INSERT statement to InsertStatement and provide the parameters used by both statements. MINA runs the INSERT only when the UPDATE affects zero rows. If you need an atomic upsert across concurrent executions, use your database's native upsert statement.

To page through a query, enable Process result in subsets and use a stable `ORDER BY`. Start with Row offset zero. Pass the previous result's nextRowOffset to the next call and stop when lastSubset is true. Maximum rows caps the total rows across those pages. Each call re-executes the query.

PostgreSQL samples

- Run [postgresql-setup.sql](/samples/postgresql-setup.sql) in a disposable PostgreSQL database. It creates `mina_demo.orders` and the `mina_demo.order_total` procedure.

- In Studio Samples, open PostgreSQL CRUD or PostgreSQL Procedure and Paging. Set the JDBC URL, username, and password in the local environment. Use Debug to run the example Sub Task or run it by task ID in an exported Python archive.

- CRUD upserts a demo row, updates it with a version guard, queries the JSONB field, and deletes that row. The other sample reads the first page and calls the procedure with an INOUT numeric argument.

PostgreSQL accepts libpq connection strings and JDBC-style PostgreSQL URLs. Configure PostgreSQL TLS mode and a trusted root certificate for verified TLS. A configured Schema becomes the search path. Use named `:parameter` bindings to avoid ambiguity with PostgreSQL's JSONB question-mark operator. Declare JSON or JSONB, ISO date/time, and decimal types for driver adaptation. To retain exact decimal digits from a mapped value, provide it as a string. Use JDBC Query with `SELECT` for PostgreSQL functions and Call Procedure for `CALL` operations. PostgreSQL statement timeout applies when configured on the activity.

Updating the guide

From the repository root, install the documentation authoring dependencies into your chosen Python environment:

```
python -m pip install -r scripts/documentation-requirements.txt
cd frontend
npm run docs:build
npm run docs:check
node --test scripts/documentation.test.mjs
```

If Python is not on PATH, set `MINA_DOCS_PYTHON` to its executable path before running `docs:build`. The Python dependencies are build-time only, not required to read the site or PDF.

`npm run build` checks that the generated documentation matches the selected source files. After changing those sources, regenerate the guide. The site and PDF are copied from `frontend/public/help` to `frontend/dist/help` by Vite and included in the normal Studio distribution. Rebuild the frontend and installer to deliver updated documentation to installed machines. Keep the whole help directory together when copying it for offline use; opening `index.html` directly also works without a server.

The guide is local to the Studio host. Making it reachable on a shared enterprise URL requires publishing these static assets through your approved web server and access controls; this change does not create a public internet endpoint.

Reference source: docs/DOCUMENTATION.md

XML JSON and Data parse and render activities

Data

Guide

XML JSON and Data parse and render activities

All six Parse/Render XML, JSON and Data activities use one implementation in Studio, Direct Python archives and Engine Python archives. Configuration contains design selectors and checkboxes. Map documents, paths, encodings, delimiters, fixed offsets, root names, JSON formatting and other runtime values in Input -> formatOptions. A mapped option takes precedence over the checkbox, activity default or shared Data Format value. Existing text settings appear as editable Input mappings; Studio migrates them when you edit the mappings. Attach project or inline schemas in the Input/Output Editor.

XML

Parse XML accepts text, base64 binary or dynamic input. Map binary input to `xmlBinary.bytes` and optionally supply `xmlBinary.forceEncoding` and `xmlBinary.byteCount`. Without an override, Parse XML uses the byte order mark or XML declaration. Attributes use `@name`, schema-defined repeating elements are arrays even with one occurrence, and ordered mixed text and child elements use `#content`.

Render XML accepts a parsed wrapper or a mapped schema tree. In Configuration, choose text or binary and set the default validation, declaration, indentation and namespace options. In Input, map `byteEncoding` for binary output and `formatOptions.cdataElements` for CDATA. The CDATA picker can insert paths from the selected input schema. Binary output includes `xmlBytes`, plus the existing MINA `binaryContent`, `bytes`, `byteCount` and `encoding` fields.

Validate parsed output and Validate input tree perform XSD validation, including required children, attributes and datatypes. Select the entry XSD as the activity schema. Keep its `xs:include` and `xs:import` dependencies as project schema assets with the relative filenames used by `schemaLocation`. An import without a location resolves only when there is one project XSD for that namespace. The schema tree and both Python archives use the same project assets.

The resolver is offline: URLs, local machine paths, traversal outside the project, DTDs, entities, XSD redefine and override are rejected. XML input is limited to 64 MiB; each XSD to 8 MiB and a dependency graph to 32 MiB. Namespace URIs and mixed text are preserved structurally; serialized prefixes, whitespace and lexical formatting need not match the original bytes.

JSON

Select the conversion mode under Configuration -> Schema type:

- Native JSON is the existing MINA mode. It parses and renders objects, arrays and primitive values directly. It supports offline JSON Schema validation with local references, and can omit null object fields without changing array positions.

- Generic converts between a JSON string and an XML string without a schema. With Badgerfish selected, namespaces use `@xmlns`, attributes use `@name` and text uses `$`. Generic Render JSON treats XML text as strings unless Infer numeric and boolean values is enabled.

- XSD converts between JSON and a schema-defined XML tree. Select an XSD in Parse JSON's Output Editor or Render JSON's Input Editor. Validation is on by default. Ignore additional JSON fields filters unknown input fields when validation is enabled. An array declared by the XSD remains an array even when only one value is present.

New JSON activities default to XSD. Existing activities without a saved mode continue to use Native JSON. The JSON root styles are None, JSON with Root and Anonymous Array. For Anonymous Array in XSD mode, the root must have one repeating child. Use null for empty values represents empty containers as nil when there is an XML container. Parse JSON rejects malformed JSON and non-finite numbers; its duplicate-key policy supports First wins, Last wins and Error. Render JSON can format the result and exposes the serialized output as `jsonString`.

Delimited and fixed-width Data

Parse Data accepts a string or runtime file path with a selected encoding. Map `text` or `fileName` in Input. Use `skipHeaderCharacters` to bypass a preamble, `noOfRecords` to limit a batch (`-1` means all records), and `startRecord` for a zero-based manual offset. If you leave the offset unmapped, each batch advances the same activity's cursor within that task execution. Use `done` to exit a loop and `nextRecord` when supplying your own offset. Continue on record errors

returns invalid field values or column counts in `ErrorRows.error`; an invalid document structure, such as an unterminatedquoted field, still fails.

Configure the format inline or add a JSON schema asset under Project Explorer -> Schemas with a `dataFormat` object. Select it as Shared Data Format on both activities. For example:

```
{"dataFormat":{"format":"delimited","delimiter":",","header":true,"fields":"id,name","fieldTypes":"integer,string","strictColumns":true}}
```

An optional `schemaId` links the shared format to a project row schema. Parse Data exposes typed `records`, `Rows.schema`, record counts and the XML representation. Render Data accepts mapped records and exposes the rendered value as `text`, `Text` and `content`.

To reuse an existing layout, right-click Schemas and choose Import XSD or Data Format. Select a Data Format XML resource (`.dataformat`, `.dataformatresource` or `.dataformatsharedresource`) or a ZIP containing it and its companion XSDs. ZIP entries retain their relative paths. The importer converts delimited and fixed layouts into reusable MINA assets and links supported Parse/Render Data references in imported processes.

A source module property bound to the Data Format column separator becomes a MINA environment property and an editable Input -> formatOptions -> Column separator mapping. Import the ZIP with its `default.substvar` and, if available, `local.substvar`, `dev.substvar`, `qa.substvar`, `pre.substvar` or `production.substvar`. A profile value overrides the default, which falls back to the resource's literal separator if present. If no value is available, set the imported property before running. Select the imported Data Format on both activities to add its mapping without overwriting custom mappings. Unsupported substitution targets and ambiguous or secret profile values are reported during import.

Quoted CSV values retain delimiters, escaped quotes and embedded line breaks. Literal multi-character delimiters and any-character separator sets also support quoting. Render Data exposes both `content` and `text` for mapping into HTTP bodies, files or subsequent activities. Headers, line endings and final separators are configurable.

Fixed-width records use positive widths or zero-based `start:end` offsets (end excluded), so columns can have gaps. The optional fixed line length includes the separator. Numeric fields use left padding and preserve significant trailing zeroes. Strict mode rejects over-width values; permissive mode retains truncation. Single-character delimited and fixed-width files are read record by record; the multi-character delimiter reader has a 64 MiB limit. Use Mapper for hierarchy and complex transformations.

Dynamic HTTP and REST invocation settings

HTTP Request and REST Invoke Configuration keeps the display name and shared connection selector. Request-specific settings are mapped in Input -> RestInputRequest:

```
RestInputRequest
 Config
 host, port, scheme, requestURI, Method, RequestBody, bodyFormat
 QueryParameters, URIParameters, QueryString, FilePath
 timeout, activityTimeoutSeconds, followRedirects, raiseForStatus
 openApiDocument, operationId, openApiBaseUrl
 openApiReferenceDocuments, openApiAllowedRefOrigins
 validateRequest, requestSchema, validateResponse, responseSchema
 Headers
 accept, content-type, Accept-Charset, Accept-Encoding, Cookie, pragma
 CustomHeaders[] -> name, value
 mimeEnvelopeElement
 mimePart[] -> mimeHeaders, binaryContent | textContent | fileName
```

Map constants, properties, upstream values or functions. String literals require quotes; booleans use `true`/`false`. Contract/schema fields accept JSON objects or quoted JSON/YAML/XSD text as appropriate. Operation IDs select the contract operation, whose method/path are authoritative. Empty custom schema overrides use the contract.

Previously saved Configuration values appear as typed Input mappings without a manual migration. Listener routing, server connections and security policies remain startup settings because they must exist before a request arrives. Keep external-reference origins restricted to trusted values; empty disables network reference retrieval.

Verification

The tests cover binary encoding overrides, CDATA, XML namespaces, XSD import/include resolution and access boundaries, Generic and XSD JSON conversion, Data Format import, dynamic Input mappings, rejected records, batching, multiline CSV and fixed offsets. All six activities run in fresh Direct and Engine archives and are compared with Studio Run and Debug.

Imported processes

Import a compatible process ZIP from Studio. MINA can retain XML and JSON Parse/Render steps and their original XSLT input bindings. Selected XSD elements are resolved by namespace and local name, including schemas embedded in the process. When a connector blocks full process conversion, the import can recover supported Mapper and format activities as separate subtasks. Supply each recovered activity's original input variables by name. Studio reports unbound Parse Data resources and empty input bindings.

From `backend`, with its Python environment:

```
python -m unittest tests.test_format_activities tests.test_http_dynamic_contracts
tests.test_ai_exception_data tests.test_raw_python tests.test_engine_export tests.test_activity_packs
-q
```

References: [Python CSV newline handling](https://docs.python.org/3/library/csv.html) and [lxml parsing and validation](https://lxml.de/apidoc/lxml.etree.html).

Selecting the XSD entry structure

When an XSD contains several global elements or named complex types, choose the required Structure beside the schema selector. This choice controls the activity's Input/Output tree, XML root, schema-derived record fields and runtime validation. Other declarations stay available as dependencies and as choices; they are not shown as additional mapping roots. The selected entry is retained in Run, Debug and both Python export formats. Imported schema text is hidden; inline source editing is available explicitly through Edit inline schema.

Reference source: docs/PARSE_RENDER_ACTIVITIES.md

Transformation and mediation capabilities

Mapping

Guide

Transformation and mediation capabilities

The Mapper remains the transformation activity. General activities provide durable message composition and recovery without reintroducing Mediation.

Mapper schemas and suggestions

Select one XSD structure

After choosing a project XSD, use Target structure to select one global element or named complex type. The Input and Output trees show only that entry and its descendants. A named complex type receives a document wrapper named after the type; choose an existing global element when its published XML root name is required. Source schemas and named source views have the same selection.

This selection is also available for task input/return interfaces (including Call Sub Task), Parse/Render XML, JSON and Data contracts, and HTTP/REST request and response schemas. Each activity retains its own choice when saved, reopened, run, debugged or exported as either Python archive format.

Selected project schemas display their filename and structure controls instead of a populated XSD textbox. Edit inline schema explicitly opens inline content editing. Opening an existing schema resource starts on Schema Structure; XSD Source remains available for deliberate source editing.

MINA retains the original declarations for changing selections and resolving references. Its selection metadata is a namespaced XSD annotation attribute; validation compiles the selected declaration and its dependencies. Unselected entry roots are excluded from the mapping tree and generated API schema. Missing types used by the selected structure remain validation errors.

The Mapper supports multiple named views of one source, each with its own schema, optional XSD complex type, and occurrence range. Keep schema files at their project-relative paths when importing or editing the contract. The Mapper resolves imports and includes from project assets. A repeating element stays an array in the structured result even when the generated XML has one occurrence; the XML itself retains the original element structure.

In Visual AI Mapper -> AI mapping options, choose Exact names or a similarity strategy. Select a source or target branch to limit matching to that branch, then set each side's search depth from 1 to 10 (default 4). Leave a side on Entire schema to search all fields on that side. Existing manual mappings are preserved.

Drop a repeating structure

On Mapper Input, drag a repeating complex source onto a repeating complex target and choose Finish after selecting a mapping mode:

- Make a copy of each copies the whole collection. The target shows one [Copy-Of] row with its source expression and hides the children. Existing child mappings in that occurrence are replaced. Clear the copy to show the children again. Use this when the source items already match the target structure.

- For each adds a [For-Each] row for the collection and keeps the item and its children visible. Matching paths are filled in without regard to case. Children display expressions relative to the current item, such as `Name` or `Address.City`, and remain editable.

- SmartMap builds an editable loop and also matches child paths that differ by underscores or hyphens. Ambiguous matches are left unmapped.

The same three choices are available on other activity Input tabs and in Edit mappings when both structures expose repeating records. This includes Call Sub Task inputs and End return mappings when the task has a schema. Drops onto simple fields still map the value directly.

To add another source for a repeating target, right-click the row and choose Duplicate mapping. This works in Mapper and other activity Input editors, including Call Sub Task and End. You can duplicate an unmapped repeating field too. The new occurrence starts with empty bindings. Drop a different collection onto it and choose a mapping mode for complex records. The original keeps its source and child bindings. At runtime the original items come first, followed by the configured duplicates in order. An occurrence with no source adds no items.

Right-click a complex element and choose Delete mapping to clear its binding and all mappings below it, even when only its children are mapped. Nested repeating occurrences within that branch are cleared too. The schema structure stays available for remapping. This works in Mapper and other activity Input editors, including HTTP responses, Call Sub Task and End. Deleting a duplicate occurrence keeps the original and sibling occurrences unchanged.

Use Map & Test to check the generated result before running or exporting the flow.

XML expressions and bindings

In Mapper Configuration, choose the XPath version and expand XML expressions and runtime binding. Set the XML source to a mapping path such as `${last.xmlString}`, or leave it empty to use the previous activity's XML output. Add namespace prefixes and variables as JSON objects. For example, `{"ord":"urn:orders"}` binds a namespace and `{"minimum":"${input.minimum}"}` binds a value to `$minimum`.

In Input, enter an expression such as `xpath('string(/ord:orders/ord:order[@id=$minimum]/@id)')`. The XPath expression builder can test a raw formula against sample XML or a JSON envelope and apply it to the selected target. Choose a function to display its arguments. Drag a sample XML field into an argument, or select the field and use the argument's button. In XPath 2.0, the target cast action wraps the current formula in the target's supported XML Schema constructor. XPath node selections return sequences. Use an XPath scalar function when the target expects one value. Map & Test accepts raw XML or a JSON input envelope. XPath resource loading is disabled. XPath 2.0 regex functions run in an isolated worker with a deadline.

Import an `.xsl` or `.xslt` file or edit the binding directly to select manual XSLT. A manual XSLT 1.0 or 2.0 binding publishes `xmlString` and structured XML values. Select a target XSD to validate the generated XML. Referenced XSDs must be present in project schemas under their declared locations. Stylesheets run with file and network access disabled. XML inputs, stylesheets, and parsed results are limited to 8 MiB per document. XSLT 2.0 runs in a separate SaxonC worker with a scrubbed environment and a deadline. The configured XML variables also supply stylesheet parameters. XSLT 2.0 accepts XML-valued parameters as document nodes.

In Configuration, use Source type overrides to coerce specific source paths before formulas and XML parameter binding. For example, `{"input.order.amount":"decimal"}` converts the input amount to a number.

In Input -> Source coercions and cardinality, create named views of the same source path. Set optional, exactly one, repeating, or at least one. Select a project schema to copy its contract into the view, or enter an inline contract. For XML, a named complex type must be valid for the source element's declaration. For JSON, the selected contract validates each occurrence. Choose a view or one of its fields from Data Sources. Occurrence ranges and type validation apply when a rule reads that view. Source views run in visual mapping mode.

In the XPath builder, use Check XPath 1.0 quick fix or Check XPath 2.0 quick fix to compare the two evaluations. Apply a passing fix to the selected visual mapping. Other mappings keep their existing XPath version. A matching sample result does not establish equivalence for every possible input.

In Input -> Open visual AI Mapper, use Download XSLT to save the active stylesheet or generate XSLT 2.0 from direct rules. Generation supports XML element paths, scalar constants, If, Choose, For-Each, For-Each-Group, nested loops, and duplicate loop occurrences with mapped children. It supports trim, normalize space, and case conversion. A selected XSD must have one root and a uniform element qualification policy. Supply an XML sample with the named document root for testing. Use Open XSLT to restore the editable mappings from a generated file. Use generated XSLT binds execution to the current visual rules. Generated XML binding preferences controls optional field existence checks, suppression when a nil source maps to an optional non-nillable target, and copying `xsi:nil` to nillable targets. Generated conditions accept XML field paths, scalar literals, comparisons, and logical operators. Custom functions and mapping statements outside this subset require a handwritten stylesheet.

The XPath builder provides extension functions such as `ext:trim`, `upper-case`, `lower-case`, `left`, `right`, `concat-sequence`, `xor`, `substring-before-last`, `substring-after-last`, `timestamp`, `base64-to-hex`, and `hex-to-base64`. `project:getModuleProperty` reads scalar MINA project properties supplied to the execution. Java system properties,

proprietary extensions, and custom Java XPath functions require a separate migration.

Legacy process import

Open a compatible process file or ZIP through Studio's project import. Supported sequences, simple scopes, conditional XPath links, no-match links, error links, static subtask calls, Mapper bindings, XML render, file activities, and basic outbound HTTP requests become MINA activities. Supported HTTP, JMS and Timer event definitions become native starters. HTTP XML reply bindings become response activities. The importer also reads supported HTTP client and server settings, non-XA JDBC resources and static SQL activities, and JMS text-message activities with direct or JNDI connections. Passwords are represented by empty deployment properties. Project XSDs are included. Check the import messages in Studio for processes and settings that need review. Unsupported processes can contribute standalone Mapper tasks for their recoverable bindings. Supply original activity variables as task input for these standalone tasks. REST/SOAP service descriptors, explicit BPEL join expressions, XA transactions, vendor-specific JDBC/JMS features, and proprietary transport policies still need manual conversion.

Shared Smart Mapper learning

Use Share reviewed mappings in Visual AI Mapper after reviewing direct field mappings. The Studio server stores field paths and schema fingerprints. Recommendations for the same source and target schema rank shared field pairs first. Disabled rules, pending suggestions, formulas, and literal values are excluded from learning. The catalog is stored in the Studio server's data directory and is available to Studio users connected to that server.

Both Python archive formats include `requirements-mapper.txt` when they use XML bindings. Install those dependencies in the target runtime before running the archive.

Mapper tests, templates and contracts

Open Map & Test and expand Saved transformation tests. Give each case a name, an input JSON value and an independently reviewed expected JSON value. Save the case and run the suite. Failures show field paths, expected/actual values and contract errors in dark red. Tests remain in the activity config and survive project save/export/import. They do not call connectors.

Versioned templates and schema changes exports a `.mtemplate.json` file containing schemas, mapping rules, lookup tables, execution policies, custom functions and test cases. Templates have a format version and content checksum; the checksum detects corruption, not authenticity. Import a template into another Mapper. Project schema IDs are cleared because IDs are project-specific. Local history retains the last 20 exported versions; changed content requires a new version. Restore a version, review its contract and rerun its tests.

Capture a contract baseline before changing the selected schema. Compare the current contract to see added/removed/changed fields, affected target mappings and raw contract changes. This reports changes; it is not a full backward compatibility proof for every schema dialect.

The portable validator now checks JSON `pattern`, `patternProperties`, common formats, conditional contracts (`if`/`then`/`else`), dependent required fields, property names and `contains` counts, in addition to the previous checks. Supported asserted formats are date, date-time, time, IPv4, IPv6, UUID, email, hostname and URI; unknown formats remain annotations. Pattern matching has a 100 ms timeout and requires the `regex` package, declared in deployment qualification when a schema uses patterns. XSD restrictions support enumeration, length bounds, numeric bounds and digit constraints. XSD patterns use the portable regex dialect with full matching; XML Schema-specific regex syntax and complete JSON Schema/XSD conformance are not claimed.

Lookups, enrichment and joins

Save named JSON lookup tables under Lookup tables and execution limits:

```
{"countries": {"US": "United States", "IN": "India"}}
```

Use `lookupTable(${input.country}, "countries", "Unknown")` in Mapper, or append a `lookupTable("countries", "Unknown")` pipeline step. Keys are case-sensitive. `lookup(value, table, default)` accepts an object table directly; `enrich(record, fields)` merges objects with enrichment fields taking precedence; `joinBy(left, right, leftKey, rightKey)` produces matching merged records; `chunks(records, size)` splits an array into batches. Missing join keys do not match. Many-to-many joins stop at 100,000 result records.

Large data and cancellation

Mapper has evaluation-step, execution-time and output-size limits. Defaults are 100,000 evaluation steps and 30 seconds. Execution runs in a worker thread; workflow cancellation signals the evaluator at subsequent evaluation checks. These are cooperative limits, not preemption of every individual function or vendor call. Input has target-field search and collapse/expand controls, so users can focus on a small portion of a large schema without losing mappings.

Both Python archive formats include:

```
python run_tests.py
Get-Content records.jsonl | python transform_jsonl.py --activity main/map --batch-size 100
```

`run_tests.py` runs saved suites and exits nonzero when any case fails or no saved suites exist. `transform_jsonl.py` processes one JSON record per line and prints one mapped JSON record per line. It runs Mapper alone, without connectors. Records are limited to 1 MB; batches default to 16 MB of serialized input/output and mapped records default to a 1 MB output limit. Use a smaller batch if the batch quota is exceeded. Output written before an error remains written; stage the output before publishing if all-or-nothing delivery is needed.

Durable message activities

Use an explicit Message namespace for each integration and the same Durable state file for cooperating activities. `MINA_MESSAGE_STATE_FILE` can provide the file globally; the default is `message-state.sqlite3` under `MINA_DATA_DIR` or the user's `.mina` folder. SQLite transactions coordinate processes sharing the same local file. For containers, mount durable storage. This is not a distributed database for multiple hosts or network filesystems. Payloads are capped at 1 MB, aggregate groups at 64 MB and state capacity defaults to 256 MB. Size accounting is an approximate SQLite page budget.

- Deduplicate Message claims a stable message key and returns `accepted`

and `claimToken`. Route on `accepted`: process accepted messages, skip or acknowledge duplicate deliveries according to the connector policy.

- Message Receipt completes the exact claim token after downstream success

or marks it failed on an error path. Stale/expired tokens cannot complete a newer claim. Acknowledge the broker only after successful work and receipt.

- Replay Messages lists failed/expired candidates. `requeue` makes one

explicit key claimable again; it does not execute downstream activities. Feed the returned stored payload back through the normal task deliberately. `purge` removes expired completed keys and old completed aggregates; it keeps failed and pending messages. Completed keys default to seven-day retention.

- Aggregate Messages appends messages with a stable message ID and

correlation key. Duplicate IDs do not increase the group count. When the expected count is reached, `ready=true` returns the records once; subsequent deliveries do not emit again. Check `ready` before forwarding. After a timeout, `flush` explicitly emits a partial group. A completed group needs a new correlation/window key for the next aggregation.

- Split Records creates record batches from an array for use with the

existing For Each group. It is an in-memory activity; use the JSONL tool for streaming file transformations.

These primitives support reliable processing but cannot guarantee exactly-once external side effects. A crash between an external write and its receipt may require replay; make the downstream write idempotent using the stable key. Live broker/provider qualification remains necessary for production deployment.

Visual mapper and generated previews

Open Visual AI Mapper from Mapper Configuration. The left pane contains editable source test data and available source paths, the middle pane contains saved mappings and the complete selected target schema field list, and the right pane displays generated output after Run mapping test. Named XSD types and local JSON Schema references use the same field parser as the Input tab. Source expressions remain editable, including upstream activity references and functions. Schema text is available under the collapsed schema sections.

Map & Test also displays generated output directly after running. JSON targets show formatted JSON; XSD targets show an XML preview with repeating elements and the target namespace. The preview does not change the Mapper runtime object or exported archive behavior; use Render XML or render-xml when the downstream activity requires a serialized XML value. Test data and the latest output are retained when saving the visual mapper.

Reference source: docs/TRANSFORMATION_CAPABILITIES.md

Activity debugging and testing

Studio

Guide

Activity debugging and testing

Job Data tree

The ribbon's Job Data window shows processes, call activities, and nested subtask activities in an expandable tree (initially expanded). Select a process or activity to inspect its captured input and output. Data is keyed by process and activity, not by a flattened activity ID. Completed calls record the actual called task, including dynamically resolved calls. Unexecuted activities are labelled as not captured. Repeated invocations show the latest captured values for that task/activity; this is not a per-invocation execution-history viewer.

Windows installations must be rebuilt and updated with the new installer to receive these UI/backend changes; restarting an older installation is insufficient.

During an active debug session, a dedicated Debug ribbon appears underneath the Studio ribbon. It provides Continue, Pause, Stop, Step In/Over/Out, Jump In/Out, Run to Selected, Breakpoints, Watches, Variables, Evaluate, Job Data, Edit Payload, Preview Inputs, Test Activity, and Mock Output. Data/tool buttons open the relevant editor section; real tests and mocks still require explicit submission there. Commands are enabled according to session state. The ribbon disappears when the session completes, fails, or stops; it is not shown during ordinary Run mode.

Choose Run -> Start Activity Testing to create a paused debug session and open the test bench. Unlike ordinary Debug, this does not automatically run the flow or start event listeners. The existing Debug Tools window also contains the bench.

Use the searchable process/activity tree on the left of the test bench. Expand a process or subprocess and select its activity; enter the input in the right pane. Each target retains its own input draft while the dialog remains open. Selecting a Call Sub Task row selects the call activity itself; expanding its arrow reveals the called subprocess and its activities. Nested calls can be expanded the same way, and selecting a nested activity targets its owning subprocess. Search includes nested activities. Circular references stop with a notice. Dynamic calls show a configured fallback, if any, labelled as potentially different from the runtime target; unresolved targets are not guessed.

Selecting a process targets its unique entry activity (or Start); Run once still executes only that entry activity, not the entire process. If the entry is ambiguous, select an activity explicitly. Independent tests can target another process in the debug project without moving the paused flow. Mock stepping is restricted to the current paused process and activity.

Select an activity and enter a JSON payload (including arrays, strings, numbers, or null), or choose raw text/XML. The test payload is available as both `input` and `last` for this independent test. Earlier activity outputs and environment properties remain available for mappings.

- Preview inputs resolves configuration and mappings without executing the activity.

- Run once (real) executes the selected activity without advancing the paused

flow. Inspect resolved inputs, output, logs, duration, and failure details.

- Use mock output & step supplies the payload as the current paused activity's

output, skips its actual connector execution, and follows its normal transitions. Use this to simulate a receiver or downstream dependency without contacting it.

- Optional field overrides are a JSON object keyed by the activity's configuration

field names, such as `message` or `topic`. They are applied after mappings as literal values. Preview first to verify the inputs.

- Optional assertions use runtime conditions, for example `${last.count} > 0`.

Results distinguish execution failures from assertion failures.

The variable editor accepts whole `input`, `last`, and `vars` replacements, as well as nested paths such as `input.customer.id`. Replacing `input` does not also replace `last`; edit each explicitly when needed. `vars` must remain an

object.

Safety and scope

Real tests use the configured connections and can publish, acknowledge, write, or call external systems. Use test environments and credentials. They are not an external-side-effect sandbox or an automatic rollback mechanism. Independent tests do not inherit a paused group's database transaction or lock context; use normal flow stepping to test those lifecycles.

Single-activity tests default to a ten-second timeout. A timeout cannot guarantee cancellation of work already submitted to a provider or blocking vendor driver. Mocks bypass the activity, not the surrounding group's execution semantics. Input overrides apply to configuration-driven activity fields; special task or interface mapping behavior still follows the activity's runtime implementation.

The most recent full result and the last 30 result summaries are retained for the debug session. Test payloads and mock results do not modify or export into the project. This bench complements breakpoints, watches, expression evaluation, variable editing, and flow stepping; real-provider integration and load testing still require the target infrastructure.

Reference source: docs/DEBUG_TESTING.md

MINA Studio debugging

Studio

Guide

MINA Studio debugging

MINA Studio supports interactive debugging for ordinary tasks and continuous event starters.

Execution and logs pane

Click Run or Debug to open Execution / Debug automatically in the lower pane, replacing the activity Configuration and Input tabs. The canvas remains visible above it. Run logs update while the task executes; debug logs update as you step or continue. Listener endpoints and generated outputs appear beside the logs.

The pane stays open while Debug is paused or a receiver is listening. It closes when execution completes, fails, or is stopped, restoring the selected activity tab. Its splitter resizes the lower pane; the expand and full-screen controls provide more space. To inspect retained logs after a session ends, choose Window -> Execution & Debug.

Click an activity on the canvas to replace the execution pane with that activity's Configuration tab. Its Input and Output tabs remain available for inspecting the mappings and schemas. Debugging continues in its current running or paused state; activity selection does not stop or restart the session. Use Execution Logs on the debug ribbon, or Window -> Execution & Debug, to return to the live logs.

When running Studio in a browser from a source checkout, the runtime serves `frontend/dist`. After updating the source, run `npm.cmd run build` from the `frontend` directory, then reload Studio in the browser (Ctrl+Shift+R in Firefox). A build written to a separate test output directory does not update the browser-served Studio. Short tasks can finish and close the pane immediately; use Window -> Execution & Debug to review their retained logs, or set a breakpoint before debugging to keep the session paused.

Execution controls

- Continue runs until completion, an enabled breakpoint, a handled exception when pause-on-error is enabled, or a manual pause.

- Step In, Step Over, and Step Out navigate activities and Call Sub Task frames.

- Run to selected continues until the activity selected on the canvas is about to execute.

- Stop cancels the debug driver and listener, rolls back open transaction groups, and releases owned native processes.

Debug Tools

Open Debug Tools from the debug bar to use the following features:

- Add an optional condition to any canvas breakpoint. Conditions use the same expression syntax as transitions, for example `${last.amount} > 100` or `contains(${last.status}, "FAIL")`.

- Enable or disable pause-on-error for handled error and retry paths.

- Add watch expressions such as `${input.orderId}`, `${last.customer.id}`, or `${vars.retryCount}`.

- Inspect the current input, previous output, process variables, process context, call stack, group iterations, and last exception.

- Evaluate an expression without modifying the running task.

- While paused, change a value under `input.*`, `last.*`, or `vars.*`. JSON values are typed; other input is stored as text.

Environment properties and connector credentials are intentionally excluded from the live-variable editor.

Continuous starters

An EMS/JMS, Kafka, Pub/Sub, SAP, timer, or file listener displays ready · waiting for event until an event arrives. Breakpoints, watches, and pause-on-error remain active for every event. Stopping the session also stops the listener and prevents it from re-arming.

Reference source: docs/DEBUGGING.md

Execution analytics

Studio

Guide

Execution analytics

Open Run -> Execution Analytics, View -> Execution Analytics, or the analytics icon in the Execution / Debug panel.

Select the current debug execution or a completed run. The window provides:

- Invocation and failure counts, summed activity duration and completed-run wall time.

- Recent-run duration graph and slowest-activity bar graph.

- Ordered invocation history, including repeated activity executions.

- Per-activity total, mean, minimum, maximum and nearest-rank P95 durations.

- Search by activity, task or type; click an activity name to open its configuration.

- CSV timing summaries, JSON metadata and a standalone HTML graph report. Open the HTML report in a browser to print or save as PDF.

Reports omit activity inputs, outputs, credentials and configuration. Timings include connector waits and retries. Subprocess durations include child execution, so summed activity times can overlap; they are not process wall time. Debug timings exclude time spent paused at breakpoints. Mocked debug steps measure the mock execution rather than a real connector call.

Execution history is held in memory by the running Studio backend and retains up to 100 runs. A backend restart clears this history; export reports to retain them. Older runs captured before this feature do not contain detailed timing records. These reports describe the Studio runtime and debugger; they do not aggregate remote Control Plane agents.

Reference source: docs/execution-analytics.md

Data and file utilities

Studio

Guide

Data and file utilities

Open Window -> XML Viewer, Window -> JSON Viewer, or Window -> Compare Files. These tools use a dedicated Studio workspace above the orchestration editor, so a utility can remain open without changing the active Task.

Large-file behavior

The desktop application gives the UI an opaque file handle rather than unrestricted filesystem access. It reads only the selected 256 KB, 1 MB, or 4 MB window. Previous and Next move through the file without retaining earlier windows. Browser mode applies the same behavior with `Blob.slice()`.

Each active window is editable. Desktop saves stream the untouched prefix and suffix through a temporary file and then replace the original, so even a very large file is not assembled in renderer memory. Navigation is locked while edits are unsaved. Browser mode downloads the edited result because web pages cannot overwrite the selected local file directly.

The utilities do not require an existing file. New XML, New JSON, New left, and New right create scratch documents where content can be typed or pasted directly. Save As converts a scratch document into a filesystem-backed document. Ctrl+S saves the editor that currently has focus.

Complete XML or JSON files up to 16 MB can be validated and pretty-printed as a whole. Larger files use a tolerant pretty printer for the current bounded window, avoiding an unbounded parse tree or full-file string in renderer memory.

XML Viewer

- Opens XML, XSD, and WSDL files.

- Creates blank XML documents for direct typing or clipboard paste.

- Pretty-prints well-formed complete documents.

- Decode < and > changes `<` and `>` into visible angle brackets in the display.

- Wraps long or minified lines by default so payload content stays visible; wrapping can be toggled from the toolbar.

- Searches the current file window.

- Saves edits or formatting back to the selected desktop file and provides Revert for unsaved changes.

JSON Viewer

- Opens JSON and JSON Lines files.

- Creates blank JSON documents for direct typing or clipboard paste.

- Pretty-prints and validates complete JSON documents.

- Uses bounded-window formatting for very large payloads.

- Wraps long or minified lines by default.

- Searches the current file window.

- Saves edits or formatting back to the selected desktop file and provides Revert for unsaved changes.

Compare Files

- Accepts any two file types.

- Allows either side to be a newly created document instead of an imported file.

- Uses side-by-side aligned text rows for text files.

- Marks added, removed, and changed rows separately.

- Automatically displays binary or unknown content as hexadecimal and printable ASCII.

- Supports a differences-only view and synchronized window navigation.

- Allows individual difference rows-or all differences in the current window-to be selected and copied left-to-right or right-to-left.

- Provides a two-pane editing mode and independent Save left and Save right actions.

- Detects external file changes before saving and refuses to overwrite newer content.

- Renders at most 10,000 comparison rows per window to prevent unusually short-line files from creating an unbounded number of UI elements.

Reference source: docs/FILE_UTILITIES.md

Runtime logging

Operations

Guide

Runtime logging

Studio records Run, Debug, listener, job, activity, transition, retry, exception, and application lifecycle events in the Execution / Debug panel. The disk button in that panel reloads saved records after a run or after Studio restarts.

Each environment properties file contains the user-configurable property:

`runtime.logDirectory`

Set it independently in `local.properties`, `dev.properties`, `qa.properties`, `pre.properties`, and `production.properties`. Run, Debug, listeners, and Stop use the selected runtime environment and write beneath that environment's configured directory. For example, setting the development value to `D:\IntegrationLogs\dev` produces:

`D:\IntegrationLogs\dev\<project-name>\<project-name>.log`

Relative values are resolved beneath the Mina data directory. Environment variables and `~` are expanded. When the property is blank, each project has an isolated structured log at the default location:

`<Mina data directory>/logs/<project-name>/<project-name>.log`

Log and audit timestamps use Arizona time by default (`America/Phoenix`, shown with the `-07:00` offset). Set `MINA_LOG_TIMEZONE` to another IANA timezone when a deployment requires a different display timezone. Scheduler and token expiry calculations continue to use UTC internally.

The active file automatically rolls at 10 MB. Four numbered archives are retained by default (`<project-name>.log.1` through `<project-name>.log.4`), so an individual file never grows without bound. Each line is UTF-8 JSON and includes the project, timestamp, level, message, and available run, correlation, task, activity, duration, and exception context. Failed package builds also write a `packaging` record with a traceback to this log, and the API returns the log path with the error. The exact resolved path is shown in Studio's Execution / Debug panel after clicking the disk button.

Packaged desktop installations place the Mina data directory under the Studio user-data location configured through `MINA_DATA_DIR`. Administrators may override logging without changing code:

- `MINA_RUNTIME_LOG_DIR`: process-wide fallback root when the active environment's `runtime.logDirectory` property is blank.

- `MINA_PROJECT_LOG_MAX_BYTES`: maximum bytes per active or archived file; default `10485760` (10 MB).

- `MINA_PROJECT_LOG_BACKUP_COUNT`: number of rolling archives; default `4`.

Payload bodies are written only when the activity's Automatic payload logging option is enabled. Passwords and secrets should never be deliberately mapped into log messages.

Reference source: docs/RUNTIME_LOGGING.md

MINA Control Plane and Data Plane: one-stop guide

Operations

Guide

MINA Control Plane and Data Plane: one-stop guide

This is the authoritative setup and operations entry point. Use one program, `scripts/linux/mina-setup.py`, with separate INI files for the Control Plane and each Data Plane. The older shell entry points are compatibility aliases. Do not combine instructions from older layouts. No systemd or systemctl is required.

1. Provision the Linux hosts

Ask the Unix team to provide an installation directory owned by the account that will run MINA, for example `/opt/mina`. Run the installation and processes as that account. Each host needs Linux, Python 3.12+ with venv/pip, Bash and network access to the approved Python package mirror, or a complete Linux wheelhouse. `curl` is useful for manual health checks. The Control Plane source build additionally uses PyInstaller, installed by the build script. Build and deployment must use the same Linux architecture and a compatible Linux distribution/glibc.

Data Planes need the Control Plane URL and owner credential for registration and the current remote-agent API. Use an internal HTTPS reverse proxy for remote access; the Control Plane listener itself is HTTP. Allow access to the configured Control Plane port/proxy and to the application connector endpoints. The setup script does not install OS packages, configure firewalls or create a TLS proxy.

For Java/JDBC/SAP/JMS connectors, provide JDK 17+ and licensed vendor JARs plus Linux native libraries where required. Windows `.dll` files and Windows virtual environments are not Linux runtimes. Set `[runtime] build_java_bridge=true` to compile the Java bridge automatically. Native ODBC, AMQP 1.0 and optional DB2 prerequisites must be supplied on hosts whose applications use them.

2. Prepare the software on Windows

From the repository root in PowerShell:

```
cd D:\Integration-tool\IntegrationFabric
py -3 scripts/linux/mina-setup.py bundle --output MinaLinuxSetup.zip
scp .\MinaLinuxSetup.zip minauser@control-plane-host:/tmp/
```

The bundle contains `administrator/`, `backend/`, `scripts/`, Java bridge source, `drivers/` when present, the two INI templates and this guide under `software/`. It excludes local databases, logs, virtual environments, caches, Windows build outputs and release archives. Review vendor driver files before transfer. The bundle sets shell scripts to LF and removes UTF-8 BOMs; the Linux entry also normalizes copied shell scripts automatically. No `dos2unix`, script editing or `chmod +x` step is required to invoke the setup entry or its subscripts.

The Linux Administrator executable is built on Linux. A Windows executable cannot be used. To use a prebuilt Linux release instead, copy the entire `MinaAdministrator-<version>-Linux-x64.tar.gz` separately and set its absolute path in `[setup] administrator_archive`; set `build_administrator=false`.

3. Extract and configure the Control Plane

On the Control Plane host, as the MINA account:

```
mkdir -p /opt/mina
python3.12 -m zipfile -e /tmp/MinaLinuxSetup.zip /opt/mina
cd /opt/mina/software
umask 077
mkdir -p /opt/mina/config
cp scripts/linux/mina-control-plane.ini /opt/mina/config/control-plane.ini
```

Edit only `/opt/mina/config/control-plane.ini`:

- `install_root=/opt/mina`, `source_root=/opt/mina/software` and the available

Python command/path under `[setup]`.

- `version` for the Linux Administrator build/release.

- `[control-plane] host` is a bind address; `0.0.0.0` binds all interfaces. Set

`port`, a strong `api_key`, and a separate stable `secret_key` for encryption. `CHANGE_ME` values are rejected. Preserve the encryption key across upgrades.

- Set `pip_index_url` to the approved package mirror, or `wheelhouse` to an

absolute directory with all required Linux wheels and build dependencies. With a wheelhouse, pip uses `--no-index`. Blank values use the configured pip defaults. The INI contains plain values without shell quoting or interpolation: `#`, `%`, `$` and `;` inside a value are preserved. Put comments on separate lines.

- `start=true` starts and health-checks the server after installation;

`start=false` installs it without starting it.

- Keep `install_local_runtime=false` when applications run on separate Data Planes.

Enable it only when the Control Plane host must also run local applications.

```
chmod 600 /opt/mina/config/control-plane.ini
python3.12 scripts/linux/mina-setup.py control-plane --config /opt/mina/config/control-plane.ini
--check
python3.12 scripts/linux/mina-setup.py control-plane --config /opt/mina/config/control-plane.ini
```

Execute from `/opt/mina/software` as shown. Absolute script/config paths also work from another directory. The default action is `setup`: it builds or reads the Linux archive, installs the complete bundle, retains existing Control Plane data, copies the private INI, starts the listener and waits for `/api/health`. Data Plane registration is a separate step. Do not append `&` to setup: it already starts the persistent process in the background and reports failures. Open the configured reverse-proxy URL, or `http://<control-plane-host>:19080/` on the permitted internal network, to use the Control Plane browser UI.

Default layout:

```
/opt/mina/
 software/ copied source and setup scripts
 config/ private operator-maintained INI files
 control-plane/ Linux executable, internal libraries, installed INI
 control-plane-data/ repository, encrypted secrets and audit state
 runtime/ Data Plane Python venv and runtime adapter (when installed)
 agent/<plane-id>/ isolated packages, agent configuration, PID and logs
 drivers/ operator-supplied Linux vendor drivers
 logs/control-plane/ administrator.log
 logs/runtime/ runtime logs
 run/ Control Plane PID
```

Optional INI keys `home`, `data_dir`, `log_dir`, `pid_dir`, and `runtime_command` under `[control-plane]`, and `data_dir`, `log_dir`, `driver_home`, `java_bridge_home`, `java`, `build_java_bridge`, `install_db2` under `[runtime]` allow explicit paths/options. Use absolute Linux paths. `runtime_command` accepts the existing application/environment placeholders; leave it absent for the generated local adapter. With separate Data Planes, the agent launches applications on its own host, not through the Control Plane's local runtime command.

4. Set up each Data Plane separately

Copy the same software ZIP to the Data Plane host and extract it as in step 3. On that host:

```
cd /opt/mina/software
umask 077
mkdir -p /opt/mina/config
cp scripts/linux/mina-data-plane.ini /opt/mina/config/customer-plane.ini
```

Edit the new INI:

- `[setup] install_root`, `source_root`, `python`, mirror/wheelhouse and `start`.

- `[control-plane] control_plane_url` is the externally reachable URL (HTTP or

HTTPS), `admin_key` matches the Control Plane owner key, and optional `ca_bundle` points to a PEM trust bundle for an internal HTTPS CA. TLS verification remains enabled. Include required CA certificates; never use a verification bypass.

- `[data-plane] id`, `name`, `host`, `namespace`, `region`, `available_capacity`,

`heartbeat_seconds`, `agent_version` and `capability_name`. IDs and namespaces use letters, digits, underscores and hyphens. `host` identifies the runtime host in inventory; the setup must be executed on that host.

- Leave `state_dir` empty to derive `/opt/mina/agent/<id>`. Different planes must

not share state directories. Applications, PIDs and logs are isolated by plane.

- Configure the vendor driver directory and Java bridge options under `[runtime]`.

Copy the Linux vendor libraries into the configured driver directory before starting connector applications. `install_db2=false` omits optional `ibm-db`; set true only when the required DB2 build/runtime prerequisites are available.

- Team/user creation is optional: leave their `id` fields empty to manage them

through the UI. When using `[delivery-team]`, set its `id` and `name`; an empty `scopes_json` adds this plane/namespace to existing scopes. An explicit JSON array replaces the team's full scope list. When using `[user]`, set a unique user ID, `team_id`, role and scope. Empty `resource_id` uses this namespace.

```
chmod 600 /opt/mina/config/customer-plane.ini
python3.12 scripts/linux/mina-setup.py data-plane --config /opt/mina/config/customer-plane.ini --check
python3.12 scripts/linux/mina-setup.py data-plane --config /opt/mina/config/customer-plane.ini
```

This installs the host runtime, registers/updates the Data Plane, provisions its Integration Runtime capability, optionally creates the team/user, and starts its reconciliation/heartbeat agent. It does not install another Control Plane. The agent downloads assigned application archives, starts/stops runtime workers, reports health and handles graceful shutdown. Inspect its agent log and the Control Plane Data Planes view to confirm ONLINE status; successful registration alone is not proof that application deployments are healthy.

For another plane, copy the INI to a different filename and change its ID, name, host and namespace. If `state_dir` is explicit, give it a unique path. Run the same command on that plane's host. Separate planes can share one installed runtime on the same Linux host; stop affected agents before upgrading that shared runtime.

5. Lifecycle commands and upgrades

From `/opt/mina/software`, use the same entry and same INI with `--action`:

```
python3.12 scripts/linux/mina-setup.py control-plane --config /opt/mina/config/control-plane.ini
--action status
python3.12 scripts/linux/mina-setup.py control-plane --config /opt/mina/config/control-plane.ini
--action stop
python3.12 scripts/linux/mina-setup.py control-plane --config /opt/mina/config/control-plane.ini
--action start
python3.12 scripts/linux/mina-setup.py data-plane --config /opt/mina/config/customer-plane.ini
--action status
python3.12 scripts/linux/mina-setup.py data-plane --config /opt/mina/config/customer-plane.ini
--action stop
python3.12 scripts/linux/mina-setup.py data-plane --config /opt/mina/config/customer-plane.ini
--action start
```

`restart` is available for both roles. To change installed software or INI values, stop the relevant process, back up state and the private INI/encryption key, replace the software tree, update the version/configuration and run `setup` again. Setup refuses to overwrite a running installation. Stops are graceful; Data Plane shutdown also stops managed runtime workers. The setup does not delete deployment state or generate a replacement encryption key. Keep the installation root and private configuration paths unchanged unless deliberately migrating the service.

`--check` validates the INI and source location without installing or starting anything; it does not prove that the network, OS libraries or package mirror are ready. API update errors other than 404 stop setup instead of creating duplicate resources. A failure leaves logs and completed setup steps available for diagnosis; correct the INI/prerequisite and rerun after stopping any affected process.

6. Troubleshooting and backup

- Control Plane: `/opt/mina/logs/control-plane/administrator.log` and

`--action status`; check configured listener/proxy and `/api/health`.

- Data Plane: `/opt/mina/agent/<id>/agent.log`, its application logs and

`--action status`. Agent PID checks identify the expected script/configuration rather than signalling an unrelated reused PID.

- 401/403: verify the owner credential and intended permissions; it is never

printed by the setup program. 400/409: inspect conflicting namespaces/scopes or immutable capability identifiers before rerunning.

- Mirror/offline failures: supply Linux wheels matching Python and architecture,

including PyInstaller dependencies for source builds. Windows wheels cannot replace them. The mirror must provide the versions in the shipped requirements.

- Vendor failures: verify Linux native library compatibility, JDK/bridge and

driver locations. OS prerequisites and licensed drivers are not downloaded.

- Protect INIs with mode 600. Back up `control-plane-data/`, the stable encryption

key, private INIs, Data Plane state and any required vendor libraries together. Restore the same key with the data; changing it makes existing secrets unreadable.

7. Compatibility entries

Use the Python entry for Windows-to-Linux transfers. `setup-mina-linux.sh` and `00-setup-and-register.sh` now delegate to Control Plane setup only. They read `MINA_CONFIG_FILE`; edit version and paths in the INI rather than supplying a positional version. `build-administrator.sh` delegates to the Linux build script. Numbered registration helpers and `deploy-application.sh` remain available for advanced operations. They are subscripts, not additional installation guides.

8. Control Plane model and operator reference

Purpose and operating model

MINA Control Plane is the self-hosted management plane for MINA data planes, capabilities, applications, resources, access assignments, observability, and immutable deployment packages. A data plane represents an on-premises runtime host or Kubernetes runtime boundary. Capabilities are provisioned into a data-plane namespace, while applications are deployed to a selected data plane, namespace, and compatible Integration Runtime capability.

MINA separates control and data planes and provides namespace-scoped capabilities and applications, platform resources, role assignments, health/heartbeat inventory, application lifecycle, audit, and observability dashboards.

Organization and delivery-team isolation

The built-in Technology Team is the permanent Control Plane owner. It registers data planes, provisions capabilities, configures platform resources, creates delivery teams, assigns principals, issues or revokes delivery credentials, and can govern all assets. Set `MINA_ADMIN_API_KEY` for its production credential.

Each data delivery team must be assigned one or more exclusive `{data plane, namespace}` scopes. The same namespace cannot be assigned to two active delivery teams. Packages, extracted package storage, deployments, environment requirements, encrypted secrets, runtime logs, lifecycle operations, and application observability carry an immutable `teamId`. Backend authorization returns `404` when another delivery team probes an asset identifier, preventing both access and asset discovery.

Technology Team can upload a package on behalf of a delivery team by selecting its name beside Upload package. Delivery automation uses a one-time credential issued from Delivery teams and sends it in `X-Control-Plane-Key`. Only a SHA-256 hash of that credential is stored. An Application Manager token can upload, deploy, manage secrets, run lifecycle operations, and inspect its own team assets. An Application Viewer token is read-only. Delivery credentials cannot access Control Plane overview, data-plane registration, capabilities, resources, team/access administration, global monitoring, or audit APIs.

For production, configure corporate identity-provider groups at the reverse proxy or identity gateway and exchange their authenticated team identity for scoped Control Plane credentials. Never expose the Technology Team key to delivery pipelines.

Control Plane does not rebuild a Studio project. A package already contains the selected starter tasks and every reachable called task, shared connection, XSD/JSON schema, resource, property profile, and deployment descriptor selected during packaging. Cloud packages keep their generated Docker/Kubernetes descriptors and can be assigned only to a Kubernetes data plane. On-premises packages can run locally through the configured command adapter.

Configuration

Variable	Default	Meaning
`MINA_ADMIN_HOST`	`0.0.0.0`	HTTP bind address
`MINA_ADMIN_PORT`	`9080`	HTTP port
`MINA_ADMIN_DATA_DIR`	`administrator/data`	Repository, state, encrypted secrets, logs, and audit location
`MINA_ADMIN_API_KEY`	empty	Owner credential sent in `X-Admin-Key` or `X-Control-Plane-Key`. Credential-free owner access is restricted to direct loopback clients and same-origin browser requests. Remote and reverse-proxy clients must supply an owner key or scoped team token.
`MINA_ADMIN_SECRET_KEY`	generated local key	Stable encryption passphrase; supply from a secret manager in clustered/production installs
`MINA_ADMIN_RUNTIME_COMMAND`	empty	Administrator-approved runtime command template
`MINA_ADMIN_MAX_PACKAGE_MB`	`250`	Maximum uploaded archive size
`MINA_ADMIN_MAX_EXPANDED_MB`	`1024`	Maximum expanded package size
`MINA_ADMIN_MAX_PACKAGE_FILES`	`10000`	Maximum archive members

The runtime command supports `{application}`, `{package}`, `{environment}`, `{deployment_id}`, and `{instance_id}` placeholders. The desktop installer includes a separate `MINAWorker` executable for this purpose. Example:

```
export MINA_ADMIN_RUNTIME_COMMAND='mina-runtime --application {application} --environment
{environment}'
```

On Windows desktop installations, configure the worker executable rather than the Studio sidecar:

```
$worker = 'C:\Program Files\MINA Studio\resources\runtime\MINAWorker\MINAWorker.exe'
$command = "`\"$worker`\" --application `\"{application}`\" --environment `\"{environment}`\""
[Environment]::SetEnvironmentVariable('MINA_ADMIN_RUNTIME_COMMAND', $command, 'Machine')
```

The runtime receives `MINA_APPLICATION_DIR`, `MINA_ENVIRONMENT`, `MINA_DEPLOYMENT_ID`, `MINA_INSTANCE_ID`, and decrypted deployment secret values in its process environment. The command comes only from trusted Administrator configuration; package contents cannot provide an executable command.

Package and deployment workflow

- In Studio Packaging, select the starter tasks, environments, target, archive format (`.mpkg`, `.zip`, `.tar.gz`, or `.ear`), and deployment files. Only selected starter tasks and their recursively called Sub Tasks are packaged. Legacy `.ifpkg` uploads remain supported during the compatibility period.

- Select Export archive for an offline bundle, or enter the Control Plane URL, credential, team, data plane, namespace, capability, deployment environment, and required secrets and select Deploy to Control Plane. Studio builds the archive once, uploads those exact bytes, creates the deployment, and reports the Control Plane deployment ID/state. Credentials and secret values are transient and are not saved in the project or package.

- Alternatively, upload an exported archive in Applications. Administrator rejects traversal paths, links/devices, duplicate paths, oversized expansion, unsupported formats, missing project/task artifacts, and invalid manifests.

- Review checksum, target, profiles, selected task metadata, and secret requirements.

- For local on-premises deployment, Start uses `MINA_ADMIN_RUNTIME_COMMAND`. Generated packages contain Linux `install.sh`/`deploy.sh` and Windows `install.ps1`/`deploy.ps1`/`start.ps1` assets. If the command adapter is not configured, startup fails visibly rather than reporting a false running state.

- For cloud deployment, the Control Plane records the desired application, environment, replicas, namespace, and capability. The registered Kubernetes data-plane agent applies the generated Dockerfile, ConfigMap, Secret, Deployment, Service, HPA, and Kustomize assets; Studio does not incorrectly invoke the local command adapter for cloud targets.

- Use Details for instance PIDs and logs. Stop performs normal termination; Kill is forced; Restart stops and recreates desired instances; Undeploy removes deployment secrets and the active inventory record.

For a private CA, keep TLS verification enabled and supply the CA PEM path in Studio. Disabling verification is provided only for isolated development systems.

Legal lifecycle transitions are enforced. Package deletion is blocked while any non-undeployed deployment references it.

Application health is evaluated per deployment. For the local command adapter, Control Plane verifies each managed runtime PID. Remote and Kubernetes agents report `HEALTHY`, `DEGRADED`, `UNHEALTHY`, or `UNKNOWN` in the `deploymentHealth` object of their data-plane heartbeat; Control Plane does not infer application health merely because the data plane is online. Health checks can be disabled per deployment.

Starter Task start/stop changes the deployment's desired starter set. A running local deployment is restarted with `MINA_ENABLED_STARTERS` containing only enabled task IDs. Remote and Kubernetes runtime agents reconcile the same desired state and may report their observed task state with the health heartbeat. Whole-application lifecycle state and individual Starter Task state remain separate.

Control-plane model and screens

- Overview: fleet, capability, deployment, request, and recent activity summaries.

- Data planes: local, remote on-premises, and Kubernetes registrations; namespace, capacity, tag, tunnel, heartbeat, and health inventory.

- Capabilities: namespace-scoped capability provisioning and status. Application deployment requires an Integration Runtime capability.

- Applications: searchable package/deployment repository; archive task and Starter Task inventory; health, instance details and logs; deploy, start, stop, configure, redeploy, undeploy, and safe package deletion; deployment and configuration revision history; individual Starter Task desired-state controls.

- Environments & secrets: exportable packaged profiles, JSON profile editing/upload, optional immediate redeployment, and required secret names. Password values are rejected from profile uploads because secret values remain encrypted and deployment-scoped.

- Observability: control-plane request/error totals plus data-plane and application CPU, memory, instance, and state telemetry.

- Resources: reusable global or data-plane-scoped resource definitions. Secret-valued properties are masked in list responses and audit entries.

- Access control: platform and team principals with Owner, Team Admin, Capability Manager, Application Manager, and Application Viewer roles, optionally scoped to a data plane and namespaces.

- Delivery teams: exclusive namespace allocation, asset counts, and one-time scoped automation credentials. Control Plane access remains disabled for delivery teams.

- Audit trail: provisioning, registration, deployment, resource, access, secret, and lifecycle operations with time, outcome, target, and detail.

REST API summary

- `GET /api/health`, `/api/control-plane/overview`, `/api/monitoring`, `/api/observability`, `/api/audit`

- `GET|POST /api/data-planes`; `GET|DELETE /api/data-planes/{id}`; `POST /api/data-planes/{id}/heartbeat`

- `GET|POST /api/capabilities`; `DELETE /api/capabilities/{id}`

- `GET|POST /api/resources`; `DELETE /api/resources/{id}`

- `GET|POST /api/access/principals`

- `GET|POST /api/teams`; `PUT|DELETE /api/teams/{id}`

- `POST /api/teams/{id}/tokens`; `DELETE /api/teams/{id}/tokens/{tokenId}`

- `GET /api/session`

- `GET /api/applications`

- `GET|POST /api/packages`; `GET|DELETE /api/packages/{artifact}/{version}`

- `GET /api/packages/{artifact}/{version}/tasks`

- `GET|PUT /api/packages/{artifact}/{version}/environments/{environment}`

- `GET|POST /api/deployments`; `GET /api/deployments/{id}`

- `PUT /api/deployments/{id}/configuration`; `GET /api/deployments/{id}/health`

- `POST /api/deployments/{id}/starters/{taskId}/{start|stop}`

- `PUT /api/deployments/{id}/secrets`

- `POST /api/deployments/{id}/{start|stop|restart|kill|undeploy}`

- `GET /api/deployments/{id}/logs`

- `GET /api/revisions/{package|deployment}/{id}`

Security, recovery, and troubleshooting

- Put Control Plane behind TLS/reverse proxy and set an API key. Restrict the data directory to the service identity.

- Permission enforcement is performed on every backend API call. UI filtering is informational and is not the security boundary.

- Back up the entire data directory together with the external `MINA_ADMIN_SECRET_KEY`. Losing or changing the key makes stored secrets unreadable.

- Rotate a secret with the secret API while stopped, then restart. The API returns secret names/configuration state only.

- A `FAILED` deployment preserves its error and logs. Correct the adapter or configuration and select Restart.

- `No runtime adapter is configured` means package validation and deployment creation are working, but `MINA_ADMIN_RUNTIME_COMMAND` has not been supplied.

- Control Plane PID reconciliation detects local processes that exited unexpectedly. Data-plane registration and heartbeat APIs provide the management-plane inventory. Remote command execution still requires a trusted data-plane agent/tunnel; this repository does not silently execute remote commands or claim a remote application is running without that adapter.

Application operations

Open Applications to manage deployments or uploaded packages. Existing configuration, starter controls, logs, revision history, and health information remain available through Manage on an application card.

Views and finding applications

- Cards shows application identity, environment, owning team, instance state, and reported health.

- Compact provides denser application cards.

- Status board groups deployments by lifecycle state. It is not a drag-to-deploy board: state changes require explicit lifecycle actions.

- Combine search, lifecycle state, data plane, environment, and team filters. Needs attention selects failed deployments or unhealthy/degraded health reports.

- Star applications and enable Favorites to focus on them. Favorites and view preferences are stored locally in this browser, not shared with other operators.

- Ctrl+K (Command+K on macOS) opens application search, unless a dialog is open.

Bulk lifecycle operations

Select individual deployments or Select visible, then choose Start, Stop, or Restart. Only eligible selected deployments are included in the confirmation dialog. Review the names, environments, and teams before confirming. Stop/restart may interrupt processing.

The client re-fetches each deployment before sending its operation, executes requests sequentially, and reports accepted, skipped, or failed results separately. A failure does not prevent later targets from being attempted. Accepted requests are not proof of healthy execution; inspect updated deployment status and health. Closing the result dialog does not cancel requests already sent. Changing filters clears hidden selections. There is deliberately no bulk delete.

UI eligibility follows the Technology Team/Application Manager model. Every request still uses the existing authenticated API and server-side authorization; client-side controls do not grant permissions.

Themes

Use Theme in the top header: Ocean dark, Slate dark, Light, or System. System follows the operating system preference. The choice persists locally per browser origin and applies to navigation, forms, dialogs, application views, and telemetry surfaces. Windows desktop and browser clients use the same served UI; their local preferences may differ.

Updating an installation

Deploy the updated `administrator/web` assets together, including `index.html`, `operations-workspace.js`, and `operations-workspace.css`. Refresh the browser or reload the Windows Control Plane client. No database migration is required. Do not replace your existing administrator data directory, environment settings, or credentials.

Application inspector and operations

The Control Plane home page summarizes running applications, data-plane connectivity, unhealthy/stopped deployments, and recent operations. The Applications workspace lets an operator search and filter deployments and packages, then inspect an application without leaving the list.

The application inspector exposes lifecycle actions, health and instances, environment/instance/health-check configuration, write-only secret updates, Starter Task controls, runtime logs, revision history and revert, configuration comparison, backup, and restore. Existing package upload, archive inspection, and deployment flows remain available.

The Telemetry page shows CPU, memory, and disk readings reported by agents, with bounded trends (up to 720 samples per data plane, sampled no more than once every 15 seconds). It also shows task outcomes and recent Control Plane request errors. Blank readings mean the agent has not reported that metric; they are not treated as zero.

Operators can create or delete alert rules for offline data planes, failed deployments, unhealthy applications, and CPU/memory/disk thresholds. Resource rules report `NO_DATA` when a host has not sent the selected metric. Rules are evaluated in the Control Plane UI/API; external notification delivery is not implemented by these rules.

Python data-plane agents install `psutil` to report host CPU, memory, and disk. If `psutil` is unavailable, the agent still reports disk space and its local process count. A remote on-premises Python agent forwards a bounded tail of each instance log. Kubernetes pod-log forwarding is not yet implemented; local deployments are read from the Control Plane host.

Optional Windows client

The Linux Control Plane remains the server. The existing browser UI remains available at the same HTTPS address. This client adds a Windows application; it does not install Python, start a local Control Plane or change server permissions.

Build on Windows

From `frontend`, run `npm ci` if dependencies are not installed, then:

```
npm run control-plane:installer
# Or an unpacked build:
npm run control-plane:unpacked
# Override the version without editing application code:
powershell -File ../scripts/build-control-plane-desktop.ps1 -Version 1.2.0
```

Installer: `frontend/release-control-plane/MINAControlPlane-<version>-Setup.exe`. Unpacked executable: `frontend/release-control-plane/win-unpacked/MINA Control Plane.exe`. These are separate from the Studio installer and Administrator server archive. Release signing requires your organization's Windows code-signing configuration; unsigned development installers may trigger Windows SmartScreen warnings.

Connect

- Update the Linux server's `administrator/web/admin.js` from this version (or

redeploy the updated Control Plane). Sign-in and rename dialogs now support both browsers and Electron. Older servers using `window.prompt` are incompatible.

- Expose the existing Control Plane through HTTPS with a certificate trusted by

Windows. A reverse proxy can terminate TLS and forward to the existing service. Serve at the origin root, e.g. `https://mina.company.com`, not `/control-plane`. Enable authentication and preserve existing API routes, uploads and downloads.

- Install and launch MINA Control Plane on Windows, enter that HTTPS address,

then enter your existing Control Plane credential when requested.

- Use Connection -> Reload / reconnect after a connection failure, or

Sign out / change server to end the local session. Closing the app does not stop remote applications or cancel operations already accepted by the server.

Security and limitations

Only the server address is stored in `%APPDATA%/MINAControlPlaneDesktop/server.json`. Remote browsing uses an isolated, non-persistent session. Its cookies, credentials and local storage are not persisted across app restarts; changing server creates a new session. Browser credential storage behavior is unchanged. Local sign-out does not revoke a server-issued credential; revoke credentials through server administration.

Remote pages have no Node.js or desktop IPC access. Sandboxing, context isolation, certificate validation and web security are enabled. HTTP and invalid certificates are rejected; there is no bypass switch. Cross-origin navigation, pop-ups and device permissions are blocked. External identity-provider redirects and cross-origin CDNs are not supported by this initial client. Existing same-origin Control Plane credential authentication, uploads, exports and management APIs remain available.

Network outages interrupt the desktop view, not deployed workloads. Reload when connectivity returns. This is an online client, not an offline administration replica.

Development and tests

`npm run control-plane:desktop` launches from source. `npm run control-plane:test` checks URL policy, isolation settings, independent packaging and dialog compatibility. The entry point is `frontend/control-plane-desktop/main.cjs`; the connection screen is `connect.html`, `connect.js` and `connect.css`. Remote content has no preload. The local screen's narrow IPC bridge is validated against its exact sender/frame. Follow release qualification with a real HTTPS Linux server: sign-in, team isolation, package upload/download, start/stop, network interruption, reconnect and local sign-out.

Reference source: docs/LINUX_SETUP.md

Python archive modes

Deployment

Guide

Python archive modes

Studio exposes two distinct Python exports. Both contain Python application files and `manifest.json` for Control Plane package metadata; neither includes project, task, or resource JSON descriptors.

- Export direct Python / Deploy direct Python compiles supported task transitions into async Python state machines. The generated task source contains direct `if`/`elif` flow decisions, inline group conditions/loops, `asyncio.gather` for success fan-out, and typed Python references for property and activity bindings. It does not bundle the MINA workflow engine. The exporter rejects unsupported semantics with activity names instead of silently substituting an engine-backed implementation. Supported paths include Start, End, Scheduler, Log, Mapper (including functions, conditions, and repeating targets), Call Sub Task, global and transition Catch, Throw, Rethrow, General activities (Empty, Assign, Sleep, Checkpoint, shared variables, and External Command), File, FTP/SFTP, hosted and outbound HTTP/REST/SOAP, HTTP Response, XML/JSON/flat-data parse and render, Excel Read, Transform, Python/Java Invoke, JDBC, Snowflake, AMQP, Kafka, Pub/Sub, EMS/JMS send, receive, request/reply and reply, and supported SAP operations and IDoc parsing. Scheduler, Kafka, Pub/Sub, EMS, JMS, SAP, HTTP, REST, and SOAP starters remain active until stopped. SAP IDoc listeners support direct JCo and EMS/JMS/Kafka messaging sources. Single-entry/single-exit If, For Each, Iterate, While, Repeat, Repeat-on-Error, JDBC transaction, and critical-section groups compile to explicit Python control flow and may be nested with mixed group types. Client-acknowledge JMS receivers use a persistent bridge session and Confirm Message can acknowledge an active delivery.

- Python compatibility archive (engine-backed) retains the full Python runtime engine and the existing wider activity/group behavior. It remains deployable but is not a full direct-code conversion. The package manifest records `pythonSource.mode: engine` so consumers can tell which implementation they received.

The standard `.mpkg` and `.ear` exports remain available. Python archives require separately licensed vendor drivers where the underlying connection needs them (for example SAP JCo or JMS/EMS). A Python source package does not replace those vendor libraries. Legacy `.ifpkg` packages remain importable.

Parallel branches and joins

Multiple eligible success transitions start parallel branches. When all branches converge on a shared activity, Run, Debug, and Python exports wait for the branches and execute that shared activity once. Nested forks join at their own convergence before reaching an enclosing join. Partial joins also wait for their participating branches: if two of three branches merge first, that intermediate activity runs once before its result joins the third branch. An unhandled branch failure prevents the shared continuation; Run and Python exports let sibling branches settle before reporting the failure. Debug steps branches individually and applies the same join barrier.

Each branch retains the task input and receives its own flow value and variables. Activity outputs remain available for explicit downstream mappings. The default value at a join comes from the last branch in transition order, independent of completion timing; business objects are not implicitly merged.

For example, connect Kafka and Pub/Sub branches to the same EMS activity to send to EMS after both complete. If EMS is instead a third outgoing branch, it runs in parallel and the common continuation waits for all three. Completion follows each connector's settings: Kafka `waitForDelivery=true` waits for broker acknowledgement; `false` completes after local enqueue. Regenerate deployed Python archives after upgrading Studio's execution behavior.

Engine-backed Python archives select connector modules from the packaged activities and groups instead of copying every engine file. All included Sub Tasks participate in selection, and JDBC transaction groups include the JDBC adapter. The exporter follows adapter imports transitively (including Java bridge dependencies and the JDBC adapter's Snowflake support). Shared scheduling, model, mapping/expression, and timestamp modules remain in every engine-backed archive. Optional connectors load only when executed; unused shared connection definitions do not cause their connector modules to be bundled. Standard JSON-based EAR archives do not contain this Python engine.

Direct archives are capability-linked from the persisted project model and selected task closure. The linker never checks project names, sample identifiers, known canvas layouts, or a catalog of previously exported applications. It records the exact activity operations and group types in `application/capabilities.py`, removes unreachable execution branches, specializes connector functions to the operations used by the project, and omits unreferenced activity modules, connector modules, native adapters, inbound servers, shared resources, schemas, and properties. Referenced shared resources are followed transitively, as are XSD `schemaLocation` and JSON Schema `$ref` dependencies, so dependency pruning does not produce an incomplete package. Its registry is required to cover every activity operation advertised by the direct exporter; adding a future supported Studio activity without linker metadata fails the exporter and its coverage test instead of silently generating an incomplete application. A property-driven resource, schema, or SAP messaging-provider selection conservatively retains compatible candidates because the selected object can change between packaged environments. Packaging never removes something required by a selected environment merely to make the archive smaller.

For a successfully compiled direct archive, run `python exported.pympkg --task TASK_ID --input '{"key":"value"}'` on a standard Python host. No extraction or source edit is needed. Alternatively, extract the archive and use `python run.py` as a Python-script job entry point (including a Databricks Python script task); pass `--environment PROFILE` when multiple profiles are packaged, and `--task TASK_ID` for a single task. Without `--task`, the generated application runs the selected starters; receiver starters keep polling until stopped. If no environment is specified, the first packaged profile is used.

Run `python run.py --check` (or `python exported.pympkg --check`) in the target data plane before deployment. The generated preflight checks only the dependencies used by that application, returns machine-readable JSON, and exits with code `2` when a required package, Java runtime, or external project artifact is missing. The Control Plane manifest also exposes the preflight command and dependency inventory.

For qualification against real providers, run `python run.py --qualify-task TASK_ID --input '{"test":true}' --iterations 1000 --concurrency 10`. Optional thresholds include `--max-error-rate`, `--max-p95-ms`, and `--max-memory-growth-mb`; `--duration-seconds` turns the same command into a bounded soak test. The command exercises the actual exported task and produces JSON evidence containing throughput, success/failure counts, latency percentiles, bounded error samples, and Python memory growth. It exits with code `3` when a threshold fails. Provider failover is tested by running this command while performing an approved broker/database/SAP failover in the test environment.

Real connectors require runtime provisioning, not source edits. Set `MINA_JAVA_BRIDGE_HOME` to the installed Java bridge directory containing `classes`, and `MINA_DRIVER_HOME` to a directory containing `sap/` and/or `jms/` licensed vendor JARs. Rebuild the bridge from this source before deploying client-acknowledge JMS flows. Place the SAP JCo native library in the SAP driver directory or on the platform native library path. A Java runtime must be available via `MINA_JAVA` or `PATH`. Depending on the generated preflight inventory, install `aiokafka`, `google-cloud-pubsub` and `google-auth`, `openpyxl`, `paramiko`, `snowflake-connector-python`, `pika`, or `azure-servicebus`. Provide connection URLs, credentials, and target-network access through the selected environment profile or environment variables. Missing packages and artifacts are reported before application startup.

The direct export is not a blanket compatibility promise for every future Studio feature. Grouped parallel fan-out still requires an explicit top-level branch boundary because a deterministic in-group join is not represented in the saved graph. Live ECC/JCo, EMS/JMS provider, AMQP, Snowflake, failover, and soak results cannot be asserted without those systems; use the generated qualification command in the target test environment and retain its evidence before approving production. The automated suite covers hosted REST, JMS request/reply correlation, SAP-over-Kafka, mixed nested groups, mock/in-memory providers, SQLite transactions, and bridge compilation. The saved `order-integration` project passes direct compiler static validation, which is not itself a live SAP qualification. Export reports unsupported graph behavior instead of generating a package with silently different behavior.

Reference source: docs/PYTHON_EXPORT.md

MINA build and distribution

Deployment

Guide

MINA build and distribution

Product outputs

MINA has three independently distributable components:

- Studio - an Electron desktop IDE for Windows. It embeds the React designer and a packaged Python sidecar.

- Control Plane - a Linux or Windows management plane for data planes, capabilities, applications, resources, access, observability, and package deployment.

- Runtime package - a portable `.mpkg`, `.tar.gz`, or `.ear`-compatible archive produced by Studio for either an on-premises or Kubernetes data plane.

Windows installation progress

The assisted Windows installers display a checklist during setup. Studio lists the app, local runtime, and background worker. Control Plane lists the desktop app. The installer checks off the bundled components together after extraction, then marks Windows registration and shortcuts complete.

Build Studio on Windows

Prerequisites:

- Windows 10 or Windows 11 x64

- Node.js satisfying Vite's requirement (`20.19+` or `22.12+`)

- Python 3.11 x64 available through the `py` launcher

- Internet access on the build machine for the first dependency installation

Run from a normal PowerShell or Command Prompt. PowerShell is used internally by the npm build script, but installed Studio users do not need PowerShell, Node.js, or Python.

```
cd D:\Integration-tool\MINA\frontend
npm ci
npm run desktop:installer -- -Version 2.4.0
```

Output:

```
frontend\release\MINAStudio-2.4.0-Setup.exe
```

`-Version` is injected into Electron Builder at build time; `package.json` is not edited. It must be a semantic version. CI can set `$env:MINA_VERSION = '2.4.0'` and run `npm run desktop:installer` without passing the argument. When neither is supplied, the version in `frontend/package.json` is used as the fallback.

The installer is machine-wide so different Windows users can launch Studio in parallel. Each user gets an independent local runtime port and workspace data directory under that user's application-data profile.

Useful development commands:

```
npm run desktop:dev # Vite UI in an Electron window with the Python sidecar
npm run desktop:unpacked # Build without creating the installer
npm run desktop:installer # Build the sidecar and final Setup.exe
```

Corporate TLS certificate or proxy

`UNABLE_TO_GET_ISSUER_CERT_LOCALLY` from npm and `CERTIFICATE_VERIFY_FAILED` from pip mean the build machine does not trust the CA used by an HTTPS-inspecting corporate proxy. This is download trust, not Windows

executable signing. Obtain the organization's root and intermediate CA chain from IT as one PEM file. Do not commit thecertificate to this repository.

Configure both Node/npm and Python/pip from PowerShell, then open a new terminal:

```
$ca = 'C:\Certificates\company-ca-chain.pem'
npm.cmd config set cafile "$ca" --location=user
npm.cmd config set strict-ssl true --location=user
py -3.11 -m pip config --user set global.cert "$ca"
[Environment]::SetEnvironmentVariable('NODE_EXTRA_CA_CERTS', $ca, 'User')
[Environment]::SetEnvironmentVariable('PIP_CERT', $ca, 'User')
```

For the current terminal, set the values as well:

```
$env:NODE_EXTRA_CA_CERTS = $ca
$env:PIP_CERT = $ca
$env:REQUESTS_CA_BUNDLE = $ca
```

Keep SSL verification enabled. If the company requires an explicit proxy, obtain its URL from IT and additionally configure `npm config set https-proxy <proxy-url> --location=user`, `HTTPS_PROXY`, and `ELECTRON_GET_USE_PROXY=1`.

After a failed `npm ci`, close running Node/Electron processes and delete the incomplete `frontend\node_modules` directory before retrying. The desktop build script now stops at the first failed npm, pip, Electron, or PyInstaller command instead of reporting a misleading successful sidecar build.

The desktop runtime is built with Python 3.11. Install the standard x64 distribution from python.org; a Microsoft Store execution alias that appears in `py -0p` but cannot launch is not sufficient. If `backend\.venv` uses another Python version, the installer build now preserves it and creates a dedicated Python 3.11 environment under `backend\build\.venv311`. The AMQP 1.0 connector uses the prebuilt `python-qpid-proton-wheel` distribution because the upstream `python-qpid-proton` 0.40 source release does not provide a CPython 3.11 Windows wheel and otherwise requires Microsoft Visual C++ Build Tools. Before PyInstaller starts, the build imports every external connector module and fails if a package or native DLL is unavailable.

Database shared connections accept either the Studio host/port/database fields or vendor JDBC URLs and translate them to the corresponding native Python adapter. The packaged runtime includes adapters for SQLite, PostgreSQL, MySQL/MariaDB, Oracle, IBM Db2, Databricks SQL, Snowflake, and SQL Server. SQL Server additionally requires Microsoft ODBC Driver 18 (or 17) to be installed on the runtime machine; the Studio detects installed drivers and reports the available names. Databricks supports personal access tokens, OAuth machine-to-machine, and interactive OAuth user-to-machine authentication with SQL warehouse hostname, HTTP path, catalog, and schema configuration.

Real SAP ECC connections

SAP shared connections use the licensed SAP Java Connector (JCo). New SAP connections use the external adapter by default; the local mock remains available as an explicit design-time option. JCo is not included in the installer. Place `sapjco3.jar` and its matching `sapjco3.dll` in the configured SAP driver directory (or the default `C:\ProgramData\MINA Studio\drivers\sap`). No Python SAP package or separate NW RFC SDK installation is required.

The repository includes `requirements-sap.txt` as a reminder. The Java bridge loads JCo at runtime, so the JCo JAR and DLL remain outside source control and are reported in the connection-test response.

See SAP_INTEGRATION.md for the supported IDoc/ALE, RFC/BAPI, transaction, lifecycle, and production certification behavior.

Ensure the SDK's native library directory is on `PATH` before starting Studio. For an application-server connection, configure `ashost`, `sysnr`, `client`, `user`, and `passwd`; for a message-server/logon-group connection, configure `mshost`, `sysid`, `group`, `client`, `user`, and `passwd`. SNC connections additionally require the SNC partner name and SAP Cryptographic Library path. The Test Connection action calls `STFC_CONNECTION` and reports the native RFC error when authentication, routing, authorization, or network setup fails.

After PyInstaller finishes, the build launches that exact packaged runtime on an isolated loopback port and requires a successful `/api/health` response before Electron Builder is allowed to create an installer. Installed Studio startup writes the runtime path, port, stdout, stderr, spawn errors, and exit status to `%APPDATA%\MINA Studio\logs\runtime-startup.log`. Startup errors include the log path and its latest output instead of only showing a generic readiness timeout.

The Windows Studio window opens while the local runtime starts and switches to the welcome screen after the runtime passes its health check. Launching Studio again in the same Windows profile brings the existing window forward. If startup is slow, the same log records elapsed time for the startup page, runtime readiness, and Studio page load.

The build compiles the complete backend with Python 3.11 before freezing it. PyInstaller then explicitly collects every module under the backend `app` package and verifies `app.main` both before and after freezing. This catches incompatible Python syntax early and avoids environment-dependent namespace/package discovery producing an executable that fails with `ModuleNotFoundError: app.main`.

The unpacked executable is:

```
frontend\release\win-unpacked\MINA Studio.exe
```

For production distribution, configure a company `.pfx` code-signing certificate through environment variables. A signing certificate is optional for producing an installer, but recommended for publisher identity and SmartScreen reputation:

```
$env:WIN_CSC_LINK = 'C:\Certificates\mina-signing.pfx'
$env:WIN_CSC_KEY_PASSWORD = '<password-from-secret-store>'
npm.cmd run desktop:installer
```

Do not put the certificate or password in `package.json` or source control. Electron Builder automatically discovers `WIN_CSC_LINK`/`WIN_CSC_KEY_PASSWORD` (and supports `CSC_LINK`/`CSC_KEY_PASSWORD` as fallbacks).

Build Control Plane on Windows

```
cd D:\Integration-tool\MINA
.\scripts\build-administrator.ps1 -Version 2.4.0
```

Or from the `frontend` directory:

```
npm run administrator:windows -- -Version 2.4.0
```

Outputs:

```
administrator\dist\MinaAdministrator\MinaAdministrator.exe
administrator\release\MinaAdministrator-2.4.0-Windows-x64.zip
```

The ZIP is a PyInstaller one-directory distribution. Copy or extract the complete `MinaAdministrator` directory, including its `_internal`, `web`, and `bin` content. Copying only `MinaAdministrator.exe` will not work reliably.

Run it:

```
$env:MINA_ADMIN_DATA_DIR = "D:\MINAAdmin\data"
& .\administrator\dist\MinaAdministrator\MinaAdministrator.exe
```

Open `http://localhost:9080`. To listen on a different address or port, set `MINA_ADMIN_HOST` and `MINA_ADMIN_PORT`.

Build and install Control Plane on Linux

Use the [Control Plane and Data Plane one-stop guide](LINUX_SETUP.md) for transfer packaging, INI configuration, installation and lifecycle commands.

Run Control Plane as a container

```
docker build -f Dockerfile.administrator --build-arg MINA_VERSION=2.4.0 -t mina-administrator:2.4.0 .
docker run -d --name mina-admin -p 9080:9080 \
 -v mina-admin-data:/var/lib/mina/administrator \
 mina-administrator:2.4.0
```

Create deployment packages in Studio

Open Packaging in Project Explorer or select Package on the ribbon. Configure:

- Artifact name and version

- Target: On-premises Linux or Cloud / Kubernetes

- Target environment

- Archive: `.mpkg`, `.tar.gz`, or `.ear`

An on-premises package contains an Administrator deployment descriptor. A cloud package contains a Docker build input and Kubernetes deployment manifest. Password property values are removed from both outputs and replaced by deployment-time secret requirements.

The `.mpackage` file is the editable Studio project package. The `.mpkg` file is the immutable deployment artifact. Studio and Control Plane continue accepting legacy `.ifproject`, `.ifpackage`, and `.ifpkg` files during the compatibility period.

Vendor Java connector bridge

Desktop builds require JDK 17 or newer (`javac` and `jlink`). The build bundles a minimal Java runtime automatically; target machines do not need Java installed. EMS, Microsoft SQL Server JDBC, and Oracle JDBC JAR placement is documented in VENDOR_DRIVERS.md.

Reference source: docs/BUILDING_AND_DISTRIBUTION.md

Vendor driver installation

Deployment

Guide

Vendor driver installation

MINA Studio bundles its own minimal Java runtime and connector bridge. Vendor-licensed JARs are intentionally not distributed in the installer.

Vendor native client libraries (`.lib`, `.dll`, `.so`, `.dylib`) and license keys are also external prerequisites. They are ignored by Git under `drivers/`, and artifact verification rejects known SAP/EMS/JMS binaries in release payloads. Keep local drivers in the configured driver folder; source cleanup must not delete customer-owned local copies. See [distribution compliance](../compliance/README.md) for release SBOMs, notices and remaining permission/source-access review.

The default machine-wide driver root on Windows is:

```
C:\ProgramData\MINA Studio\drivers
```

Set `MINA_DRIVER_HOME` to override the complete root, or enter a Driver JAR directory in an individual shared connection. Source checkouts also search `D:\Integration-tool\MINA\drivers` as a development fallback.

EMS / JMS

Place the EMS client files in:

```
C:\ProgramData\MINA Studio\drivers\jms\tibjms.jar
C:\ProgramData\MINA Studio\drivers\jms\jakarta.jms-api-2.0.3.jar
```

For older EMS releases, use the matching `jms-2.0.jar` supplied with that EMS installation. Add `tibjmsufo.jar` for unshared-state failover. Do not mix JARs from different EMS releases.

The EMS direct factory class is supplied by the runtime default. If overriding it, use the exact fully qualified class name from your licensed driver documentation. Java class names are provider identifiers and must not be renamed.

Direct mode uses the EMS URL, username, and password. JNDI mode additionally uses the configured initial-context factory, provider URL, credentials, and connection-factory JNDI name.

Microsoft SQL Server

Download the Microsoft JDBC Driver for SQL Server and place the Java 11-compatible JAR in:

```
C:\ProgramData\MINA Studio\drivers\jdbc\sqlserver\mssql-jdbc-<version>.jre11.jar
```

The driver class is `com.microsoft.sqlserver.jdbc.SQLServerDriver`. SQL Server Authentication is fully Java-based and does not need ODBC. Windows Integrated Authentication additionally requires Microsoft's architecture-matched authentication DLL; username/password authentication is the portable default.

Oracle Database

Place the Oracle JDBC driver in:

```
C:\ProgramData\MINA Studio\drivers\jdbc\oracle\ojdbc11.jar
```

Optional Oracle companion JARs such as `orai18n.jar`, `ucp.jar`, or Oracle PKI libraries can be put in the same directory when the selected Oracle features require them. The driver class is `oracle.jdbc.OracleDriver`. Thin JDBC mode does not require Oracle Instant Client.

Build prerequisite

The installer build machine needs JDK 17 or newer because `desktop:installer` compiles the bridge and creates the bundled Java runtime. End-user machines do not need Java installed.

Build the bridge alone with:

```
powershell -ExecutionPolicy Bypass -File scripts\build-java-bridge.ps1
```

The connection-test response reports `loadedJars`, making it possible to verify exactly which vendor files were loaded.

Reference source: docs/VENDOR_DRIVERS.md

MINA environment variables

Deployment

Guide

MINA environment variables

This is the reference list of environment variables recognized by the current MINA source, Studio sidecar, and Control Plane. Variable names are case-sensitive on Linux and Kubernetes. Values containing passwords, private keys, API keys, or tokens must be supplied by the operating system, service manager, or cloud secret store; do not commit them to a project or container image.

Runtime and Studio sidecar

Variable	Default	Scope and purpose
`MINA_PORT`	`8787`	Backend HTTP port used by `run_sidecar.py`. Studio normally assigns this automatically.
`MINA_LOG_LEVEL`	`info`	Uvicorn process log level, for example `debug`, `info`, `warning`, or `error`.
`MINA_DATA_DIR`	`backend/data`	Runtime data root for projects, SAP TID files, and default project logs.
`MINA_RUNTIME_LOG_DIR`	`<data-dir>/logs`	Process-wide fallback log root when `runtime.logDirectory` is blank.
`MINA_PROJECT_LOG_MAX_BYTES`	`10485760`	Maximum size of each project log file before rotation.
`MINA_PROJECT_LOG_BACKUP_COUNT`	`4`	Number of rotated project log files retained.
`MINA_LOG_TIMEZONE`	`America/Phoenix`	IANA display timezone for runtime, debugger, and audit timestamps. Arizona is emitted with `-07:00`.
`MINA_WORKER_THREADS`	`auto`	Runtime worker-thread setting; otherwise environment property `advanced.workerThreads` or `advanced.threadCount` is used.
`MINA_APPLICATION_DIR`	unset	Application directory supplied to a deployed runtime by the Control Plane.
`MINA_ENVIRONMENT`	`local` or deployment value	Active runtime environment.
`MINA_DEPLOYMENT_ID`	unset	Deployment identifier supplied by the Control Plane.
`MINA_INSTANCE_ID`	unset	Runtime instance identifier supplied by the Control Plane.
`MINA_BUILD_VERSION`	`source`	Build/version value included in startup diagnostics.
`MINA_STATIC_DIR`	bundled static directory	Override location of backend-served frontend assets.
`MINA_DRIVER_HOME`	platform search locations	SAP JCo/native driver directory override. Must contain matching `sapjco3.jar` and native library.
`MINA_JAVA`	automatic Java lookup	Java executable override for the SAP/JMS Java bridge.
`MINA_JAVA_BRIDGE_HOME`	repository/bundled bridge location	Java bridge installation directory override.
`MINA_AI_MODEL`	`gpt-5`	AI Builder model name when `OPENAI_API_KEY` is configured.
`OPENAI_API_KEY`	unset	Enables the OpenAI AI Builder provider. Store as a secret.

The Electron Studio sets `MINA_PORT`, `MINA_DATA_DIR`, `MINA_LOG_LEVEL`, and `MINA_BUILD_VERSION` for its child runtime. It also supports these Studio-only process variables:

Variable	Purpose
`MINA_PYTHON`	Python executable override for development sidecar startup.
`MINA_DEV_URL`	Frontend development URL loaded by Electron instead of the local production URL.

Control Plane

Variable	Default	Scope and purpose
`MINA_ADMIN_HOST`	`0.0.0.0`	Control Plane HTTP bind address.
`MINA_ADMIN_PORT`	`9080`	Control Plane HTTP port.
`MINA_ADMIN_DATA_DIR`	`administrator/data`	Control Plane state root: packages, deployments, machines, secrets, audit, and logs. Mount this directory persistently.
`MINA_ADMIN_API_KEY`	unset	Technology Team API key. When set, protected APIs require `X-Admin-Key` or `X-Control-Plane-Key`. Store in a secret manager.
`MINA_ADMIN_SECRET_KEY`	generated local key	Stable encryption key for stored deployment secrets. Required from a durable secret store for production and clustered deployments.
`MINA_ADMIN_RUNTIME_COMMAND`	unset	Trusted command template used to start an on-premises runtime. Supports `{application}`, `{package}`, `{environment}`, `{deployment_id}`, and `{instance_id}`.
`MINA_ADMIN_MAX_PACKAGE_MB`	`250`	Maximum uploaded package size in megabytes.
`MINA_ADMIN_MAX_EXPANDED_MB`	`1024`	Maximum expanded package size in megabytes.
`MINA_ADMIN_MAX_PACKAGE_FILES`	`10000`	Maximum archive member count.
`MINA_ADMIN_VERSION`	packaged value or `development`	Control Plane version override.
`MINA_ADMIN_WEB`	bundled `web` directory	Override location of Control Plane frontend files.

The Linux service wrapper also recognizes:

Variable	Default	Purpose
`MINA_ADMIN_HOME`	`/opt/mina/control-plane`	Installed Control Plane directory.
`MINA_ADMIN_PID_DIR`	`/opt/mina/run`	PID file directory used by `bin/minaadmin`.
`MINA_ADMIN_LOG_DIR`	`/opt/mina/logs/control-plane`	Service-wrapper stdout/stderr log directory.

Build and packaging variables

Variable	Purpose
`MINA_VERSION`	Default version passed to Studio or Control Plane build scripts when no explicit version argument is supplied.

Variable	Purpose
`WIN_CSC_LINK` / `WIN_CSC_KEY_PASSWORD`	Windows Electron signing certificate and password. Keep both outside source control.
`CSC_LINK` / `CSC_KEY_PASSWORD`	Electron Builder fallback names for Windows signing.

`MINA_VERSION` and signing variables are build-time settings; they are not needed on an installed runtime machine.

Automatically injected deployment variables

When the Control Plane starts an on-premises deployment, it injects `MINA_APPLICATION_DIR`, `MINA_ENVIRONMENT`, `MINA_DEPLOYMENT_ID`, and `MINA_INSTANCE_ID`. It also injects the deployment's decrypted secret values using their configured names. These values should not be manually duplicated in a package or checked into source control.

Windows

For the current PowerShell session:

```
$env:MINA_DATA_DIR = 'C:\ProgramData\MINA\runtime-data'
$env:MINA_RUNTIME_LOG_DIR = 'D:\IntegrationLogs'
$env:MINA_LOG_TIMEZONE = 'America/Phoenix'
$env:MINA_DRIVER_HOME = 'C:\SAP\jco'
python -m uvicorn app.main:app --host 127.0.0.1 --port 8787
```

For a persistent machine-level setting, run PowerShell as Administrator and use `[Environment]::SetEnvironmentVariable('NAME', 'VALUE', 'Machine')`. Restart the service or application after changing environment variables.

Linux and systemd

Create a root-readable environment file such as `/etc/mina/runtime.env`:

```
MINA_DATA_DIR=/opt/mina/runtime/data
MINA_RUNTIME_LOG_DIR=/opt/mina/logs/runtime
MINA_LOG_TIMEZONE=America/Phoenix
MINA_DRIVER_HOME=/opt/mina/drivers
MINA_ENVIRONMENT=dev
```

Reference it from the service unit:

```
[Service]
EnvironmentFile=/etc/mina/runtime.env
```

Protect files containing secrets with `chmod 600` and set the service account as owner. Run `systemctl daemon-reload` and restart the service after changes.

Docker and Kubernetes

Use ordinary configuration values for paths and ports, and use Docker/Kubernetes Secrets for credentials:

```
env:
 - name: MINA_ENVIRONMENT
 value: dev
 - name: MINA_LOG_TIMEZONE
 value: America/Phoenix
 - name: MINA_DATA_DIR
 value: /var/lib/mina/runtime
 - name: MINA_ADMIN_API_KEY
 valueFrom:
 secretKeyRef:
 name: mina-admin-secrets
 key: api-key
 - name: MINA_ADMIN_SECRET_KEY
 valueFrom:
 secretKeyRef:
```

```
name: mina-admin-secrets
key: encryption-key
```

Persist `MINA_DATA_DIR` and `MINA_ADMIN_DATA_DIR` on volumes. For SAP JCo, provide the licensed matching JCo JAR/native library through a protected volume or image layer and set `MINA_DRIVER_HOME` to that directory. Do not print secret environment values in startup diagnostics.

Operational notes

- Environment variables are read during process startup; restart the affected

process after changing them.

- Relative paths are resolved from the process/application data location and

should be replaced with absolute paths in production.

- `MINA_LOG_TIMEZONE` changes display timestamps only. Scheduler and token

expiry calculations continue to use UTC internally.

- Existing log files are not rewritten when the timezone changes.

- To inspect effective non-secret settings on Windows use `Get-ChildItem Env:`;

on Linux use `env` or `systemctl show <service> --property=Environment`.

Reference source: docs/ENVIRONMENT_VARIABLES.md

SAP ECC integration runtime guide

Connectors

Guide

SAP ECC integration runtime guide

MINA uses the licensed SAP Java Connector (JCo) supplied by the customer. The connector is not redistributed with Studio. This implementation interoperates with SAP ECC through public RFC/JCo contracts. MINA is an independent product and does not include third-party proprietary connector implementations.

MINA itself is not SAP-certified merely because it uses JCo. SAP permits JCo to connect external applications to SAP in both directions, but the customer must download it from SAP, comply with named-user/solution licensing, and must not redistribute the connector. Record the JCo, Java, OS, and CPU architecture reported by Test Connection as part of every production approval.

Supported integration patterns

Pattern	Runtime behavior
IDoc Listener	Persistent registered JCo server, tRFC TID duplicate protection, bounded delivery queue, one workflow per IDoc in a package, enclosing transaction commit only after every IDoc succeeds
RFC/BAPI Listener	Persistent registered JCo server for the selected function, request/reply export and table response mapping, rollback when the task fails or omits Reply
Invoke RFC/BAPI	Persistent, bounded JVM worker pool; JCo destination pooling; import, changing, nested structure, and table parameters; request/reply, tRFC, and qRFC transport
Post IDoc / IDoc Reader posting	XML or rendered input is converted to EDI_DC40 and EDI_DD40 and sent with `IDOC_INBOUND_ASYNCHRONOUS`; tRFC or qRFC is selected at transport level
IDoc Parser / Converter / Renderer	Metadata-driven, byte-position fixed-width conversion with physical E2 to logical E1 normalization, configurable code page, strict/replace invalid-byte policy, and delimiter-safe flat records
Read Table	`RFC_READ_TABLE` field, filter, paging, and named-record output
Dynamic Connection	A dedicated persistent JVM and thread-bound JCo context across activities; explicit commit/end, rollback on failure/expiry, bounded session count, and duplicate-session rejection
IDoc acknowledgment / confirmation	Invokes the customer-configured SAP acknowledgment RFC because this function is solution-specific

ALE distribution is implemented through the IDoc listener/posting paths. SAP partner profiles, ports, process codes, message types, RFC destinations, and authorizations remain SAP Basis configuration and are not generated by Studio.

Required runtime files

Install a matching x64 JCo distribution in one of these locations:

```
C:\ProgramData\MINA Studio\drivers\sap\sapjco3.jar
C:\ProgramData\MINA Studio\drivers\sap\sapjco3.dll
```

On Linux use `libsapjco3.so` next to `sapjco3.jar`, or select an explicit SAP driver directory in the shared connection. The packaged Java runtime and JCo native library must have the same architecture. Do not mix files from different JCo releases.

Use a currently supported SAP JCo 3.1 patch and a JDK/JRE listed for that patch in SAP Note 2786882. As of September 2026 SAP publishes JCo 3.1.14 and lists Java 8, 11, 17, 21, and 25; re-check SAP's support page when qualifying a build.

ECC configuration checklist

- Create the RFC user with only the required S_RFC and IDoc authorizations.

- Configure SM59 for the Studio Program ID, gateway host, and gateway service.

- Configure WE21, WE20, message types, process codes, and partner profiles for

ALE/IDoc traffic.

- For qRFC, configure and monitor the queue in SMQ1/SMQ2 and set a non-empty

SAP queue name in the activity.

- Make each concurrently deployed registered server identity unambiguous.

- Keep the TID store on durable writable storage. Never share one TID file

between unrelated Program IDs.

- Validate `STFC_CONNECTION`, fetch the actual IDoc/RFC metadata, and save the

selected metadata with the project before packaging.

Resource and failure behavior

- Listener event queues and stderr history are bounded. Backpressure reaches

the JCo callback instead of permitting unbounded Python heap growth.

- Cancellation does not leave blocked Python executor threads. Runtime shutdown

closes native listener processes and removes their temporary descriptors.

- The JCo server waits for the workflow result. Success commits the delivery;

task failure, cancellation, missing RFC Reply, or reply timeout rolls it back so SAP can retry according to its transaction policy.

- TID state is persisted atomically. A previously committed or in-flight TID is

rejected as a duplicate; SAP confirmation removes it.

- Single-call BAPI auto-commit executes `BAPI_TRANSACTION_COMMIT` in the same

JCo context as the BAPI call.

- A Dynamic Connection with `transactional=true` pins all calls carrying its

`sessionID` to one worker/JCo context. `contextEnd=true` closes the context; with `autoCommit=true` it first executes `BAPI_TRANSACTION_COMMIT`. Failure, expiry, or normal termination without `commitOnTerminate=true` rolls back.

- External outbound calls use persistent JVM workers. `outboundWorkerCount`

controls Python-side worker concurrency; `poolCapacity`, `peakLimit`, and `connectionExpirationMilliseconds` configure the JCo destination pool in each worker. Changing any connection setting, including credentials, creates a distinct pool identity without storing plaintext credentials in its key.

Flat IDoc and messaging modes

In JSON mode, the IDoc Parser and Converter expose one schema-shaped document under `SAPIDoc`, for example `SAPIDoc.ARTMAS05.IDOC`. Map `SAPIDoc` to a Kafka JSON payload to send that document once. Older direct runtime mapping paths such as `SAPIDoc.IDOC` and `ARTMAS05.IDOC` remain available as lookup aliases; they are not additional serialized JSON branches. Actual repeating IDocs and segments remain in the payload.

The built-in parser reads canonical EDI_DD40 byte fields (`SEGNAM`, `MANDT`, `DOCNUM`, `SEGNUM`, `PSGNUM`, `HLEVEL`, and `SDATA`) and EDI_DC40 control fields. Set `idocEncoding` to the actual sender code page. Keep `invalidCharacterPolicy=strict` for production unless replacement characters are an accepted business rule. When SDATA may contain CR/LF, configure a unique `idocRecordDelimiter`; that delimiter is authoritative and embedded line breaks remain data. With no delimiter, canonical 1063-byte records are split by width.

If the separately licensed `sapidoc3.jar` is installed, Test Connection reports its presence and version. Selecting the SAP JIDocLib parser engine validates IDoc XML with SAP's repository-backed `IDocXMLProcessor`; flat EDI_DD40 input continues to use the built-in byte parser because SAP documents the public IDoc class-library processor as an XML processor.

IDoc Listener supports these sources:

- `NoMessaging`: the direct persistent JCo RFC server, including durable TID

handling and SAP transaction acknowledgment.

- `EMS` or `JMS`: consumes one queue message through the selected shared

connection and publishes its body to the parser. Client acknowledgment is exposed through the standard Confirm Message activity.

- `Kafka`: consumes one topic record through the selected Kafka resource and

publishes its value to the parser. Manual offset confirmation uses the same Confirm Message contract.

For broker-backed modes, select `messagingResourceId` and `messagingDestination`. SAP-to-broker transaction atomicity is owned by the upstream bridge; MINA does not claim an XA transaction spanning SAP tRFC and an independent EMS/JMS/Kafka broker.

Production certification

Unit and mock tests verify serialization, lifecycle, transaction routing, and bounded queues, but they cannot certify a particular ECC/JCo combination. Before production, run an environment test with the exact JCo version and architecture:

- sustained IDoc batches containing the largest expected SDATA payloads;

- concurrent RFC/BAPI calls at the configured maximum connection count;

- SAP and Studio restart during in-flight tRFC/qRFC deliveries;

- duplicate TID, reply timeout, malformed IDoc, network partition, and expired

credential scenarios;

- a 24-72 hour soak while recording process RSS, JVM heap/GC, thread count,

handle count, queue depth, throughput, and SAP SM58/SMQ1/SMQ2 state.

Do not accept the connector for production until those measurements remain stable and every failed transaction is demonstrably recoverable.

The repository includes a repeatable qualification runner and an intentionally non-secret example configuration:

```
Copy-Item docs\sap-soak-config.example.json sap-soak-config.local.json
# Edit the local copy with QA-only credentials and parameters, then run:
backend\.venv\Scripts\python.exe scripts\sap-production-soak.py `
 --config sap-soak-config.local.json --output sap-soak-report.json
```

Set `terminateWorkerEveryCalls` to a positive number for controlled worker-JVM failure injection. Run separate 24-72 hour profiles for sRFC, transactional BAPI, tRFC/qRFC IDoc posting, and inbound listeners. The generated report holds throughput, failures, latency percentiles, and worker process IDs; correlate those IDs with OS/JVM monitoring and SAP SM58/SMQ1/SMQ2 evidence. Never commit the local configuration because it contains credentials.

Reference source: docs/SAP_INTEGRATION.md

EMS/JMS sender reuse in Run mode

Connectors

Guide

EMS/JMS sender reuse in Run mode

Run previously included the unique job execution ID in the persistent sender cache key. Each incoming event therefore opened another Java worker/EMS connection. After eight cached configurations, additional jobs fell back to one-shot JVMs. Debug used a stable session ID and did not normally encounter this problem.

The runtime now gives EMS/JMS an application-scoped sender identity, separate from the job ID. Jobs keep unique IDs for logging and tracing. Subtasks and parallel branches inherit their caller's sender scope; Debug retains its own session scope. Different applications and runtime instances remain isolated. Connection settings still participate in the cache key, so changed credentials/settings are not ignored.

Studio project Stop cancels execution and force-closes that application's senders. Restart can establish a fresh sender. Idle eviction, bounded caching, serialized JMS session access, and the existing no-replay-on-uncertain-send behavior remain.

The fix applies to Windows and Linux execution through WorkflowRuntime, including new engine-based exports. Rebuild/reinstall Studio and regenerate/redeploy engine archives to replace older bundled code. Direct Python connector exports use a different connection path and are not modified by this fix.

Verification uses the actual Java bridge and a local test JMS provider: twelve separate Run jobs retain one sender, application Stop is isolated, restart works, and provider failures are not automatically replayed. Debug, scheduling, runtime, and engine-export regression suites also pass. Real EMS throughput still requires measurement against the target broker; no fixed delivery latency is guaranteed.

For diagnosis inspect `senderReused`, `providerSendMs`, and `publishTiming` (`queueMs`, `bridgeMs`, `totalMs`) in send output. A healthy warm send should reuse its sender. Upstream event rate, broker persistence, network latency, and payload logging can still constrain end-to-end application throughput.

Reference source: docs/EMS_RUN_CONNECTION_REUSE.md

Kafka publishing performance

Connectors

Guide

Kafka publishing performance

An explicit Input mapping replaces the selected message field, including an object-valued `${last}` default. It does not merge earlier EMS, Pub/Sub, or Kafka publish receipts into that message. Map a parser's document branch (for example `${IDoc-Parser.SAPIDoc.ARTMAS05}`) to send only the IDOC. Kafka's String serializer encodes mapped objects and arrays as JSON text; existing strings and bytes retain their content. The JSON serializer also encodes objects as JSON.

Studio and engine-backed exports reuse Kafka producers and run publishing on a dedicated four-thread executor. Long-running receivers, SAP/JCo calls, and other default-executor work therefore do not hold up Kafka enqueue operations. Producer cleanup closes this executor with the runtime. Direct Python exports reuse an asynchronous producer per connection/configuration and event loop.

Publish output includes `publishLatencyMs` plus:

- `publisherQueueMs` (Studio/engine): time waiting for a Kafka worker thread.

- `producerSetupMs`: producer lookup/creation time; direct Python includes initial broker connection setup and waiting for the creation lock.

- `sendAndWaitMs`: serialization/enqueue/delivery work after producer lookup; this is not a pure network-latency measurement.

- `producerReused` (direct Python): whether the existing asynchronous producer was reused.

With `waitForDelivery` enabled, completion includes broker acknowledgement. Kafka transactions also wait for delivery and commit in the Studio engine. With it disabled, `queued: true` and `deliveryConfirmed: false` indicate local queue acceptance, not confirmed broker delivery. This change does not alter acknowledgement settings or transaction semantics.

For remaining delays, compare the first publish with subsequent publishes using the same connection. High setup time on the first direct Python publish can involve metadata, DNS, TLS, or authentication. High send/wait time can include configured linger, queue backpressure, broker replication, retries, and network latency. Capture these timing fields alongside the effective producer settings and broker metrics before changing reliability options. Rebuild installed Studio and regenerate exported Python archives to use the updated implementation.

The automated tests cover thread-pool saturation, producer reuse, queued and acknowledged sends, and transaction initialization reuse. They do not establish throughput or latency against a real Kafka broker.

Reference source: docs/KAFKA_PUBLISHING.md

REST APIs and shared HTTP connections

Connectors

Guide

REST APIs and shared HTTP connections

Use HTTP Client Connection for HTTP Request / REST Invoke and HTTP Server Connection for HTTP Listener / REST Receiver. These presets create reusable HTTP resources with explicit client/server roles. Existing HTTP resources continue to work; choose a role when editing them. A server resource cannot be used for an outbound request, and a client resource cannot host a listener.

MINA API Blueprint: load YAML and host an API

Open API Blueprint on the Studio ribbon. Load a `.raml` RAML 1.0 API, or a `.yaml`, `.yml` or `.json` OpenAPI contract, or paste its contents. Load example supplies a small two-operation API. Choose Validate YAML to check syntax, contract structure, unique routes/operation IDs, path parameters, schemas and any declared response examples. Missing operation IDs receive stable generated IDs. YAML duplicate keys and aliases are rejected; ordinary YAML strings such as `on`/`off` remain strings. Numeric response keys are normalized to status-code strings.

For a multi-file contract, choose Load API folder, then select Entry API document. The folder's relative paths are preserved, so references such as `models/order.yaml` and local JSON Pointers can be bundled without fetching URLs or reading files outside the selected folder. Import accepts up to 16 supplied source files, each at most 8 MiB, with a 32 MiB upload total and an 8 MiB bundled-contract limit. RAML folders can include RAML/YAML/JSON, XML/XSD examples, text and Markdown. Change remote references to matching local relative paths before importing; the wizard does not download URLs. Switching entry documents invalidates the previous validation and operation selection while retaining edits to previously selected files.

Validation checks every declared request/response media schema, response codes, parameter locations and duplicates, route placeholders, callback structure, and mock examples against their corresponding media schemas. Named/referenced examples, explicit JSON null examples, and schema-level examples are supported. Example response headers use declared examples or schema defaults. Declared 204/205 responses can be mocked without a body. Generated names remain unique even when routes differ only in case or callbacks share a display name. Contracts are bounded to 256 paths, 512 operations, and 1024 generated operation/callback tasks.

Generated mock responses preserve JSON strings and null as JSON values in Studio, Run, Debug, and exported Python applications. Hyphenated URI parameter names, such as `{order-id}`, are routed correctly. Hosted Swagger resolves named examples from imported documents and retains response header declarations; Postman exports preserve explicit null request examples.

Use Find operations, Select all, and Clear selection for larger contracts. Filtering does not deselect hidden operations. Each operation indicates its available mock status, or that it will return 501 until implemented. Creating an API remains separate from implementing its business logic; loading a contract never generates that logic automatically.

Select the operations to implement, then choose a saved HTTP server connection or enter a new bind host, port and protocol. HTTPS hosting requires the server's own certificate/private key or a saved connection with an identity store. The selected connection settings are copied into a dedicated API server resource, preserving property bindings and authentication settings. The contract is stored once on that resource and remains editable there.

Choose initial response behavior explicitly. 501 placeholders is the default: importing an interface does not implement business logic. Declared response examples hosts schema-validated mock examples for testing; operations without examples still return 501. Each selected operation becomes an editable Starter Task containing REST Receiver -> HTTP Response. Replace the placeholder/example response with your integration logic and enable response validation when implementing the operation.

Select API package to choose an existing task package. New package creates a root folder; New subpackage creates a child of the selected package. Names use the same validation as Project Explorer, and spaces become hyphens. Generated operation tasks, contract callback helpers when present, and the API-specific server connection are saved in the selected package. New folders are only saved when the API is created; closing the wizard leaves the project unchanged.

Create & host API saves and starts all selected routes together, showing their actual endpoint URLs. There is no separate Create API tasks button in this wizard. If hosting fails, the saved tasks remain available and Host API retries without duplicating them. Run on any task in that Blueprint hosts its sibling API tasks; Stop releases the listener ports. Unrelated starter tasks do not join the Blueprint. Both Python archive formats host the same generated routes when running the application.

The wizard imports RAML 1.0, OpenAPI 3 and Swagger 2 contracts. The activity contract editor separately supports explicit external-reference configuration. Create new API creates a separate design; Update existing API changes a selected Blueprint in place through review. When retaining multiple API versions, give them distinct routes/base paths or ports so an application-wide Python deployment can host them together. The popup stays within the viewport and scrolls its content independently of its action controls.

RAML traits, libraries and resource types

Choose the API root beginning with `#%RAML 1.0`, rather than a Library/Trait/ResourceType fragment. Load API folder supplies local `!include` documents and `uses` libraries, including nested namespaces. The importer expands parameterized traits (`is`) and resource types (`type`), inheritance, optional methods such as `post?`, reserved `resourcePath`, `resourcePathName` and `methodName` parameters, and standard naming transformations such as `!singularize`, `!pluralize` and case conversions. Explicit declarations override inherited values; inherited collections are merged. An optional method is expanded only when that method exists on the resource or through nonoptional inheritance.

Supported types include objects, arrays, inherited/namespaced types, unions, nullable values, primitive constraints, date/time formats, XML metadata and included JSON schemas. Nested routes, URI/query/header parameters, media types, responses, examples, Basic/OAuth2 security and single-credential Pass Through security convert into the shared runtime contract. Explicit `protocols: [HTTPS]` requires a TLS connection. Configure the actual credentials and TLS identity on the server shared connection; importing security declarations does not generate secrets.

The saved resource contains normalized OpenAPI, so Swagger, Postman, Run, Debug and both Python archive formats use the same expanded contract without a RAML parser at deployment. Keep the original source folder under version control and reload it to update the API; the wizard does not regenerate RAML source from edited OpenAPI. The Studio backend additionally installs the pinned `inflection` dependency from `backend/requirements.txt` for template transformations.

This is a bounded offline RAML importer, not a claim of complete RAML specification certification. Overlays/extensions, whole-object `queryString`, unsupported security mechanisms, custom validation facets and unsupported type facets must be converted before import. Missing files, cyclic expansion, unknown types/templates, YAML aliases/duplicate keys and unsupported declarations produce validation errors. No import reads arbitrary disk paths, fetches URLs or executes custom YAML tags.

Safely update an existing API

- Stop Run/Debug. Open API Blueprint, choose Update existing API and select its API-specific server connection.

- Load the revised contract or source folder, validate it, and select the operations to retain or add. Keep `(operationId)` stable in RAML when renaming a route; otherwise matching falls back to the existing method/path.

- Choose Preview update. The first preview saves pending Studio edits. Inspect added, changed, unchanged and removed operations, conflicts and warnings. Response behavior controls only newly generated operations.

- Acknowledge the review and choose Apply reviewed update. Host API then starts the updated implementation.

Matched tasks retain their IDs, names, packages, activities, transitions, groups, input/output schemas, business logic and mappings. Existing response mappings are preserved rather than replaced with examples. The dedicated connection retains its ID, transport, authentication and property bindings; its normalized contract changes. New operations receive new editable flows. Callback helpers retain stable IDs and custom mappings; obsolete callback helpers are retained with a warning so existing Call Sub Task references remain valid.

Removal of an implemented operation blocks apply instead of silently deleting its flow. Resolve removals explicitly or retain the operation in the contract. Ambiguous matches and object-name collisions also block apply. Schema changes require review and may require manual business-mapping changes; preserving a mapping does not guarantee that it satisfies a changed schema.

Apply is rejected while that project's Run/Debug session is active. A preview token binds the exact contract and selection to the saved project revision. If another editor changes the saved project, apply returns a conflict; Refresh preview reads the latest saved revision without writing an old Studio snapshot over it, and requires review again. A temporary project-directory snapshot restores the prior saved project if saving the update fails. If restoration itself fails, the server reports the retained recovery directory; keep that directory for recovery. This protects ordinary save failures, rather than guaranteeing recovery from every machine/storage failure.

Default confidentiality

On an HTTP client/shared connection, enable Default confidentiality / system trust to require HTTPS with server-certificate and hostname verification. System root trust remains available; a configured trusted-certificate folder augments it. No client identity certificate is needed unless the provider requires mutual TLS. An HTTP URL or disabled certificate verification conflicts with default confidentiality and fails configuration. When no port is configured, HTTPS uses 443; explicit ports are respected. The setting is also available as the boolean environment property `connections.http.defaultConfidentiality`.

This is MINA's Python equivalent of default TLS confidentiality, using the runtime host's system trust rather than a Java `cacerts` file. Certificate-folder trust applies to HTTP Request, REST Invoke and SOAP Request/Reply, in Studio and both Python exports.

Design and run a REST API

HTTP Send Response input headers

Every HTTP Send Response activity exposes these optional fields by default in Input, for both HTTP Listener and REST Receiver flows:

- HTTPHeaders -> Content-Type, complex StatusLine, and Cookies. Content-Type accepts common media types or a custom value. Cookies accepts one complete Set-Cookie string or an array of strings; map an array in Studio, for example `["session=abc; Path=/; HttpOnly", "locale=en; Path=/"]`. Each cookie is emitted separately.

- DynamicHTTPHeaders -> Headers* -> headerName, headerValue. Choose Add header for independent entries, duplicate the repeating Headers structure, map a collection, or use For Each. Each populated entry needs a name and a value; an empty string is a valid value. Completely empty entries are ignored. Header names are case-insensitive. Repeated names produce separate header lines; Content-Type must have a single value.

Legacy `headers`, `contentType` and `statusCode` mappings remain supported. HTTPHeaders overrides matching legacy settings. DynamicHTTPHeaders overrides matching standard/legacy headers, except Set-Cookie values append to Cookies. Unmapped optional fields keep the existing defaults. Response media types must still satisfy the endpoint contract and the caller's Accept header. Header names and values are validated; newline/control-character injection and malformed status lines fail the activity. The transport retains control of Content-Length, Transfer-Encoding and Connection. These mappings run identically in Studio Run/Debug and Direct/Engine Python archives.

Expand HTTPHeaders -> StatusLine in Input to map each optional child independently:

Field	Type	Example Input expression	Behavior
`httpVersion`	string	`"HTTP/1.0"` or `${input.version}`	Selects the actual response version on a native HTTP/1 listener (1.0 or 1.1). On HTTP/2 or HTTP/3 it must match the negotiated protocol. The `HTTP/` prefix is optional.
`statusCode`	integer	`201` or `${Receive.query.status}`	Overrides the legacy statusCode when supplied. Leave unmapped to retain the legacy value or default 200. Numeric literals have no quotes.

Field	Type	Example Input expression	Behavior
`reasonPhrase`	string	`"Stored by integration"` or `${input.description}`	Sends a custom reason phrase on native HTTP/1 listeners, including an explicitly empty

Field	Type	Example Input expression	Behavior
			string. Omit to use the standardstatus description. Newlines,control characters andnon-Latin-1 text are rejected.

Map the whole StatusLine object when an upstream activity already supplies these fields. An empty object or omitted children keeps the existing numeric status. Legacy full status-line string mappings, such as `"HTTP/1.0 201 Stored by integration"`, also apply their version and reason rather than discarding them. Mapped settings remain visible in Output -> statusLine and execution/debug data. Hosted final responses require status codes 200-599; 1xx informational responses are not sent by this activity.

`statusCode` controls the actual response status. Native HTTP/1 listeners now apply the mapped version and reason to the wire response. For example, version `"1.0"`, status `201` and reason `"Stored by integration"` send `HTTP/1.0 201 Stored by integration`. This works over HTTP and HTTPS. HTTP/1.0 responses use length-delimited bodies and `Connection: close`; header-only responses retain their normal bodyless semantics. File responses retain their known content length. Each connection/request has independent settings, so custom phrases cannot leak to another response or modify global status tables.

Listener/connection	Version override	Reason override
Native HTTP/1 listener (server connection version 1.1), HTTP or HTTPS	HTTP/1.0 or HTTP/1.1	Custom Latin-1 text or explicitly empty string
Negotiated HTTP/2	Must be HTTP/2 (2.0 alias accepted)	Omit or use empty string; nonempty phrases are rejected
Negotiated HTTP/3	Must be HTTP/3 (3.0 alias accepted)	Omit or use empty string; nonempty phrases are rejected
HTTP/1 fallback on a connection configured for HTTP/2 or HTTP/3, or an HTTP/1 ASGI preview/host without MINA's extension	Matching HTTP/1.1 only	Unsupported; omit the field

HTTP/2 and HTTP/3 carry a numeric `:status`, not a textual reason phrase ([HTTP/2](https://www.rfc-editor.org/rfc/rfc9113.html#section-8.3.2), [HTTP/3](https://www.rfc-editor.org/rfc/rfc9114.html#section-4.3.2)). A response mapping cannot upgrade or downgrade an established HTTP/2/3 connection: configure the server shared connection and client for that protocol instead. Unsupported settings produce an HTTP_RESPONSE 500 before response headers are committed, and the server log records the rejected setting. The workflow may already have run before its response is checked. Protocol-error responses use the transport's normal status line. Studio Run, Debug and both Python archive formats share these rules; ASGI-based previews cannot claim native status-line support unless their host advertises it.

Request and listener setup

For HTTP Request and REST Invoke, map the contract, operation ID, reference documents/origins and request/response validation settings under Input -> RestInputRequest.Config. Existing configuration values are shown as typed mappings. Configuration keeps the display name and shared connection selector. JSON/XSD validation and contract selection can use properties or upstream values; XSD validation uses offline lxml validation with self-contained schemas. Generated archives include the required dependencies. Listener routing stays in startup configuration.

Create a Starter Task and choose HTTP Listener or REST API Receiver. In endpoint setup, select the HTTP server shared connection, methods, route, supported request and response content types, and any request or response schema. Use Create new connection in the shared connection picker if needed. Studio creates the listener or receiver and an HTTP Send Response activity. Schema controls display the selected schema name. Selecting a project schema copies its current definition, so reselect it after editing that definition when the endpoint needs the update.

The receiver starts a task when its configured method and path match a request. Run starts a real listener on the shared connection's host and port, and Stop releases the port. Multiple exported receiver tasks can share a port when their transport

settings agree and their routes do not conflict.

Use HTTP Response on the same workflow path to reply to a REST Receiver's caller. Studio may name the generated node HTTP-Send-Response. Set the status code and map the response body and any headers there.

Select every method you want the receiver to accept. In Request parameters, use URI path to type a placeholder already in the route, such as `{id}` in `/orders/{id}`. Add Query entries for values such as `?details=true` and Custom header entries for values such as `X-Tenant`. Set the type and mark any required entries. The receiver returns HTTP 400 if a required value is missing or a declared value has the wrong type. The workflow receives the values in `pathParameters`, `query`, and `headers`, with header names normalized to lowercase. OpenAPI and Postman include these declarations.

Run and Debug show the live listener address. If the connection has a public API base URL, Studio shows that separately as the deployment address. Selecting HTTP on the shared server connection disables TLS and switches HTTP/2 back to HTTP/1.1.

REST Invoke and HTTP Request support URI parameters, default and activity headers/query parameters, JSON, text/XML, form, binary and file bodies, buffered/chunked transfer, response status validation, and optional request/response schema validation. XML requires a text body and the appropriate media type. REST invocation returns `statusCode`, `headers`, `body`, `elapsedMs`, and `httpVersion`; Untouched legacy HTTP Request activities retain their payload response; new requests and requests edited through the Input tree return the response envelope.

Optionally paste an OpenAPI 3 or Swagger 2 JSON/YAML contract and select its exact operation ID. The operation defines the HTTP method/path, required parameters, media types and request/response schemas. Invocation uses the connection's base URL when supplied, otherwise the contract server URL. A receiver combines its base path with the operation path; without a configured base path it uses the contract server's path. Body validation defaults to enabled. Parameter validation includes required values, arrays, objects, cookies, numeric/string constraints and formats. Operation-level parameters override path-level definitions. Response schemas select exact status, status class (for example `4XX`), then `default`, including error responses before status faults are raised.

Local references work offline. Supply external documents in Supplied external reference documents as a JSON object mapping document URI to document object or YAML text. For network references, configure Contract base URL for relative paths and explicitly list exact HTTPS origins in Allowed external reference HTTPS origins. No origins means no remote retrieval; redirects, URL credentials and implicit file access are forbidden. Retrieval uses the shared client's TLS trust, with document/count/size bounds; schema validation itself never fetches URLs. YAML aliases are rejected. The contract limit is 8 MiB, with at most 16 referenced documents and 32 MiB downloaded references. Mapping expressions do not execute YAML code. MIME bodies continue to use the explicit Input tree; OpenAPI does not automatically manufacture MIME mapping rules or perform browser authorization/consent.

The Studio preview URL uses the same validation and authentication as the real listener. It cannot establish mutual TLS for an individual project; use the configured HTTPS endpoint to test client certificates. Preview signatures use the project-relative listener path. The real endpoint's HTTP/TLS transport settings apply when Run hosts the dedicated listener or an exported application runs.

Request and response trees

Configuration holds the shared HTTP client connection and optional API contract and schema validation settings. Request values belong in Input, under `RestInputRequest`:

- `Config`: host, port, scheme, requestURI, Method, RequestBody, bodyFormat, query and URI parameters, QueryString, FilePath, timeout in milliseconds, and request policy overrides. Unmapped connection settings supply defaults.

- `Headers`: accept, content-type, Accept-Charset, Accept-Encoding, Cookie and pragma.

- `CustomHeaders`: choose Add header to create a row with separate name and value mappings. Add or remove rows independently, or map a source collection with For Each. Repeated custom headers retain all values. Duplicate entries for a nonrepeating standard header fail validation. Header names and values must be ASCII and cannot contain line breaks.

- `mimeEnvelopeElement`: repeating `mimePart` entries containing `mimeHeaders` and exactly one of binaryContent (base64), textContent or fileName. MIME headers include content-type, content-transfer-encoding, content-id and content-disposition. File names refer to files available on the runtime host. Multipart messages are buffered by default; `Config.streamRequest` enables bounded streaming within the configured size limit.

Legacy request configuration values appear as Input mappings when opened; editing saves them into the new structure. String constants require quotes; numbers and booleans use native literals, while expressions supply runtime values. Output exposes `RestOutputResponse` with statusLine (httpVersion, statusCode, reasonPhrase), body, asciiContent, binaryContent, Headers, repeating DynamicHeaders and parsed MIME parts. The runtime also retains the envelope's statusCode, headers, body, elapsedMs and httpVersion fields. Input and Output trees show optional and repeating cardinalities.

The request input uses the `RestInputRequest` root. Studio, Direct Python and Engine Python exports use the same request and MIME implementation.

For a request to `/orders/{id}`, map `Config.URIParameters` to an object such as `{"id":"A/B"}` and `Config.QueryParameters` to an object such as `{"tag":["open","priority"]}`. The slash in the ID is encoded as part of the path value, and both tags are sent. To send `X-Tenant: west`, open CustomHeaders, choose Add header, and map the name to `"X-Tenant"` and the value to `"west"`. A mapped query value overrides the same key from the connection or request URI.

Connection configuration

Area	Implemented configuration
Addressing	Client/server role, protocol, host, port, base URL and base path; activity path and methods
Confidentiality	System trust with default confidentiality, custom trust with a configured store, HTTPS enforcement, hostname verification, TLS 1.2/1.3 bounds, TLS 1.2 cipher list
Certificates	PEM trust certificates, PEM certificate chain/private key and key password, PKCS12 identity/trust store passwords, JKS store password and identity alias, CRL, optional/required inbound client certificates
Client timeouts	Separate connection, socket/read, write and pool wait timeouts in milliseconds; legacy second-based values remain accepted
Pooling	Total and per-host limits, persistent connections, idle expiry, disabled pooling, cookie persistence and response buffer size
Delivery	Redirect switch, bounded connection retries and retry delay, request/response size limits, expected content/accept types and success status codes
Proxy	Proxy URL or host/port and optional credentials
Server limits	Minimum/maximum QTP worker settings, concurrency and queue limits, queue timeout, accept backlog, persistent connection idle timeout, body read timeout, header/body size limits and IP/CIDR allowlist
Execution	Request execution timeout and contract validation

Zero client phase timeout means no independent phase deadline, subject to the activity timeout budget. The default activity budget is 180 seconds; synchronous client phase waits and response consumption are bounded, but this is not a hard wall-clock cancellation of a Python thread. Connection failures are retryable; configured status retries additionally require idempotency safeguards. Read failures are not automatically replayed. Host/port settings must refer to an address the runtime can actually bind or reach.

QTP is a Jetty concept. MINA implements its minimum/maximum settings with Python authentication workers, a bounded admission queue and an asynchronous execution gate. It does not embed Jetty or duplicate its scheduling semantics. HTTP/1.1 listeners use Uvicorn. Selecting HTTP version `2` on a server connection enables Hypercorn with HTTPS ALPN negotiation for HTTP/2 and HTTP/1.1. HTTP/2 listeners require HTTPS and use the same trust, identity and mutual-TLS configuration. TLS 1.3 cipher selection follows Python/OpenSSL defaults. Certificate revocation uses configured CRLs; OCSP fetching and Java security-provider configuration are not implemented.

PEM and PKCS12 work in Python. JKS conversion requires Java `keytool` on PATH or under JAVA_HOME. A selected JKS identity key must use the store password; convert stores with a distinct key password to PKCS12 first. Certificate/key paths are external deployment requirements; MINA does not bundle private keys or certificate stores into source archives.

Trust certificates from a property-bound folder

Set the string property `connections.http.trustedCertificateFolder` to your certificate folder, for example `D:\MINA\certificates\trusted` on Windows or `/etc/mina/certificates/trusted` on Linux. In the HTTP shared connection, bind Trusted root / chain certificate folder to `${properties.connections.http.trustedCertificateFolder}` using the property picker. Select that HTTP connection for HTTP Request, REST Invoke or SOAP Request/Reply. Enable HTTPS and keep certificate verification enabled.

Place trusted root certificates directly in the folder. Intermediate chain certificates are optional: normally the HTTPS server supplies them, but you can add them to the same folder if needed. A complete path to a trusted root is still required; an intermediate alone does not bypass root verification. Server/client identity certificates and private keys remain separate settings for hosting HTTPS or mutual TLS.

The folder accepts `.pem`, `.crt`, `.cer` and `.der` files, including PEM bundles and binary DER certificates. It requires no OpenSSL rehash operation. Subfolders are not scanned; unrelated file extensions are ignored. Empty, missing, unreadable or invalid certificate folders produce a configuration error. Keep private keys out of the trust folder. Limits are 256 certificate files, 8 MiB per file and 32 MiB total. Folder trust is added to system trust and any explicitly configured trust store.

New outbound calls detect certificate file additions, removals and file modification times and create a fresh TLS client when needed. Existing requests finish with their original TLS context. Inbound servers load folder trust at startup; restart them after changing client CA certificates. Configure an environment-specific property value and provision the folder on each deployment host. Studio and both Python archive formats resolve the same property and use the same folder-loading implementation; the folder itself remains an external deployment requirement.

Authentication

Scheme	Client	Server
Basic	Preemptive or challenge-based username/password	Constant-time credential comparison; empty credentials rejected
Digest / NTLM	HTTP challenge authentication	Not implemented
Bearer	Supplied access token	Supplied token comparison
OAuth2	Client credentials, supplied authorization code with optional PKCE verifier, refresh token; cached token exchange	HTTPS token introspection; active/expiry checks and required scopes
JWT	Supplied JWT, or generate a signed token with claims, issuer, audience and lifetime	Verify configured algorithm/key, expiration, issuer/audience when configured, clock skew
HMAC	Sign method, exact request path/query, timestamp, nonce and SHA256 body digest	Signature, timestamp and nonce replay checks
LDAP	Not an outbound REST authentication scheme	Basic credentials authenticated by verified LDAPS or StartTLS, service search/user DN template, optional required group
Certificate	TLS client identity	Required and verified mutual TLS

JWT algorithms are explicitly limited to HS256/384/512, RS256/384/512 and ES256/384. Asymmetric signing uses a PEM private key; verification uses a PEM public key. Configure strong shared secrets and the expected issuer/audience for your application.

OAuth token and introspection endpoints require HTTPS. Authorization-code acquisition/consent is performed outside MINA; provide the returned code, redirect URI and verifier when applicable. Token refresh is automatic when the provider supplies a refresh token. LDAP search escapes usernames, rejects anonymous binds and always verifies its TLS connection. LDAP group checking uses `memberOf` and depends on directory support.

HMAC uses MINA's explicit signing format, not AWS SigV4 or a vendor-specific HMAC policy. Send `X-MINA-Key-Id`, `X-MINA-Timestamp`, `X-MINA-Nonce`, and `Authorization: HMAC <base64 signature>`. The signed text is five newline-separated values: uppercase method, exact encoded path/query, Unix timestamp, nonce, and hexadecimal SHA256 body digest. Select SHA256/384/512 for the keyed signature. Signed requests cannot follow redirects. Select policy storage `redis` and bind Redis policy connection URL to a protected property for atomic nonce claims across replicas; `memory` is local to one runtime. Redis failures reject protected requests with 503 rather than silently disabling replay protection. Use `rediss://` for encrypted Redis connections.

Browser access, key rotation and throttling

Configure exact Allowed browser origins on a server connection to enable CORS. Credentialed CORS rejects wildcard origins. Preflight checks the requested method and headers before authentication; actual requests still authenticate normally, and allowed-origin authentication failures include CORS headers. CORS is a browser policy, not an alternative to authentication. Configure required browser headers and exposed response headers explicitly.

For JWT issuer key rotation, set an explicit HTTPS JWT verification JWKS URL, asymmetric signature algorithm, expected issuer and audience. The endpoint comes from configuration, never from token headers. Keys are cached for 300 seconds by default; an unknown `kid` can refresh after a default 10-second cooldown. Discovery is bounded to 1 MiB/128 keys and uses configured TLS trust. Symmetric JWKS algorithms, mismatched algorithms and unsuitable keys are rejected. Missing keys or discovery failures never bypass signature checks.

Set Requests per client/window above zero to enable fixed-window throttling. The key uses the effective client IP and route. Exceeding the limit returns 429 and `Retry-After`. Redis storage shares counts across replicas. Memory storage applies within one runtime. Set Trusted proxy IPs / CIDRs if a trusted proxy supplies client addresses.

Retries and circuit breakers

`retryCount` bounds retries. Retry response status codes is empty by default; explicitly select statuses such as `429,502,503,504` when appropriate. Response retries require an idempotent method, a mapped `Idempotency-Key`, or the explicit non-idempotent retry override. An idempotency header only helps when the destination enforces it. Backoff grows exponentially with optional jitter; `Retry-After` seconds/HTTP dates are honored within the configured maximum delay and remaining activity budget. Read failures are not automatically replayed. All retry attempts share one activity budget.

Set Circuit breaker failures above zero to open a runtime-local circuit after consecutive connectivity/server failures. Circuit recovery delay controls when one half-open probe may run; success closes the circuit and failure reopens it. Configuration/client validation errors do not count as server failures. Circuit state is local, while optional Redis throttling/replay state is shared.

Binary bodies and large files

Listeners preserve binary request bytes and expose base64 `binaryContent`, complete repeated `headerPairs`, and scalar-or-list query/header values. `?tag=a&tag=b` preserves both tags. Ambiguous duplicate Authorization/Host/Content-Type/Content-Length/Origin headers are rejected. Existing routes return 405 with `Allow` for unsupported methods; GET routes support HEAD and listeners answer OPTIONS automatically.

File request bodies (`Config.FilePath`) stream from disk. Set `Config.streamRequest = true` to stream MIME attachments; streaming MIME supports base64/binary/7bit/8bit transfer encoding and bounded nested envelopes. Streamed uploads require authentication that does not internally buffer the body: preemptive Basic, Bearer, OAuth2, JWT, HMAC or certificates. Challenge-authentication modes that require buffering reject streaming requests explicitly.

For downloads, map `Config.responseFile` to a runtime filename. Data streams into a sibling temporary file; successful completion publishes it atomically. Existing destinations are protected unless `Config.overwriteResponseFile = true`. Failures and size-limit errors remove the partial file. Output body contains `fileName` and `size`. Use buffered mode when response-schema validation is required; file mode cannot bypass that validation. Set `Config.responseBodyMode = "binary"` for a buffered byte body; structured output retains base64 `binaryContent`.

For large inbound bodies, select Inbound body storage = file and an existing writable Temporary upload directory on the server connection. The flow receives `fileName`/`size`; consume or move the file during that request. Temporary uploads are deleted on success, failure, cancellation or disconnect. This mode spools the complete wire entity, including a multipart entity, without buffering it or incrementally extracting its parts. Use memory mode for JSON/XSD body validation or automatic MIME-part decoding.

HTTP Response can map either `body`, base64 `binaryContent`, or `fileName` for streamed output. Repeating response-header values such as `Set-Cookie` remain separate headers. Size bounds still apply in all storage modes.

Swagger documentation and Postman

Studio's Samples -> HTTP & REST examples opens twenty editable host/caller projects directly. Each opening creates a new copy. Cards show provider dependencies and setup requirements. The Samples footer downloads the API setup guide and helper kit, including certificate preparation and local OAuth fixtures. For secure examples, prepare the helper kit and set the opened project's local `demo.directory` property to its generated `local` folder. No machine-specific certificate paths or generated private keys are bundled in the gallery.

Hosted HTTP and REST listeners automatically serve MINA API Explorer, with bundled Swagger UI and Try it out, at `http(s)://host:port/_mina/docs`. Run and Debug show the actual listener endpoint, an Open Swagger link and a Download Postman collection button. The download is a Postman v2.1 collection JSON; import it into Postman and supply any credential variables locally. The API Explorer also has a collection download button. No CDN is needed.

The OpenAPI document is at `/_mina/openapi.json` and the collection at `/_mina/postman.json`. These describe the currently hosted routes, including the selected API Blueprint operations, effective base path, parameters, request/response schemas, examples and authentication. Generic HTTP listeners receive generated operation descriptions. Swagger testing invokes the actual integration. In Debug it pauses at configured breakpoints; use Continue/Step in Studio to finish the request. One request can be debugged at a time; additional requests return 503. HTTP activity timeouts still apply; a timed-out debug request stops that debug session so a stale paused job cannot affect another request.

On the HTTP shared connection, Host Swagger documentation disables/enables documentation (enabled by default). `/_mina` is reserved when enabled. API documentation access defaults to authenticated: the specification and collection use the connection's authentication, while the HTML/assets are available to display the access panel. Public documentation access must be selected explicitly and does not bypass API authentication. Public API base URL optionally supplies an externally reachable origin and gateway prefix for generated links and collections. Configure your gateway to forward the documentation paths as well as the API routes.

Basic/LDAP credentials, access tokens and HMAC secrets entered in the API Explorer stay in that tab and are not saved in generated collections. Postman collections use empty credential variables. HMAC signing is supplied for raw JSON/text requests; multipart/file signing requires your own signing client. Browser HMAC signing needs HTTPS or localhost. Configure mutual-TLS client certificates separately in your browser/Postman, and trust the hosting server's TLS certificate. Browser security and CORS policies still apply. Outbound HTTP Request and REST Invoke activities call providers; they do not themselves host Swagger routes.

Swagger UI assets and their Apache-2.0 license/notices are included in both Direct and Engine Python archives. To update bundled assets after upgrading the pinned npm dependency, run `npm run swagger:assets` from `frontend` and rerun the API documentation/export tests.

Python deployment details

Studio, Direct Python archives and Engine Python archives use the same HTTP transport, authentication and listener implementation. Generated dependency checks include HTTPX, Uvicorn and the security libraries required by the configured connections. Provision external certificate/key/store files on the execution host, and supply exported secrets through the deployment environment/property bindings. Export retains endpoint URLs and key-file locations while removing secret values.

Direct archives can run with `python run.py` after extraction or as a Python zip application. Engine archives run with `python -m application.main`. Existing Administrator deployment commands continue to apply. Optional modules include PyJWT/cryptography, ldap3, httpx-ntlm and h2. Test against your own directory, identity provider, certificate chain and proxy before deployment; local automated tests exercise protocol behavior and failures, not access to external enterprise services.

Reference documentation and scope

Use the activity's Documentation tab for its own Configuration, Input, Output, and Errors reference. In Help -> MINA Documentation, search for HTTP and REST activity verification for the supported test matrix and remaining standards gaps.

OpenAPI security and multipart forms

Select a server connection that can enforce the imported operation's security. MINA uses operation-level OpenAPI security, or inherits the top-level declaration if omitted. Configure OpenAPI API key secrets as a protected JSON property keyed by security scheme name. API keys can be read from headers, query parameters or cookies. MINA requires all keys named in a selected alternative, together with its configured Basic/LDAP, Bearer/JWT/OAuth2 or mutual TLS requirements where applicable. The listener refuses startup if it cannot enforce any alternative. A shared connection supports one HTTP Authorization scheme at a time.

For a contract with `multipart/form-data`, map RequestBody by form property name. Arrays generate repeated parts, nested objects are JSON, and `format: binary` fields take base64. OpenAPI `encoding.contentType` controls the part type. Map multipartPartHeaders as an object keyed by form property when the contract declares extra headers. Declared defaults fill omitted outbound headers. MINA validates required headers and their schemas on both sides. Reserved framing and content headers remain controlled by the serializer. The receiving workflow gets typed `body` and `mimeEnvelopeElement`. Contract-driven forms are buffered and bounded.

Outbound query parameters with OpenAPI `allowReserved: true` preserve URI-safe reserved characters and valid percent escapes. Delimiters that could create another query parameter or a URL fragment remain escaped.

Shared HTTP policies

- Set Listener error response format to problem for RFC 9457 `application/problem+json` errors produced by the listener. Existing connections default to the original JSON error shape. Workflow responses still use the body and content type mapped in HTTP Send Response.

- Set Trusted proxy IPs / CIDRs only for proxies you control that maintain the `X-Forwarded-For` chain. MINA checks the immediate peer and walks the chain from right to left. The effective address appears in `clientIp` and applies to IP allowlists and rate limits. The original peer remains in `remote`.

- Enable Propagate HTTP trace context on the inbound and outbound connections to link version 00 `traceparent` headers across a workflow. The listener exposes `traceId`. This propagates identifiers and does not export trace spans.

- Enable Conditional GET and HEAD and map `ETag` or `Last-Modified` response headers to return 304 when the request validators match. The workflow still runs. The option does not cache responses or enforce write preconditions.

- HTTP Write Preconditions checks inbound headers against mapped datastore validators and returns `allowed` and `statusCode` (200, 412, or 428). Branch before any mutation. Use a version predicate or transaction lock in the actual write to prevent a concurrent change between the check and mutation.

- Shared public response cache reuses fresh public HTTP 200 GET responses in the runtime process. It partitions entries by destination and request headers, honors explicit freshness and skips authenticated, conditional, range and file requests. A successful mutation through a cache-enabled connection invalidates that origin's entries. Replicas each have their own cache.

- For OAuth client assertions, select private_key_jwt and set the registered PEM key path, asymmetric algorithm, and optional key ID. The runtime signs a short-lived assertion for the configured HTTPS token or introspection URL. Provide the signing key separately on the runtime host.

HTTP/3, event streams, and callbacks

Select protocol 3 for native HTTP/3. Hosting binds UDP and TCP on the configured port and needs an HTTPS PEM certificate plus unencrypted PEM private key. TCP also serves HTTP/2 and HTTP/1.1. Native outbound HTTP/3 requires direct, verified HTTPS and supports buffered requests and replies. Proxies, file streaming, mutual TLS, custom CRLs and custom cipher suites are outside the HTTP/3 implementation.

Use Read Server-Sent Events to read a bounded batch with an optional Last-Event-ID. Use WebSocket Exchange with a `ws://` or `wss://` URL to send an optional message and collect bounded replies. Binary replies are base64. These activities close their connection when finished. They do not publish SSE streams, host WebSocket endpoints, or reconnect automatically.

API Blueprint imports OpenAPI webhooks as selectable listeners under `/_webhooks/<name>`. Configure authentication on the server connection before hosting them. Callbacks on selected operations generate reusable Sub Tasks. Call one explicitly with `callbackUrl` and `body` and set the `connections.http.callbackAllowedOrigins` property to the approved HTTPS origins. Use the contract expression shown in the task description to map the URL. MINA validates the callback

contract, verifies TLS, and disables redirects. The generated task performs one workflow-managed delivery rather than creating an automatic subscription or durable delivery queue.

Deployment qualification

The activity-level suite `tests.test_http_activity_e2e` exercises HTTP Listener and REST Receiver through both HTTP Request and REST Invoke, returning through HTTP Response on real local listeners. It also covers Studio contract preview, HMAC encoded request targets, HTTPS preview enforcement, multiple content types, strict JSON, bodyless responses and fresh Direct/Engine archives.

REST contracts retain all declared request/response content types and validate against the actual Content-Type, using exact media types before type wildcards and `*/*`. Swagger exposes all declared formats. Set HTTP Response's content type explicitly when returning a representation other than the configured default; the workflow chooses the response representation. Explicit request-schema overrides remain effective. Optional absent request bodies are accepted; required bodies cannot be bypassed with Send Body Mode = NEVER/AUTO.

In HTTP Listener and REST Receiver setup or Configuration, check one or more Request content types and Response content types. You can add custom types alongside the common ones. No selection keeps automatic behavior. With multiple response types, MINA uses the caller's `Accept` header and its quality values; the first checked format is the default. An explicitly mapped HTTP Send Response content type must be one of the offered types and acceptable to the client. For XML output, map an XML document or an object with one root element. With mixed media types, a standalone XSD applies to XML and a standalone JSON Schema applies to JSON; use an OpenAPI contract for separate schemas by media type. Swagger lists each selected format, and the Postman export includes a request for each selected request format.

URL-encoded form fields use declared primitive schema types before validation, including repeated primitive arrays. Direct archives also accept the default HTTP Response operation, matching runtime behavior.

When body format is omitted, outbound text/XML content is sent as text and URL-encoded form content is encoded as form data. Explicit body format settings retain precedence. Text responses such as `42` remain strings. Malformed nonempty JSON and NaN/Infinity are rejected; binary response mode retains raw content. HEAD, 204, 205 and 304 responses skip body schema validation because they have no response content. TRACE is supported as an optional API operation; CONNECT proxy tunneling is not offered as an API method.

Studio preview resolves methods and paths from the selected API contract, including HEAD, OPTIONS and 405 behavior. Signed previews use the actual encoded public request target. A connection requiring HTTPS must use an HTTPS endpoint; plain HTTP preview cannot bypass confidentiality.

An explicit HTTPS server scheme always enables TLS, even if a legacy TLS checkbox value is false. A missing server certificate/key then fails startup instead of silently opening a plaintext listener.

Run `python -m unittest tests.test_http_capabilities tests.test_http_request_tree tests.test_http_production` from `backend` with its virtual environment active. The suite covers local TLS/mTLS, HTTP/2 negotiation, HTTP contracts, binary/repeating values, streaming/file cleanup, policy behavior and Python exports. Redis protocol calls are tested with an injected store; provision and exercise the actual Redis service, issuer/LDAP, certificates, proxy and upstream API in staging.

Export Direct or Engine Python and install the archive's generated dependencies. HTTP contract support requires `jsonschema`, `PyYAML` and `regex`; HTTP/2 hosting adds `hypercorn`/`h2`; distributed policies add `redis`. Keep certificate files and secrets external. Supply the Redis URL through the generated secret binding, never an unprotected source literal.

Use the Direct archive's `--qualify-task <task-id>` runner against the staging API with representative payloads, concurrency, latency/error thresholds and memory limits. Test the Engine archive with the same fixtures and deployment load harness. Repeat under upstream outages, token/key rotation, Redis failure and deployment restart. Record the report with the release. Automated regression success is evidence for the implemented behavior, not a guarantee about an untested production environment or third-party service.

Reference source: docs/HTTP_REST_CAPABILITIES.md

HTTP and REST activity verification

Connectors

Guide

HTTP and REST activity verification

The verification covers HTTP Listener, REST Receiver, HTTP Request, REST Invoke and HTTP Response in Studio, native Python listeners, Debug, Swagger/Postman generation and both Python archive formats. Live checks use local providers and generated local certificates. External enterprise services still need validation in their deployment environment.

Activity combinations

`backend/tests/test_http_activity_e2e.py` sends real requests through these four flows. Each receives a JSON request, calls a separate local provider with a mapped body/custom header, and returns the provider response through HTTP Response.

Inbound activity	Outbound activity	Response activity
HTTP Listener	HTTP Request	HTTP Response
HTTP Listener	REST Invoke	HTTP Response
REST Receiver	HTTP Request	HTTP Response
REST Receiver	REST Invoke	HTTP Response

Confirmed gaps corrected

Gap	Resulting behavior
Only one OpenAPI request/response representation retained	Keep all declared representations; choose the schema from actual Content-Type, with exact/type-wildcard/global-wildcard precedence
Text/XML bodies implicitly JSON-quoted	Infer text, XML and URL-encoded form serialization when no explicit body format is supplied
Numeric-looking plain text parsed as JSON numbers	Preserve text responses as strings
HEAD/204/304 rejected by payload schema validation	Skip response-body validation when HTTP forbids response content
Optional missing body rejected by an object schema	Validate supplied bodies; enforce presence independently for required bodies
URL-encoded numeric/boolean/array fields validated as raw strings	Decode primitive form fields using their declared schema types before validation
Required mapped body could be suppressed after validation	Apply Send Body Mode before validating required presence
Malformed JSON silently returned as text	Reject malformed nonempty JSON and non-finite JSON numbers; explicit binary mode keeps raw content
Non-finite OAuth introspection expiry accepted	Require a finite expiry later than the current time when exp is present
Preview ignored contract-derived methods	Resolve effective contract paths/methods and use the shared listener protocol handler
Preview did not mirror HEAD/OPTIONS/405 behavior	Use the same method dispatch as native listeners
HMAC preview rewrote the signed target	Verify the actual public encoded path/query sent by the client

Gap	Resulting behavior
HTTP preview could bypass an HTTPS connection setting	Reject plain HTTP preview for HTTPS-required connections
Explicit HTTPS scheme could be contradicted by a false TLS flag	HTTPS scheme always enables TLS; missing server identity fails startup instead of opening a plaintext listener
TRACE exposed inconsistently; CONNECT offered as an API method	Support TRACE through contracts, hosting, invocation and documentation; remove CONNECT tunneling from editor choices
Default HTTP Response accepted at runtime but rejected by Direct export	Allow the unambiguous default HTTP Response operation in Direct archives

Swagger documents every declared content type. The workflow still chooses its response representation: set HTTP Response Content-Type to the representation actually returned. This does not automatically implement Accept-header negotiation or business logic.

Capability coverage

Capability	Evidence
Connection/activity/Input-tree configuration resolution	Request-tree tests and the live activity matrix
HTTP/HTTPS; root-folder trust; hostname verification; mutual TLS	Local TLS tests and runnable sample checks
Basic, Bearer/JWT, OAuth2 and HMAC	Authentication tests; local Basic/JWT/HMAC and OAuth token/introspection sample services
Digest, non-preemptive Basic, NTLM/LDAP integration paths	Existing transport tests; LDAP/provider calls use injected test clients
Path/query/header serialization and repeated values	Contract, request-tree and production HTTP tests
JSON, XML/XSD, plain text, form, binary and MIME	Contract, protocol, streaming and activity-level tests
Status/header propagation, HEAD, OPTIONS, method rejection	Listener and preview tests
Timeouts, connection limits, queues and cancellation cleanup	Transport/server and production tests
File streaming, MIME limits and upload/download cleanup	Production streaming tests
Retry limits, idempotency controls, circuit breaking, CORS, IP rules and throttling	Policy/production tests; distributed store calls are injected
HTTP/2 over TLS	Live local ALPN/HTTP2 test
Swagger, OpenAPI and Postman export	API documentation tests and exported live API tests
Live HTTP Debug pause/continue	API documentation Debug test
Direct/Engine archive compatibility	Fresh archive subprocesses hosting actual API routes; text contract and offline docs checks
Studio Samples	Installed template checks and executable sample verifier

Reproduce the checks

From `backend`, with its Python environment active:

```
python -m unittest tests.test_http_activity_e2e tests.test_api_docs tests.test_api_blueprint
tests.test_http_capabilities tests.test_http_request_tree tests.test_http_production
tests.test_raw_python tests.test_engine_export tests.test_activity_packs tests.test_debug_testing
tests.test_debug_input_capture -q
```

From the repository root, after sample certificate preparation:

```
backend/.venv/Scripts/python.exe samples/http-rest-examples/verify_examples.py
```

On Linux use the corresponding environment's Python executable. The sample verifier opens loopback ports 9001-9003 and 9443-9449; keep these free. Fixtures are shut down afterward. From `frontend`, run the Node test suite and `npm run build` to verify editor contracts and the distributable Studio assets.

Deployment boundaries

The checks do not certify a particular external OAuth issuer, LDAP directory, proxy, Redis service, certificate/key rotation process or production load profile. Test those with actual staging infrastructure and representative payloads, concurrency, expiry and outage cases.

This is a request/response API implementation. The table below identifies the remaining protocol and contract boundaries. Certificate and authentication settings must be available on the execution host when deploying an exported application.

Standards coverage and deployment limits

The activities cover ordinary HTTP/REST request and response integrations. This table separates protocol defects from capabilities needed by particular API contracts or deployment environments.

Priority	Gap	Current behavior and impact
Security scope	Imported OpenAPI security	Hosting rejects unenforceable requirements. A supported alternative can combine multiple OpenAPI API keys with the connection's Authorization scheme and mutual TLS. The connection supports one Authorization scheme at a time.
Corrected	HTTP 205 Reset Content	Listeners suppress response content, and clients skip body processing and validation. Generated OpenAPI omits content for 205.
Supported for buffered forms	OpenAPI serialization and multipart	Outbound query values honor `allowReserved` for safe wire characters. Contract-driven forms apply and validate declared non-reserved part headers and content types.
Implemented as a setting	Problem Details	Opt in to RFC 9457 responses for listener-generated errors. Existing connections retain their legacy JSON format.
Implemented as settings	Trace context and proxy trust	Version 00 W3C trace context can follow inbound and outbound activities. Explicit proxy CIDRs enable trusted X-Forwarded-For client attribution. Span export and other forwarded-header formats are outside this feature.
Implemented as a setting	OAuth client authentication	The runtime can sign a private-key JWT assertion for the configured HTTPS endpoint. Real provider registration, credential rotation and interoperation remain staging checks.
Opt-in	Conditional requests and caching	Conditional GET/HEAD uses mapped validators. A dedicated activity evaluates write preconditions before a datastore write. The bounded process cache serves fresh public GETs and does not perform stale serving or distributed invalidation.
Supported with limits	Callbacks and webhooks	Named webhooks become hosted endpoints. Callback Sub Tasks require explicit workflow calls and an HTTPS allowlist. Durable subscriptions and automatic callback delivery remain outside the scaffold.
Bounded clients	SSE and WebSocket	Activities read SSE events or exchange WebSocket messages and close after their configured limits. Inbound SSE publishing and WebSocket hosting are outside these activities.

Priority	Gap	Current behavior and impact
Native transport	HTTP/3	QUIC hosting and direct buffered clients use TLS 1.3. Mutual TLS, custom CRLs or cipher suites, proxies, and outbound file or event streaming are outside HTTP/3 support.

Standards references

- [OpenAPI media type matching](https://spec.openapis.org/oas/v3.1.2.html#request-body-object)

- [OpenAPI parameter serialization and security requirements](https://spec.openapis.org/oas/v3.1.2.html)

- [HTTP 205 Reset Content and conditional requests](https://www.rfc-editor.org/rfc/rfc9110.html#section-15.3.6)

- [HTTP caching](https://www.rfc-editor.org/rfc/rfc9111.html)

- [HTTP/3](https://www.rfc-editor.org/rfc/rfc9114.html)

- [Problem Details for HTTP APIs](https://www.rfc-editor.org/rfc/rfc9457.html)

- [OAuth JWT client authentication](https://www.rfc-editor.org/rfc/rfc7523.html#section-2.2)

- [W3C Trace Context](https://www.w3.org/TR/trace-context/)

Reference source: docs/HTTP_REST_ACTIVITY_VERIFICATION.md

MINA security review - 2026-09-22

Security

Guide

MINA security review - 2026-09-22

Outcome and scope

Production blockers found, particularly for shared/multi-team deployments. Reviewed source at commit `676c0bf` with the existing working-tree changes present. This is a security-focused review across Studio/runtime, debugger, Control Plane, local/Kubernetes agent, Electron IPC, package handling, connector execution, secret storage, and selected dependencies. It is not an exhaustive certification that all code or dependencies are safe. No application security fixes were applied in this review. Only this report was added.

Checks used static source inspection, mocked in-process authorization checks, synthetic credentials, installed-package metadata, upstream advisories, and `npm audit --json --ignore-scripts`. No real credentials were printed, no live deployments were modified, and no destructive or resource-exhaustion tests ran.

Severity below assumes the affected service is reachable or the stated attacker role exists. Firewall, reverse proxy, OS permissions, and deployment topology can reduce exposure, but do not fix the underlying application boundary.

Findings

F1 - Critical: unauthenticated Control Plane owner access by default

Evidence: `administrator/app/main.py:66,141-157` uses an empty API key by default and returns the Technology Team Owner identity when no credential is provided. `administrator/run_admin.py:6` binds to `0.0.0.0:9080` by default.

Impact: Anyone able to reach an unconfigured instance can obtain governance and deployment privileges, including access to agent deployment responses that contain decrypted deployment secrets. The risk is not restricted to localhost.

Validation: With `API_KEY` mocked to empty, an anonymous synthetic request resolved to `roles=['Owner']`. No live server configuration was changed.

Fix: Fail closed when credentials are missing. Restrict initial bootstrap to loopback plus a one-time credential; require authenticated setup before exposing network access. Keep management APIs behind TLS and an authenticated boundary.

F2 - Critical: deployed application code inherits management credentials and host privileges

Evidence: `administrator/app/main.py:1238-1243` copies the full administrator environment into application subprocesses. `administrator/agent/main.py:173-184` does the same for agent deployments. Agent configuration reads `MINA_AGENT_KEY` from that environment (`administrator/agent/main.py:39`). Raw Python deployments are started under the same OS identity, not a separate tenant security boundary.

Impact: A malicious or compromised deployed application can read management credentials such as the admin API key, secret-encryption key, or agent key when configured via environment. Agent endpoints currently require a Technology Team credential, not a narrowly scoped per-plane identity. Consequently an inherited credential can expose other teams' deployments and secrets. The same OS identity also permits access to other application files accessible to the service account; clearing environment variables alone is insufficient isolation.

Validation: Source-confirmed inheritance and privileged agent authorization. No real credential extraction or malicious deployment was attempted.

Fix: Explicit child-process environment allowlist, per-plane scoped credentials, and isolated low-privilege application identities/containers with separate filesystem access. Never expose management or encryption keys to application code. Treat packages, scripts, and vendor drivers as executable code requiring approval.

F3 - High: deployment move bypasses team namespace authorization

Evidence: `administrator/app/main.py:1735-1748` checks deployment ownership, the target plane's registered namespaces, and the capability, but omits `team_can_use_namespace`. Initial deployment creation checks this permission at `administrator/app/main.py:1160`.

Impact: An Application Manager can move its stopped deployment to a registered namespace not assigned to its team, bypassing the intended deployment-placement boundary. Availability of valid plane/capability IDs is required.

Validation: A mocked Team A request successfully moved its own deployment to `team-b`; a mocked `team_can_use_namespace` returning false was never called. All persistence/audit functions were mocked; no real deployment moved.

Fix: Apply the same team/namespace/capability authorization to move, redeploy, restore/revert, and start. Recheck immediately before execution as permissions may change after deployment creation. Add negative cross-team route tests.

F4 - High: Studio management/runtime APIs have no authentication boundary

Evidence: `backend/app/main.py:575-617` exposes project listing, creation, editing, and execution without authentication dependencies. The middleware near line 80 controls caching, not access. Workflows can execute Python and commands (`backend/app/runtime.py:2030,2089`).

Impact: A caller who can access the runtime HTTP endpoint can read projects and configure/execute operations with the runtime account's permissions. The packaged sidecar binds to loopback (`backend/run_sidecar.py`), which limits remote exposure but does not authenticate other local users/processes. Exposing this same API for browser access without an authenticated proxy broadens the risk. Cross-origin browser exploitation was not established in this review.

Validation: Unauthenticated in-process requests to project listing and a synthetic memory-only connection test both returned HTTP 200. No shell commands or real connector operations were invoked through these endpoints.

Fix: Per-launch authenticated desktop session credentials, Origin/Host checks, and explicit authenticated user access/RBAC for browser deployments. Keep untrusted workflow code out of the privileged Studio/management process.

F5 - Medium: project exports and workspace files contain plaintext secrets

Evidence: `backend/app/store.py:46-58` serializes resource configurations and property values without encryption. `backend/app/main.py:689-700` writes full project/resource/property models into the project export without redaction.

Impact: Shared `.mpackage` exports and workspace backups can disclose database, EMS, cloud, and other connector credentials. This concerns project exports; it does not imply every deployment archive's secret handling is identical.

Validation: A synthetic password in a mocked project resource was present in the exported `project.json`. No real project secret was read or displayed.

Fix: Redact secret values by default on project export, preserve secret references, and make any sensitive export an explicit protected operation. Use OS-backed or managed secret storage and restrict workspace ACLs.

F6 - High: Studio project import has unbounded memory/decompression exposure

Evidence: `backend/app/main.py:1475-1494` calls `await file.read()` with no application limit and decompresses manifest/project ZIP entries without checking uncompressed size or compression ratios. By contrast, Control Plane and agent archive handlers implement explicit member and expanded-size limits.

Impact: A reachable import endpoint can be used to consume large amounts of memory/CPU using oversized or highly compressed uploads, disrupting Studio and its runtime. A reverse-proxy compressed-body limit alone is insufficient.

Validation: Source-confirmed. No ZIP bomb or large-memory attack was executed.

Fix: Limit upload bytes, entry count, expanded size and per-entry size before materialization; use bounded streaming reads, concurrency limits, and quotas.

F7 - Medium: installed Python web dependencies have known advisories

Evidence: The inspected backend environment has FastAPI `0.115.12`, Starlette `0.46.2`, and python-multipart `0.0.20`. Both backend and administrator requirements pin FastAPI `0.115.12` and python-multipart `0.0.20`.

- Starlette's multipart rollover can block the event loop: CVE-2025-54121,

fixed in `0.47.2`. The installed `0.46.2` is affected. [Maintainer advisory](https://github.com/Kludex/starlette/security/advisories/GHSA-2c2j-9gv5-cj73).

- python-multipart has Content-Disposition parameter interpretation issues

covered by CVE-2026-53537, fixed in `0.0.30`. [Maintainer advisory](https://github.com/Kludex/python-multipart/security/advisories/GHSA-vffw-93wf-4j4q).

- The separate python-multipart arbitrary-file-write advisory CVE-2026-24486

requires non-default `UPLOAD_DIR`/`UPLOAD_KEEP_FILENAME` settings. Those settings were not found in the reviewed application, so this is not being claimed as a demonstrated arbitrary-file-write exploit here. [Advisory](https://github.com/advisories/GHSA-wp53-j4wj-2cfg).

Fix: Upgrade to a mutually compatible, patched FastAPI/Starlette/multipart set and regression-test uploads, forms, and packaged builds. Scan resolved Python dependencies and shipped artifacts in CI rather than only requirements text.

Additional hardening observations

- Electron uses context isolation, disabled Node integration, and sandboxing,

which are positive. However, file-write IPC accepts caller-provided paths (`frontend/electron/main.cjs:158-170`) without sender-frame authorization, and navigation/window-opening restrictions were not found. A compromised renderer could abuse the bridge. A renderer compromise was not demonstrated here.

- Control Plane UI stores its credential in localStorage. Reduce exposure with

scoped/session credentials and a suitable CSP; do not assume HTML escaping alone provides a complete browser security boundary.

- TLS enforcement, firewall rules, Windows ACLs, Unix permissions, Kubernetes

RBAC/network policies, and supply-chain/signing controls need deployment-level verification. They cannot be certified from this source checkout.

- Existing archive path/member checks, Control Plane encrypted secret storage,

hashed team tokens, ownership checks on many routes, and default SSH host-key rejection are useful controls. Their presence does not mitigate the specific bypasses above.

Dependency-check results and limitations

`npm audit --json --ignore-scripts` completed successfully and reported zero known advisories across 478 dependency entries. This is not a claim that the frontend or Electron bridge is vulnerability-free.

`pip-audit` is not installed in the backend environment. No package installation or application dependency mutation was performed. Python checks were limited to installed metadata and selected verified upstream advisories, not a full resolved dependency audit. Vendor SAP/EMS/JDBC JARs, native libraries, bundled Java/Python, and the actual Windows installer were not fully audited.

Recommended order

- Before shared deployment: close anonymous Control Plane access and isolate

application processes from management credentials and filesystem privileges.

- Fix namespace authorization on move and audit every lifecycle route.

- Authenticate Studio APIs and restrict network exposure.

- Patch the web dependency stack and bound import/upload processing.

- Redact/export secrets safely, harden Electron IPC/navigation, and add security

regression tests plus complete Python/JAR/native artifact scans.

No live-server compromise was demonstrated or required. The findings are enough to block a claim that the current product is safe for untrusted multi-team hosting.

Reference source: docs/SECURITY_REVIEW_2026-09-22.md

Web dependency security update

Security

Guide

Web dependency security update

The Studio backend, browser runtime, and Administrator now pin the same reviewed web stack: FastAPI 0.141.1, Starlette 1.6.0, and python-multipart 0.0.32. This replaces FastAPI 0.115.12 and python-multipart 0.0.20 and explicitly pins Starlette instead of relying on an indirect dependency.

Upstream references:

- [Multipart upload rollover denial of service](https://github.com/Kludex/starlette/security/advisories/GHSA-2c2j-9gv5-cj73)

- [URL-encoded form limits](https://github.com/Kludex/starlette/security/advisories/GHSA-82w8-qh3p-5jfq)

- [Windows StaticFiles UNC handling](https://github.com/Kludex/starlette/security/advisories/GHSA-wqp7-x3pw-xc5r)

- [Multipart disposition parsing](https://github.com/Kludex/python-multipart/security/advisories/GHSA-vffw-93wf-4j4q)

Applying the update

For source installations, stop the service, install its requirements with its own Python environment, then restart it. From the repository root on Windows:

```
& .\backend\.venv\Scripts\python.exe -m pip install -r backend/requirements.txt
& .\backend\.venv\Scripts\python.exe scripts/verify-web-dependencies.py
& .\administrator\.venv\Scripts\python.exe -m pip install -r administrator/requirements.txt
& .\administrator\.venv\Scripts\python.exe scripts/verify-web-dependencies.py
```

For a browser-only installation use `backend/requirements-browser.txt` instead. For packaged installations, rebuild and redistribute the desktop/Administrator installer. Already installed executables retain their old bundled dependencies. Existing deployment environments must update their own Python dependencies too. Do not disable certificate verification to install dependencies.

Windows packaging scripts now verify the installed web versions before bundling; dependency-install failures stop packaging rather than reusing stale packages.

Verification and limits

Security regressions cover multipart upload functionality, offloaded disk rollover, URL-encoded field count/size enforcement, consistent dependency pins, and stale build rejection. Backend runtime and Administrator tests pass with the new stack. This is not a full security certification or remediation of other audit findings. Request/body size limits, authentication, and deployment hardening remain necessary.

Local environment caveats: the Administrator virtual environment points to a missing Python 3.11 installation, so Administrator tests were run with the updated backend environment. Repair/recreate that environment before building it. The backend Python 3.14 environment also reports platform compatibility issues for httptools, PyYAML, watchfiles, and websockets during `pip check`; these packages were not changed by this update. A clean supported build environment and installer smoke test are still required before release.

Reference source: docs/WEB_DEPENDENCY_SECURITY.md